LinkStudio

A modular physical-layer communication simulator

Theory and User Manual



Wireless · Wireline · Optical · Coding · Synchronization · MIMO

Engine v0.4 · October 25, 2026

Contents

About this manual vii

I Using and extending LinkStudio 1

1 Overview and quick start 2
1.1 What LinkStudio is . . . 2
1.2 Installation . . . 3
1.3 Three ways to run a simulation . . . 3
1.4 Repository map . . . 4

2 The simulation engine 5
2.1 Signals . . . 5
2.2 Blocks . . . 5
2.3 The registry . . . 7
2.4 Graphs . . . 7
2.5 Subsystems . . . 9
2.6 Simulations and stop criteria . . . 9
2.7 Sweeps . . . 10
2.8 Metadata propagation . . . 11
2.9 Scenarios . . . 11

3 Graphical editor, API and command line 13
3.1 The graphical editor . . . 13
3.2 The HTTP and WebSocket API . . . 16
3.3 Command line . . . 17

4 Developer guide: writing blocks 18
4.1 A complete example: free-space path loss . . . 18
4.2 Conventions . . . 19
4.3 Testing a block against theory . . . 19
4.4 Adding a scenario . . . 20

II Theory and implementation 21

5 Baseband signal model, noise and Monte-Carlo estimation 22
5.1 The complex envelope . . . 22
5.2 Additive white Gaussian noise . . . 22
5.3 Monte-Carlo estimation of error rates . . . 24
5.4 Closed-form references . . . 25

6 Digital modulation and demodulation 26
6.1 Constellations . . . 26
6.2 Optimal detection . . . 27
6.3 Error probability over AWGN . . . 28

7 Pulse shaping and matched filtering **29**
7.1 Linear modulation . . . 29
7.2 The Nyquist criterion . . . 29
7.3 The matched filter . . . 29
7.4 Discrete-time implementation . . . 30
7.5 Eye diagrams . . . 31

8 Radio front-end impairments **33**
8.1 Carrier frequency and phase offsets . . . 33
8.2 Oscillator phase noise . . . 33
8.3 IQ imbalance . . . 34
8.4 Power amplifier nonlinearity . . . 34
8.5 Analog-to-digital conversion . . . 35
8.6 Timing offset . . . 36

9 Fading channels and equalization **37**
9.1 The multipath channel . . . 37
9.2 Rayleigh and Rician fading . . . 37
9.3 Tapped delay lines and 3GPP profiles . . . 39
9.4 Path loss and link budgets . . . 40
9.5 Error probability with fading . . . 41
9.6 Equalization . . . 41

10 Synchronization **44**
10.1 The preamble . . . 44
10.2 Symbol timing recovery . . . 44
10.3 Frame synchronization and coarse CFO . . . 47
10.4 Carrier phase tracking . . . 47

11 OFDM **49**
11.1 Cyclic prefix and circular convolution . . . 49
11.2 Subcarrier layout . . . 50
11.3 Channel estimation and equalization . . . 50
11.4 Blind timing and frequency synchronization . . . 51
11.5 Peak-to-average power ratio . . . 52

12 Multiple antennas **54**
12.1 The flat-fading MIMO channel . . . 54
12.2 Detection for spatial multiplexing . . . 54
12.3 Diversity . . . 55

13 Channel coding **57**
13.1 Cyclic redundancy check . . . 57
13.2 LFSR sequences and scrambling . . . 57
13.3 Interleaving . . . 58
13.4 Block codes: repetition and Hamming . . . 58
13.5 Convolutional codes and the Viterbi algorithm . . . 58
13.6 LDPC codes . . . 60
13.7 Polar codes . . . 62
13.8 Comparison . . . 63

14 Wireline communications **65**
14.1 Line codes . . . 65
14.2 The cable as a transmission line . . . 65
14.3 Crosstalk . . . 69
14.4 Full duplex: hybrid echo and echo cancellation . . . 69
14.5 SerDes equalization . . . 69
14.6 Discrete multitone (DSL) . . . 71

15 Optical fiber communications **73**
15.1 Signals, power and noise . . . 73
15.2 The optical fiber . . . 75
15.3 Semiconductor lasers . . . 82
15.4 The VCSEL . . . 86
15.5 External modulators . . . 90
15.6 Photodetection . . . 93
15.7 IM/DD system performance . . . 95
15.8 Short-reach links: VCSELs and multimode fiber . . . 98
15.9 Optical amplifiers . . . 103
15.10 Multi-span links and the OSNR . . . 105
15.11 Raman amplification . . . 107
15.12 Coherent systems . . . 108
15.13 Nonlinear interference and the GN model . . . 115
15.14 Wavelength-division multiplexing . . . 118
15.15 Free-space optics . . . 119
15.16 Summary of the optical models . . . 121

16 Validation **122**
16.1 Running the checks . . . 122
16.2 Physics checks . . . 122
16.3 Known limitations . . . 124

III Reference **125**

A Block reference **126**
A.1 SOURCES . . . 126
A.2 ENCODERS . . . 126
A.3 BIT PROCESSING . . . 127
A.4 MAPPERS . . . 127
A.5 MULTIPLEXERS . . . 127
A.6 FRAMING . . . 127
A.7 MODULATORS . . . 128
A.8 FILTERS AND RESAMPLERS . . . 129
A.9 AMPLIFIERS AND ATTENUATORS . . . 129
A.10 TRANSMISSION MEDIA . . . 129
A.11 IMPAIRMENTS . . . 130
A.12 DEMODULATORS . . . 131
A.13 SYNCHRONIZATION . . . 132
A.14 EQUALIZERS . . . 132
A.15 DETECTORS AND ESTIMATORS . . . 133
A.16 DEMULTIPLEXERS . . . 133

A.17 DEMAPPERS 133
A.18 DECODERS 133
A.19 MEASUREMENTS 134

B Metadata keys **135**

C Scenario library **137**

Bibliography **139**

About this manual

LinkStudio is a block-based simulator of the physical layer (PHY) of communication systems. A simulation is a directed graph of blocks: sources, modulators, pulse shapers, channels, synchronizers, equalizers, decoders and meters. The same engine covers radio links, copper lines and optical fiber. It is driven from Python, from YAML scenario files, from the command line or from a browser-based editor.

This manual has two goals and is organized accordingly.

Part I – Using and extending LinkStudio. How the engine works (signals, blocks, graphs, Monte-Carlo runs, sweeps, scenarios) and how to drive it from the GUI, the CLI and the Python API. It ends with a developer guide for writing new blocks.

Part II – Theory and implementation. One chapter per subsystem. Every chapter first develops the theory with the notation used in the code, then shows *where* and *how* it is implemented, and finally validates the implementation against a closed-form result. Part II can be read as a compact textbook on digital transmission whose equations can all be checked in the code.

How to read the chapters of Part II

Each theory chapter follows the same pattern:

1. **Model.** The physical or mathematical model, derived with the assumptions stated explicitly.
2. **Implementation.** A blue *In the code* box names the blocks, files and metadata involved. Short excerpts show the lines that realize the key equations, for example the split-step solver of the nonlinear Schrödinger equation (Listing 15.4). Excerpts are extracted automatically from the source, so their file names and line numbers match the code base this manual was built from.
3. **Validation.** A figure comparing the simulator with theory. All measured curves in this manual (markers in the plots) are produced by running the real blocks with `manual/tools/generate.py`; solid lines are closed-form results.

Green *Key idea* boxes summarize a result; orange *Watch out* boxes point at common mistakes or at modelling limitations.

Typographic conventions

`awgn`	a block type (as written in scenarios and in the GUI palette)
`snr_db`	a block parameter
`noise_var`	a metadata key travelling with a signal
`linkstudio/phy/optical.py`	a file, relative to `backend/`
SAMPLES	a signal kind (Section 2.1)

Notation

$x(t)$, $x[n]$	complex envelope (continuous, sampled)	f_s	sample rate
T, $R_s = 1/T$	symbol period, symbol rate	N_{sps}	samples per symbol
E_s, E_b	energy per symbol, per information bit	N_0	one-sided noise PSD
M, $k = \log_2 M$	constellation size, bits per symbol	R	code rate
$\mathcal{CN}(\mu, \sigma^2)$	circular complex Gaussian	$\mathrm{Q}(x)$	Gaussian tail function
$(\cdot)^*$, $(\cdot)^{\mathsf{T}}$, $(\cdot)^{\mathsf{H}}$	conjugate, transpose, Hermitian	$\star$	convolution
$\mathcal{F}\{\cdot\}$	Fourier transform	$\mathbb{E}[\cdot]$	expectation

Fourier transforms follow the NumPy sign convention used everywhere in the code:

$$X(\omega) = \int x(t)\, e^{-j\omega t}\, dt, \qquad x(t) = \frac{1}{2\pi} \int X(\omega)\, e^{j\omega t}\, d\omega. \tag{1}$$

This convention fixes the signs of the dispersion operator and of the nonlinear phase in the fiber model (Section 15.2.5). The discrete transforms are `np.fft.fft` and `np.fft.ifft`; they are unitary (`norm="ortho"`) in OFDM and DMT.

Rebuilding this manual

```
python manual/tools/generate.py      # run the simulator, extract excerpts and tables
cd manual && latexmk -pdf linkstudio-manual.tex
```

Re-running the generator after a code change updates every figure, excerpt and reference table, so the manual stays consistent with the code.

Part I

Using and extending LinkStudio

Chapter 1

Overview and quick start

1.1 What LinkStudio is

LinkStudio simulates the physical layer of a communication link at the level of *complex-envelope samples*. A link is described as a graph of blocks. Every block is a small, self-contained model (a QAM mapper, an 80-km fiber span, a Viterbi decoder, a BER meter) with typed inputs and outputs and validated parameters. The engine executes the graph on *batches* of independent frames and accumulates metrics until a stop criterion is met.

Figure 1.1 shows the layers of the code base. They depend on each other only downwards:

- the **core** knows nothing about communications;
- the **PHY library** knows nothing about the user interface;
- the **API** turns graphs and results into JSON for the browser front end.

The same scenario therefore runs identically from a Python script, the CLI or the GUI.

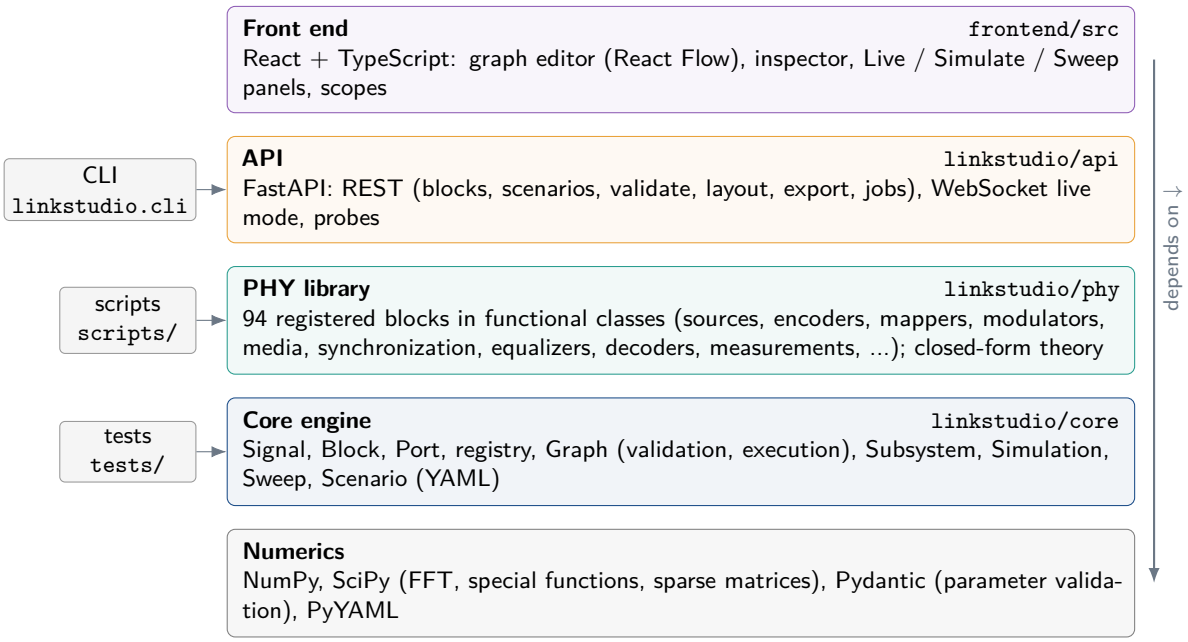


Figure 1.1: Layers of LinkStudio. Paths are relative to the repository; Python packages live in `backend/`.

1.1.1 Capabilities at a glance

Domain	Models
Modulation	PSK, square QAM, PAM, custom constellations with Gray labels; hard, exact and max-log soft demapping
Waveforms	RRC/RC/Gaussian/rectangular pulses, matched filter, rate changes, OFDM with pilots and CP, DMT with water-filling
Channels	AWGN (SNR, E_s/N_0, E_b/N_0), Rayleigh/Rician flat fading, Jakes Doppler, 3GPP EPA/EVA/ETU, MIMO with antenna correlation
RF front end	carrier offset, phase noise, IQ imbalance, PA models (Rapp, Saleh, clipping), quantizer, timing offset
Synchronization	Schmidl–Cox frame sync, Gardner timing loop with Oerder–Meyr start, decision-directed PLL, CP-based OFDM sync
Receivers	one-tap, ZF/MMSE FIR, LMS/NLMS, FFE+DFE, CTLE, echo canceller, ZF/MMSE/ML MIMO detection, Alamouti
Coding	CRC, scrambler, interleavers, repetition, Hamming, convolutional + Viterbi, puncturing, QC-LDPC (BP), polar (SC)
Wireline	NRZ, Manchester, AMI, MLT-3, 4B/5B, 64B/66B, skin-effect cables, ANSI NEXT/FEXT, impulse noise, jitter, ADSL/VDSL DMT
Optical	laser/EML with chirp and RIN, IQ Mach–Zehnder, NLSE/Manakov fiber with PMD, EDFA with ASE, WDM, PIN/APD, coherent receiver and DSP, FSO turbulence
Metrics	BER with confidence interval, SER, FER/BLER, EVM/MER, power/PAPR, OSNR, iteration counts

1.2 Installation

LinkStudio needs Python 3.11 or newer. The front end is already built in `frontend/dist`; Node.js is only needed to modify it.

```
cd backend
pip install -e ".[ui,viz,dev]"    # engine + web server + plotting + tests
python -m pytest                   # the test-suite (about 170 tests)
python scripts/main.py --quick     # runs every demo and compares with theory
```

1.3 Three ways to run a simulation

1. The graphical editor. `python -m linkstudio serve` opens `http://127.0.0.1:8000`. Pick a scenario from the browser (Chapter 3), press *Start live* and move the `snr_db` slider of the `awgn` block: constellation, spectrum and BER update while the simulation runs.

2. A YAML scenario from the command line. Scenario files describe the graph and how to run it (Section 2.9):

```
python -m linkstudio run scenarios/tutorials/01_bpsk_awgn.yaml
python -m linkstudio blocks              # list all block types
python -m linkstudio describe fiber      # parameters and ports of one block
```

3. The Python API. The snippet below builds the graph of Figure 1.2 and measures the BER of QPSK at $E_b/N_0 = 6\,\mathrm{dB}$. The result is about 2.4×10^{-3}, the value of $\mathrm{Q}(\sqrt{2E_b/N_0})$.

```
import linkstudio as cs
```

```
g = cs.Graph("qpsk")
g.create("src", "random_bits", n_bits=2000)
g.create("map", "mapper", scheme="psk", order=4)
g.create("awgn", "awgn", snr_db=6.0)                    # Eb/N0 by default
g.create("demap", "demapper", method="hard")
g.create("ber", "ber_meter")
g.chain("src", "map", "awgn", "demap", "ber.rx")      # default ports
g.connect("src", "ber.tx")                              # explicit edge

result = cs.Simulation(g, batch_size=64, seed=1).run(
    stop=cs.StopCriteria.errors("ber.bit_errors", 200))
print(result.summary())
```

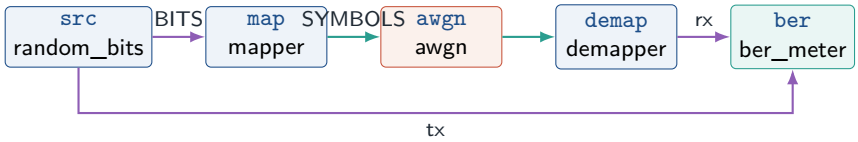


Figure 1.2: The graph built by the quick-start snippet. Edge colours follow the signal kind.

1.4 Repository map

`backend/linkstudio/core/`	engine: `signal.py`, `block.py`, `registry.py`, `graph.py`, `composite.py`, `simulation.py`, `sweep.py`, `scenario.py`
`backend/linkstudio/phy/`	block library; `dsp/` holds shared numerical helpers (filters, constellations, LFSR, analysis), `theory.py` closed-form references
`backend/linkstudio/api/`	web server: `app.py`, `editor.py` (GUI ↔ graph), `jobs.py`, `probes.py`
`backend/scenarios/`	ready-made scenarios grouped by category (Chapter C)
`backend/scripts/`	`main.py` and one demo per subsystem, each checking theory
`backend/tests/`	pytest suite (engine, DSP, physics, API, every scenario)
`frontend/`	React application
`docs/`	short Markdown notes (architecture, engine, GUI)
`manual/`	this manual and its generator

Chapter 2

The simulation engine

The engine (`linkstudio/core`) is about one thousand lines of Python and contains no communication theory. It provides five abstractions:

- *signals* that flow along edges;
- *blocks* that transform them;
- *graphs* that connect blocks;
- *simulations* that run graphs batch after batch;
- *sweeps* that repeat simulations over a parameter grid.

This chapter describes each of them and the rules they enforce.

2.1 Signals

A `Signal` (`core/signal.py`) is an immutable record with four fields:

`data` a NumPy array. Axis 0 is always the *batch* (independent frames simulated in parallel) and the last axis is time, bit index or subcarrier index. Multi-dimensional kinds add axes in between, as in Figure 2.1.

`kind` the semantic type of the data, a flag of the `Kind` enumeration (Table 2.1).

`fs` the sample or symbol rate in Hz, when it is meaningful.

`meta` a dictionary of metadata that travels downstream with the data (Section 2.8).

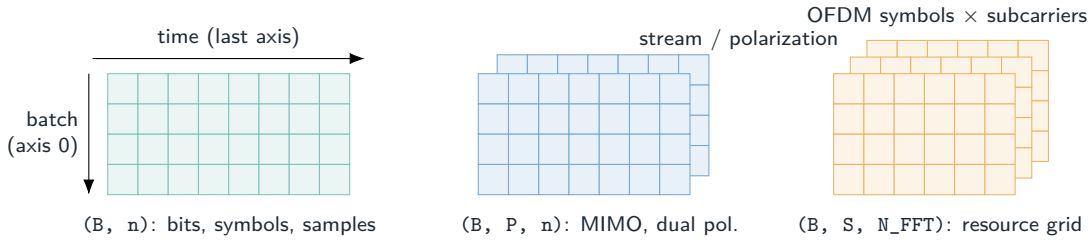


Figure 2.1: Array conventions. Every block may assume that axis 0 is the batch and the last axis is time (or frequency for grids).

Signals are never modified in place. A block derives a new signal with `x.evolve(new_data, key=value)`. This keeps the input's kind, rate and metadata and merges the new keys, so information added upstream is preserved unless a block deliberately overrides it.

2.2 Blocks

A block (`core/block.py`) declares three things:

- a `Params` model, a Pydantic class whose fields carry types, defaults, bounds and descriptions;
- named, typed `inputs` and `outputs` (`Port` objects);

Table 2.1: Signal kinds. Kinds are bit flags, so a port can accept several of them (`Kind.BITS | Kind.LLR`). The colours are those used by the GUI.

Kind	dtype	Meaning
BITS	`uint8`	hard bits in $\{0, 1\}$
LLR	`float64`	log-likelihood ratios, $L = \ln P(b{=}0)/P(b{=}1)$ (positive means 0)
SYMBOLS	`complex128`	one sample per symbol (constellation domain)
SAMPLES	`complex128/float64`	oversampled waveform; optical fields in $\sqrt{\mathrm{W}}$
GRID	`complex128`	OFDM resource grid (frequency domain)
CSI	`complex128`	channel state information; the form is given by `csi_type`
FLAGS	`bool`	one flag per frame (e.g. CRC check)

- a `process(inputs, ctx)` method mapping one batch of input signals to output signals.

Measurement blocks also override `metrics()`.

Ports carry the accepted (or produced) kinds, a `required` flag for inputs and, for outputs, an optional `same_as` reference: `Port.like("in")` produces the same kind as input `in`. The GUI uses this information to colour handles and to refuse invalid connections, and the graph validator uses it to propagate kinds (Section 2.4.1).

2.2.1 Life cycle

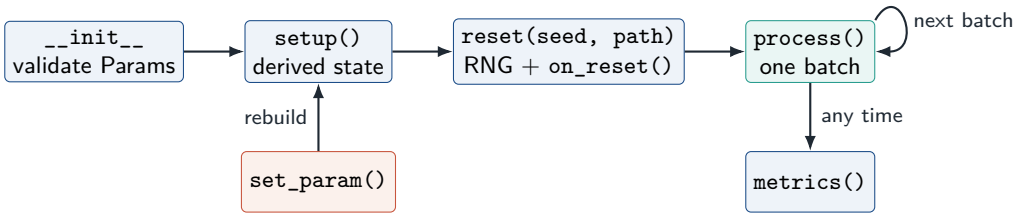


Figure 2.2: Block life cycle. `setup()` rebuilds derived state (filter taps, code matrices, trellises) after construction and after every parameter change. `on_reset()` clears accumulators before a run.

2.2.2 Reproducible randomness

Every block owns a private NumPy generator. On reset it is re-seeded from the master seed of the run *and* the block's hierarchical path:

Code 2.1: Seeding the random generator of a block. `linkstudio/core/block.py`, lines 104–116

```
104 def reset(self, seed: int, path: str) -> None:
105     """Re-seed the block random generator and clear runtime state.
106
107     The generator is derived from the master ``seed`` and the block ``path``
108     (its hierarchical name), so every block gets an independent but
109     reproducible random stream.
110     """
111     self.rng = np.random.default_rng([seed, zlib.crc32(path.encode())])
112     self.__dict__["_recording"] = set()
113     try:
114         self.on_reset()
```

```
    finally:
        self.__dict__["_state_keys"] = tuple(sorted(self.__dict__.pop("_recording")))
```

Because the CRC-32 of the path enters the seed sequence, two `awgn` blocks named differently draw independent noise. The same graph with the same seed produces bit-identical results, whatever the execution order of unrelated blocks and whether the run starts from the GUI, the CLI or a script.

2.3 The registry

`@register_block("type_name", category=..., subcategory=..., label=...)` adds a class to a global registry (`core/registry.py`). Registered blocks can be created by name (`create_block`, `Graph.create`) and are described by `block_catalogue()`. The catalogue contains the documentation, the ports and the JSON Schema of the parameters. The GUI builds its palette and its parameter forms from this catalogue, and Chapter A of this manual is generated from it. The *category* is the functional class of the block in the palette (`encoder`, `modulator`, `medium`, `measurement`, ...); the optional *subcategory* splits the large classes (code families, wireless / wireline / optical media) and *label* is the readable name.

2.4 Graphs

A `Graph` holds named blocks and edges. Edges are stored as a map destination input $\mapsto$ source output, so every input has at most one driver, while an output may fan out to any number of inputs. Ports are addressed as `"block.port"`; omitting the port selects the block's first declared port, which makes `g.chain("src", "map", "awgn")` a compact way to wire a pipeline. Parameters are addressed as `"block.param"`, and `"/"` descends into subsystems (`"rx/demap.method"`).

2.4.1 Validation

`Graph.validate()` enforces three rules and returns the resolved kind of every output:

1. every *required* input is driven, either by an edge or, inside a subsystem, by an external input;
2. the graph is acyclic;
3. along every edge the produced kind intersects the accepted kinds.

Kinds are resolved in topological order. An output declared with `same_as` takes the kind actually delivered to the referenced input, intersected with the kinds that input accepts. An `interleaver` fed with LLRs therefore produces `LLR`, and the validator can reject an LLR output connected to a block that only accepts symbols, before anything is executed.

The topological order is computed with Kahn's algorithm:

Code 2.2: Topological sort of the block graph. linkstudio/core/graph.py, lines 193–215

```
def _toposort(self) -> list[str]:
    succ: dict[str, set[str]] = {n: set() for n in self.blocks}
    for dst, src in self.edges.items():
        if dst.block == src.block:
            raise GraphError(f"Block '{dst.block}' is connected to itself.")
        succ[src.block].add(dst.block)
    indeg = {n: 0 for n in self.blocks}
    for targets in succ.values():
        for t in targets:
```

```
            indeg[t] += 1
    ready = [n for n in self.blocks if indeg[n] == 0]
    order: list[str] = []
    while ready:
        n = ready.pop(0)
        order.append(n)
        for t in sorted(succ[n], key=list(self.blocks).index):
            indeg[t] -= 1
            if indeg[t] == 0:
                ready.append(t)
    if len(order) != len(self.blocks):
        cyclic = [n for n in self.blocks if n not in order]
        raise GraphError(f"The graph contains a cycle involving: {cyclic}")
    return order
```

Ties are broken by insertion order, so the execution order is deterministic. Feedback loops (PLLs, adaptive filters, decision feedback) live *inside* blocks, where they can run at the sample rate. The graph itself carries one batch at a time from sources to sinks.

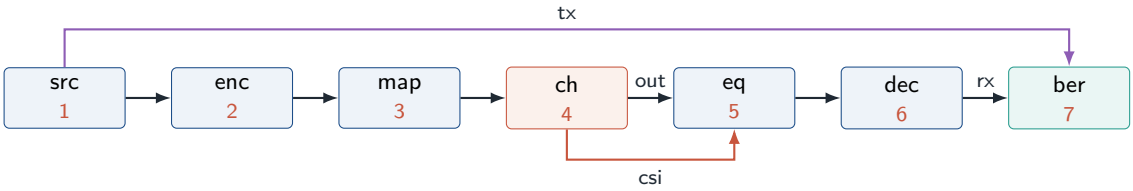


Figure 2.3: A graph with fan-out (`src.out` drives `enc.in` and `ber.tx`) and a side channel (`ch.csi`). Red numbers are the execution order.

2.4.2 Execution

`Graph.execute(ctx)` walks the blocks in order, gathers their inputs from the signals already produced, checks the delivered kinds at run time and calls `process`:

Code 2.3: The core of the executor. `linkstudio/core/graph.py`, lines 240–265

```
for name in self.order():
    block = self.blocks[name]
    inputs: dict[str, Signal] = {}
    for pname, port in block.input_ports.items():
        ep = Endpoint(name, pname)
        src = self.edges.get(ep)
        if src is not None:
            sig = produced[src]
        elif ep in external:
            sig = external[ep]
        else:
            continue
        if sig.kind not in port.kinds:
            raise BlockError(f"Input '{ep}' accepts {port.kinds.label()} but received {sig.
    kind.label()}.")
        inputs[pname] = sig
    try:
        outputs = block.process(inputs, ctx)
    except (BlockError, GraphError):
        raise
    except Exception as exc:  # add context, keep the original traceback
        raise BlockError(f"Block '{name}' ({block.type_name}) failed: {exc}") from exc
    for pname in block.output_ports:
        if pname not in outputs:
```

```
263                 raise BlockError(f"Block '{name}' did not produce output '{pname}'.")
264             produced[Endpoint(name, pname)] = outputs[pname]
265             values[f"{name}.{pname}"] = outputs[pname]
```

All produced signals are returned keyed by `"block.port"`; internal signals of subsystems appear as `"sub/block.port"`. These keys are what the GUI probes and what `Simulation.run(capture=...)` returns. Exceptions raised inside a block are re-raised as `BlockError` with the block name, so the user sees which block failed and why.

2.5 Subsystems

A `Subsystem` (`core/composite.py`) wraps a whole graph as a single block. Its ports map to chosen inner ports, and its output kinds are those resolved by validating the inner graph. A transmitter or a receiver can thus be built once and reused in many scenarios; the same mechanism will let a future network layer treat a whole PHY link as one node. Parameters and metrics of inner blocks are reached with the `"/"` separator: `rx/dec.max_iter`, `rx/dec/avg_iterations`.

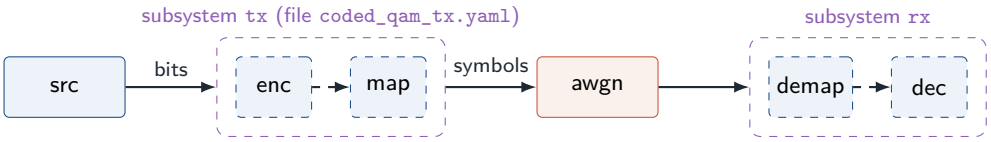


Figure 2.4: Subsystems nest graphs; from outside they behave as ordinary blocks.

2.6 Simulations and stop criteria

A `Simulation` (`core/simulation.py`) runs a graph batch after batch. One batch contains `batch_size` independent frames, which NumPy processes as a single vectorized operation, so a larger batch amortizes the Python overhead. Meter blocks accumulate counts across batches. The run stops at the first satisfied condition of a `StopCriteria`:

Code 2.4: Stop criteria are checked after every batch. linkstudio/core/simulation.py, lines 35–47

```
35 def check(self, n_batches: int, elapsed_s: float, metrics: dict[str, float]) -> str | None:
36     if n_batches < self.min_batches:
37         return None
38     if self.metric is not None and self.threshold is not None:
39         if self.metric not in metrics:
40             raise GraphError(f"Stop metric '{self.metric}' not found. Available: {sorted(
       metrics)}")
41         if metrics[self.metric] >= self.threshold:
42             return f"{self.metric} >= {self.threshold:g}"
43     if n_batches >= self.max_batches:
44         return "max_batches"
45     if self.max_time_s is not None and elapsed_s >= self.max_time_s:
46         return "max_time"
47     return None
```

The classic rule for error-rate estimation is `StopCriteria.errors("ber.bit_errors", 100)`: stop once 100 errors have been counted. Section 5.3 shows why this gives a relative accuracy of about $\pm 20\,\%$ independent of the BER itself. `run` also accepts an `on_batch` callback, which the web server uses to report progress and to cancel jobs.

2.6.1 Multi-core runs

`run(..., workers=N)` spreads the batches over N worker processes (`core/parallel.py`); `Sweep.run(workers=N)` simulates N points at a time. A pool with one process per CPU core is started on first use and then shared by all runs. Batches are sent in chunks sized to take about 0.1 s, which keeps the inter-process overhead small.

Each worker rebuilds the graph from its specification, and batch k is seeded from (seed, k). The worker returns the accumulator state of every block (`Graph.export_state`: the attributes set by `on_reset` plus `\_last*` values). The coordinator merges the states in batch order into its own copy of the graph (`merge_state`: numbers add up, lists are concatenated, `\_last*` values are replaced) and checks the stop criterion after every merged batch. The result therefore depends neither on the number of workers nor on the order in which batches finish:

Code 2.5: Batches are merged in order; the stop rule sees the same sequence as in a serial run. linkstudio/core/parallel.py, lines 140–145

```
140 while reason is None and next_merge in ready:
141     state, captured = ready.pop(next_merge)
142     graph.merge_state(state)
143     captures = captured or captures
144     next_merge += 1
145     reason = done(next_merge, graph.metrics())
```

A parallel run is reproducible, but it is a different random realization from the serial run with the same seed, where each block draws all batches from one stream. Blocks that keep memory from one batch to the next restart at every batch. Sweep points are independent serial runs, so a parallel sweep returns exactly the records of a serial one.

2.7 Sweeps

A `Sweep` (`core/sweep.py`) runs a fresh simulation at every point of a parameter grid. In `grid` mode the points are the Cartesian product of the value lists; in `zip` mode the lists are paired element-wise. It returns a table with one record per point, which can be filtered, printed or written to CSV.

Code 2.6: Each sweep point sets the parameters and runs a new simulation.linkstudio/core/sweep.py, lines 140–146

```
140 for i, point in enumerate(points):
141     if should_stop is not None and should_stop():
142         break
143     for key, value in point.items():
144         self.graph.set_param(key, value)
145     sim = Simulation(self.graph, self.batch_size, self.seed if self.same_seed else self.
        seed + i)
146     res = sim.run(n_batches=self.n_batches, stop=self.stop, on_batch=keep_going)
```

Common random numbers By default every point reuses the same seed (`same_seed=True`). Two points then see the same bits and the same noise realizations, scaled differently. If X and Y are the estimates at two neighbouring points,

$$\operatorname{Var}(X - Y) = \operatorname{Var} X + \operatorname{Var} Y - 2\operatorname{Cov}(X, Y),$$

and the positive covariance created by shared randomness makes the *differences* between points much less noisy. The curves look smooth and comparisons between configurations are fair. Use different seeds only when the points must be statistically independent.

2.8 Metadata propagation

Physical quantities that a downstream block needs but cannot infer from the samples travel in `Signal.meta`. Examples:

- the constellation used by the mapper;
- the oversampling factor;
- the nominal signal power;
- the noise variance after equalization;
- the accumulated chromatic dispersion of a fiber link.

Each block copies the metadata of its input and adds or overrides what it changes. Table 2.2 follows the most important keys through a pulse-shaped link. The mechanism is what lets the `awgn` block set E_b/N_0 correctly whatever comes before it, and lets a soft demapper compute exact LLRs without being told the SNR. Chapter B lists all keys.

Table 2.2: Metadata along a pulse-shaped QPSK link with N_{sps} samples per symbol and $E_s/N_0 = \gamma$. The waveform power is per sample; symbols have unit energy.

after block	`power`	`sps`	`noise_var`	also sets
`mapper`	1	1	–	`constellation`, `bits_per_symbol`
`pulse_shaper`	$1/N_{\text{sps}}$	N_{sps}	–	`pulse_taps`, `delay`
`awgn` (`esn0`)	$1/N_{\text{sps}}$	N_{sps}	$N_0 = 1/\gamma$	
`matched_filter`	1	1	$1/\gamma$	`n_symbols`
`demapper`	uses `constellation` and `noise_var` for exact LLRs			

Watch out Metadata describes the *nominal* signal. A block that changes the signal in a way it cannot describe (a CTLE, an unknown channel) sets the affected keys to `None`, and downstream blocks fall back to measuring. When you write a block, propagate or invalidate every key your processing changes.

2.9 Scenarios

A scenario (`core/scenario.py`) is a graph plus its run configuration, stored as YAML:

```yaml
name: QPSK over AWGN
category: Tutorials            # used by the GUI scenario browser
description: ...
seed: 1
batch_size: 64
blocks:
  src:   {type: random_bits, params: {n_bits: 2000}}
  map:   {type: mapper, params: {scheme: psk, order: 4}}
  awgn:  {type: awgn, params: {snr_db: 6}}
  demap: {type: demapper, params: {method: hard}}
  ber:   {type: ber_meter}
  rx:    {type: subsystem, file: subsystems/coded_qam_rx.yaml}   # optional
chain: [src, map, awgn, demap, ber.rx]      # default-port connections
edges:
  - src.out -> ber.tx                        # explicit connections
simulation: {stop: {metric: ber.bit_errors, threshold: 200, max_batches: 500}}
sweep: {params: {awgn.snr_db: [0, 2, 4, 6, 8]}}   # optional: run a sweep instead
```

```
probes: [{key: awgn.out, view: constellation}]   # GUI scopes
layout: {src: [0, 0], map: [230, 0]}             # GUI positions
```

Files saved from the GUI contain only non-default parameters, so they stay short and readable and remain usable from the CLI. Chapter C lists the scenarios shipped with LinkStudio.

Chapter 3

Graphical editor, API and command line

3.1 The graphical editor

`python -m linkstudio serve` starts the web server and opens the editor in the browser. Figure 3.1 shows its layout.

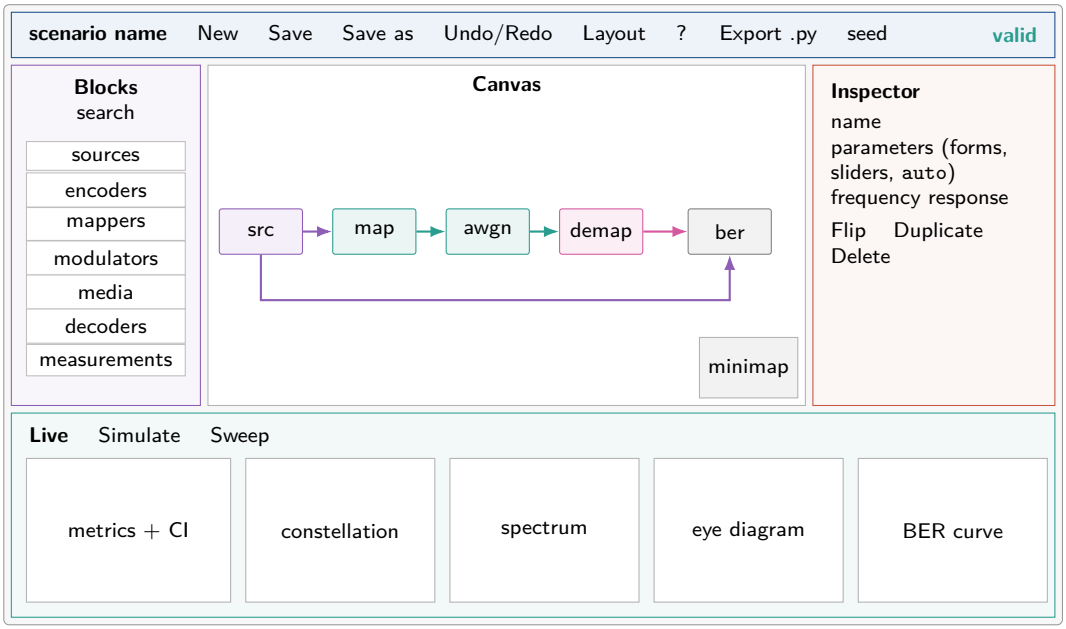


Figure 3.1: Layout of the editor: palette, canvas, inspector and the run dock with its three modes.

Palette. The 94 blocks are grouped by functional class in signal-chain order: sources, encoders, bit processing, mappers, multiplexers, framing, modulators, filters and resamplers, amplifiers and attenuators, transmission media, impairments, then the receiver classes (demodulators, synchronization, equalizers, detectors and estimators, demultiplexers, demappers, decoders) and the measurements. Complementary classes (encoders and decoders, mappers and demappers, ...) share a colour. Only the large classes are split into subclasses (code families, media); the others are flat lists. Every block draws its function on the canvas: codewords with parity, the shift register and trellis of convolutional codes, the Tanner graph, the interleaver permutation drawn from its seed (or the block matrix numbered in write order), the scrambler LFSR with its real taps and seed, pulse shapes, constellations, and the physical blocks (adders, mixers, antennas, fibers, lasers) as drawings without a frame.

Editing.

- **Adding blocks:** drag a block from the palette or double-click it.
- **Connecting:** drag from an output handle to an input handle. Handle colours are the signal kinds of Table 2.1; incompatible connections are refused, and dashed handles are optional inputs.

- **Selection:** click a block; Shift or Ctrl + click adds blocks, Shift + drag on the empty canvas selects the blocks in a box, Ctrl+A selects all and Esc clears the selection. With several blocks selected a bar over the canvas offers *Align row* (the links between them become straight), *Align column*, *Distribute* (equal gaps), *Flip*, *Duplicate* and *Delete*.
- **Clipboard:** Ctrl+C, Ctrl+X and Ctrl+V copy, cut and paste blocks with the connections between them; the copy is placed at the mouse and pasting again steps the copies. Ctrl+D duplicates, Delete deletes, the arrow keys move the selection (Shift + arrow by a larger step).
- **Direction:** a block faces right (inputs on the left, outputs on the right) or left. F, or *Flip* in the inspector, turns the selected blocks around. A block dragged to the other side of the blocks it is linked to turns around by itself, so a chain can run right to left or turn back on itself without moving any port. The block drawings are mirrored with the block; their text stays readable and plots against time or frequency keep their axes.
- **Moving ports:** Ctrl + drag a handle to move it anywhere on the block border, e.g. for feedback paths. Port places and directions are saved in the scenario (`port_layout`).
- **Alignment:** while a block is dragged it snaps so that its connected ports line up with the ports at the other end (straight links), or else so that its edges or centre line up with a neighbouring block; a guide line shows the alignment.
- **Automatic layout** (Ctrl+L) arranges the blocks along the signal flow (Figure 3.2). Each block goes in the column given by its longest distance from a source, and sources that feed late blocks are pulled forward. The longest path (the main chain) stays on one straight line. Branches such as training sequences, channel estimates and meters go in lanes under it, ordered by barycentre sweeps to reduce crossings. A long chain wraps into rows sized from the shape of the canvas and run alternately left to right and right to left (a horseshoe); the blocks of the reversed rows face left. Scopes and meters that tap a link from their bottom side sit above the gap after the block they tap. Unconnected blocks go in a last row.
- **Links** are drawn with orthogonal segments routed around the blocks (A* search on a sparse grid of block edges and channels, with a penalty per bend), so they never pass behind a block. All links are routed together. Where links of different nets would run on top of each other along a channel, their shared segments are moved apart so that each link stays visible.
- **Inspector:** parameter forms are generated from each block's schema. Bounded and dB-valued parameters get sliders, and `auto` leaves a value to upstream metadata. For linear blocks the inspector also plots the *frequency response* (magnitude, phase or group delay): the model transfer function of cables, CTLE, lasers and VCSELs, photodiodes, single- and multimode fibers and pulse shapers, and an estimate $\hat{H} = S_{xy}/S_{xx}$ of one realization for random multipath channels.
- **Undo:** Ctrl+Z and Ctrl+Y undo and redo any change: blocks, connections, parameters, positions, port places and directions, scopes.
- **Shortcuts:** ? (or the ? button) lists them all. Besides those above: Ctrl+S saves, Ctrl+Shift+S saves as, Ctrl+O opens the scenario browser, Ctrl+B and Ctrl+I show or hide the block palette and the inspector. Saving keeps the run settings (`simulation`, `sweep`) and the ports of a subsystem file, so a scenario saved from the GUI runs the same from the command line.

The run buttons are disabled while the graph is invalid, and the validator's message is shown next to them.

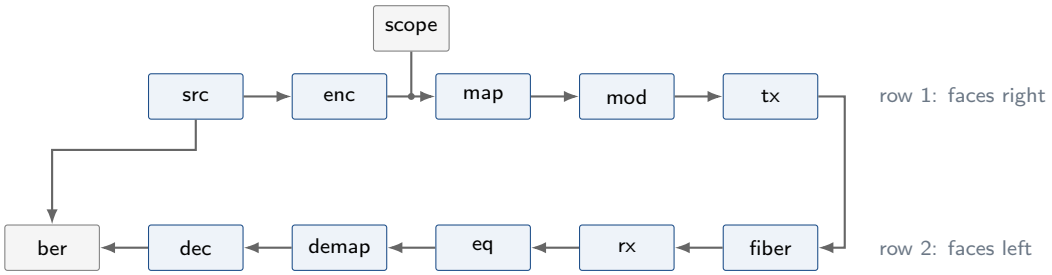


Figure 3.2: Automatic layout of a long chain: it wraps into a second row that runs right to left with its blocks facing left. A scope tapping a link from its bottom side sits above it.

Modes.

Live simulates one frame per step (batch size 1), 1–30 steps per second, and streams every probe; the metrics accumulate across steps. Parameter changes are applied between steps and the meters restart, so the effect of a slider is visible immediately. Structural edits restart the session. The rate only sets how often the scopes refresh, not the results.

Simulate runs a Monte-Carlo job in the background, either for a fixed number of batches or until a metric reaches a threshold. *frames/batch* sets the batch size (saved in the scenario) and *cores* the number of worker processes (Section 2.6.1). It shows the final metrics (BER with its 95 % confidence interval) and the probes of the last batch.

Sweep runs one or two nested parameter sweeps and plots any metric, with an optional theoretical curve; with several cores the points run in parallel. Curves can be kept and overlaid, and the results exported as CSV or PNG.

The Scope block. Signals are displayed by the `scope` block (class *measurements*), an instrument with a screen. Its inputs enter from the bottom side: drop the scope from the palette onto a link, or drag one of its free inputs onto a link, to tap the output that feeds it. The tap is drawn as a junction dot just after that output. A scope has up to four inputs; the selector in its corner chooses the displayed one, and the bottom-right corner resizes it. The plot selector under its name (parameter `view`) lists the views that suit the displayed signal:

Kind	Views (the first is the `auto` choice)
SAMPLES	spectrum (Welch), constellation, eye diagram, time
SYMBOLS	constellation, time, spectrum
GRID	constellation, resource-grid magnitude
CSI	impulse / frequency response
LLR	histogram, time
BITS	time

A view that does not suit the signal falls back to the first one of its kind. After a *Simulate* run every view is already computed, so switching needs no new run; in *Live* the next frame shows the new view, and the measurements are not restarted. Older scenarios with one scope type per view (`scope_constellation`, ...) are converted to `scope` with the matching view when they are loaded.

Axes carry physical units. The server sends values in base SI units together with their unit, taken from the `unit` metadata: $\sqrt{\text{W}}$ for optical fields, A for photocurrents, none for normalized baseband signals. The editor picks the SI prefix (ps, GHz, mA, ...) so that tick labels stay short. Spectra are absolute power spectral densities (dBm/Hz for optical fields, one-sided for real signals). The eye diagram shows time in seconds over two unit intervals and its vertical axis is centred on the mean of the traces and spans all of them.

3.2 The HTTP and WebSocket API

The editor talks to the FastAPI application in `linkstudio/api/app.py`. Everything it does can also be scripted.

Method	Path	Purpose
GET	`/api/health`	server version and number of CPU cores
GET	`/api/blocks`	block catalogue: documentation, ports, parameter JSON Schema
GET	`/api/scenarios`	scenario list with name, category, description, block count
GET/PUT	`/api/scenarios/{path}`	load / save a scenario in editor format
POST	`/api/validate`	validate an editor graph; returns errors and port kinds
POST	`/api/layout`	server-side layout (the editor lays out locally)
POST	`/api/export/python`	standalone Python script equivalent to the graph
POST	`/api/response`	frequency response of one block (magnitude, phase, group delay)
POST	`/api/jobs`	start a simulate or sweep job
GET	`/api/jobs/{id}`	job status, progress, partial and final results
POST	`/api/jobs/{id}/cancel`	cancel a job
WS	`/api/live`	interactive live mode

Jobs run in a thread pool (`api/jobs.py`); the `workers` entry of the job configuration sends the batches or sweep points to that many worker processes. Their results contain JSON-safe metrics (NaN becomes `null`), the probe data of the last batch and, for sweeps of a single parameter whose value matches a known modulation, theoretical BER curves. A failed job reports the message `failed` with the details in a separate `detail` field.

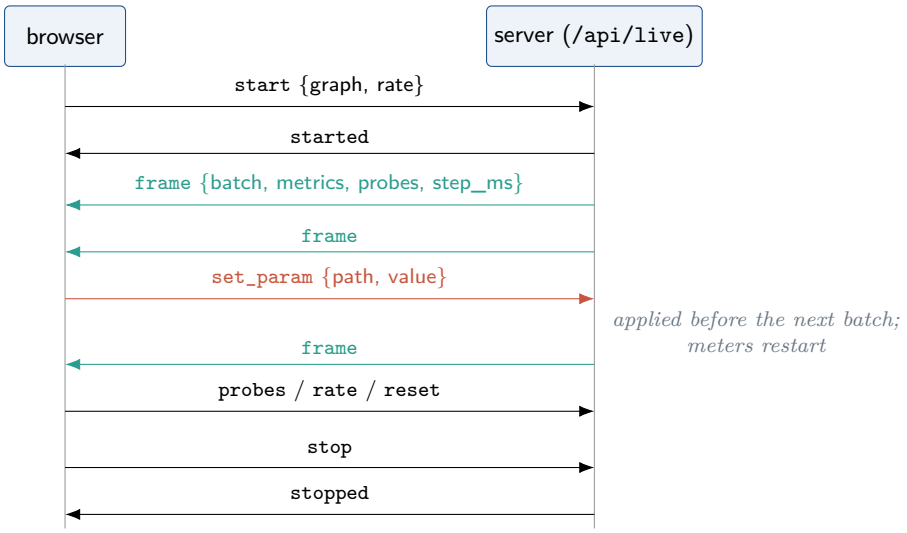


Figure 3.3: Message flow of the live mode. The server steps the simulation in a worker thread and paces frames to the requested rate.

3.3 Command line

`python -m linkstudio blocks [--category C]`	list block types
`python -m linkstudio describe TYPE`	ports, parameters and documentation of a block
`python -m linkstudio run FILE.yaml [--csv OUT]`	run a scenario (simulation or sweep)
`python -m linkstudio serve [--host H] [--port P] [--no-browser]`	start the editor
`python scripts/main.py [--quick]`	run all demos with theory checks

The demo scripts in `scripts/` (one per subsystem) are short, readable examples of the Python API. Each prints simulated values next to theory and returns a pass/fail status.

Chapter 4

Developer guide: writing blocks

Adding a model to LinkStudio means writing one block class. The engine, the CLI, the scenario loader and the GUI pick it up automatically once it is registered. This chapter builds a complete example and lists the conventions a block must respect to behave like the built-in ones.

4.1 A complete example: free-space path loss

Suppose we want a block that attenuates a radio signal by the free-space loss between two isotropic antennas at distance d and carrier frequency f_c (Friis):

$$L_{\mathrm{fs}} = \left(\frac{4\pi d f_c}{c}\right)^2, \qquad L_{\mathrm{fs}}\big|_{\mathrm{dB}} = 20\log_{10} d + 20\log_{10} f_c - 147.55. \tag{4.1}$$

The amplitude is scaled by $L_{\mathrm{fs}}^{-1/2}$. The nominal power metadata and any noise already present must be scaled by L_{fs}^{-1}, so that a downstream `awgn` block in `nominal` mode still computes the SNR from the right reference.

Code 4.1: A new block, e.g. in `linkstudio/phy/channel/pathloss.py`.

```python
import numpy as np
from pydantic import Field

from linkstudio.core import Block, BlockParams, Kind, Port, RunContext, Signal,
    register_block

C_LIGHT = 299_792_458.0


@register_block("free_space_loss", category="medium", subcategory="Wireless", label="Free-
    Space Loss")
class FreeSpaceLoss(Block):
    """Friis free-space path loss between isotropic antennas."""

    class Params(BlockParams):                          # validated, serializable, shown in
    the GUI
        distance_m: float = Field(100.0, gt=0, description="Link distance.")
        carrier_hz: float = Field(2.4e9, gt=0, description="Carrier frequency.")

    inputs = {"in": Port(Kind.SYMBOLS | Kind.SAMPLES)}
    outputs = {"out": Port.like("in")}                  # same kind as the input

    def setup(self) -> None:                            # derived state, rebuilt on parameter
    change
        p = self.params
        self.loss = (4 * np.pi * p.distance_m * p.carrier_hz / C_LIGHT) ** 2

    def on_reset(self) -> None:                         # runtime state, cleared before a run
        self._frames = 0

    def process(self, inputs: dict[str, Signal], ctx: RunContext) -> dict[str, Signal]:
        x = inputs["in"]
        meta = {k: x.meta[k] / self.loss for k in ("power", "noise_var") if x.meta.get(k)
    is not None}
        self._frames += x.batch_size
```

```
31          return {"out": x.evolve(x.data / np.sqrt(self.loss), **meta)}
32
33      def metrics(self) -> dict[str, float]:
34          return {"loss_db": 10 * np.log10(self.loss), "frames": self._frames}
```

To register it, import the module in the package `__init__.py` (`linkstudio/phy/channel/__init__.py`). From then on:

- `g.create("pl", "free_space_loss", distance_m=500)` works from Python;
- `{type: free_space_loss, params: {distance_m: 500}}` works in YAML;
- the block appears in the GUI palette under *Transmission media* → *Wireless* as *Free-Space Loss*, with a form for its two parameters;
- `pl.distance_m` can be swept;
- `pl.loss_db` is reported among the metrics.

4.2 Conventions

C1. Shapes. Accept any leading batch shape. Operate on the last axis and use `...` indexing, so the block works for `(B, n)` as well as `(B, P, n)` arrays.

C2. Vectorize over the batch. Loops over samples are acceptable for adaptive algorithms, but each iteration should process all frames at once (see the LMS loop in Listing 9.6).

C3. Kinds. Declare the narrowest accepted kinds; use `Port.like` when the output kind follows the input.

C4. Metadata. Propagate with `evolve`. Update the keys your processing changes (`power`, `noise_var`, `sps`, `delay`, ...) and set them to `None` when you cannot describe the result.

C5. Randomness. Draw only from `self.rng`; never use NumPy's global generator. This keeps runs reproducible (Section 2.2).

C6. Real signals stay real. Wireline and IM/DD chains are real-valued; do not return complex arrays for real inputs.

C7. Errors. Raise `BlockError` with a message telling the user how to fix the problem (*"Timing recovery needs an oversampled input (sps >= 2)"*).

C8. Metrics accumulate. Sum counts in `process`, compute ratios in `metrics`, clear them in `on_reset`.

4.3 Testing a block against theory

The test-suite follows one rule: a physical model is validated against a closed-form result, not against itself. `tests/helpers.py` provides `uncoded_link` and `ber_at`. A typical test reads:

```
def test_awgn_ber_matches_theory():
    g = uncoded_link("qam", 16, snr_db=10.0, n_bits=4 * 1200)
    assert np.isclose(ber_at(g, errors=300), theory.ber_qam(16, 10.0), rtol=0.2)
```

With 300 errors the relative standard deviation of the estimate is about $1/\sqrt{300} \approx 6\,\%$, so a $20\,\%$ tolerance makes false alarms negligible but still detects a 1 dB error. For our example the natural check is the metric itself: at 100 m and 2.4 GHz, $L_{\mathrm{fs}} = 80.05$ dB. Chapter 16 lists the physics checks of the whole library.

4.4 Adding a scenario

Save a graph from the GUI, or write a YAML file under `backend/scenarios/<category>/` with `name`, `category` and `description`. The test `tests/test_scenarios.py` loads and runs every scenario file, so a broken scenario is caught automatically.

Part II

Theory and implementation

Chapter 5

Baseband signal model, noise and Monte-Carlo estimation

Every chapter of Part II relies on three conventions:

- a signal is represented by its sampled complex envelope;
- noise is circular complex Gaussian with a variance tied to N_0;
- performance is estimated by counting errors.

This chapter makes the three precise.

5.1 The complex envelope

A real band-pass signal centred on the carrier f_c can always be written as

$$s(t) = \Re\left\{ x(t)\, e^{j2\pi f_c t} \right\} = x_I(t)\cos 2\pi f_c t - x_Q(t)\sin 2\pi f_c t, \qquad x(t) = x_I(t) + j\, x_Q(t), \tag{5.1}$$

where $x(t)$ is the *complex envelope*. If s occupies $|f \pm f_c| < B/2$, then x is a low-pass signal of bandwidth $B/2$. It can be sampled at $f_s \geq B$, whatever the carrier frequency. Linear time-invariant band-pass systems act on x through their low-pass equivalent impulse response. A carrier frequency error Δf becomes a rotation $x(t)e^{j2\pi\Delta f t}$, and an oscillator phase error $\phi(t)$ becomes $x(t)e^{j\phi(t)}$.

> **In the code** All SYMBOLS and SAMPLES signals are sampled complex envelopes $x[n] = x(n/f_s)$ with `Signal.fs`$= f_s$. Two exceptions are worth noting:
>
> - wireline and IM/DD electrical signals are *real* baseband signals ($x_Q \equiv 0$), and the blocks keep their arrays real;
> - optical fields are envelopes around the optical carrier, scaled in $\sqrt{\mathrm{W}}$ (Chapter 15).
>
> Powers are *envelope* powers, $P = \mathbb{E}|x[n]|^2$. The factor $\frac{1}{2}$ relating envelope and band-pass powers cancels in every SNR, so it is never applied.

5.2 Additive white Gaussian noise

5.2.1 From N_0 to a sample variance

Thermal noise at the receiver input is modelled as white with two-sided band-pass power spectral density $N_0/2$. Its complex envelope $w(t)$ is circular complex Gaussian with two-sided PSD N_0 (in envelope units). Observed through any filter of noise bandwidth f_s and sampled at f_s, it gives independent samples

$$w[n] \sim \mathcal{CN}(0, \sigma^2), \qquad \sigma^2 = N_0 f_s, \qquad \mathbb{E}\big[(\Re w)^2\big] = \mathbb{E}\big[(\Im w)^2\big] = \sigma^2/2. \tag{5.2}$$

The code works in units where the sample period is the time unit, so N_0 and σ^2 coincide numerically. The metadata key `noise_var` is σ^2, the noise variance per complex sample (or per symbol after matched filtering).

5.2.2 SNR definitions

Let P be the signal power per sample and N_{sps} the number of samples per symbol. The energy per symbol is $E_s = P\,N_{\text{sps}}T_s$ and the noise PSD is $N_0 = \sigma^2 T_s$, so

$$\frac{E_s}{N_0} = \frac{P\,N_{\text{sps}}}{\sigma^2}. \tag{5.3}$$

A symbol carries $k = \log_2 M$ coded bits. A code of rate R and N_s spatially multiplexed streams turn these into kRN_s information bits per channel use, so

$$\frac{E_b}{N_0} = \frac{E_s}{kRN_sN_0} \quad\Longrightarrow\quad \sigma^2 = \frac{P\,N_{\text{sps}}}{(E_b/N_0)\,kRN_s}. \tag{5.4}$$

The `awgn` block implements the three definitions of Table 5.1. N_{sps}, k, R and N_s are read from the metadata set by the pulse shaper (or OFDM modulator), the mapper, the encoders and the stream demultiplexer. Equation (5.4) is therefore correct for any chain, which is why the coded and MIMO curves of later chapters can be compared with uncoded theory on the same E_b/N_0 axis.

Table 5.1: SNR modes of the `awgn` block.

`mode`	meaning of `snr_db`	noise variance σ^2
`snr`	signal-to-noise ratio per sample	P/SNR
`esn0`	E_s/N_0	$PN_{\text{sps}}/(E_s/N_0)$
`ebn0`	E_b/N_0 (default)	$PN_{\text{sps}}/\big((E_b/N_0)\,kRN_s\big)$

The reference power P is, in order of priority:

1. the parameter `signal_power`;
2. the nominal power `power` carried by the metadata (`power_reference=nominal`, the textbook definition of *average* SNR);
3. the power measured over the batch.

The nominal reference matters for fading: the average SNR is defined by the transmitted power, not by the instantaneous power after a deep fade.

Code 5.1: Noise variance from the SNR definition (`awgn.py`). `linkstudio/phy/channel/awgn.py`, lines 49–72

```
49 def noise_variance(self, x: Signal) -> float:
50     p = self.params
51     if p.mode == "thermal":
52         if not x.fs:
53             raise BlockError("Thermal noise needs the sample rate (fs) of the signal.")
54         return 1.380649e-23 * p.temperature_k * 10 ** (p.noise_figure_db / 10) * x.fs
55     if p.signal_power is not None:
56         power = p.signal_power
57     elif p.power_reference == "nominal" and x.meta.get("power") is not None:
58         power = float(x.meta["power"])
59     else:
60         power = float(np.mean(np.abs(x.data) ** 2))
61     snr = 10 ** (p.snr_db / 10)
62     if p.mode == "snr":
63         return power / snr
64     sps = x.meta.get("sps", 1)
65     if p.mode == "esn0":
66         return power * sps / snr
```

```
    k = p.bits_per_symbol or x.meta.get("bits_per_symbol")
    if k is None:
        raise BlockError("Eb/N0 mode needs bits_per_symbol (mapper upstream or parameter)."
    )
    rate = p.code_rate or x.meta.get("code_rate", 1.0)
    # Spatial multiplexing carries several symbols per channel use.
    return power * sps / (snr * k * rate * x.meta.get("spatial_rate", 1))
```

Code 5.2: Adding the noise. Real inputs get $\sigma^2/2$ per real dimension. linkstudio/phy/channel/awgn.py, lines 74–82

```
def process(self, inputs: dict[str, Signal], ctx: RunContext) -> dict[str, Signal]:
    x = inputs["in"]
    n0 = self.noise_variance(x)
    if np.iscomplexobj(x.data):
        noise = self.rng.standard_normal((*x.data.shape, 2)).view(np.complex128)[..., 0]
    else:  # real baseband (wireline, IM/DD): N0/2 per real dimension
        noise = self.rng.standard_normal(x.data.shape)
    y = x.data + np.sqrt(n0 / 2) * noise
    return {"out": x.evolve(y, noise_var=x.meta.get("noise_var", 0.0) + n0)}
```

Remark. For a real signal only the in-phase noise component matters, with variance $\sigma^2/2 = N_0/2$. A real antipodal (NRZ, 2-PAM) signal then has the same error probability as BPSK, as it should. This is tested in `tests/test_wireline_optical.py`.

5.3 Monte-Carlo estimation of error rates

5.3.1 The estimator and its accuracy

Let N bits be observed and K of them be in error. If errors are independent with probability p, then $K \sim \text{Binomial}(N, p)$ and the natural estimator $\hat{p} = K/N$ is unbiased with

$$\operatorname{Var}\hat{p} = \frac{p(1-p)}{N}, \qquad \frac{\sqrt{\operatorname{Var}\hat{p}}}{p} = \sqrt{\frac{1-p}{Np}} \approx \frac{1}{\sqrt{\mathbb{E}K}}. \tag{5.5}$$

The relative accuracy depends only on the number of *errors*, not on the number of bits. Stopping a simulation after a fixed number of errors (Section 2.6) therefore gives the same relative accuracy at every SNR: 100 errors give about $\pm 10\,\%$ standard deviation, and $\pm 20\,\%$ at 95 % confidence.

5.3.2 The Wilson score interval

The normal approximation $\hat{p} \pm z\sqrt{\hat{p}(1-\hat{p})/N}$ fails when K is small: with $K = 0$ it gives a zero-width interval. The Wilson interval [18] instead inverts the test $|\hat{p} - p| \le z\sqrt{p(1-p)/N}$ for the unknown p. Squaring gives the quadratic $(1 + z^2/N)p^2 - (2\hat{p} + z^2/N)p + \hat{p}^2 \le 0$, whose roots are

$$p_{\pm} = \frac{\hat{p} + \dfrac{z^2}{2N} \pm z\sqrt{\dfrac{\hat{p}(1-\hat{p})}{N} + \dfrac{z^2}{4N^2}}}{1 + \dfrac{z^2}{N}}, \qquad z = 1.96 \text{ for } 95\,\%. \tag{5.6}$$

The interval is never empty, stays inside $[0, 1]$ and is accurate even for a handful of errors. `ber_meter` reports it as `ber_ci_low` and `ber_ci_high`, and the GUI shows it next to the BER.

Code 5.3: The Wilson interval in `metrics.py`. linkstudio/phy/metrics.py, lines 13–21

```
def wilson_interval(errors: int, trials: int, z: float = 1.96) -> tuple[float, float]:
    """95 % Wilson score confidence interval of an error probability."""
```

```
    if trials == 0:
        return float("nan"), float("nan")
    p = errors / trials
    denom = 1 + z * z / trials
    centre = (p + z * z / (2 * trials)) / denom
    half = z * np.sqrt(p * (1 - p) / trials + z * z / (4 * trials * trials)) / denom
    return float(max(centre - half, 0.0)), float(min(centre + half, 1.0))
```

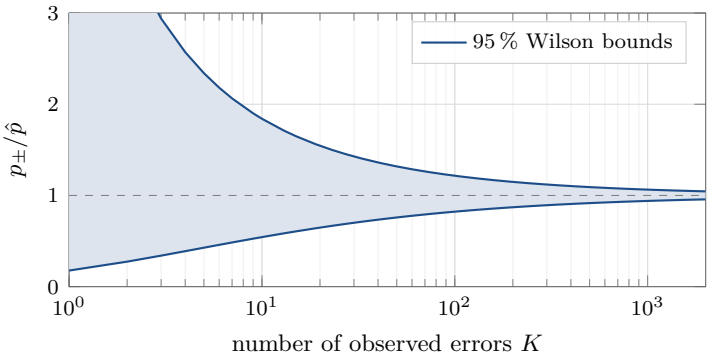


Figure 5.1: Width of the 95 % confidence interval relative to the estimate, as a function of the number of observed errors (computed with `wilson_interval` at $p = 10^{-4}$). Ten errors leave a factor of about two of uncertainty; one hundred errors bring it to ±20 %.

Correlated errors The binomial model assumes independent errors. After a decoder, or in block fading, errors come in bursts within a frame. The interval is then optimistic, because the effective number of independent observations is closer to the number of erroneous *frames*. For coded systems, judge the accuracy by `frame_errors` as well.

5.4 Closed-form references

`linkstudio/phy/theory.py` collects the closed-form results used in tests, demos, the GUI and this manual. All functions accept arrays and SNRs in dB.

`ber_bpsk`, `ber_pam`, `ber_qam`	exact Gray-coded BER over AWGN (Section 6.3)
`ser_psk`, `ber_psk`	Craig's integral for M-PSK
`ber_bpsk_rayleigh`, `ber_bpsk_mrc`, `ber_rayleigh`	fading and diversity (Section 9.5)
`ber_hamming_hard`	post-decoding BER of Hamming codes
`ook_q_factor`, `q_to_ber`, `osnr_to_esn0_db`	optical receivers (Chapter 15)
`gaussian_pulse_broadening`	dispersion of a Gaussian pulse
`shannon_capacity`, `ebn0_min_db`	information-theoretic limits

Chapter 6

Digital modulation and demodulation

6.1 Constellations

A memoryless modulator maps each group of k bits $\boldsymbol{b} = (b_0, \dots, b_{k-1})$ to one of $M = 2^k$ complex points, $s = \mathcal{S}(\boldsymbol{b})$. The label $\ell = \sum_i b_i 2^{k-1-i}$ is read MSB first and indexes the table of points. LinkStudio builds four families, all normalized to unit average energy $\mathbb{E}|s|^2 = 1$:

$$\text{M-PAM:} \quad s_\ell = \frac{2i - (M-1)}{\sqrt{(M^2-1)/3}}, \quad \ell = g(i), \tag{6.1}$$

$$\text{M-QAM:} \quad s_\ell = \frac{a_{\ell_I} + j\, a_{\ell_Q}}{\sqrt{2(M-1)/3}}, \quad \ell = (\ell_I, \ell_Q),\ a.\ \text{Gray } \sqrt{M}\text{-PAM levels}, \tag{6.2}$$

$$\text{M-PSK:} \quad s_\ell = e^{\,j(2\pi i/M + \phi_0)}, \quad \ell = g(i), \quad \phi_0 = \pi/4 \text{ for QPSK, else } 0, \tag{6.3}$$

where $g(i) = i \oplus \lfloor i/2 \rfloor$ is the binary-reflected Gray code. Adjacent points therefore differ in exactly one bit. The denominators are the average energies of the unnormalized PAM and QAM grids, $\frac{1}{M}\sum_i (2i - M + 1)^2 = (M^2-1)/3$ per dimension. Custom constellations are normalized the same way.

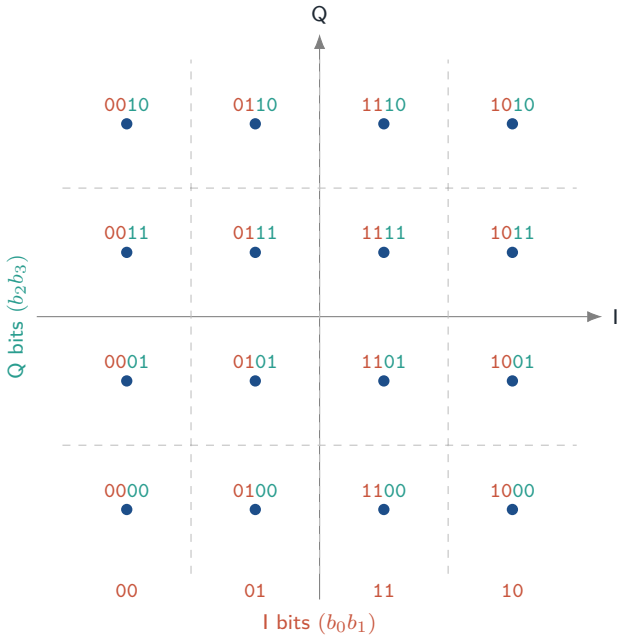


Figure 6.1: 16-QAM with the labels produced by `qam(16)`. The first half of the label selects the in-phase level and the second half the quadrature level, each with a Gray 4-PAM labelling. Dashed lines are the decision boundaries.

Code 6.1: Square QAM as the product of two Gray PAMs (`dsp/constellations.py`).
linkstudio/phy/dsp/constellations.py, lines 118–125

```
118 side = int(round(np.sqrt(order)))
119 if side * side != order or side & (side - 1):
120     raise ValueError(f"Square QAM needs M = 4^n, got {order}.")
```

```
half = int(np.log2(side))
levels = _pam_levels(side)
labels = np.arange(order)
points = levels[labels >> half] + 1j * levels[labels & (side - 1)]
return Constellation(points, name=f"{order}-QAM")
```

In the code `mapper` (`phy/modulation.py`) builds the constellation in `setup()` and maps frames of nk bits to n symbols. It sets `constellation`, `bits_per_symbol`, `power`= 1, `sps`= 1 and the symbol rate `symbol_rate` as `fs`. `demapper` takes the constellation from the metadata unless one is given explicitly.

6.2 Optimal detection

For the observation $y = s + w$ with $w \sim \mathcal{CN}(0, \sigma^2)$ and equiprobable symbols, the maximum-likelihood decision is the nearest point:

$$\hat{s} = \arg\max_{s \in \mathcal{S}} p(y \mid s) = \arg\max_{s} \frac{1}{\pi\sigma^2} e^{-|y-s|^2/\sigma^2} = \arg\min_{s \in \mathcal{S}} |y - s|^2. \tag{6.4}$$

`Constellation.nearest` evaluates all M distances in chunks of 2^{16} symbols, which bounds memory. `method=hard` returns the label bits of the nearest point.

6.2.1 Bit log-likelihood ratios

Soft-input decoders need, for every bit, the a-posteriori log-likelihood ratio. With $\mathcal{S}_i^{(b)}$ the subset of points whose i-th label bit equals b, and equiprobable bits,

$$L(b_i) = \ln \frac{P(b_i = 0 \mid y)}{P(b_i = 1 \mid y)} = \ln \sum_{s \in \mathcal{S}_i^{(0)}} e^{-|y-s|^2/\sigma^2} - \ln \sum_{s \in \mathcal{S}_i^{(1)}} e^{-|y-s|^2/\sigma^2}. \tag{6.5}$$

Keeping only the dominant term of each sum gives the max-log approximation

$$L(b_i) \approx \frac{1}{\sigma^2} \Big(\min_{s \in \mathcal{S}_i^{(1)}} |y - s|^2 - \min_{s \in \mathcal{S}_i^{(0)}} |y - s|^2 \Big), \tag{6.6}$$

which is exact for BPSK and loses only a few tenths of a dB with good codes. A positive LLR means bit 0 throughout LinkStudio. The variance σ^2 may differ from symbol to symbol, for example after zero-forcing equalization of a fading channel; `noise_var` may therefore be an array broadcastable to the symbols.

Code 6.2: Exact (log-sum-exp) and max-log LLRs. `linkstudio/phy/dsp/constellations.py`, lines 72–86

```
k = self.bits_per_symbol
flat = y.reshape(-1)
nv = np.broadcast_to(np.asarray(noise_var, dtype=np.float64), y.shape).reshape(-1)
llr = np.empty((flat.size, k))
zero = self.bit_labels == 0  # (M, k)
for s in range(0, flat.size, _CHUNK):
    metric = -np.abs(flat[s:s + _CHUNK, None] - self.points[None, :]) ** 2
    metric /= np.maximum(nv[s:s + _CHUNK, None], 1e-300)
    for i in range(k):
        m0, m1 = metric[:, zero[:, i]], metric[:, ~zero[:, i]]
        if method == "exact":
            llr[s:s + _CHUNK, i] = logsumexp(m0, axis=1) - logsumexp(m1, axis=1)
        else:
            llr[s:s + _CHUNK, i] = m0.max(axis=1) - m1.max(axis=1)
return llr.reshape(*y.shape[:-1], -1)
```

Key idea The LLR scale matters. Equalizers update `noise_var` with their noise enhancement ($\sigma^2/|h|^2$ for a one-tap ZF equalizer, $\sigma^2\|\boldsymbol{w}\|^2$ for an FIR equalizer, the post-detection variance for MIMO). The decoder downstream then receives LLRs with the correct reliability without any manual tuning.

6.3 Error probability over AWGN

With $\mathrm{Q}(x) = \frac{1}{2}\operatorname{erfc}(x/\sqrt{2})$ the Gaussian tail, the classic results implemented in `theory.py` are:

- **BPSK and Gray QPSK**: $P_b = \mathrm{Q}\left(\sqrt{2E_b/N_0}\right)$. QPSK is two orthogonal BPSK channels sharing the symbol energy.
- **Gray M-PAM**, exact (Cho and Yoon [4]): with $L = M$, $m = \log_2 L$ and $\gamma_b = E_b/N_0$,

$$P_b = \frac{1}{m}\sum_{q=1}^{m}\frac{1}{L}\sum_{i=0}^{(1-2^{-q})L-1}(-1)^{\lfloor i2^{q-1}/L\rfloor}\Big(2^{q-1} - \Big\lfloor\frac{i2^{q-1}}{L} + \frac{1}{2}\Big\rfloor\Big)\operatorname{erfc}\Big((2i+1)\sqrt{\tfrac{3m\gamma_b}{L^2-1}}\Big). \quad (6.7)$$

- **Square Gray M-QAM**: the in-phase and quadrature components are independent $\sqrt{M}$-PAM signals carrying half the bits each, so $P_b^{\mathrm{QAM}}(M) = P_b^{\mathrm{PAM}}(\sqrt{M})$ at the same E_b/N_0.
- **M-PSK**: the symbol error rate is Craig's integral [5]

$$P_s = \frac{1}{\pi}\int_0^{(M-1)\pi/M}\exp\Big(-\frac{E_s}{N_0}\frac{\sin^2(\pi/M)}{\sin^2\theta}\Big)d\theta, \qquad P_b \approx P_s/k, \quad (6.8)$$

  where the Gray approximation $P_b \approx P_s/k$ is tight at moderate SNR.

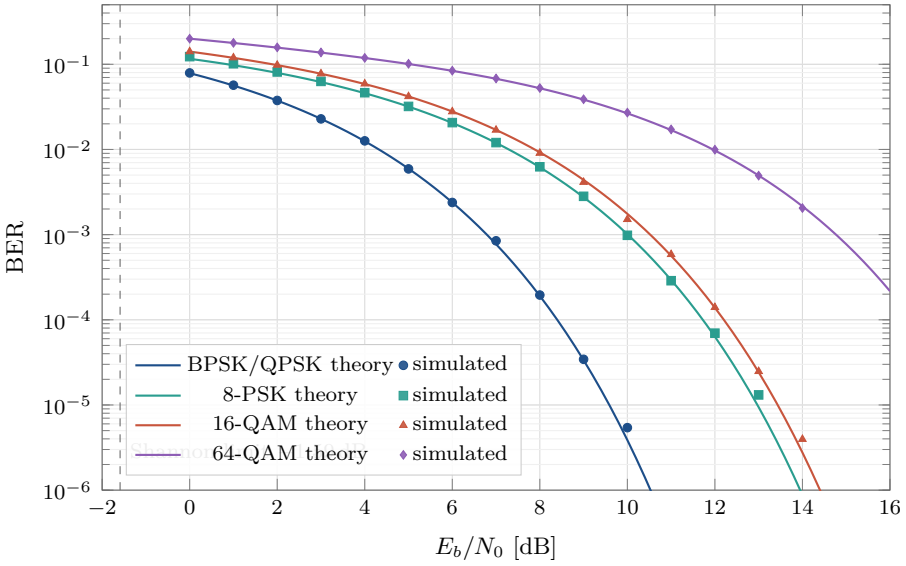


Figure 6.2: Bit error rate over AWGN: simulated (markers, chain `random_bits`→`mapper`→`awgn`→`demapper`→`ber_meter`, up to 200 errors per point) and exact theory (lines). Points with fewer than 10 errors are omitted.

The ultimate limit. For a spectral efficiency η bit/s/Hz, the AWGN capacity $C = \log_2(1 + \mathrm{SNR})$ with $\mathrm{SNR} = \eta E_b/N_0$ requires $E_b/N_0 \geq (2^\eta - 1)/\eta$, which tends to $\ln 2 = -1.59\,\mathrm{dB}$ as $\eta \to 0$ (`ebn0_min_db`). The gap between this limit and the uncoded curves of Figure 6.2 is what channel coding recovers (Chapter 13).

Chapter 7

Pulse shaping and matched filtering

7.1 Linear modulation

A linearly modulated waveform is a train of pulses weighted by the symbols,

$$x(t) = \sum_{m} a_m \, g(t - mT). \tag{7.1}$$

For i.i.d. zero-mean symbols with $\mathbb{E}|a_m|^2 = E_a$, the power spectral density is

$$S_x(f) = \frac{E_a}{T} \, |G(f)|^2, \tag{7.2}$$

so the spectrum is shaped entirely by the pulse. The receiver filters with $h(t)$ and samples the output at $t = mT + \tau_0$. With $p = g \star h$ the end-to-end pulse,

$$y(mT + \tau_0) = a_m \, p(\tau_0) + \underbrace{\sum_{l \neq m} a_l \, p((m-l)T + \tau_0)}_{\text{intersymbol interference}} + (w \star h)(mT + \tau_0). \tag{7.3}$$

7.2 The Nyquist criterion

The ISI vanishes for every symbol sequence if and only if $p(nT) = \delta[n]$ (taking $\tau_0 = 0$). By the Poisson summation formula this is equivalent to the frequency-domain condition

$$\sum_{k=-\infty}^{\infty} P\!\left(f - \frac{k}{T}\right) = T \qquad \forall f. \tag{7.4}$$

The narrowest solution is the ideal low-pass of bandwidth $1/(2T)$, the sinc pulse, which is not realizable and decays only as $1/t$. The *raised-cosine* (RC) family trades bandwidth for decay. Its spectrum has a cosine roll-off of relative width β:

$$P_{\mathrm{RC}}(f) = \begin{cases} T, & |f| \le \frac{1-\beta}{2T}, \\ \frac{T}{2}\left[1 + \cos\left(\frac{\pi T}{\beta}\left(|f| - \frac{1-\beta}{2T}\right)\right)\right], & \frac{1-\beta}{2T} < |f| \le \frac{1+\beta}{2T}, \\ 0, & \text{otherwise}, \end{cases} \tag{7.5}$$

$$p_{\mathrm{RC}}(t) = \operatorname{sinc}\frac{t}{T} \, \frac{\cos(\pi \beta t/T)}{1 - (2\beta t/T)^2}. \tag{7.6}$$

The occupied bandwidth is $(1+\beta)R_s$ and the pulse decays as $1/t^3$.

7.3 The matched filter

Among all receive filters, the one maximizing the SNR at the sampling instant follows from the Cauchy–Schwarz inequality. For white noise of PSD N_0,

$$\mathrm{SNR} = \frac{\left|\int G(f)H(f)\,df\right|^2}{N_0 \int |H(f)|^2 df} \le \frac{\int |G(f)|^2 df}{N_0} = \frac{E_g}{N_0}, \tag{7.7}$$

with equality for $H(f) = G^*(f)$, that is $h(t) = g^*(-t)$. To have both a matched filter and zero ISI, the RC response is split equally between transmitter and receiver. Each side uses the *root-raised-cosine* (RRC) pulse, $|G_{\mathrm{RRC}}(f)|^2 = P_{\mathrm{RC}}(f)$, whose impulse response is

$$g_{\mathrm{RRC}}(t) = \frac{\sin\left(\pi \frac{t}{T}(1-\beta)\right) + 4\beta \frac{t}{T}\cos\left(\pi \frac{t}{T}(1+\beta)\right)}{\pi \frac{t}{T}\left(1 - (4\beta \frac{t}{T})^2\right)}, \tag{7.8}$$

with the finite limits $1-\beta+4\beta/\pi$ at $t = 0$ and $\frac{\beta}{\sqrt{2}}\left[(1+\frac{2}{\pi})\sin\frac{\pi}{4\beta} + (1-\frac{2}{\pi})\cos\frac{\pi}{4\beta}\right]$ at $t = \pm T/(4\beta)$. These special points are handled explicitly in the code:

Code 7.1: RRC taps with their singular points (`dsp/filters.py`). linkstudio/phy/dsp/filters.py, lines 19–36

```
def rrc_taps(sps: int, span: int, rolloff: float) -> np.ndarray:
    """Root-raised-cosine pulse, unit energy."""
    t = _time_axis(sps, span)
    b = rolloff
    if b == 0:
        return _unit_energy(np.sinc(t))
    h = np.empty_like(t)
    eps = 1e-9
    at_zero = np.abs(t) < eps
    at_sing = np.abs(np.abs(4 * b * t) - 1) < eps
    rest = ~(at_zero | at_sing)
    tt = t[rest]
    h[rest] = (np.sin(np.pi * tt * (1 - b)) + 4 * b * tt * np.cos(np.pi * tt * (1 + b))) /
    (
        np.pi * tt * (1 - (4 * b * tt) ** 2))
    h[at_zero] = 1 - b + 4 * b / np.pi
    h[at_sing] = b / np.sqrt(2) * ((1 + 2 / np.pi) * np.sin(np.pi / (4 * b))
                                   + (1 - 2 / np.pi) * np.cos(np.pi / (4 * b)))
    return _unit_energy(h)
```

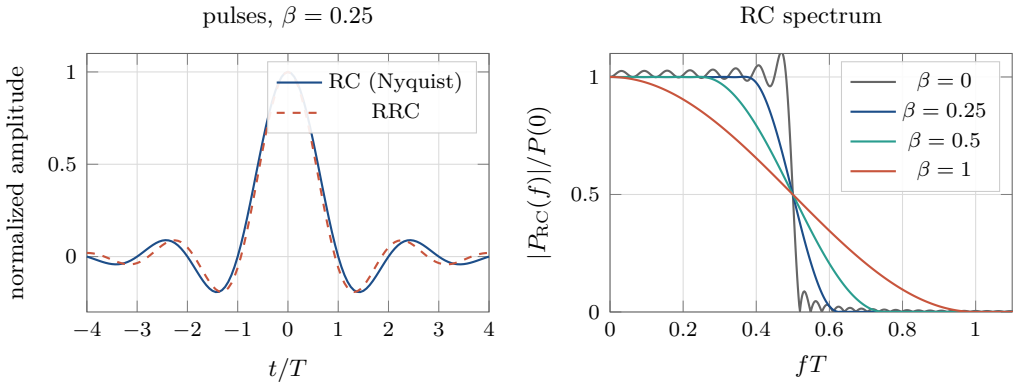


Figure 7.1: Left: the RC pulse crosses zero at every non-zero multiple of T, while the RRC pulse alone does not. Right: spectra of the truncated RC taps produced by `rc_taps` (span 32 symbols). All curves pass through 0.5 at $fT = 0.5$, as required by Equation (7.4).

7.4 Discrete-time implementation

In the code `pulse_shaper` upsamples by N_{sps} (zero stuffing) and convolves with unit-energy taps spanning `span` symbols. It records the taps in `pulse_taps`, the group delay $(L-1)/2$ in `delay`, and the power per sample $1/N_{\mathrm{sps}}$ (unit symbol energy spread over

N_{sps} samples).
`matched_filter` convolves with $g^*[-n]$ (or another pulse), samples at the known delay and rescales.

Code 7.2: The pulse shaper and its metadata. `linkstudio/phy/waveform/pulse_shaping.py`, lines 46–53

```
def process(self, inputs: dict[str, Signal], ctx: RunContext) -> dict[str, Signal]:
    x = inputs["in"]
    sps = self.params.sps
    y = fir_filter(upsample(x.data, sps), self.taps)
    return {"out": Signal(y, Kind.SAMPLES, fs=x.fs * sps if x.fs else None, meta={
        **x.meta, "sps": sps, "pulse_taps": self.taps, "n_symbols": x.length,
        "power": (x.meta.get("power") or 1.0) / sps,  # unit-energy pulse spreads Es over sps samples
        "delay": x.meta.get("delay", 0.0) + (len(self.taps) - 1) / 2})}
```

With unit-energy taps the peak of the end-to-end pulse is $p[0] = \sum_n |g[n]|^2 = 1$, and the noise gain of the matched filter is $\sum_n |h[n]|^2 = 1$. The block nevertheless normalizes by the actual peak and scales `noise_var` by $\sum |h|^2 / |p[0]|^2$, so that mismatched receive filters (an RC receive filter, a rectangular integrate-and-dump) also yield symbols at nominal amplitude with the correct noise variance:

Code 7.3: Normalization of the matched filter output. `linkstudio/phy/waveform/pulse_shaping.py`, lines 96–106

```
y = fir_filter(x.data, rx_taps)
n_sym = x.meta.get("n_symbols", x.length // sps)
start = int(round(x.meta.get("delay", 0.0) + (len(rx_taps) - 1) / 2))

noise_gain = float(np.sum(np.abs(rx_taps) ** 2))
if p.normalize and tx_taps is not None:
    overall = np.convolve(tx_taps, rx_taps)
    peak = overall[int(round((len(tx_taps) - 1) / 2 + (len(rx_taps) - 1) / 2))]
    y = y / peak
    noise_gain /= abs(peak) ** 2
noise = {"noise_var": x.meta["noise_var"] * noise_gain} if x.meta.get("noise_var") is not None else {}
```

Following Equation (5.3), the AWGN block sets $\sigma^2 = P N_{\text{sps}} / (E_s/N_0) = 1/(E_s/N_0)$ on the waveform. After the matched filter the symbols have unit energy and noise variance σ^2, so the symbol SNR equals E_s/N_0, the matched-filter bound. This is verified in `tests/test_physics.py::test_pulse_shaped_link_matches_theory`: the BER of an RRC-shaped link equals the symbol-rate theory.

Truncation Taps are truncated to `span` symbols. Short spans raise the spectral side lobes and leave a small residual ISI that grows as $\beta \to 0$. For dense WDM or very small roll-offs, use spans of 32 symbols or more (the coherent optical scenarios use 32).

7.5 Eye diagrams

Overlaying traces of two symbol periods shows the ISI at a glance (Figure 7.2). After the RRC transmit filter alone the eye is partially closed, because an RRC pulse is not a Nyquist pulse. After the matched filter all traces pass through ± 1 at the sampling instant.

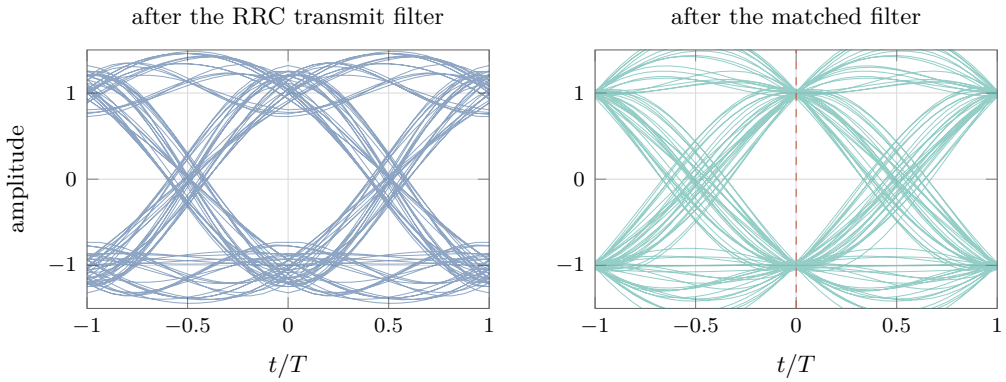


Figure 7.2: Eye diagrams of binary PAM with RRC pulses ($\beta = 0.35$, 16 samples/symbol, 30 dB SNR), cut from the outputs of `pulse_shaper` and `matched_filter` (`decimate=false`). The dashed line is the sampling instant used by the matched filter.

Chapter 8

Radio front-end impairments

Real transmitters and receivers distort the signal before any propagation effect. This chapter models the impairments implemented in `phy/channel/impairments.py`. Figure 8.1 shows their signatures on a 16-QAM constellation.

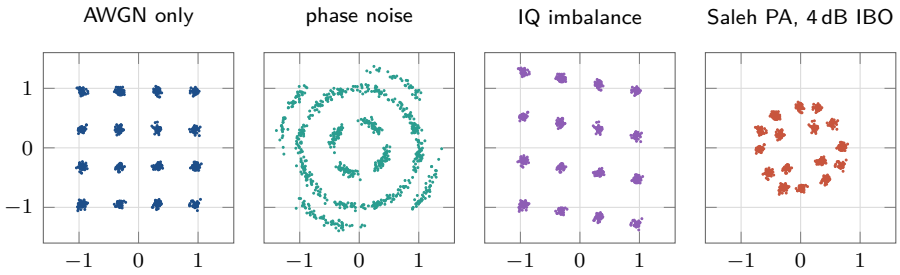


Figure 8.1: 16-QAM at $E_s/N_0 = 26\,\mathrm{dB}$ through `phase_noise` (150 Hz linewidth at 1 MBd), `iq_imbalance` (1.5 dB, 8°) and `power_amplifier` (Saleh). Phase noise smears points along circles; IQ imbalance turns the square grid into a parallelogram; the PA compresses and rotates the outer points.

8.1 Carrier frequency and phase offsets

A local oscillator at $f_c + \Delta f$ with phase ϕ_0 turns the received envelope into

$$y[n] = x[n]\, e^{\,j(2\pi \Delta f\, n/f_s + \phi_0)}. \tag{8.1}$$

`carrier_offset` applies this rotation (`cfo_hz`, `phase_deg`, or a uniformly random phase per frame). Synchronization (Chapter 10) estimates and removes it.

8.2 Oscillator phase noise

A free-running oscillator accumulates phase errors. The standard model, and the one used for lasers in Chapter 15, is a Wiener process: the instantaneous frequency deviation is white, so the phase performs a random walk. In discrete time,

$$\phi[n] = \phi[n-1] + \varepsilon[n], \qquad \varepsilon[n] \sim \mathcal{N}\!\left(0,\ \frac{2\pi\Delta\nu}{f_s}\right). \tag{8.2}$$

Proposition 8.1. The field $e^{j\phi(t)}$ of a Wiener phase with increment variance $2\pi\Delta\nu|\tau|$ has a Lorentzian spectrum of full width at half maximum $\Delta\nu$.

Proof. $\phi(t+\tau) - \phi(t) \sim \mathcal{N}(0, 2\pi\Delta\nu|\tau|)$, and $\mathbb{E}e^{jX} = e^{-\operatorname{Var} X/2}$ for a Gaussian X, so the autocorrelation is $R(\tau) = e^{-\pi\Delta\nu|\tau|}$. Its Fourier transform is

$$S(f) = \frac{\Delta\nu/(2\pi)}{f^2 + (\Delta\nu/2)^2},$$

which falls to half its peak at $f = \pm\Delta\nu/2$. □

Code 8.1: Wiener phase noise (`phase_noise`). `linkstudio/phy/channel/impairments.py`, lines 74–79

```
74 def process(self, inputs: dict[str, Signal], ctx: RunContext) -> dict[str, Signal]:
75     x = inputs["in"]
76     fs = _require_fs(x, "Phase noise")
77     var = 2 * np.pi * self.params.linewidth_hz / fs
78     phi = np.cumsum(self.rng.standard_normal(x.data.shape) * np.sqrt(var), axis=-1)
79     return {"out": x.evolve(x.data * np.exp(1j * phi))}
```

The phase variance grows linearly with time. What matters for a receiver is therefore the product $\Delta\nu\, T$ of linewidth and symbol duration: carrier recovery must track the phase faster than it wanders.

8.3 IQ imbalance

In a quadrature down-converter the Q branch may have a gain $g \neq 1$ and a phase error φ relative to the I branch. Writing the two branch outputs and recombining them gives the widely-linear model

$$y = K_1 x + K_2 x^{*} + d, \qquad K_1 = \frac{1 + g\, e^{-j\varphi}}{2}, \qquad K_2 = \frac{1 - g\, e^{j\varphi}}{2}, \tag{8.3}$$

where d is a DC offset. The conjugate term creates a mirror image of the signal. Its strength is measured by the image rejection ratio $\mathrm{IRR} = |K_1|^2/|K_2|^2$; for $g = 1.5\,\mathrm{dB}$ and $\varphi = 8^\circ$ it is about 19 dB. On a constellation it shears the square grid (Figure 8.1).

Code 8.2: IQ imbalance and DC offset (`iq_imbalance`). `linkstudio/phy/channel/impairments.py`, lines 117–124

```
117 def process(self, inputs: dict[str, Signal], ctx: RunContext) -> dict[str, Signal]:
118     x = inputs["in"]
119     p = self.params
120     g = 10 ** (p.amplitude_db / 20)
121     phi = np.deg2rad(p.phase_deg)
122     k1 = (1 + g * np.exp(-1j * phi)) / 2
123     k2 = (1 - g * np.exp(1j * phi)) / 2
124     return {"out": x.evolve(k1 * x.data + k2 * np.conj(x.data) + complex(p.dc_i, p.dc_q))}
```

8.4 Power amplifier nonlinearity

A memoryless band-pass nonlinearity acts on the envelope $x = r e^{j\theta}$ as

$$y = A(r)\, e^{\,j(\theta + \Phi(r))}, \tag{8.4}$$

where $A(r)$ is the AM/AM curve and $\Phi(r)$ the AM/PM curve. The input is first scaled so that its mean power lies `input_backoff_db` below the saturation level $A_{\mathrm{sat}} = 1$ (input back-off, IBO). Three models are implemented:

$$\text{Rapp (solid state):} \quad A(r) = \frac{r}{\left(1 + r^{2p}\right)^{1/(2p)}}, \qquad \Phi = 0, \tag{8.5}$$

$$\text{Saleh (TWTA):} \quad A(r) = \frac{r}{1 + \beta_a r^2}, \qquad \Phi(r) = \frac{\alpha_\phi r^2}{1 + \beta_\phi r^2}, \tag{8.6}$$

$$\text{ideal clipper:} \quad A(r) = \min(r, 1), \qquad \Phi = 0. \tag{8.7}$$

The Saleh AM/AM curve is divided by α_a so that all models have unit small-signal gain; the default constants are those of Saleh's original TWT fit [6]. The smoothness p of the Rapp model interpolates between a soft compression ($p = 1$) and the ideal clipper ($p \to \infty$).

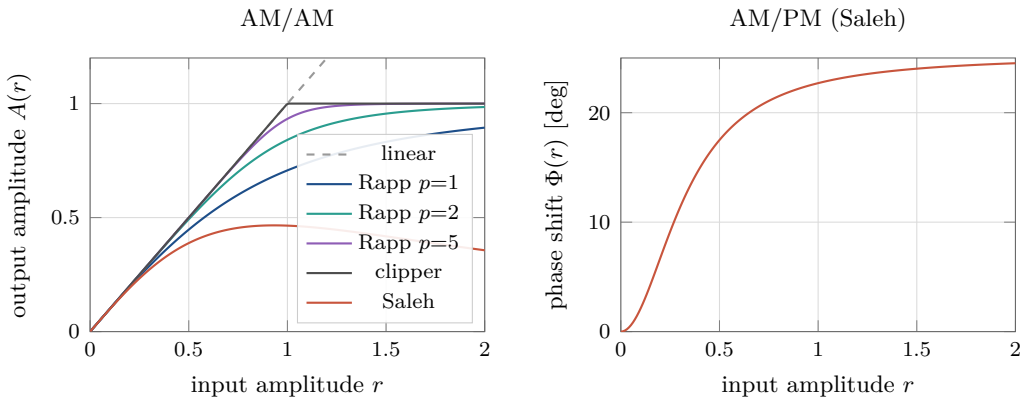


Figure 8.2: PA characteristics measured by driving the `power_amplifier` block with an amplitude ramp (back-off chosen so that the block's input scaling is unity).

Code 8.3: Back-off scaling and the AM/AM, AM/PM models. `linkstudio/phy/channel/impairments.py`, lines 150–163

```
p_in = float(np.mean(np.abs(x.data) ** 2))
scale = np.sqrt(10 ** (-p.input_backoff_db / 10) / max(p_in, 1e-300))
u = x.data * scale
r, theta = np.abs(u), np.angle(u)
if p.model == "rapp":
    a = r / (1 + r ** (2 * p.smoothness)) ** (1 / (2 * p.smoothness))
    y = a * np.exp(1j * theta)
elif p.model == "saleh":
    # Divided by alpha_a so that the small-signal gain is 1, like the other models.
    a = r / (1 + p.saleh_beta_a * r ** 2)
    dphi = p.saleh_alpha_p * r ** 2 / (1 + p.saleh_beta_p * r ** 2)
    y = a * np.exp(1j * (theta + dphi))
else:
    y = np.minimum(r, 1.0) * np.exp(1j * theta)
```

Watch out The nonlinearity generates spectral regrowth (out-of-band emission) and an in-band error that is *not* Gaussian. For OFDM, whose envelope is nearly Gaussian (Section 11.5), an IBO of several dB is needed. Compare the `power_meter` PAPR with the back-off.

8.5 Analog-to-digital conversion

`quantizer` models a uniform mid-rise ADC with b bits on each of I and Q. The full scale A is either given or set to `loading_factor`$\times$ the RMS value σ_x of each component. The step is $\Delta = 2A/2^b$. For a signal that does not clip, the quantization error is approximately uniform with variance $\Delta^2/12$, giving

$$\mathrm{SQNR} = \frac{\sigma_x^2}{\Delta^2/12} = \frac{3 \cdot 2^{2b}}{(A/\sigma_x)^2} \;\Rightarrow\; \mathrm{SQNR}_{\mathrm{dB}} \approx 6.02\,b + 4.77 - 20\log_{10}\frac{A}{\sigma_x}. \tag{8.8}$$

With the default loading factor of 4 (clipping probability about 6×10^{-5} for a Gaussian component), an 8-bit ADC gives about 41 dB.

Code 8.4: Uniform mid-rise quantization with clipping. `linkstudio/phy/channel/impairments.py`, lines 191–198

```
levels = 1 << p.bits
```

```
step = 2 * fs / levels

def q(v: np.ndarray) -> np.ndarray:
    idx = np.clip(np.floor(v / step), -levels // 2, levels // 2 - 1)
    return (idx + 0.5) * step

return {"out": x.evolve(q(x.data.real) + 1j * q(x.data.imag))}
```

8.6 Timing offset

`timing_offset` delays the signal by $d = \lfloor d \rfloor + \mu$ samples. The integer part is a shift. The fractional part uses a Blackman-windowed sinc interpolator of 33 taps, $h[n] = \operatorname{sinc}(n - 16 - \mu)\, w[n] / \sum_n \operatorname{sinc}(n - 16 - \mu)\, w[n]$, which is accurate as long as the signal is oversampled. The frame length is preserved, so a delay pushes the tail out of the frame; the `preamble` block appends guard symbols for this reason.

Chapter 9

Fading channels and equalization

9.1 The multipath channel

In a radio channel the transmitted signal reaches the receiver along many paths with different delays τ_l, complex gains and Doppler shifts. The low-pass equivalent channel is a linear time-varying filter,

$$y(t) = \sum_l a_l(t)\, x(t-\tau_l) + w(t), \qquad h(\tau;t) = \sum_l a_l(t)\,\delta(\tau-\tau_l). \tag{9.1}$$

Two scales characterize it:

- the *delay spread* σ_τ, whose inverse is the coherence bandwidth $B_c \approx 1/(5\sigma_\tau)$;
- the maximum Doppler shift $f_D = v f_c/c$, whose inverse sets the coherence time $T_c \approx 0.423/f_D$.

A signal of bandwidth $B \ll B_c$ sees *flat* fading (a single complex gain). If $B \gtrsim B_c$ the fading is frequency selective and causes ISI.

9.2 Rayleigh and Rician fading

When a path is the superposition of many unresolved scatterers with random phases, the central limit theorem makes its gain circular complex Gaussian, $h \sim \mathcal{CN}(0,1)$. The envelope $|h|$ is then Rayleigh distributed and the power $|h|^2$ exponential. A line-of-sight component of relative power K (the Rician factor) adds a deterministic term:

$$h = \sqrt{\frac{K}{K+1}}\, e^{j\theta_0} + \sqrt{\frac{1}{K+1}}\, h_{\mathrm{Ray}}, \qquad \mathbb{E}|h|^2 = 1. \tag{9.2}$$

9.2.1 Time correlation: the Clarke–Jakes model

For isotropic scattering in the horizontal plane, a path arriving at angle α is shifted by $f_D\cos\alpha$. Averaging over a uniform α gives [19] the autocorrelation and the U-shaped Doppler spectrum

$$R_h(\tau) = \mathbb{E}[h(t+\tau)h^*(t)] = J_0(2\pi f_D \tau), \qquad S_h(f) = \frac{1}{\pi f_D\sqrt{1-(f/f_D)^2}},\ |f| < f_D. \tag{9.3}$$

LinkStudio generates it with the sum-of-sinusoids model of Zheng and Xiao [27]:

$$h[n] = \sqrt{\tfrac{2}{M}} \sum_{i=1}^{M} \left(\cos\psi_i + j\sin\psi_i\right)\cos(2\pi f_D \cos\alpha_i\, n/f_s + \phi), \qquad \alpha_i = \frac{2\pi i - \pi + \theta}{4M}, \tag{9.4}$$

with θ, ϕ and the ψ_i independent and uniform on $[-\pi,\pi)$. Jakes' original deterministic model is not stationary. Fully random arrival angles are stationary but need many sinusoids before a single realization behaves like the ensemble. The Zheng–Xiao choice spreads the M angles evenly over a quarter circle with one random offset. The process is wide-sense stationary, $E|h|^2 = 1$, the in-phase and quadrature parts are uncorrelated, and the autocorrelation matches Equation (9.3) closely with only $M = 8$ to 16 sinusoids (16 by default). The test suite checks R_h against J_0 to within 0.02 and the I/Q cross-correlation.

Code 9.1: Sum-of-sinusoids Rayleigh process (`channel/fading.py`).
linkstudio/phy/channel/fading.py, lines 68–86

```python
def sum_of_sinusoids(rng: np.random.Generator, batch: int, n: int, doppler_norm: float,
                     n_sinusoids: int = 16) -> np.ndarray:
    """Time-varying Rayleigh process with Jakes (Clarke) Doppler spectrum, E|h|^2 = 1.

    Zheng & Xiao (2003) sum-of-sinusoids generator: M sinusoids with arrival angles
    alpha_n = (2 pi n - pi + theta) / (4 M) and independent random phases. Unlike the
    original Jakes model it is wide-sense stationary, and its autocorrelation approaches
    J0(2 pi f_D tau) per realization with few sinusoids. ``doppler_norm`` is f_D / f_s.
    """
    m = n_sinusoids
    wd = 2 * np.pi * doppler_norm * np.arange(n)
    theta = rng.uniform(-np.pi, np.pi, (batch, 1))
    phi = rng.uniform(-np.pi, np.pi, (batch, 1))
    psi = rng.uniform(-np.pi, np.pi, (batch, m))
    alpha = (2 * np.pi * np.arange(1, m + 1) - np.pi + theta) / (4 * m)  # (batch, m)
    arg = wd[None, None, :] * np.cos(alpha)[:, :, None] + phi[:, :, None]
    xc = np.sum(np.cos(psi)[:, :, None] * np.cos(arg), axis=1)
    xs = np.sum(np.sin(psi)[:, :, None] * np.cos(arg), axis=1)
    return (xc + 1j * xs) * np.sqrt(2 / m)
```

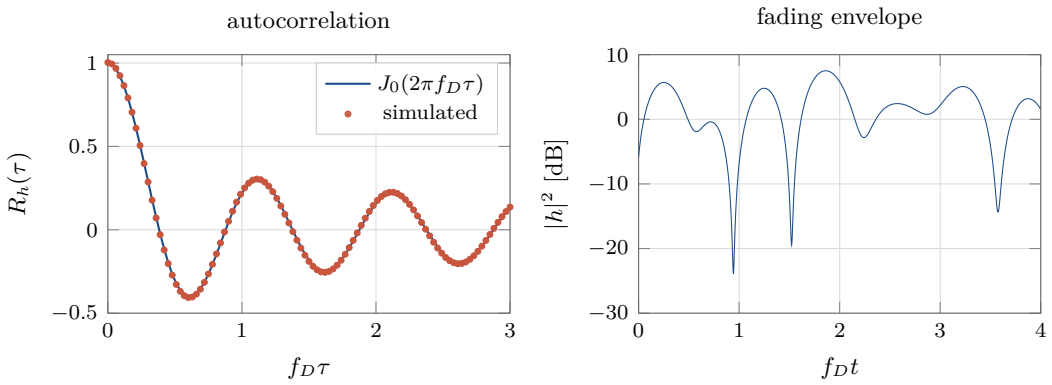


Figure 9.1: Left: autocorrelation of `sum_of_sinusoids` (400 realizations, $f_D/f_s = 0.01$) against Equation (9.3). Right: a realization; fades deeper than 20 dB occur about once per wavelength travelled.

In the code `flat_fading` multiplies the input by h and publishes the true gains on its `csi` output (`csi_type="gain"`). Three time-variation modes are available:

- `block`: one coefficient per frame (quasi-static);
- `fast`: independent per sample (ideal interleaving);
- `jakes`: correlated with Doppler `doppler_hz`; needs `fs`.

Code 9.2: Fading generation and the Rician line-of-sight term.linkstudio/phy/channel/fading.py, lines 115–126

```python
if p.variation == "block":
    h = complex_normal(self.rng, (B, 1))
elif p.variation == "fast":
    h = complex_normal(self.rng, (B, n))
else:
    if not x.fs:
        raise BlockError("Jakes fading needs the sample rate (fs) of the signal.")
```

```
    h = sum_of_sinusoids(self.rng, B, n, p.doppler_hz / x.fs, p.n_sinusoids)
if p.model == "rician":
    k = 10 ** (p.k_factor_db / 10)
    los = np.exp(1j * self.rng.uniform(0, 2 * np.pi, (B, 1)))
    h = np.sqrt(k / (k + 1)) * los + np.sqrt(1 / (k + 1)) * h
```

9.3 Tapped delay lines and 3GPP profiles

`multipath` implements $y(t) = \sum_l h_l(t)\, x(t - \tau_l)$ on the sample grid. A delay $\tau_l f_s$ that is not an integer cannot be applied by shifting samples. Rounding it would merge taps: at LTE rates (15.36–30.72 MHz) the EPA taps at 30, 70, 90 and 110 ns fall onto two samples, and the frequency correlation of the channel changes. The block therefore applies each path through a band-limited fractional delay: a Hann-windowed sinc of ± 8 samples, normalized to unit energy. Over 75 % of the Nyquist band it delays by the exact fractional amount within 5 % (checked in the tests). Integer delays stay single taps, and `fractional_delays=false` restores rounding. The profile is normalized to unit total power. Taps are Rayleigh (or static with `fading=false`), constant per frame or each with an independent Doppler process. The 3GPP extended models of TS 36.104 [20] are built in:

Model	delays [ns]	relative powers [dB]
EPA	0, 30, 70, 90, 110, 190, 410	0, −1, −2, −3, −8, −17.2, −20.8
EVA	0, 30, 150, 310, 370, 710, 1090, 1730, 2510	0, −1.5, −1.4, −3.6, −0.6, −9.1, −7, −12, −16.9
ETU	0, 50, 120, 200, 230, 500, 1600, 2300, 5000	−1, −1, −1, 0, 0, 0, −3, −5, −7

Their RMS delay spreads are 43, 357 and 991 ns.

5G NR profiles. TR 38.901 §7.7.2 [28] defines TDL-A, B and C (NLOS) and TDL-D and E (LOS) with normalized delays: the tabulated profiles have unit RMS delay spread. The delays are scaled to the wanted spread DS (`delay_spread_ns`, from 10 ns "very short" to 1000 ns "very long"), $\tau_n = \tau_{n,\mathrm{model}}\,\mathrm{DS}$. Computing the RMS spread of the transcribed tables gives 1.000 for A, B, C and E and 0.994 for D, a check on the 23–24 tap values that the tests repeat. In TDL-D and TDL-E the first tap holds a LOS path (power −0.2 dB and −0.03 dB) on top of a Rayleigh component at the same delay. The block gives the LOS path a fixed amplitude with random phase; in `jakes` mode it also has the Doppler shift $0.7 f_D$ of the standard. The first tap is then Ricean with $K_1 = 13.3$ dB (TDL-D) or 22 dB (TDL-E). The tests estimate K_1 from 4000 frames and recover 13.3 dB within 1 dB.

The channel impulse response is published on `csi`: `csi_type="cir"` with shape (B, L) for block fading, or `"cir_tv"` with shape (B, L, n) for time-varying taps.

Code 9.3: Band-limited fractional delay used for every tap. `linkstudio/phy/channel/fading.py`, lines 51–60

```
whole = int(np.floor(delay))
frac = delay - whole
if frac < 1e-9:
    return whole, np.ones(1)
k = np.arange(-half_width + 1, half_width + 1)
taps = np.sinc(k - frac) * (0.5 + 0.5 * np.cos(np.pi * (k - frac) / half_width))
first = whole + k[0]
if first < 0:
    taps, first = taps[-first:], 0
return first, taps / np.linalg.norm(taps)
```

9.4 Path loss and link budgets

Fading describes fluctuations around a mean received power. The mean itself follows from the distance through the path loss. `path_loss` offers four models.

- **Free space** (ITU-R P.525 [29]): $L = 20\log_{10}(4\pi d f_c/c)$, i.e. $32.45 + 20\log_{10} f_{\mathrm{MHz}} + 20\log_{10} d_{\mathrm{km}}$ dB.
- **UMa, UMi street canyon, InH office** (TR 38.901 Table 7.4.1-1 [28]), f_c in GHz and d_{3D} in metres. LOS uses a dual-slope law. In UMi, for example,

$$\mathrm{PL}_{\mathrm{LOS}} = \begin{cases} 32.4 + 21\log_{10} d_{3D} + 20\log_{10} f_c, & d_{2D} \le d'_{\mathrm{BP}} \\ 32.4 + 40\log_{10} d_{3D} + 20\log_{10} f_c & \\ \quad -9.5\log_{10}\left(d'^2_{\mathrm{BP}} + (h_{\mathrm{BS}} - h_{\mathrm{UT}})^2\right), & d_{2D} > d'_{\mathrm{BP}} \end{cases} \tag{9.5}$$

  with breakpoint $d'_{\mathrm{BP}} = 4h'_{\mathrm{BS}}h'_{\mathrm{UT}}f_c/c$ and effective heights $h' = h - 1\,\mathrm{m}$. This is the distance at which the ground-reflected ray starts cancelling the direct one, so the slope changes from about 2 to 4. NLOS is the larger of the LOS value and a steeper NLOS law, e.g. $35.3\log_{10} d_{3D} + 22.4 + 21.3\log_{10} f_c - 0.3(h_{\mathrm{UT}} - 1.5)$ for UMi. Shadow fading is log-normal with $\sigma_{\mathrm{SF}} = 4/7.82\,\mathrm{dB}$ (UMi LOS/NLOS), $4/6\,\mathrm{dB}$ (UMa) and $3/8.03\,\mathrm{dB}$ (InH).

With `condition=probabilistic` each frame is LOS with the probability of Table 7.4.2-1, e.g. $P_{\mathrm{LOS}} = 18/d + e^{-d/36}(1 - 18/d)$ for UMi beyond 18 m. The UMa breakpoint uses $h_E = 1\,\mathrm{m}$, the standard's value for user heights below 13 m.

Code 9.4: Path loss and shadow-fading standard deviation of TR 38.901.
linkstudio/phy/channel/propagation.py, lines 35–56

```
d2d = np.asarray(d2d_m, dtype=float)
d3d = np.sqrt(d2d ** 2 + (h_bs - h_ut) ** 2)
lf, ld = np.log10(fc_ghz), np.log10(d3d)
if scenario in ("uma", "umi"):
    dbp = 4 * (h_bs - 1.0) * (h_ut - 1.0) * fc_ghz * 1e9 / C0
    if scenario == "uma":
        pl1 = 28.0 + 22 * ld + 20 * lf
        pl2 = 28.0 + 40 * ld + 20 * lf - 9 * np.log10(dbp ** 2 + (h_bs - h_ut) ** 2)
        nlos = 13.54 + 39.08 * ld + 20 * lf - 0.6 * (h_ut - 1.5)
        sigma = (4.0, 6.0)
    else:
        pl1 = 32.4 + 21 * ld + 20 * lf
        pl2 = 32.4 + 40 * ld + 20 * lf - 9.5 * np.log10(dbp ** 2 + (h_bs - h_ut) ** 2)
        nlos = 35.3 * ld + 22.4 + 21.3 * lf - 0.3 * (h_ut - 1.5)
        sigma = (4.0, 7.82)
    pl_los = np.where(d2d <= dbp, pl1, pl2)
else:
    pl_los = 32.4 + 17.3 * ld + 20 * lf
    nlos = 38.3 * ld + 17.30 + 24.9 * lf
    sigma = (3.0, 8.03)
pl = np.where(los, pl_los, np.maximum(pl_los, nlos))
return pl, np.where(los, sigma[0], sigma[1])
```

Absolute noise. A link budget needs the noise in watts. `awgn` in `mode=thermal` adds $N_0 = k_B T F$ per hertz, i.e. variance $k_B T F\, f_s$ per complex sample, with noise figure F (`noise_figure_db`) and $T = 290\,\mathrm{K}$. The signal must then carry absolute power. The mapper's unit power is read as 1 W (30 dBm) and the transmit power is set with `gain` (`gain_db` $= P_{\mathrm{tx}}[\mathrm{dBm}] - 30$). At one sample per symbol $E_s/N_0 = P_{\mathrm{rx}}/(k_B T F R_s)$. A test runs QPSK at 7 dBm over 2 km of free space at 2.4 GHz into a 5 dB noise figure and checks the BER against theory at that E_s/N_0.

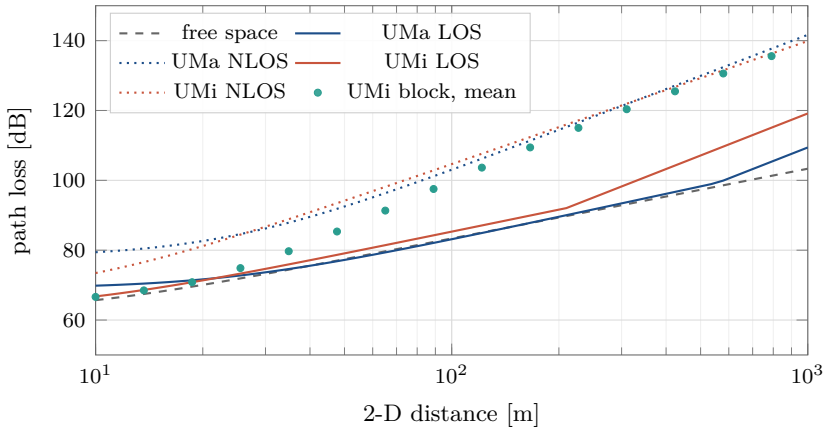


Figure 9.2: TR 38.901 path loss at 3.5 GHz with default heights (UMa 25 m, UMi 10 m, UT 1.5 m). The UMa LOS curve bends at the breakpoint $d'_{\mathrm{BP}} \approx 560$ m (UMi: 210 m). Markers: mean loss of `path_loss` (UMi, probabilistic LOS, shadowing on, 4000 frames). Close to the base station they follow LOS; far away, NLOS.

The *Urban micro-cell link budget* scenario combines `path_loss` and thermal noise: sweep the distance and watch the LOS fraction and the frame error rate.

9.5 Error probability with fading

With coherent detection and perfect channel knowledge, the instantaneous SNR of BPSK in flat fading is $\gamma = |h|^2 E_b/N_0$, exponentially distributed with mean $\bar{\gamma}$. Averaging the AWGN error probability over it gives

$$\bar{P}_b = \int_0^\infty \mathrm{Q}\left(\sqrt{2\gamma}\right) \frac{e^{-\gamma/\bar{\gamma}}}{\bar{\gamma}}\, d\gamma = \frac{1}{2}\left(1 - \sqrt{\frac{\bar{\gamma}}{1+\bar{\gamma}}}\right) \xrightarrow{\bar{\gamma}\to\infty} \frac{1}{4\bar{\gamma}}. \tag{9.6}$$

The error rate now falls only as $1/\mathrm{SNR}$ instead of exponentially: 10 dB more SNR buys one decade of BER. Diversity, the subject of Section 12.3, restores a steeper slope. `ber_rayleigh` performs the same average numerically for any AWGN curve. Figure 12.2 compares Equation (9.6) with the chain `flat_fading`→`awgn`→`one_tap_equalizer`.

9.6 Equalization

9.6.1 One-tap equalization

For flat fading, $y = hs + w$, and dividing by the known gain gives $\hat{s} = s + w/h$. The noise variance becomes $\sigma^2/|h|^2$ symbol by symbol, and `one_tap_equalizer` writes exactly this array to `noise_var`, so the soft demapper weights faded symbols correctly. The `phase` variant only derotates, which is sufficient for PSK.

9.6.2 Linear FIR equalizers designed from the channel

For a frequency-selective channel with impulse response $\boldsymbol{h} = (h_0, \ldots, h_{L-1})$, a linear equalizer $\boldsymbol{w}$ of N taps produces the combined response $\mathbf{H}\boldsymbol{w}$, where $\mathbf{H}$ is the $(N + L - 1) \times N$ convolution matrix of $\boldsymbol{h}$. Zero forcing asks the combined response to approximate a unit impulse $\boldsymbol{e}_d$ at decision delay d. The MMSE design adds the noise power σ^2 as a regularizer:

$$\boldsymbol{w}_d = \arg\min_{\boldsymbol{w}} \; \|\mathbf{H}\boldsymbol{w} - \boldsymbol{e}_d\|^2 + \lambda\|\boldsymbol{w}\|^2 = (\mathbf{H}^{\mathsf{H}}\mathbf{H} + \lambda\mathbf{I})^{-1}\mathbf{H}^{\mathsf{H}}\boldsymbol{e}_d, \qquad \lambda = \begin{cases} \sigma^2 & \text{MMSE} \\ \approx 0 & \text{ZF.} \end{cases} \tag{9.7}$$

All delays are solved at once, since the columns of $(\mathbf{H}^{\mathsf{H}}\mathbf{H} + \lambda \mathbf{I})^{-1}\mathbf{H}^{\mathsf{H}}$ are the solutions for every d. The delay with the largest useful gain $(\mathbf{H}\boldsymbol{w}_d)_d$ is selected. The output is then scaled by $1/(\mathbf{H}\boldsymbol{w}_d)_d$ to remove the MMSE bias, and the noise variance becomes $\sigma^2\|\boldsymbol{w}\|^2$.

Code 9.5: Least-squares design of ZF/MMSE equalizers with delay selection (`linear_equalizer`).
`linkstudio/phy/receiver.py`, lines 87–108

```
H = np.zeros((B, Lc, n), dtype=np.complex128)
for j in range(n):
    H[:, j:j + L, j] = h
nv = x.meta.get("noise_var", 0.0)
lam = np.mean(nv) if self.params.method == "mmse" else 1e-9
A = np.conj(np.swapaxes(H, 1, 2)) @ H + lam * np.eye(n)
W = np.linalg.solve(A, np.conj(np.swapaxes(H, 1, 2)))  # (B, n, Lc): column d = taps for delay d
if self.params.delay is not None:
    if self.params.delay >= Lc:
        raise BlockError(f"Delay must be < n_taps + L - 1 = {Lc}.")
    d = np.full(B, self.params.delay)
else:
    gain = np.real(np.einsum("bij,bji->bi", H, W))  # (H W)[d, d]
    d = np.argmax(gain, axis=1)
rows = np.arange(B)
w = W[rows, :, d]  # (B, n)
bias = np.einsum("bij,bj->bi", H, w)[rows, d]
w = w / bias[:, None]
full = np.stack([np.convolve(x.data[b], w[b]) for b in range(B)])
idx = d[:, None] + np.arange(x.length)
y = np.take_along_axis(np.pad(full, ((0, 0), (0, Lc))), idx, axis=1)
new_nv = np.asarray(nv) * np.sum(np.abs(w) ** 2, axis=1)[:, None]
```

9.6.3 Adaptive equalization: LMS

When the channel is unknown, the equalizer $z[n] = \boldsymbol{w}^{\mathsf{H}}\boldsymbol{u}[n]$, with $\boldsymbol{u}[n] = (y[n], y[n-1], \dots)$, can minimize $\mathbb{E}|d[n] - z[n]|^2$ by stochastic gradient descent. The instantaneous gradient with respect to $\boldsymbol{w}^*$ is $-\boldsymbol{u}\,e^*$ with $e = d - z$, giving the LMS update and its normalized version:

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \mu\,\boldsymbol{u}[n]\,e^*[n], \qquad \mu_{\text{NLMS}} = \frac{\tilde{\mu}}{\|\boldsymbol{u}[n]\|^2 + \epsilon}, \quad 0 < \tilde{\mu} < 2. \tag{9.8}$$

The reference $d[n]$ is the known training sequence for the first `n_train` symbols and the nearest constellation point afterwards (decision-directed mode). The residual MSE measured after training is written to `noise_var`.

Code 9.6: The (N)LMS loop runs over time but processes all frames of the batch at once.
`linkstudio/phy/receiver.py`, lines 147–166

```
for t in range(N + D):
    u = ypad[:, t:t + n][:, ::-1]  # [y[t], y[t-1], ...]
    z = np.sum(np.conj(w) * u, axis=1)
    k = t - D
    if k < 0:
        continue
    out[:, k] = z
    if k < p.n_train:
        target = ref.data[:, k]
    elif p.decision_directed:
        target = const.points[const.nearest(z)]
    else:
        continue
    e = target - z
```

```
    if k >= p.n_train:
        err_power += float(np.sum(np.abs(e) ** 2))
        err_count += B
    mu = p.step_size / (1e-9 + np.sum(np.abs(u) ** 2, axis=1, keepdims=True)) if p.
    normalized \
        else p.step_size
    w += mu * u * np.conj(e)[:, None]
```

ZF versus MMSE Zero forcing inverts spectral nulls and amplifies the noise there. MMSE balances ISI against noise and tends to ZF at high SNR. The decision-feedback equalizer of Section 14.5 removes post-cursor ISI without noise enhancement, at the cost of error propagation.

Chapter 10

Synchronization

A receiver does not know when symbols start, where the frame begins, or the exact carrier frequency and phase. Figure 10.1 shows the synchronized single-carrier receiver of `phy/sync.py`, which recovers these quantities in three stages.

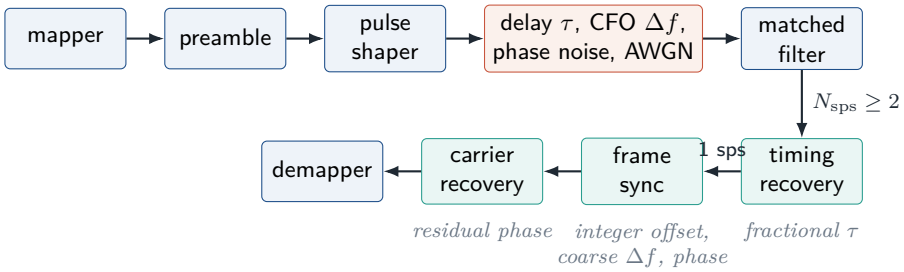


Figure 10.1: The synchronized receiver of the *Synchronized QPSK receiver* scenario.

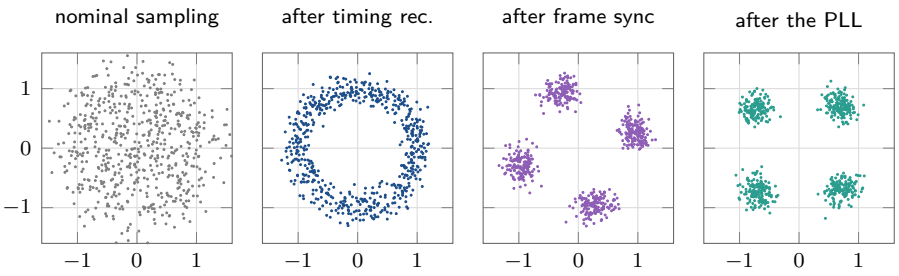


Figure 10.2: QPSK through a delay of 9.37 samples, a 3 kHz CFO at 1 MBd, 10 Hz phase noise and 12 dB E_b/N_0. Panels: sampled at the nominal instants (ISI and rotation), after Gardner timing recovery (rotating ring), after preamble-based CFO and phase correction (residual spread), after the decision-directed PLL.

10.1 The preamble

`preamble` prepends L known QPSK symbols made of two identical halves of length $h = L/2$, the Schmidl–Cox structure [9], and appends `guard` zero symbols. The repetition makes the timing metric insensitive to the CFO and yields a frequency estimate (Section 10.3). The guard gives the receiver slack for unknown delays.

10.2 Symbol timing recovery

After the matched filter the waveform is sampled at N_{sps} samples per symbol, but the optimum instants $mT + \tau$ fall between samples. Timing recovery has three parts:

- an *interpolator* that evaluates the waveform at arbitrary instants;
- a *timing error detector* (TED);
- a *loop filter* that turns detector outputs into corrections.

10.2.1 Interpolation

A cubic Lagrange (Farrow) interpolator evaluates $x(i+\mu)$ for $0 \le \mu < 1$ from four neighbours:

$$x(i+\mu) = c_0 + c_1\mu + c_2\mu^2 + c_3\mu^3, \quad \begin{aligned} c_0 &= x_0, & c_1 &= -\tfrac{1}{3}x_{-1} - \tfrac{1}{2}x_0 + x_1 - \tfrac{1}{6}x_2, \\ c_2 &= \tfrac{1}{2}x_{-1} - x_0 + \tfrac{1}{2}x_1, & c_3 &= -\tfrac{1}{6}x_{-1} + \tfrac{1}{2}x_0 - \tfrac{1}{2}x_1 + \tfrac{1}{6}x_2. \end{aligned} \tag{10.1}$$

Code 10.1: Farrow-structure cubic interpolation (`cubic_interpolate`). linkstudio/phy/sync.py, lines 36–44

```
n = x.shape[-1]
i = np.floor(t).astype(np.int64)
mu = t - i
rows = np.arange(x.shape[0])
xm1, x0, x1, x2 = (x[rows, np.clip(i + d, 0, n - 1)] for d in (-1, 0, 1, 2))
c1 = -xm1 / 3 - x0 / 2 + x1 - x2 / 6
c2 = xm1 / 2 - x0 + x1 / 2
c3 = -xm1 / 6 + x0 / 2 - x1 / 2 + x2 / 6
return ((c3 * mu + c2) * mu + c1) * mu + x0
```

10.2.2 Feedforward start: the Oerder–Meyr estimator

A linearly modulated signal is cyclostationary. The mean of $|x(t)|^2$ is periodic in T, and its first Fourier coefficient has a phase proportional to the timing offset. Over K symbols,

$$\hat{\varepsilon} = -\frac{1}{2\pi}\arg \sum_{k=0}^{KN_{\mathrm{sps}}-1} |x[k]|^2 e^{-j2\pi k/N_{\mathrm{sps}}} \qquad \text{(in symbols, modulo 1)}, \tag{10.2}$$

which needs $N_{\mathrm{sps}} \ge 3$ [8]. The block uses it to initialize the loop, which removes the long acquisition (and the risk of hang-up) of a loop started from an arbitrary phase.

Code 10.2: Oerder–Meyr initial estimate. linkstudio/phy/sync.py, lines 117–123

```
if p.initial_estimate and sps >= 3:
    # Oerder & Meyr: phase of the symbol-rate spectral line of |x|^2.
    seg = slice(int(start), int(start) + p.estimate_symbols * sps)
    k = np.arange(seg.start, min(seg.stop, L))
    line = np.sum(np.abs(data[:, k]) ** 2 * np.exp(-2j * np.pi * k / sps), axis=1)
    eps = -np.angle(line) / (2 * np.pi)  # peak position modulo one symbol
    tau = np.mod(eps - start / sps + 0.5, 1.0) - 0.5
```

10.2.3 The Gardner detector

Gardner's TED [7] uses two samples per symbol, the current strobe and the midpoint between strobes:

$$e[m] = \Re\Big\{ x^*\big((m-\tfrac{1}{2})T + \hat{\tau}\big)\,\big[x(mT+\hat{\tau}) - x((m-1)T+\hat{\tau})\big]\Big\}. \tag{10.3}$$

It is independent of the carrier phase and works before carrier recovery. Its mean response to a timing error, the *S-curve*, is zero at the correct instant and has slope K_d there (Figure 10.3). With the convention of the code, $e > 0$ means the strobes are late.

10.2.4 The second-order loop

A proportional-plus-integral loop filter makes the loop second order. It tracks a constant offset with zero error and a frequency (clock drift) with zero steady-state error. The gains follow from

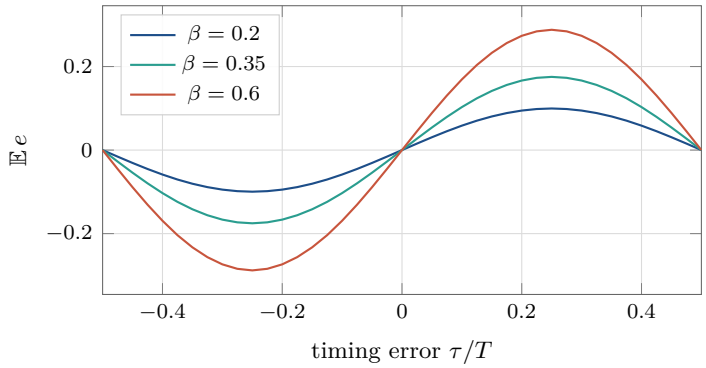


Figure 10.3: Measured S-curves of the Gardner detector for QPSK with RRC pulses (8 samples per symbol, matched-filtered output of the real blocks, interpolated with the same cubic interpolator). The slope at the origin grows with the roll-off; for $\beta = 0.35$ it is close to 1, the default `detector_gain`.

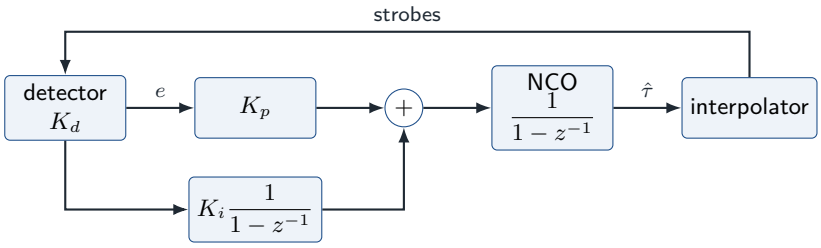


Figure 10.4: Proportional-plus-integral loop used for timing (and, with a phase detector, for the PLL).

the normalized noise bandwidth B_nT and the damping ζ [3]:

$$\theta = \frac{B_nT}{\zeta + \frac{1}{4\zeta}}, \qquad K_p = \frac{4\zeta\theta}{(1 + 2\zeta\theta + \theta^2)K_d}, \qquad K_i = \frac{4\theta^2}{(1 + 2\zeta\theta + \theta^2)K_d}. \tag{10.4}$$

Code 10.3: Second-order loop gains. `linkstudio/phy/sync.py`, lines 24–31

```
24 def loop_gains(bandwidth: float, damping: float, detector_gain: float = 1.0) -> tuple[float,
       float]:
25     """Proportional and integral gains of a second-order loop.
26
27     ``bandwidth`` is the noise bandwidth normalized to the update rate.
28     """
29     theta = bandwidth / (damping + 1 / (4 * damping))
30     denom = 1 + 2 * damping * theta + theta ** 2
31     return 4 * damping * theta / denom / detector_gain, 4 * theta ** 2 / denom /
       detector_gain
```

Code 10.4: The timing loop: interpolate, detect, filter, advance. `linkstudio/phy/sync.py`, lines 128–136

```
128 for m in range(1, n_out):
129     t = np.clip(start + (m + tau) * sps, 1, L - 3)
130     y = cubic_interpolate(data, t)
131     mid = cubic_interpolate(data, np.clip(t - sps / 2, 1, L - 3))
132     err = np.real(np.conj(mid) * (y - prev))  # > 0 when sampling late
133     integ += ki * err
134     tau -= kp * err + integ
135     out[:, m] = y
136     prev = y
```

A small B_nT averages noise better but tracks more slowly; 0.01 (the default) is a common compromise.

10.3 Frame synchronization and coarse CFO

At symbol rate, with a residual CFO of ϵ cycles per symbol, the two halves of the preamble differ only by the phase $2\pi\epsilon h$. The Schmidl–Cox metric is evaluated at every candidate start d:

$$P(d) = \sum_{k=0}^{h-1} r^*[d+k]\, r[d+k+h], \qquad M(d) = \frac{|P(d)|^2}{\left(\sum_k |r[d+k+h]|^2\right)^2}, \qquad \hat{\epsilon} = \frac{\arg P(\hat{d})}{2\pi h}. \tag{10.5}$$

The estimate is unambiguous for $|\epsilon| < 1/(2h)$. Because $M(d)$ has a plateau, its maximum is used only for the CFO estimate. After derotation, the frame start is located by cross-correlation with the known preamble, $\hat{d} = \arg\max_d \left|\sum_k r[d+k]\, p^*[k]\right|$. The CFO is then refined at the exact position, and the common phase is taken from the correlation peak. Finally the payload symbols are extracted.

Code 10.5: Schmidl–Cox CFO, correlation timing and refinement (`frame_sync`).
`linkstudio/phy/sync.py`, lines 191–212

```
191 if self.params.correct_cfo:
192     cand = np.arange(D + 1)[:, None] + np.arange(h)  # (D+1, h)
193     P = np.sum(np.conj(data[:, cand]) * data[:, cand + h], axis=-1)  # (B, D+1)
194     R = np.sum(np.abs(data[:, cand + h]) ** 2, axis=-1)
195     d0 = np.argmax(np.abs(P) ** 2 / np.maximum(R, 1e-12) ** 2, axis=1)
196     eps = np.angle(P[b_idx, d0]) / (2 * np.pi * h)  # cycles per symbol
197     data = data * np.exp(-2j * np.pi * eps[:, None] * n_idx)
198
199 win = np.arange(D + 1)[:, None] + np.arange(L)  # (D+1, L)
200 corr = np.sum(data[:, win] * np.conj(pre), axis=-1)  # (B, D+1)
201 d = np.argmax(np.abs(corr), axis=1)
202
203 if self.params.correct_cfo:  # refine with the exact preamble position
204     first = d[:, None] + np.arange(h)
205     fine = np.angle(np.sum(np.conj(data[rows, first]) * data[rows, first + h], axis=-1)) /
        (2 * np.pi * h)
206     data = data * np.exp(-2j * np.pi * fine[:, None] * n_idx)
207     eps = eps + fine
208 pwin = d[:, None] + np.arange(L)
209 phase = np.angle(np.sum(data[rows, pwin] * np.conj(pre), axis=-1))
210
211 idx = d[:, None] + L + np.arange(n_pay)
212 out = data[rows, idx] * np.exp(-1j * phase)[:, None]
```

10.4 Carrier phase tracking

The residual phase (CFO residue and phase noise) is tracked by a decision-directed PLL. For $y = s\, e^{j\Delta\phi} + w$ and a correct decision $\hat{s}$,

$$e = \frac{\Im\{y\, \hat{s}^*\}}{|\hat{s}|^2} = \sin\Delta\phi + \text{noise} \approx \Delta\phi, \tag{10.6}$$

so the detector gain is 1. The same second-order loop as above updates the phase estimate every symbol. Like every decision-directed scheme it has an M-fold phase ambiguity, which the preamble phase estimate resolves beforehand.

Code 10.6: Decision-directed PLL (`carrier_recovery`). `linkstudio/phy/sync.py`, lines 246–252

```
246 for n in range(N):
247     y = x.data[:, n] * np.exp(-1j * phi)
248     d = const.points[const.nearest(y)]
249     err = np.imag(y * np.conj(d)) / np.maximum(np.abs(d) ** 2, 1e-12)
250     integ += ki * err
251     phi += kp * err + integ
252     out[:, n] = y
```

Watch out The PLL bandwidth must exceed the phase-noise linewidth relative to the symbol rate. It must also be small enough that decision errors at low SNR do not make it slip cycles. A cycle slip rotates all subsequent decisions by $2\pi/M$ and appears as a burst of errors until the next preamble. Differential encoding or pilot symbols are the classic remedies.

Chapter 11

OFDM

Orthogonal frequency-division multiplexing turns a frequency-selective channel into many parallel flat channels, each equalized by a single complex division. This chapter derives the property, describes the subcarrier layout and the pilot-based receiver of `phy/waveform/ofdm.py`, and examines the two weaknesses of OFDM: synchronization and peak-to-average power ratio.

11.1 Cyclic prefix and circular convolution

An OFDM symbol carries N frequency-domain values X_k (the subcarrier symbols). The transmitter computes the unitary inverse DFT and prepends the last N_{cp} samples (the *cyclic prefix*):

$$x[n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X_k \, e^{j2\pi kn/N}, \qquad n = -N_{\mathrm{cp}}, \dots, N-1. \tag{11.1}$$

Proposition 11.1. If the channel impulse response has length $L \le N_{\mathrm{cp}}+1$, then after removing the prefix and applying the unitary DFT,

$$Y_k = H_k X_k + W_k, \qquad H_k = \sum_{l=0}^{L-1} h_l \, e^{-j2\pi kl/N}, \qquad W_k \sim \mathcal{CN}(0, \sigma^2) \text{ i.i.d.} \tag{11.2}$$

Proof. Thanks to the prefix, the N retained received samples equal the *circular* convolution of x with h: $\boldsymbol{y} = \mathbf{C}\boldsymbol{x} + \boldsymbol{w}$, with $\mathbf{C}$ circulant. Every circulant matrix is diagonalized by the DFT, $\mathbf{C} = \mathbf{F}^{\mathsf{H}} \operatorname{diag}(H_0, \dots, H_{N-1}) \mathbf{F}$ with $\mathbf{F}$ the unitary DFT matrix. Hence $\mathbf{F}\boldsymbol{y} = \operatorname{diag}(H)\mathbf{F}\boldsymbol{x} + \mathbf{F}\boldsymbol{w}$, and $\mathbf{F}\boldsymbol{w}$ is white with the same variance because $\mathbf{F}$ is unitary. □

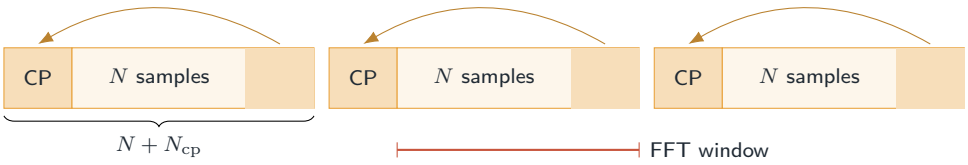


Figure 11.1: OFDM symbols with cyclic prefix. Any FFT window starting inside the prefix (after the channel transient) sees a cyclically shifted symbol, which only multiplies X_k by a linear phase.

> **In the code** `ofdm_modulator` builds an `OFDMLayout`, maps data and pilots onto the grid, applies `np.fft.ifft(..., norm="ortho")` and adds the prefix. The unitary transform keeps the noise variance per subcarrier equal to the noise variance per time sample.
> It sets `sps`$= N/N_{\mathrm{act}}$ and `power`$= (N_{\mathrm{data}}P_d + N_{\mathrm{pil}})/N$. By Equation (5.3) the AWGN block in E_s/N_0 mode then produces $\sigma^2 = 1/(E_s/N_0)$, the per-subcarrier SNR. Cyclic-prefix and pilot overheads are deliberately not counted in E_s.

Code 11.1: Resource mapping, unitary IFFT and cyclic prefix. `linkstudio/phy/waveform/ofdm.py`, lines 94–108

```
94 n_sym = -(-n // lay.n_data)
```

```
data = np.zeros((B, n_sym * lay.n_data), dtype=np.complex128)
data[:, :n] = x.data
grid = np.zeros((B, n_sym, lay.n_fft), dtype=np.complex128)
grid[..., lay.bins(lay.data_freqs)] = data.reshape(B, n_sym, lay.n_data)
grid[..., lay.bins(lay.pilot_freqs)] = lay.pilot_values
time = np.fft.ifft(grid, axis=-1, norm="ortho")
with_cp = np.concatenate([time[..., lay.n_fft - lay.cp_len:], time], axis=-1)
fs = self.params.subcarrier_spacing * lay.n_fft
data_power = (x.meta.get("power") or 1.0)
n_pilots = lay.pilot_freqs.size
meta = {**x.meta, "ofdm": lay, "n_data_symbols": n, "sps": lay.n_fft / lay.active_freqs.
    size,
        "delay": 0.0, "data_power": data_power,
        "power": (lay.n_data * data_power + n_pilots) / lay.n_fft}
frame = with_cp.reshape(B, -1)
```

11.2 Subcarrier layout

N_{act} subcarriers are used around DC. DC itself is left empty, as in Wi-Fi and LTE, and the remaining edge subcarriers act as guard bands. Every `pilot_spacing`-th active subcarrier carries a known pilot. The last active subcarrier is always a pilot, so that interpolation never extrapolates. Pilot values are a fixed pseudo-random QPSK sequence; identical pilots would add up coherently into a periodic time-domain peak.

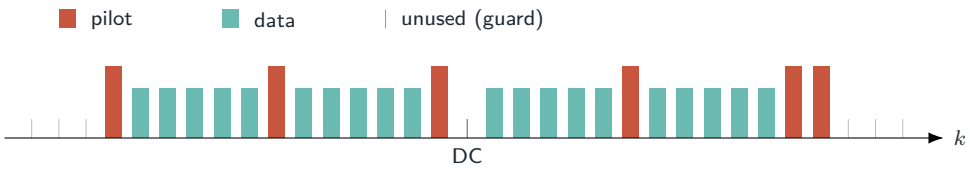


Figure 11.2: Layout with $N_{\text{act}} = 26$ active subcarriers and a pilot every 6 (the first and the last active subcarriers are pilots).

11.3 Channel estimation and equalization

11.3.1 Least squares on pilots

At pilot subcarriers Equation (11.2) gives the LS estimate $\hat{H}_p = Y_p / X_p$, with error variance σ^2 since pilots have unit modulus. The channel at data subcarriers is obtained by linear interpolation, implemented as a fixed matrix from pilot to active subcarriers. Optionally the estimates are averaged over the OFDM symbols of a frame, which lowers the estimation noise when the channel is static.

11.3.2 Removing the timing phase slope

A timing offset of δ samples inside the prefix multiplies H_k by $e^{-j2\pi k\delta/N}$. This fast phase rotation across subcarriers is poorly approximated by linear interpolation of the real and imaginary parts. The estimator therefore:

1. estimates the dominant slope from products of adjacent pilots, $\hat{s} = \arg \sum_p \hat{H}_{p+1} \hat{H}_p^* / \Delta_p$;
2. removes it;
3. interpolates the now slowly varying response;

4. restores the slope.

Code 11.2: LS estimation with phase-slope removal (`ofdm_channel_estimator`).
`linkstudio/phy/waveform/ofdm.py`, lines 235–251

```
hp = grid.data[..., lay.bins(lay.pilot_freqs)] / lay.pilot_values
if self.params.average_symbols:
    hp = np.broadcast_to(hp.mean(axis=1, keepdims=True), hp.shape)
# Remove the linear phase of the dominant delay (timing offset inside the CP)
# before interpolating, then restore it.
pf = lay.pilot_freqs
gaps = np.diff(pf)
if gaps.size:
    step = np.bincount(gaps).argmax()
    pairs = np.nonzero(gaps == step)[0]
    slope = np.angle(np.sum(hp[..., pairs + 1] * np.conj(hp[..., pairs]), axis=-1)) / step
else:
    slope = np.zeros(hp.shape[:-1])
hp = hp * np.exp(-1j * slope[..., None] * pf)
interp = self._interp_matrix(lay)
H = np.ones((B, n_sym, n_fft), dtype=np.complex128)
H[..., lay.bins(lay.active_freqs)] = (hp @ interp.T) * np.exp(1j * slope[..., None] * lay.
    active_freqs)
```

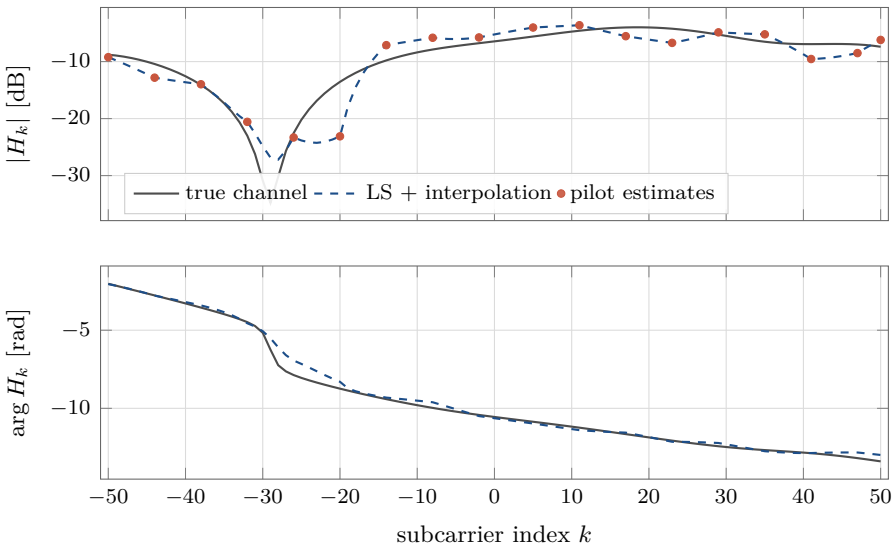


Figure 11.3: One OFDM symbol over the EVA channel ($N = 128$, 100 active subcarriers, pilot every 6, 15 kHz spacing, $E_s/N_0 = 15$ dB): true response from the channel's `csi` (`perfect` estimator) and LS estimate.

11.3.3 One-tap equalization

`ofdm_equalizer` extracts the data subcarriers and divides by the estimate, $\hat{X}_k = Y_k/\hat{H}_k$. It writes the per-subcarrier noise variance $\sigma^2/|\hat{H}_k|^2$ to `noise_var`. With coding and interleaving across subcarriers, the soft demapper then gives low weight to faded subcarriers. This is how OFDM obtains frequency diversity (see the Wi-Fi and LTE scenarios).

11.4 Blind timing and frequency synchronization

The cyclic prefix is a copy of the end of each symbol, so the received signal is correlated with itself N samples later over the prefix. Van de Beek *et al.* [10] use this redundancy. With a CFO

of ε subcarrier spacings, $r[k+N] \approx r[k] e^{j2\pi\varepsilon}$ inside the prefix, so the correlation

$$\gamma(\theta) = \sum_{m} \sum_{k=\theta+mL}^{\theta+mL+N_{\mathrm{cp}}-1} r[k]\, r^{*}[k+N], \qquad L = N + N_{\mathrm{cp}}, \tag{11.3}$$

peaks in magnitude at the symbol start, and its phase gives the fractional CFO:

$$\hat{\theta} = \arg\max_{\theta} |\gamma(\theta)|, \qquad \hat{\varepsilon} = -\frac{1}{2\pi} \arg \gamma(\hat{\theta}), \quad |\varepsilon| < \tfrac{1}{2}. \tag{11.4}$$

The block accumulates γ over all OFDM symbols of the frame with cumulative sums, which makes the search over θ linear in the frame length. It then starts the FFT window `backoff` samples early, inside the prefix, to stay clear of the channel's delay spread. The resulting linear phase is absorbed by the channel estimator. Compared with the full ML metric of van de Beek, the energy correction term is omitted; it matters only at low SNR.

Code 11.3: CP correlation, timing and fractional CFO (`ofdm_sync`).
linkstudio/phy/waveform/ofdm.py, lines 148–159

```
148 prod = r[:, :n - N] * np.conj(r[:, N:])  # r[k] r*[k+N]
149 csum = np.concatenate([np.zeros((B, 1)), np.cumsum(prod, axis=1)], axis=1)
150 n_sym = (n - N - cp - D) // L + 1
151 theta = np.arange(D + 1)
152 gamma = np.zeros((B, D + 1), dtype=np.complex128)
153 for m in range(max(n_sym, 1)):
154     start = theta + m * L
155     gamma += csum[:, start + cp] - csum[:, start]
156 d = np.argmax(np.abs(gamma), axis=1)
157 eps = -np.angle(gamma[np.arange(B), d]) / (2 * np.pi) if self.params.correct_cfo else np.
        zeros(B)
158 backoff = self.params.backoff if self.params.backoff is not None else cp // 4
159 d = np.maximum(d - backoff, 0)
```

Watch out An integer CFO (a multiple of the subcarrier spacing) shifts the whole grid and is invisible to the CP correlation. It must be resolved with a preamble or pilots, which this block does not do.

11.5 Peak-to-average power ratio

Each time sample is a sum of N_{act} independent terms. By the central limit theorem it is nearly $\mathcal{CN}(0, P)$, so $|x|^2/P$ is approximately exponential. Treating the N Nyquist-rate samples of a symbol as independent gives the classic approximation

$$P\big(\mathrm{PAPR} > \gamma\big) \approx 1 - \big(1 - e^{-\gamma}\big)^{N_{\mathrm{act}}}. \tag{11.5}$$

Remark. The modulator produces Nyquist-rate samples, which miss part of the analog peaks; with oversampling the PAPR is typically about 0.5 dB higher.

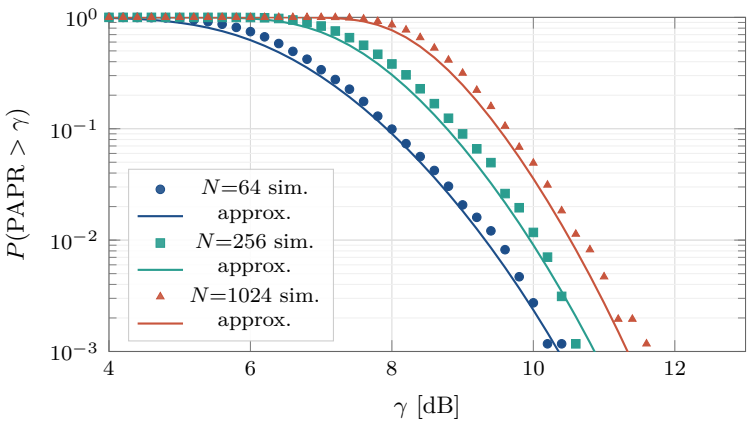


Figure 11.4: CCDF of the per-symbol PAPR of QPSK-OFDM at the output of `ofdm_modulator` (no oversampling, no prefix) and the approximation above. A PAPR of 10–12 dB forces the power-amplifier back-off discussed in Chapter 8.

Chapter 12

Multiple antennas

12.1 The flat-fading MIMO channel

With N_t transmit and N_r receive antennas and flat fading, one channel use is

$$\boldsymbol{y} = \mathbf{H}\boldsymbol{x} + \boldsymbol{w}, \qquad \mathbf{H} = \frac{1}{\sqrt{N_t}}\mathbf{G}, \quad G_{ij} \sim \mathcal{CN}(0,1), \quad \boldsymbol{w} \sim \mathcal{CN}(\mathbf{0}, \sigma^2\mathbf{I}). \tag{12.1}$$

The normalization by $\sqrt{N_t}$ keeps the total transmitted power independent of the number of antennas. Every receive antenna then sees unit average signal power, and the SNR set in the `awgn` block is the SNR *per receive antenna*. Antenna correlation follows the Kronecker model with exponential correlation matrices,

$$\mathbf{H} = \frac{1}{\sqrt{N_t}}\,\mathbf{R}_r^{1/2}\mathbf{G}\,\mathbf{R}_t^{1/2}, \qquad [\mathbf{R}]_{ij} = \rho^{|i-j|}. \tag{12.2}$$

In the code Multi-stream signals have shape (B, N_t, n).

- `stream_demux` distributes symbols round-robin over the streams and sets `spatial_rate`$= N_t$, so that E_b/N_0 accounts for the N_t symbols sent per channel use (Equation (5.4)).
- `mimo_channel` draws $\mathbf{H}$ per frame (`block`) or per symbol (`fast`) and publishes it on `csi` (`csi_type="mimo"`).
- `mimo_detector` and `alamouti_decoder` use it, and `stream_mux` restores a single stream.

12.2 Detection for spatial multiplexing

12.2.1 Zero forcing

$\mathbf{W}_{\mathrm{ZF}} = (\mathbf{H}^{\mathsf{H}}\mathbf{H})^{-1}\mathbf{H}^{\mathsf{H}}$ separates the streams exactly, $\hat{\boldsymbol{x}} = \boldsymbol{x} + \mathbf{W}\boldsymbol{w}$. Stream k then has noise variance $\sigma^2[(\mathbf{H}^{\mathsf{H}}\mathbf{H})^{-1}]_{kk}$, which becomes large when $\mathbf{H}$ is ill-conditioned. For $N_t = N_r$ each stream has only one degree of diversity.

12.2.2 MMSE and its bias

The linear MMSE estimator is $\mathbf{W} = (\mathbf{H}^{\mathsf{H}}\mathbf{H} + \sigma^2\mathbf{I})^{-1}\mathbf{H}^{\mathsf{H}}$ (unit-power symbols). Its output is biased: $\mathbb{E}[\hat{x}_k | x_k] = b_k x_k$ with $b_k = (\mathbf{WH})_{kk} < 1$.

Proposition 12.1. For the MMSE filter, the residual interference plus noise of stream k has variance $b_k(1-b_k)$. After unbiasing ($\hat{x}_k/b_k$), the noise variance is therefore $(1-b_k)/b_k$, and the post-detection SINR is $b_k/(1-b_k)$.

Proof. For the MMSE filter, the error covariance is $\mathbf{E} = \mathbb{E}[(\hat{\boldsymbol{x}} - \boldsymbol{x})(\hat{\boldsymbol{x}} - \boldsymbol{x})^{\mathsf{H}}] = \mathbf{I} - \mathbf{WH}$, so the MSE of stream k is $1 - b_k$. Write $\hat{x}_k = b_k x_k + \nu_k$ with ν_k uncorrelated with x_k (an orthogonality property of the MMSE estimator). Then $\mathrm{MSE}_k = (1-b_k)^2 + \operatorname{Var}\nu_k = 1 - b_k$, which gives $\operatorname{Var}\nu_k = b_k(1-b_k)$. □

The block outputs the unbiased symbols and writes $(1-b_k)/b_k$ to `noise_var`, so the soft demapper receives correctly scaled LLRs.

Code 12.1: ZF and unbiased MMSE detection with post-detection noise variances.
`linkstudio/phy/mimo.py`, lines 152–161

```
gram = Hh @ H
reg = n0 if self.params.method == "mmse" else 0.0
inv = np.linalg.inv(gram + reg * np.eye(nt))
s = (inv @ Hh @ yv)[..., 0]  # (B, n, Nt)
if self.params.method == "zf":
    nv = n0 * np.real(np.diagonal(inv, axis1=-2, axis2=-1))
else:
    bias = np.real(np.diagonal(inv @ gram, axis1=-2, axis2=-1))  # (W H)_kk
    s = s / bias
    nv = (1 - bias) / np.maximum(bias, 1e-12)
```

12.2.3 Maximum likelihood

$\hat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x}\in\mathcal{S}^{N_t}} \|\boldsymbol{y}-\mathbf{H}\boldsymbol{x}\|^2$ is optimal and achieves diversity N_r per stream. The block searches all M^{N_t} candidates exhaustively (limited to 4096) and outputs hard decisions.

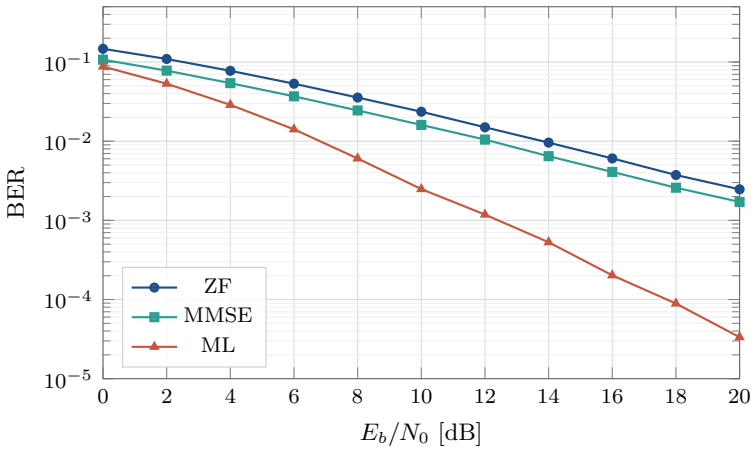


Figure 12.1: 2×2 spatial multiplexing of QPSK over fast Rayleigh fading (simulated). ZF and MMSE have diversity 1, the same slope as single-antenna Rayleigh fading, and MMSE gains about 2 dB. ML reaches diversity 2.

12.3 Diversity

12.3.1 Receive diversity: maximal-ratio combining

With one transmit antenna, ZF and MMSE reduce to maximal-ratio combining, $\hat{x} = \boldsymbol{h}^{\mathsf{H}}\boldsymbol{y}/\|\boldsymbol{h}\|^2$, whose SNR is the sum of the branch SNRs. For BPSK with L i.i.d. Rayleigh branches of mean SNR $\bar{\gamma}$ each [1],

$$\bar{P}_b = p^L \sum_{l=0}^{L-1} \binom{L-1+l}{l} (1-p)^l, \qquad p = \frac{1-\mu}{2}, \quad \mu = \sqrt{\frac{\bar{\gamma}}{1+\bar{\gamma}}}, \tag{12.3}$$

and asymptotically $\bar{P}_b \propto \bar{\gamma}^{-L}$: the slope of the curve is the diversity order.

12.3.2 Transmit diversity: the Alamouti code

Two symbols are sent over two antennas in two symbol periods [11]:

	antenna 1	antenna 2
t_1	s_1	s_2
t_2	$-s_2^*$	s_1^*

$$\Longrightarrow \qquad \begin{aligned} \tilde{s}_1 &= h_1^* r_1 + h_2 r_2^* = (|h_1|^2 + |h_2|^2)\, s_1 + \tilde{w}_1, \\ \tilde{s}_2 &= h_2^* r_1 - h_1 r_2^* = (|h_1|^2 + |h_2|^2)\, s_2 + \tilde{w}_2. \end{aligned} \tag{12.4}$$

The code is orthogonal, so the linear combiner decouples the symbols and gives full diversity 2 without channel knowledge at the transmitter. The decoder sums over any number of receive antennas. Because the transmit power is split between the antennas, the SNR is 3 dB below two-branch MRC: $\bar{P}_b^{\text{Alamouti}}(\bar{\gamma}) = \bar{P}_b^{\text{MRC},L=2}(\bar{\gamma}/2)$.

Code 12.2: Alamouti combining over N_r receive antennas. `linkstudio/phy/mimo.py`, lines 196–200

```
196 r1, r2 = y[:, :, 0::2], y[:, :, 1::2]  # (B, Nr, n/2)
197 h1, h2 = H[:, :, 0:1], H[:, :, 1:2]  # (B, Nr, 1)
198 norm = np.sum(np.abs(h1) ** 2 + np.abs(h2) ** 2, axis=1)  # (B, 1)
199 s1 = np.sum(np.conj(h1) * r1 + h2 * np.conj(r2), axis=1) / norm
200 s2 = np.sum(np.conj(h2) * r1 - h1 * np.conj(r2), axis=1) / norm
```

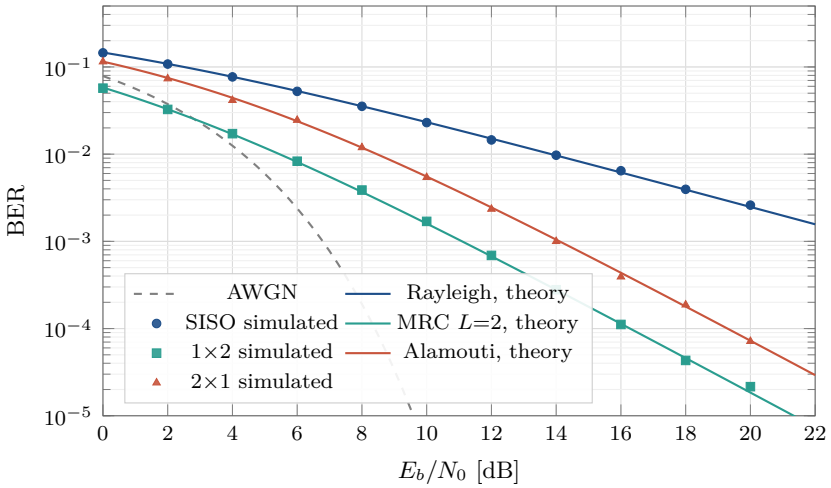


Figure 12.2: BPSK over Rayleigh fading: single antenna (Equation (9.6)), 1×2 MRC and 2×1 Alamouti (Equation (12.3)). The E_b/N_0 axis is per receive antenna. Diversity 2 doubles the slope; Alamouti runs parallel to MRC, 3 dB to the right.

Chapter 13

Channel coding

Channel coding adds structured redundancy so that the receiver can detect and correct errors. Figure 13.1 shows the coding blocks of `phy/coding/`. They can be combined freely. Every encoder multiplies the `code_rate` metadata by its own rate, including tail bits and CRC overhead, so the `awgn` block always refers E_b/N_0 to the *information* bits (Equation (5.4)).

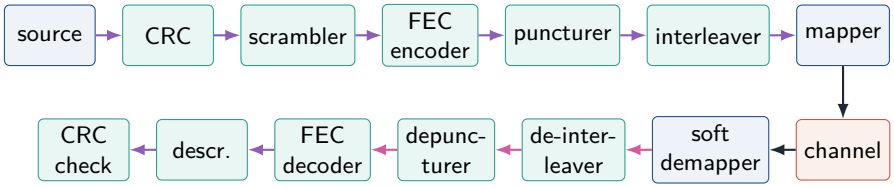


Figure 13.1: A complete coded chain. Purple edges carry bits, pink edges LLRs. The scrambler and the interleaver are their own inverses (`inverse=true` for the interleaver).

13.1 Cyclic redundancy check

A CRC with generator polynomial $g(x)$ of degree r appends the remainder $p(x) = x^r u(x) \bmod g(x)$ to the message $u(x)$; the receiver recomputes it and compares. Because polynomial division over GF(2) is linear, the parity is a linear function of the message, $\boldsymbol{p} = \boldsymbol{u}\mathbf{P} \bmod 2$. Row i of $\mathbf{P}$ is the remainder of the monomial $x^{k-1-i+r}$. Computing all of them once turns CRC encoding and checking of a whole batch into a single integer matrix product.

Code 13.1: The CRC parity matrix, built once per frame length and cached.
`linkstudio/phy/coding/crc.py`, lines 30–39

```
30 r = poly.bit_length() - 1
31 rems = np.empty(k, dtype=np.int64)
32 rem = poly ^ (1 << r)  # x^r mod g
33 for j in range(k):
34     rems[k - 1 - j] = rem
35     rem <<= 1
36     if rem >> r & 1:
37         rem ^= poly
38 shifts = np.arange(r - 1, -1, -1)
39 return ((rems[:, None] >> shifts) & 1).astype(np.int32)
```

A CRC of degree r detects all error bursts of length $\leq r$. It misses a random error pattern with probability $\approx 2^{-r}$. `crc_decoder` reports `bler`, the block error rate detected by the CRC, and outputs a validity flag per frame. CRC-8, CRC-16-CCITT, LTE CRC-24A and IEEE 802.3 CRC-32 are available.

13.2 LFSR sequences and scrambling

A linear feedback shift register produces $s[k] = \bigoplus_{t \in \mathcal{T}} s[k-t]$. When the feedback polynomial $1 + \sum_{t \in \mathcal{T}} x^t$ is primitive of degree m, the sequence is an m-sequence of period $2^m - 1$ with nearly ideal autocorrelation. The `prbs` source (PRBS-7 to PRBS-31, ITU-T O.150) and the additive `scrambler` (default $x^7 + x^4 + 1$, IEEE 802.11) are built on it. Since every output bit

depends only on bits at least $\min \mathcal{T}$ positions back, chunks of that many bits can be computed with vector operations:

Code 13.2: Vectorized LFSR generation (`dsp/lfsr.py`). linkstudio/phy/dsp/lfsr.py, lines 27–40

```
deg, step = max(taps), min(taps)
s = np.empty(deg + n, dtype=np.uint8)
s[:deg] = 1 if state is None else state
pos = deg
# Every output depends on bits at least `step` positions back, so chunks of
# `step` bits can be produced with vector operations.
while pos < deg + n:
    m = min(step, deg + n - pos)
    acc = np.zeros(m, dtype=np.uint8)
    for t in taps:
        acc ^= s[pos - t:pos - t + m]
    s[pos:pos + m] = acc
    pos += m
return s[deg:].copy(), s[n:].copy()
```

Scrambling is a XOR with the sequence and is its own inverse. On LLRs, descrambling flips the sign of the affected values. The self-synchronizing $x^{58} + x^{39} + 1$ scrambler of 64B/66B is described in Section 14.1.

13.3 Interleaving

Fading and decoders both produce errors in bursts, while most codes are designed for independent errors. An interleaver permutes the coded bits, and the de-interleaver applies the inverse permutation, so a burst on the channel is spread over many codewords. The `block` interleaver writes row by row into an $R \times (n/R)$ matrix and reads column by column. The `random` interleaver applies a seeded pseudo-random permutation.

13.4 Block codes: repetition and Hamming

The repetition code sends every bit n times. Its soft decoder simply adds the n LLRs. The Hamming code $(2^m - 1, 2^m - 1 - m)$ has as parity-check matrix $\mathbf{H}$ all non-zero m-bit columns, arranged so that the code is systematic. A single error in position j produces the syndrome $\boldsymbol{s} = \mathbf{H}\boldsymbol{e}^{\mathsf{T}} = \boldsymbol{h}_j$, which identifies the position; a lookup table maps each syndrome to the bit to flip. Decoding errors occur with ≥ 2 channel errors; the approximation

$$P_b \approx \frac{1}{n} \sum_{j=2}^{n} (j+1) \binom{n}{j} p^j (1-p)^{n-j}, \qquad p = \mathrm{Q}\left(\sqrt{2R\,E_b/N_0}\right), \tag{13.1}$$

counts the j channel errors plus at most one wrong correction (`ber_hamming_hard`).

13.5 Convolutional codes and the Viterbi algorithm

13.5.1 Encoder and trellis

A rate-$1/n$ feed-forward convolutional encoder with constraint length K computes n output bits per input bit,

$$c_j[t] = \bigoplus_{i=0}^{K-1} g_{j,i}\, u[t-i], \qquad j = 1, \ldots, n, \tag{13.2}$$

with generators given in octal: $(171, 133)_8$ is the industry-standard $K = 7$ code of IEEE 802.11, DVB and many others, with free distance $d_{\text{free}} = 10$. The encoder state is the last $K - 1$ input bits, giving 2^{K-1} states. Appending $K - 1$ zeros (`terminate`) returns the trellis to state 0.

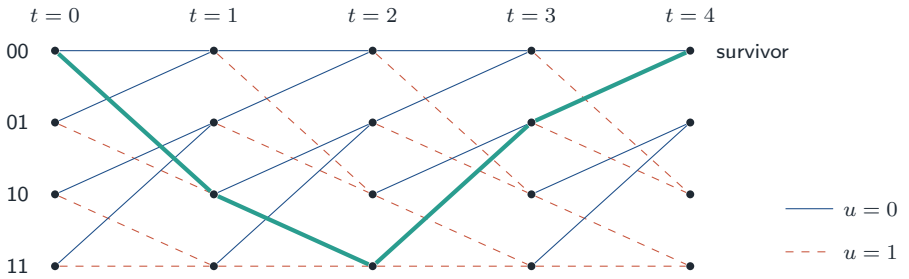


Figure 13.2: Trellis of the $K = 3$, $(7, 5)_8$ code (4 states, MSB = most recent input). The Viterbi algorithm keeps one survivor path per state and time; the green path is the decoded sequence `1 1 0 0`.

13.5.2 Maximum-likelihood sequence decoding

Given channel LLRs $L_{t,j}$ for the coded bits, the log-likelihood of a candidate codeword is, up to a constant,

$$\ln p(\boldsymbol{y} \mid \boldsymbol{c}) = \sum_{t,j} \frac{L_{t,j}}{2}(1 - 2c_{t,j}) + \text{const}, \tag{13.3}$$

a *correlation* metric. The Viterbi algorithm maximizes it over all paths of the trellis by dynamic programming. At each time, for every state it keeps the better of the two incoming paths (add–compare–select) and stores the decision. A traceback from state 0 (terminated trellis) recovers the input bits. Path metrics are renormalized every step to avoid overflow.

Code 13.3: Viterbi decoding vectorized over states and frames (`Trellis.decode`).
linkstudio/phy/coding/convolutional.py, lines 67–86

```
S = self.n_states
pm = np.full((B, S), -np.inf)
pm[:, 0] = 0.0
decisions = np.empty((T, B, S), dtype=bool)
for t in range(T):
    bm0 = 0.5 * llr[:, t, :] @ self.sign0.T  # (B, S) correlation metrics
    bm1 = 0.5 * llr[:, t, :] @ self.sign1.T
    c0 = pm[:, self.prev0] + bm0
    c1 = pm[:, self.prev1] + bm1
    d = c1 > c0
    decisions[t] = d
    pm = np.where(d, c1, c0)
    pm -= pm.max(axis=1, keepdims=True)
state = np.zeros(B, dtype=np.int64) if terminated else np.argmax(pm, axis=1)
bits = np.empty((B, T), dtype=np.uint8)
rows = np.arange(B)
for t in range(T - 1, -1, -1):
    bits[:, t] = self.input_bit[state]
    state = np.where(decisions[t, rows, state], self.prev1[state], self.prev0[state])
return bits[:, :T - (self.K - 1)] if terminated else bits
```

With hard decisions (bit inputs) the metric degenerates to the Hamming distance, losing about 2 dB (`tests/test_physics.py::test_conv_coded_soft_vs_hard`). The asymptotic coding gain of soft decoding is $10 \log_{10}(R\, d_{\text{free}}) = 7$ dB for the $K = 7$ code.

13.5.3 Puncturing

Deleting coded bits periodically raises the rate without a new decoder. The default pattern `[1,1,1,0,0,1]` turns the rate-1/2 code into rate 3/4, as in IEEE 802.11. The depuncturer reinserts the deleted positions as erasures ($L = 0$, no information) before decoding. When the original length is not available in the metadata, it is inferred as the shortest mother length whose pattern keeps exactly the received number of bits.

Code 13.4: Erasure insertion (`depuncturer`). linkstudio/phy/coding/puncture.py, lines 55–64

```
m = llr.shape[-1]
n = x.meta.get("punctured_length")
if n is None:  # shortest mother length whose pattern keeps exactly m positions
    period = len(self.params.pattern)
    big = _keep_mask(self.params.pattern, (m // sum(self.params.pattern) + 1) * period)
    n = int(np.nonzero(np.cumsum(big) == m)[0][0]) + 1
keep = _keep_mask(self.params.pattern, n)
out = np.zeros((*llr.shape[:-1], n))
out[..., keep] = llr
return {"out": Signal(out, Kind.LLR, meta=x.meta)}
```

13.6 LDPC codes

13.6.1 Structure

A low-density parity-check code [13] is the null space of a sparse matrix $\mathbf{H} \in \{0,1\}^{m \times n}$: $\mathbf{H}\boldsymbol{c}^{\mathsf{T}} = \mathbf{0}$. Its Tanner graph has a variable node per code bit and a check node per row (Figure 13.3). LinkStudio builds quasi-cyclic codes in the style of IEEE 802.11n and 5G NR: $\mathbf{H}$ is an array of $z \times z$ circulant permutation matrices (or zero blocks), described by a base matrix of shifts (Figure 13.4).

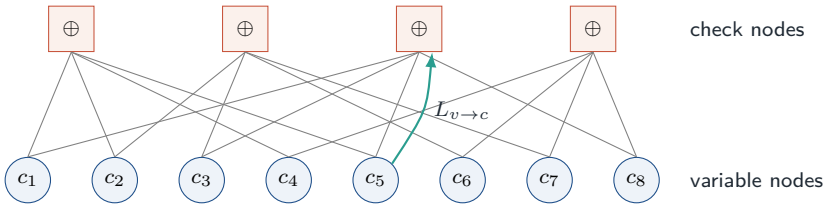


Figure 13.3: A Tanner graph. Belief propagation exchanges messages along the edges: variables tell checks what they believe, checks reply with what the other bits imply.

13.6.2 Encoding by accumulation

Split $\mathbf{H} = [\mathbf{H}_i \,|\, \mathbf{H}_p]$ and the codeword $\boldsymbol{c} = [\boldsymbol{u} \,|\, \boldsymbol{p}]$. The parity part is block dual-diagonal: identities on the diagonal and on the sub-diagonal. The parity equations $\mathbf{H}_i\boldsymbol{u}^{\mathsf{T}} \oplus \mathbf{H}_p\boldsymbol{p}^{\mathsf{T}} = \mathbf{0}$ then read, block by block,

$$\boldsymbol{p}_0 = \boldsymbol{s}_0, \qquad \boldsymbol{p}_j = \boldsymbol{p}_{j-1} \oplus \boldsymbol{s}_j, \qquad \boldsymbol{s} = \mathbf{H}_i\boldsymbol{u}^{\mathsf{T}}, \tag{13.4}$$

so the parity is a cumulative XOR of the syndrome blocks. Encoding costs one sparse product and needs no Gaussian elimination. The dual-diagonal structure also guarantees that $\mathbf{H}$ has full rank.

Code 13.5: Systematic QC-LDPC encoding. linkstudio/phy/coding/ldpc.py, lines 71–74

```
s = (self.H_info @ u.T.astype(np.int32)) % 2  # (m, B)
```

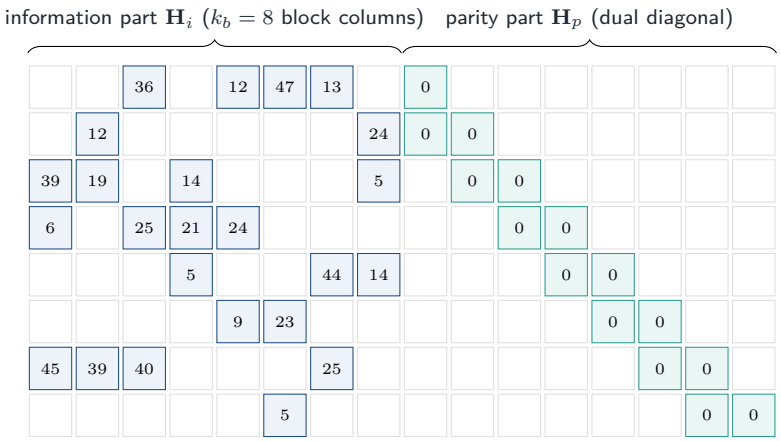


Figure 13.4: Base matrix of the default code ($k_b = m_b = 8$, $z = 48$, rate $1/2$, $n = 768$), read back from `QCLDPCCode.H`. Each number is the cyclic shift of a 48×48 identity; empty cells are zero blocks. Every information column has weight 3.

```
blocks = s.reshape(self.m // self.z, self.z, -1).astype(np.uint8)
parity = np.bitwise_xor.accumulate(blocks, axis=0).reshape(self.m, -1)
return np.concatenate([u, parity.T.astype(np.uint8)], axis=1)
```

13.6.3 Belief-propagation decoding

With channel LLRs L_v, the flooding schedule alternates two updates.

$$\text{variable to check:}\quad L_{v\to c} = L_v + \sum_{c' \in \mathcal{N}(v)\setminus c} L_{c'\to v}, \tag{13.5}$$

$$\text{check to variable:}\quad L_{c\to v} = 2\tanh^{-1} \prod_{v' \in \mathcal{N}(c)\setminus v} \tanh\frac{L_{v'\to c}}{2} \quad \text{(sum-product)}, \tag{13.6}$$

$$L_{c\to v} \approx \alpha \prod_{v'} \operatorname{sgn}(L_{v'\to c}) \cdot \min_{v' \in \mathcal{N}(c)\setminus v} |L_{v'\to c}| \quad \text{(normalized min-sum)}. \tag{13.7}$$

The min-sum rule only needs, for each check, the two smallest input magnitudes: the output to the edge holding the minimum is the second minimum, and to all others the minimum. The factor $\alpha \approx 0.8$ compensates its overestimation of reliabilities. After every iteration the hard decisions are tested against the syndrome. Frames that satisfy it stop early, and the average number of iterations is reported as a metric.

Code 13.6: The normalized min-sum check-node update, vectorized over all checks of all frames.
linkstudio/phy/coding/ldpc.py, lines 92–103

```
if method == "min_sum":
    sign = np.where(V < 0, -1.0, 1.0)
    sign[:, ~valid] = 1.0
    mag = np.abs(V)
    mag[:, ~valid] = np.inf
    i1 = np.argmin(mag, axis=-1)[..., None]
    min1 = np.take_along_axis(mag, i1, -1)
    np.put_along_axis(mag, i1, np.inf, -1)
    min2 = mag.min(axis=-1, keepdims=True)
    pos = np.arange(V.shape[-1])
    out_mag = np.where(pos == i1, min2, min1) * alpha
    out = np.prod(sign, axis=-1, keepdims=True) * sign * out_mag
```

The message passing is vectorized with a padded table of the edges of every check node (irregular row weights are padded with neutral entries) and sparse incidence matrices for the variable-node sums.

13.7 Polar codes

13.7.1 Channel polarization

Arıkan's transform [12] $\boldsymbol{x} = \boldsymbol{u}\mathbf{G}_N$, with $\mathbf{G}_N = \left[\begin{smallmatrix}1 & 0\\ 1 & 1\end{smallmatrix}\right]^{\otimes n}$, $N = 2^n$, combines N uses of a channel W into N synthetic channels $W_N^{(i)}$ seen by the bits u_i under successive decoding. As N grows they polarize: a fraction tends to noiseless channels, and the rest to useless ones. One step splits a channel with Bhattacharyya parameter Z into a worse and a better one,

$$Z(W^-) \le 2Z - Z^2, \qquad Z(W^+) = Z^2. \tag{13.8}$$

Starting from $Z = e^{-E_s/N_0}$ (BPSK over AWGN at the `design_snr_db`) and applying the recursion n times ranks the synthetic channels. The K best carry information and the others are *frozen* to 0.

Code 13.7: Reliability ranking by the Bhattacharyya recursion. `linkstudio/phy/coding/polar.py`, lines 35–39

```
def rec(z: float, size: int) -> list[float]:
    if size == 1:
        return [z]
    return rec(2 * z - z * z, size // 2) + rec(z * z, size // 2)
return np.array(rec(float(np.exp(-10 ** (design_snr_db / 10))), 1 << n_log2))
```

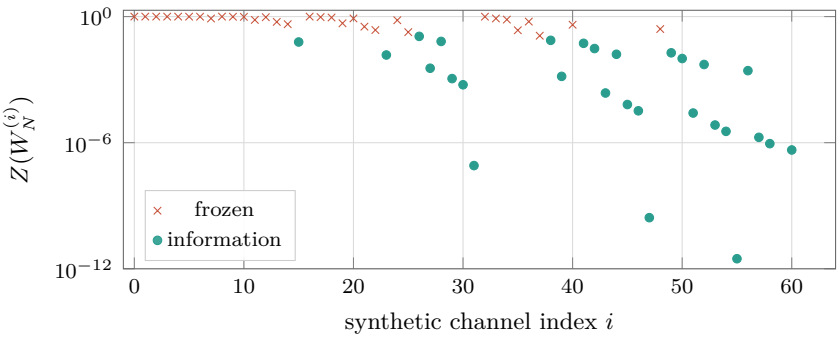


Figure 13.5: Polarization for $N = 64$, $K = 32$ at a design SNR of 0 dB (`reliability`, `frozen_mask`). Reliable channels concentrate at high indices, the frozen set at low indices.

13.7.2 Encoding and successive-cancellation decoding

Without bit reversal, $\boldsymbol{u} = [\boldsymbol{u}_a, \boldsymbol{u}_b]$ encodes recursively as $\boldsymbol{x} = [\mathrm{enc}(\boldsymbol{u}_a) \oplus \mathrm{enc}(\boldsymbol{u}_b),\ \mathrm{enc}(\boldsymbol{u}_b)]$, computed in place with $\log_2 N$ butterfly stages:

Code 13.8: The polar transform in $\log_2 N$ vectorized stages. `linkstudio/phy/coding/polar.py`, lines 44–52

```
x = u.copy()
n = x.shape[-1]
half = 1
while half < n:
    x = x.reshape(*x.shape[:-1], -1, 2, half)
```

```
    x[..., 0, :] ^= x[..., 1, :]
    x = x.reshape(*x.shape[:-3], n)
    half *= 2
return x
```

The SC decoder follows the same recursion on LLRs. For the halves $\boldsymbol{L}_1, \boldsymbol{L}_2$,

$$f(a,b) = \operatorname{sgn} a \operatorname{sgn} b \min(|a|,|b|) \ \text{(left child)}, \qquad g(a,b,\hat{x}) = b + (1-2\hat{x})\, a \ \text{(right child)}. \tag{13.9}$$

The left half is decoded first; its re-encoded decisions $\hat{\boldsymbol{x}}_a$ enter the right half. Subtrees made only of frozen bits return zeros immediately, and subtrees made only of information bits (rate-1 nodes) are decided by a direct hard decision. Both shortcuts are exact.

Code 13.9: Recursive SC decoding with rate-0 and rate-1 node shortcuts.
linkstudio/phy/coding/polar.py, lines 59–83

```
def sc_decode(llr: np.ndarray, frozen: np.ndarray) -> np.ndarray:
    """Successive-cancellation decoding. Returns the estimated u ``(B, N)``."""
    u_hat = np.zeros(llr.shape, dtype=np.uint8)

    def rec(L: np.ndarray, lo: int) -> np.ndarray:
        n = L.shape[-1]
        mask = frozen[lo:lo + n]
        if mask.all():
            return np.zeros(L.shape, dtype=np.uint8)
        if not mask.any():  # rate-1 node: direct hard decision
            x = (L < 0).astype(np.uint8)
            u_hat[:, lo:lo + n] = polar_encode(x)
            return x
        if n == 1:
            x = (L < 0).astype(np.uint8)
            u_hat[:, lo:lo + 1] = x
            return x
        h = n // 2
        L1, L2 = L[:, :h], L[:, h:]
        xa = rec(_f(L1, L2), lo)
        xb = rec(L2 + (1 - 2.0 * xa) * L1, lo + h)
        return np.concatenate([xa ^ xb, xb], axis=1)

    rec(llr, 0)
    return u_hat
```

13.8 Comparison

Key idea Soft decisions are worth about 2 dB, and iterative decoding of long sparse codes approaches capacity. The rate-1/2 limit for binary-input AWGN is 0.19 dB. The 768-bit LDPC code is about 3 dB away at 10^{-5}; longer codes get closer.

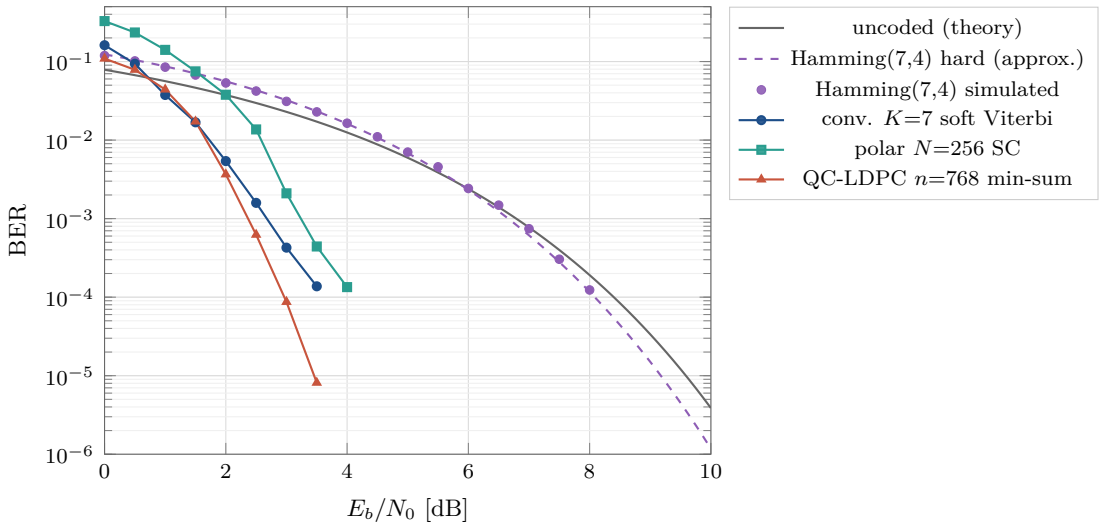


Figure 13.6: QPSK over AWGN with the codes of this chapter (rate $\approx 1/2$ except Hamming 4/7; max-log LLRs; simulated with the real blocks, up to 200 errors or 20 batches per point). At 10^{-4} the Hamming code gains well under 1 dB with hard decisions, while the convolutional and LDPC codes gain about 5 and 5.5 dB.

Chapter 14

Wireline communications

Copper links (Ethernet over twisted pair, SerDes over backplanes, DSL over telephone loops) are real-valued baseband systems. Their channel is a lossy, dispersive transmission line, and their noise is dominated by crosstalk, echo and jitter rather than thermal noise. `phy/wireline.py` models these effects; all its blocks keep real signals real.

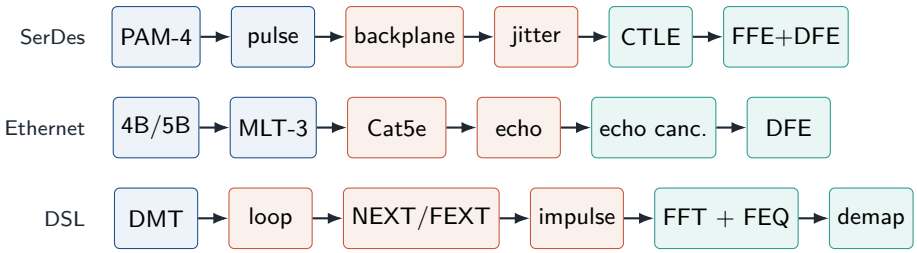


Figure 14.1: Simplified chains of the wireline scenarios (10GBASE-KR / PAM-4 SerDes, 100BASE-TX, ADSL/VDSL).

14.1 Line codes

A line code shapes the spectrum of the transmitted symbols: it removes DC (transformer coupling), guarantees transitions (clock recovery) or concentrates energy at low frequency (cable loss, emissions). `line_encoder` implements four codes:

code	rule	levels	PSD with rectangular pulses (T = bit time)
NRZ	$1 \to +1$, $0 \to -1$	2	$T\,\mathrm{sinc}^2(fT)$
Manchester	$0 \to (+1,-1)$, $1 \to (-1,+1)$	2	$T\,\mathrm{sinc}^2(fT/2)\sin^2(\pi fT/2)$
AMI	$0 \to 0$, $1 \to \pm 1$ alternately	3	$T\,\mathrm{sinc}^2(fT)\sin^2(\pi fT)$
MLT-3	each 1 steps through $0,+1,0,-1$	3	concentrated below $1/(4T)$

Manchester and AMI have a spectral null at DC. MLT-3, used by 100BASE-TX, completes a full cycle every four ones, so its fundamental is at most a quarter of the bit rate.

Block line codes. `line_block_encoder` implements two block codes.

- **4B/5B** maps each nibble to a 5-bit word with at most one leading and two trailing zeros. Long zero runs are thus excluded, at rate 0.8.
- **64B/66B** prepends the sync header `01` to each 64-bit block and scrambles the payload with the self-synchronizing polynomial $x^{58} + x^{39} + 1$, $s[n] = d[n] \oplus s[n-39] \oplus s[n-58]$. The descrambler needs no synchronization, but each channel error produces three errors after descrambling (error multiplication, tested in `tests/test_wireline_optical.py`).

14.2 The cable as a transmission line

A uniform line of length ℓ, terminated in its characteristic impedance, has transfer function $H(f) = e^{-\gamma(f)\ell}$ with propagation constant $\gamma = \sqrt{(R + j\omega L)(G + j\omega C)} = \alpha + j\beta$. Two loss mechanisms dominate:

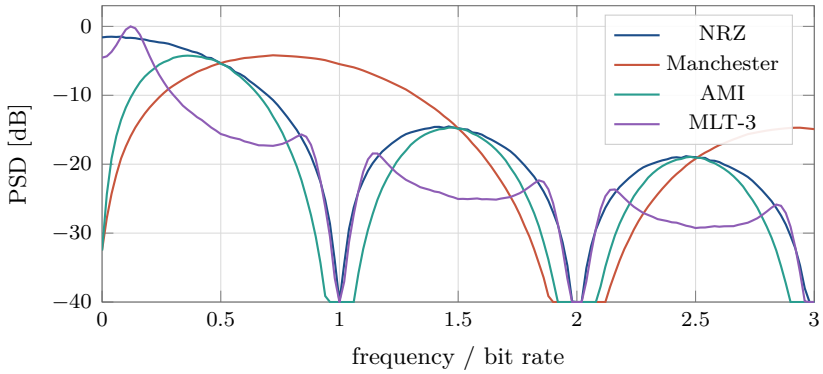


Figure 14.2: Welch spectra of random data through `line_encoder` and a rectangular `pulse_shaper`, normalized to the NRZ peak and clipped at −40 dB. Note the DC nulls of Manchester and AMI, and Manchester's doubled bandwidth.

- the *skin effect*: current crowds near the conductor surface, so $R \propto \sqrt{f}$ and the attenuation $\alpha_s \propto \sqrt{f}$;
- *dielectric loss*: $G \propto f$ gives $\alpha_d \propto f$.

For the fitted presets, in dB per km,

$$\alpha_{\mathrm{dB}}(f) = a\sqrt{f_{\mathrm{MHz}}} + b\, f_{\mathrm{MHz}} \left(+ \; c/\sqrt{f_{\mathrm{MHz}}} \right). \tag{14.1}$$

The last term belongs to the TIA-568 category formulas (Cat 5e: $c = 0.050$ per 100 m, Cat 6A: 0.25) and is held constant below 1 MHz, where the formula is not specified. It is applied as an attenuation only; it is a fraction of a dB in the Ethernet band.

Causality fixes the phase A causal, minimum-phase channel cannot have an arbitrary attenuation: the log-magnitude and the phase form a Hilbert-transform pair. For the skin effect the pair is exact and elementary. The function $e^{-k\sqrt{j\omega}}$ is analytic in the right half s-plane (it is the Laplace-domain response of a diffusion line), and

$$e^{-k\sqrt{j\omega}} = e^{-k\sqrt{\omega}\,(1+j)/\sqrt{2}}\,:$$

the $\sqrt{f}$ loss comes with an *equal* $\sqrt{f}$ phase lag. LinkStudio therefore uses

$$H(f) = \exp\Big(-\alpha_s(f)\,\ell\,(1 + j\ \mathrm{sgn}\, f) - \alpha_d(f)\,\ell \Big), \tag{14.2}$$

with α in nepers. Dropping the phase would produce a symmetric, non-causal pulse; keeping it produces the long causal tail characteristic of copper (Figure 14.4, right), which is what makes DSL and Ethernet equalizers work hard.

The dielectric term needs the same care. A loss exactly proportional to $|f|$ at all frequencies violates the Paley–Wiener condition, so it has no exact causal partner; its Hilbert transform diverges logarithmically. Physical dielectrics have a nearly constant loss tangent over a wide but finite band. The Djordjevic–Sarkar permittivity [31] captures this:

$$\varepsilon(\omega) = \varepsilon_\infty + \frac{\Delta\varepsilon}{m_2 - m_1} \log_{10} \frac{\omega_2 + j\omega}{\omega_1 + j\omega}, \qquad \omega_{1,2} = 10^{m_{1,2}}\ \mathrm{rad/s}.$$

It is a continuum of Debye relaxations, hence causal, and its loss tangent is nearly flat between ω_1 and ω_2. LinkStudio uses $m_1 = 4$, $m_2 = 12$, sets $\tan\delta$ so that the attenuation is $b\,f$ (within 1 % from 10 MHz to 1 GHz) and $\varepsilon'(1\,\mathrm{GHz}) = \varepsilon_r$ (4.0 for FR-4, 2.3 for

polyethylene, 1.5 for foamed coax). It then applies $e^{-j\omega\ell(\sqrt{\varepsilon(\omega)}-\sqrt{\varepsilon_\infty})/c}$; subtracting $\sqrt{\varepsilon_\infty}$ removes only the front delay. On the 1 m backplane the energy of the impulse response before $t = 0$ drops from 40 % (zero-phase dielectric term) to 5×10^{-7}.

Code 14.1: Cable response: skin effect with its phase, TIA term and causal dielectric (`wireline.py`). linkstudio/phy/wireline.py, lines 296–305

```
pa, pb = CABLE_PRESETS[preset]
a = pa if a is None else a
b = pb if b is None else b
f_mhz = np.abs(f) / 1e6
km = length_m / 1000
neper = np.log(10) / 20
skin = a * np.sqrt(f_mhz) * km * neper
low = LOW_FREQUENCY_TERM.get(preset, 0.0) / np.sqrt(np.maximum(f_mhz, 1.0)) * km * neper
dielectric = dielectric_response(f, b * 1e-9, length_m, DIELECTRIC_CONSTANT.get(preset,
    2.3))
return np.exp(-skin * (1 + 1j * np.sign(f)) - low) * dielectric
```

Code 14.2: Djordjevic–Sarkar dielectric response. linkstudio/phy/wireline.py, lines 268–281

```
if loss_db_m_hz == 0:
    return np.ones_like(f, dtype=np.complex128)
m1, m2 = 4.0, 12.0
c0 = 299_792_458.0
tan_d = loss_db_m_hz * c0 / (20 / np.log(10) * np.pi * np.sqrt(eps_r))
d_eps = tan_d * eps_r * (m2 - m1) * np.log(10) / (np.pi / 2)

def log_term(w):
    return d_eps / (m2 - m1) * np.log10((10 ** m2 + 1j * w) / (10 ** m1 + 1j * w))

eps_inf = eps_r - float(np.real(log_term(2 * np.pi * 1e9)))
w = 2 * np.pi * np.asarray(f, dtype=float)
n = np.sqrt(eps_inf + log_term(w))
return np.exp(-1j * w * length_m * (n - np.sqrt(eps_inf)) / c0)
```

14.2.1 Telephone loops from primary constants

DSL standards specify loops through their primary constants, not through fitted losses. ITU-T G.996.1 Annex A [30] tabulates R, L, G and C of polyethylene-insulated 26 AWG (0.4 mm) and 24 AWG (0.5 mm) cable from 1 Hz to 5 MHz. The `pic_26awg` and `pic_24awg` presets interpolate these tables at 70 °F in $\log f$. Above 5 MHz they extend them with $R \propto \sqrt{f}$, $G \propto f$ and constant L, C. The loop is a two-port with ABCD matrix

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} = \begin{pmatrix} \cosh\gamma\ell & Z_0 \sinh\gamma\ell \\ \sinh(\gamma\ell)/Z_0 & \cosh\gamma\ell \end{pmatrix}, \qquad Z_0 = \sqrt{\frac{R + j\omega L}{G + j\omega C}}, \tag{14.3}$$

between source and load resistances $Z_S = Z_L = Z$ (`termination_ohm`, 100 Ω as in G.996.1). The insertion transfer function relative to a direct connection is

$$H(f) = \frac{2Z}{AZ + B + CZ^2 + DZ}. \tag{14.4}$$

Unlike the $\sqrt{f}$ fit, this model has the right low-frequency behaviour: below about 20 kHz the loss is set by the DC resistance $R_0\ell$ and the impedance mismatch, not by $\sqrt{f}$. G.996.1 also publishes the insertion loss of its test loops. Figure 14.3 compares the model with Table 2 for the Mid-C loop (1829 m of 26 AWG): the deviation is below 0.3 dB from 20 kHz to 1.1 MHz, the rounding of the table. The *ADSL-like DMT* scenario now runs over this loop.

Code 14.3: ABCD response of a PIC loop. linkstudio/phy/wireline.py, lines 248–257

```
r, ind, g, c = pic_primary_constants(f, preset)
w = 2 * np.pi * np.maximum(np.abs(f), 1.0)  # the table starts at 1 Hz (DC behaves like 1 Hz)
zs, ys = r + 1j * w * ind, g + 1j * w * c
x = np.sqrt(zs * ys) * length_m
z0 = np.sqrt(zs / ys)
z = termination_ohm
h = 2 * z / (2 * np.cosh(x) * z + z0 * np.sinh(x) + np.sinh(x) / z0 * z ** 2)
_, l_hi, _, c_hi = pic_primary_constants(np.array([_PIC_F[-1]]), preset)
h = h * np.exp(1j * w * length_m * np.sqrt(l_hi[0] * c_hi[0]))
return np.where(f < 0, np.conj(h), h)
```

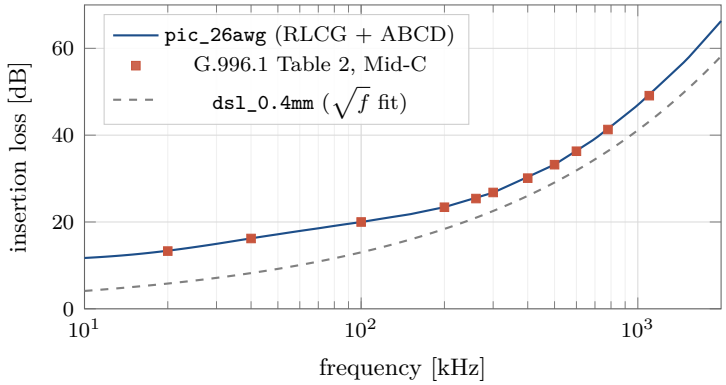


Figure 14.3: Insertion loss of 1829 m of 26 AWG cable between 100 Ω terminations: RLCG model from the G.996.1 primary constants against the values published in G.996.1 Table 2, and the earlier $\sqrt{f}$ fit for comparison.

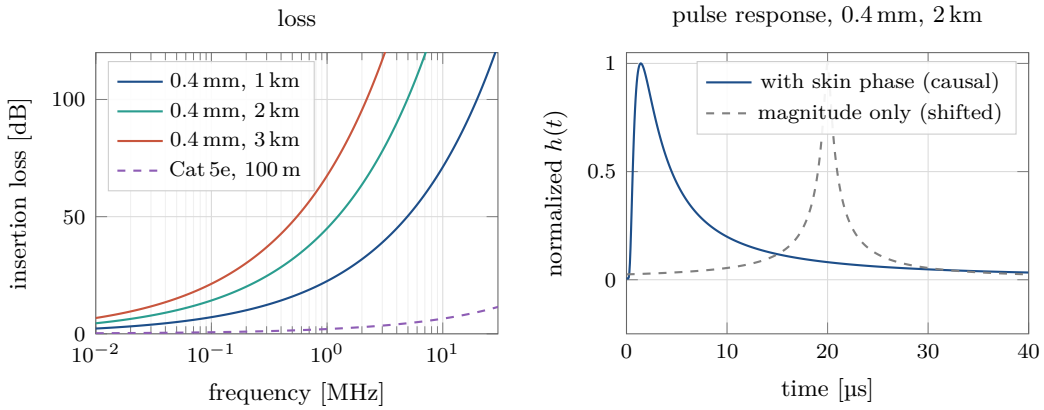


Figure 14.4: Left: insertion loss of the presets (`cable_response`). Right: impulse response at 17.664 MSa/s with the minimum-phase term (used by `copper_line`) and, for comparison, the zero-phase response with the same magnitude, shifted by 20 µs.

In the code `copper_line` applies $H(f)$ in the frequency domain, with zero padding to avoid circular wrap-around. It publishes the sampled impulse response, truncated at 99.95 % of its energy, on `csi` (`csi_type="cir"`), for `linear_equalizer` or the DMT FEQ. Presets: Cat 5e, Cat 6A, 0.4 and 0.5 mm telephone pairs (fit and G.996.1 RLCG), RG-6 coax and an FR-4 backplane trace. The RLCG presets remove only the front delay $\ell\sqrt{LC}$; the remaining dispersive delay is part of the channel.

14.3 Crosstalk

Signals on neighbouring pairs of a binder couple electromagnetically. The ANSI T1.417 [21] 99 % worst-case models for N disturbers give the coupling power transfer functions

$$|H_{\mathrm{NEXT}}(f)|^2 = 8.536 \times 10^{-15}\, N^{0.6} f^{1.5}, \tag{14.5}$$

$$|H_{\mathrm{FEXT}}(f)|^2 = 7.744 \times 10^{-21}\, N^{0.6}\, \ell_{\mathrm{ft}}\, f^2\, |H_{\mathrm{line}}(f)|^2. \tag{14.6}$$

NEXT (near end) does not depend on the length and grows with frequency. FEXT (far end) travels the whole line and is attenuated by it. `crosstalk` builds the disturbance from the spectrum of the neighbour's transmit signal (input `tx`, or the signal itself) with random phases, shapes it with the model and adds it. The crosstalk-to-signal ratio is reported.

Code 14.4: ANSI crosstalk models. `linkstudio/phy/wireline.py`, lines 357–362

```
357 fa = np.abs(f)
358 if model == "next":
359     return 8.536e-15 * n ** 0.6 * fa ** 1.5
360 length_ft = length_m * 3.281
361 line_gain = np.abs(line) ** 2 if line is not None else 1.0
362 return 7.744e-21 * n ** 0.6 * length_ft * fa ** 2 * line_gain
```

14.4 Full duplex: hybrid echo and echo cancellation

Full-duplex transceivers (1000BASE-T, DSL) send and receive on the same pair. The hybrid leaks part of the local transmit signal into the receiver as *echo*, often much stronger than the far-end signal. `hybrid_echo` adds the transmit signal filtered by a short echo path; `echo_canceller` identifies that path with an NLMS filter driven by the known local transmit signal and subtracts its estimate.

NLMS converges at a speed proportional to μ, but leaves an excess mean-square error, the misadjustment $\mathcal{M} \approx \mu/(2-\mu)$ for NLMS. That excess error is noise added to the far-end signal. The block therefore uses a *gear shift*: a large step for the first `train_samples` samples (fast acquisition, as at link start-up), then a small one (low residual). The power reduction after convergence is reported as a metric.

Code 14.5: NLMS echo cancellation with gear shifting. `linkstudio/phy/wireline.py`, lines 479–484

```
479 for n in range(N):
480     u = ref[:, n:n + T][:, ::-1]
481     e = y[:, n] - np.sum(w * u, axis=1)
482     out[:, n] = e
483     mu = self.params.train_step_size if n < self.params.train_samples else self.params.
        step_size
484     w += mu * e[:, None] * np.conj(u) / (1e-9 + np.sum(np.abs(u) ** 2, axis=1, keepdims=
        True))
```

14.5 SerDes equalization

14.5.1 Continuous-time linear equalizer

A CTLE is an analog high-pass with one zero and two poles,

$$H_{\mathrm{CTLE}}(s) = A\, \frac{1 + s/\omega_z}{(1 + s/\omega_{p1})(1 + s/\omega_{p2})}, \tag{14.7}$$

providing a peaking of about $20\log_{10}(\omega_{p1}/\omega_z)$ dB that partially inverts the channel loss (Figure 14.5). It amplifies high-frequency noise and crosstalk as well, so it is followed by a decision-feedback equalizer.

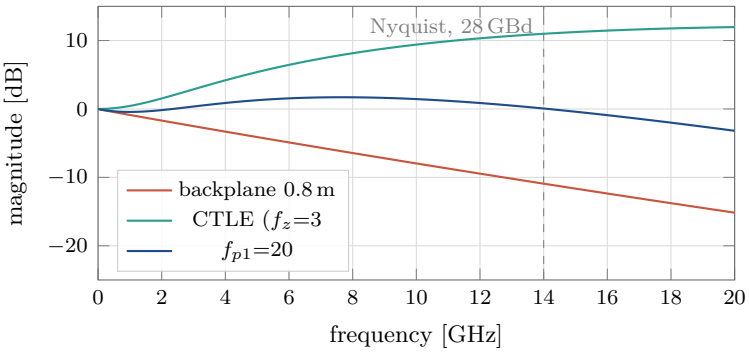


Figure 14.5: The CTLE keeps the combined response within about 2 dB up to the Nyquist frequency of a 28 GBd link (`cable_response` and `CTLE.response`). Beyond it the second pole and the growing loss take over.

14.5.2 Feed-forward plus decision-feedback equalizer

The DFE output is

$$z[n] = \sum_{i=0}^{F-1} w_i\, y[n + D - i] - \sum_{j=1}^{K} b_j\, \hat{a}[n - j], \tag{14.8}$$

where the feedback taps cancel the post-cursor ISI of past decisions $\hat{a}$. Because past decisions are noise-free when correct, the DFE removes ISI *without* noise enhancement, unlike a linear equalizer. The price is error propagation: a wrong decision feeds wrong ISI back. Both filters adapt jointly with NLMS on the stacked regressor $[\boldsymbol{u}; \hat{\boldsymbol{a}}]$, trained on `n_train` known symbols and then decision-directed.

Code 14.6: Joint NLMS adaptation of the FFE and the DFE (`dfe_equalizer`).
`linkstudio/phy/wireline.py`, lines 565–578

```
565 for n in range(N):
566     u = pad[:, n:n + F][:, ::-1]
567     z = np.sum(w * u, axis=1) - np.sum(bfb * past, axis=1)
568     out[:, n] = z
569     d = ref.data[:, n] if n < p.n_train else const.points[const.nearest(z)]
570     e = d - z
571     if n >= p.n_train:
572         err += float(np.sum(np.abs(e) ** 2))
573         count += B
574     norm = 1e-9 + np.sum(np.abs(u) ** 2, axis=1, keepdims=True) + np.sum(np.abs(past) ** 2,
        axis=1, keepdims=True)
575     w += p.step_size * e[:, None] * np.conj(u) / norm
576     if K:
577         bfb -= p.step_size * e[:, None] * np.conj(past) / norm
578         past = np.concatenate([d[:, None], past[:, :-1]], axis=1)
```

14.5.3 Jitter

Sampling-clock jitter perturbs the sampling instants, $t_n = nT_s + \Delta(n)$, with a random (Gaussian, RJ) and a sinusoidal (SJ) component expressed in unit intervals. `jitter` resamples the waveform at the perturbed instants with the cubic interpolator; RJ is drawn once per symbol and

interpolated between symbols. To first order, the error is $x'(t)\,\Delta$, which is why steep edges, that is high bandwidth, make jitter costly.

Code 14.7: Perturbed sampling instants. linkstudio/phy/wireline.py, lines 602–606

```
n_sym = N // sps + 2
rj = self.rng.standard_normal((B, n_sym)) * self.params.rj_rms_ui
rj_t = np.stack([np.interp(n / sps, np.arange(n_sym), r) for r in rj])
sj = self.params.sj_amplitude_ui * np.sin(2 * np.pi * self.params.sj_frequency_hz * n / fs)
t = n[None, :] + sps * (rj_t + sj[None, :])
```

14.6 Discrete multitone (DSL)

14.6.1 Real OFDM

DMT is OFDM for real baseband channels. Imposing Hermitian symmetry, $X_{N-k} = X_k^*$, makes the IFFT output real. Tones $k = 1, \dots, N/2 - 1$ carry independent QAM symbols; DC and Nyquist are unused. The cyclic prefix turns the loop into $N/2 - 1$ parallel flat channels, as in Chapter 11. The demodulator equalizes each tone with one division (FEQ) using the channel's impulse response.

Code 14.8: Per-tone mapping and Hermitian symmetry (`dmt_modulator`). linkstudio/phy/wireline.py, lines 686–692

```
X = np.zeros((B, n_sym, p.n_fft), dtype=np.complex128)
for b, idx in self.groups.items():
    sel = self.offsets[idx][:, None] + np.arange(b)  # (n_tones_b, b)
    X[..., self.tones[idx]] = self.consts[b].map(bits[..., sel].reshape(B, n_sym, -1))
X[..., p.n_fft - self.tones] = np.conj(X[..., self.tones])
t = np.fft.ifft(X, axis=-1, norm="ortho").real
out = np.concatenate([t[..., p.n_fft - p.cp_len:], t], axis=-1).reshape(B, -1)
```

14.6.2 Bit loading and the SNR gap

For uncoded QAM at a target error rate, the achievable bits per tone are well approximated by the *gap* formula

$$b_k = \log_2\!\Big(1 + \frac{\mathrm{SNR}_k}{\Gamma\,\gamma_m}\Big), \qquad \Gamma \approx 9.8\,\mathrm{dB}\ (P_s = 10^{-7}), \tag{14.9}$$

where γ_m is a noise margin [14]. The block computes the expected SNR_k from the cable model and the AWGN level. With the time-domain power $P = 2N_{\mathrm{tones}}/N$ and $\sigma^2 = P/\mathrm{SNR}$, $\mathrm{SNR}_k = |H_k|^2\,\mathrm{SNR}\,N/(2N_{\mathrm{tones}})$. It then rounds b_k down to the sizes the mapper supports: 0, 1 (BPSK) or an even number (square QAM).

Code 14.9: Gap-approximation bit loading. linkstudio/phy/wireline.py, lines 616–619

```
gamma = 10 ** ((gap_db + margin_db) / 10)
b = np.floor(np.log2(1 + snr / gamma)).astype(int)
b = np.clip(b, 0, max_bits)
return np.where(b >= 2, b - (b % 2), b)
```

Bit loading versus water-filling The transmit PSD is flat; only the bits adapt to the channel. True water-filling would also redistribute power, pouring more where the SNR is high. With a flat PSD the loss is small at high SNR, but it is a modelling choice to keep in mind.

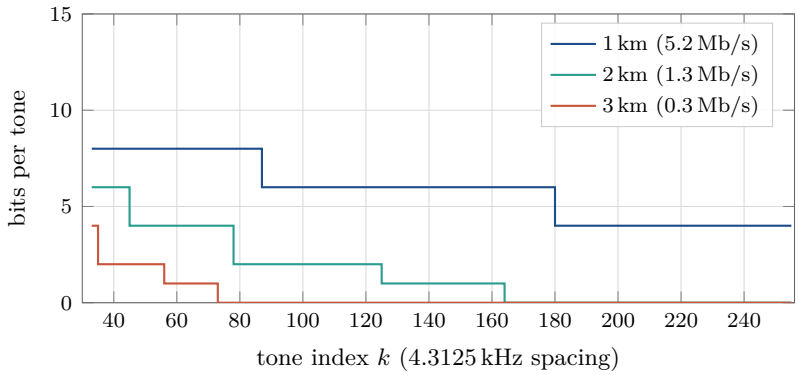


Figure 14.6: Bit loading of `dmt_modulator` on a 0.4 mm loop (ADSL downstream tones 33–255, 50 dB SNR, 9.8 dB gap, 3 dB margin). Data rates are those reported by the block.

Chapter 15

Optical fiber communications

This chapter develops the theory behind the optical blocks and shows, block by block, how the simulator implements it. The treatment follows the two parts of the optical fiber communications course on which the models are based [39]: intensity modulation with direct detection (IM/DD) for short and medium reach, and coherent dual-polarization systems with optical amplification for long haul. The VCSEL follows the rate-equation model of Shi, Qi, Wang and Hu [40]. Wherever possible the figures compare the simulated blocks with the closed-form results they are meant to reproduce.

The code is in four modules: `phy/optical.py` (fiber, modulators, amplifiers, detectors, coherent DSP, free space), `phy/lasers.py` (rate-equation lasers), `phy/multimode.py` (multimode fiber) and `phy/theory.py` (closed-form references). Figure 15.1 shows the three families of links they support.

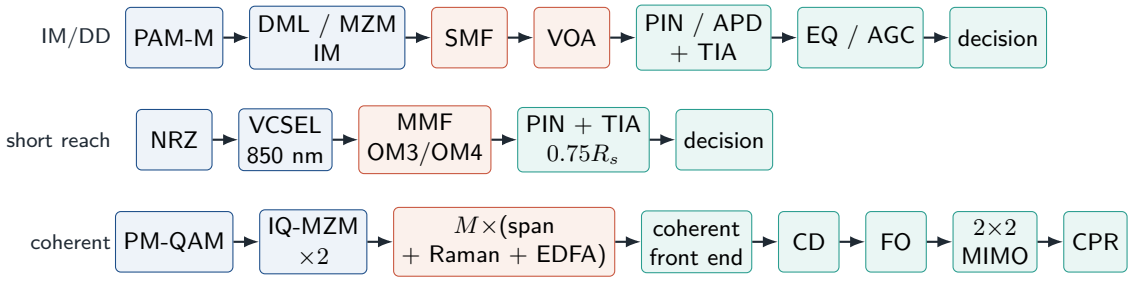


Figure 15.1: The three families of optical links: IM/DD over single-mode fiber, VCSEL links over multimode fiber inside data centers, and coherent dual-polarization transmission over amplified long-haul links.

In the code Optical signals are complex field envelopes scaled so that $|E|^2$ is the instantaneous optical power in watts. A single polarization has the shape (B, n) and dual polarization $(B, 2, n)$. Optical blocks keep the following metadata up to date:

- `wavelength_nm`, the carrier wavelength;
- `power`, the average power in W (launch, after loss, after gain);
- `cd_ps_nm` and `cd_slope_ps_nm2`, the accumulated dispersion $D\ell$ and slope $S\ell$;
- `ase_psd`, the accumulated ASE density per polarization in W/Hz;
- `unit`, $\sqrt{\mathrm{W}}$ for optical fields.

The OSNR can therefore be computed at any point of a link from `power` and `ase_psd`, without measuring the noise.

15.1 Signals, power and noise

15.1.1 Bandpass signals, analytic signal and complex envelope

An optical field is a bandpass signal around the laser frequency $f_0 \approx 193\,\mathrm{THz}$:

$$E_{rb}(t) = a(t)\cos\left(2\pi f_0 t + \phi_0(t)\right). \tag{15.1}$$

In amplitude modulation (on–off keying, PAM) only $a(t)$ carries information and the spectrum is the baseband spectrum of $a(t)$ shifted to $\pm f_0$, $E(f) = \frac{1}{2}[A(f-f_0) + A(f+f_0)]$. Phase modulation varies $\phi_0(t)$; a spurious frequency modulation $\Delta f(t)$ of the transmitter, $E = a\cos(2\pi(f_0 + \Delta f(t))t + \phi_0)$, is called *chirp*. For a real signal $|E(f)|$ is even and $\arg E(f)$ odd, so the negative frequencies carry no extra information. Two complex representations remove them:

- the *analytic signal* $E_a(t) = a(t)e^{j\phi_0(t)}e^{j2\pi f_0 t}$, with $E_{rb} = \Re\{E_a\}$;
- the *complex envelope* $\tilde{E}(t) = a(t)e^{j\phi_0(t)} = E_R(t) + jE_I(t)$, with $E_{rb} = \Re\{\tilde{E}e^{j2\pi f_0 t}\}$.

The spectrum of $\tilde{E}$ is the positive-frequency part of $E(f)$ moved to $f = 0$; it is not symmetric, because $\tilde{E}$ is complex. The real signal has power $P_{rb} = \frac{1}{2}\overline{a^2}$, the analytic signal and the envelope $P_a = P_{ce} = \overline{a^2} = 2P_{rb}$. Following the convention of optical communications the envelope is normalized so that

$$P(t) = |\tilde{E}(t)|^2 \quad [\mathrm{W}], \tag{15.2}$$

which corresponds to the physical field $\Re\{\sqrt{2}\,\tilde{E}(t)e^{j2\pi f_0 t}\}$. The tilde is dropped from now on, and $\bar{P}$ denotes the time average (for OOK with equiprobable bits and $P_0 = 0$, $\bar{P} = P_{\mathrm{peak}}/2$).

Bandwidth. A real baseband signal of bandwidth B (for example a 10 Gb/s NRZ drive, $B \approx 10$ GHz to the first null) produces a bandpass optical signal of bandwidth $2B$ around f_0 and a complex envelope of bandwidth B around 0 (two-sided $2B$). The usual digital signal is $x(t) = x_{\mathrm{bias}} + \sum_k \alpha_k s(t - kT_s)$, whose power spectral density is

$$G_x(f) = x_{\mathrm{bias}}^2\delta(f) + \frac{\sigma_\alpha^2}{T_s}|S(f)|^2. \tag{15.3}$$

Rectangular NRZ pulses give a sinc^2 with first null at the symbol rate $D = 1/T_s$; Nyquist pulses reach the minimum bandwidth $D/2$; raised-cosine pulses with roll-off ρ occupy $\frac{D}{2}(1+\rho)$. QAM modulates the in-phase and quadrature components with two PAM streams, $\tilde{E} = a(t) + jb(t)$, doubling the bit rate for the same symbol rate.

Frequency and wavelength. With $f = c/\lambda$, a small band transforms as

$$|\Delta f| = \frac{c}{\lambda^2}|\Delta\lambda|, \tag{15.4}$$

so 100 GHz is 0.8 nm at 1550 nm and 0.57 nm at 1310 nm. Table 15.1 lists the optical bands.

Table 15.1: Telecommunication wavelength bands of silica fiber.

band	range [nm]	frequency [THz]	use
first window	850	353	VCSELs and LEDs over multimode fiber
O (original)	1260–1360	238–220	zero dispersion of G.652 fiber; DMLs, data centers, access
E (extended)	1360–1460	220–205	water-peak region (low-water-peak fibers)
S (short)	1460–1530	205–196	emerging amplifiers
C (conventional)	1530–1565	196.1–191.7	lowest loss, EDFAs, long-haul WDM
L (long)	1565–1625	191.7–184.5	second EDFA band
U (ultra-long)	1625–1675	184.5–179.0	monitoring

Power levels. Ratios are given in dB, $G_{\mathrm{dB}} = 10\log_{10}(P_1/P_2)$, and absolute powers in dBm, $P_{\mathrm{dBm}} = 10\log_{10}(P/1\,\mathrm{mW})$, so cascades add: $P_{\mathrm{out,dBm}} = P_{\mathrm{in,dBm}} + G_{\mathrm{dB}} - A_{\mathrm{dB}}$.

15.1.2 White Gaussian noise

A white Gaussian process of power spectral density $N_0/2$ filtered by $H(f)$ has density $\frac{N_0}{2}|H(f)|^2$ and power

$$P_n = \frac{N_0}{2}\int_{-\infty}^{\infty} |H(f)|^2 df, \tag{15.5}$$

which is $N_0 B$ for an ideal rectangular filter of (one-sided) bandwidth B. In the simulator the noise of a block is generated over the whole simulated band f_s and filtered by the downstream blocks, so the result does not depend on the sample rate.

15.1.3 Polarization and the four quadratures

A single-mode fiber guides two degenerate modes with the same transverse profile and orthogonal polarizations. The field is therefore a vector,

$$\boldsymbol{E}(z,t) = (E_{Ix} + jE_{Qx})\hat{x} + (E_{Iy} + jE_{Qy})\hat{y}, \qquad P(z,t) = |E_x|^2 + |E_y|^2, \tag{15.6}$$

with four real degrees of freedom, the *quadratures*. The reference polarizations may be linear, circular or elliptical; birefringence rotates them randomly along the fiber and in time (minutes in buried cables, milliseconds with vibrations, tens of microseconds on aerial cables), but it preserves orthogonality and power. Direct-detection systems modulate and detect only $P(t)$ and are insensitive to polarization; coherent systems modulate and detect all four quadratures.

15.2 The optical fiber

15.2.1 Structure and modes

A fiber is a strand of pure silica 125 µm in diameter. The core has a slightly higher refractive index than the cladding (by about 0.35 %, with germanium doping), and light is confined by total internal reflection. Solving Maxwell's equations, the field is a sum of guided modes,

$$E(\boldsymbol{r},z,t) = \sum_{i=1}^{N_{\text{modes}}} E_i(z,t)\,\mathcal{E}_i(\boldsymbol{r}), \tag{15.7}$$

each with its own transverse profile $\mathcal{E}_i(\boldsymbol{r})$ and its own propagation constant. A step-index core guides a single mode (two polarizations) when

$$V = \frac{2\pi a}{\lambda}\,\text{NA} = \frac{2\pi a}{\lambda}\sqrt{n_{\text{core}}^2 - n_{\text{clad}}^2} < 2.405, \tag{15.8}$$

which at 1310–1550 nm needs a core diameter of about 10 µm. The fundamental LP_{01} mode has an approximately Gaussian intensity profile. Multimode fibers (50 or 62.5 µm cores) guide hundreds of modes travelling at different speeds; they are used with VCSELs inside data centers (Section 15.8). For single-mode fiber the transverse profile is omitted and $E(z,t)$ is normalized so that $|E(z,t)|^2$ is the power crossing the fiber section.

15.2.2 Attenuation

Absorption and scattering make the field decay as $E(z) = E(0)e^{-\alpha_E z}$, the power as $P(z) = P(0)e^{-2\alpha_E z} = P(0)e^{-\alpha z}$. In decibels the loss is linear in distance,

$$A_{\text{dB}} = 10\log_{10}\frac{P(0)}{P(z)} = \alpha_{\text{dB}} z, \qquad \alpha_{\text{dB}} = 20\log_{10}(e)\,\alpha_E = 8.686\,\alpha_E = 4.343\,\alpha, \tag{15.9}$$

and the link equation of every power budget is

$$P_{Rx,\mathrm{dBm}} = P_{Tx,\mathrm{dBm}} - \alpha_{\mathrm{dB}} L. \tag{15.10}$$

The intrinsic loss of glass combines Rayleigh scattering ($\propto \lambda^{-4}$) at short wavelengths and infrared vibrational absorption at long wavelengths, plus impurity peaks (the water peak near 1383 nm). The minimum, about 0.2 dB/km (0.17 dB/km for pure-silica-core fiber, below 0.18 dB/km for the best commercial fibers), lies in the C band, where erbium amplifiers also work. The C band spans 35 nm, about 4.4 THz. Across one channel (tens of GHz) the attenuation is constant: it scales the signal without distorting it. Across a WDM comb it varies by a few hundredths of a dB/km.

> **Key idea** Throughout the manual α without subscript is the *power* attenuation coefficient (the NLSE convention); the field coefficient is $\alpha_E = \alpha/2$. Block parameters are in dB/km. The GN model of Section 15.13 is written with the field coefficient, as in the literature.

15.2.3 Chromatic dispersion

The propagation constant β depends on frequency, polarization and power. Its frequency dependence is expanded around the carrier $\omega_0 = 2\pi f_0$ ($\Delta\omega = \omega - \omega_0$):

$$\beta(\omega) = \beta_0 + \beta_1\Delta\omega + \frac{1}{2}\beta_2\Delta\omega^2 + \frac{1}{6}\beta_3\Delta\omega^3 + \dots, \qquad E_{\mathrm{out}}(z,\omega) = E_{\mathrm{in}}(\omega)e^{-j\beta(\omega)z}e^{-\alpha_E z}. \tag{15.11}$$

The terms have distinct effects:

- $e^{-j\beta_0 z}$ is a constant phase, irrelevant;
- $e^{-j\beta_1\Delta\omega z}$ is a linear phase, a delay $t_D = \beta_1 z$ (group velocity $v_g = 1/\beta_1$), which does not distort;
- $e^{-j\frac{1}{2}\beta_2\Delta\omega^2 z}$ is quadratic in frequency and distorts: it is *chromatic dispersion* (CD).

A distortion-free channel needs $|H(f)|$ constant and $\arg H(f)$ linear on the signal band. Dispersion is an all-pass filter with quadratic phase,

$$H_F(f) = e^{-j2\pi^2\beta_2(f-f_0)^2 z}, \qquad |H_F| = 1. \tag{15.12}$$

The group delay of a phase-only filter is $\tau_g = -\frac{1}{2\pi}\frac{d\phi}{df}$; for Equation (15.12)

$$\tau_g(f) = 2\pi\beta_2(f - f_0)z, \qquad \frac{d\tau_g}{df} = 2\pi\beta_2 z, \tag{15.13}$$

so two spectral components Δf apart arrive $\Delta\tau_g = 2\pi\beta_2\Delta f\, z$ apart: different colours travel at different speeds. Fiber datasheets give instead the dispersion parameter D, defined from the delay per unit wavelength, $\tau_g(\lambda) = \tau_g(\lambda_0) + D(\lambda - \lambda_0)z$, so that

$$\Delta\tau_g = D\,\Delta\lambda\, z, \qquad D\Big[\tfrac{\mathrm{ps}}{\mathrm{nm\cdot km}}\Big] = -\frac{2\pi c}{\lambda^2}\,\beta_2\Big[\tfrac{\mathrm{ps}^2}{\mathrm{km}}\Big] \approx -0.784\,\beta_2 \quad (\lambda = 1550\,\mathrm{nm}). \tag{15.14}$$

D and β_2 have opposite signs; standard fiber has $D \approx +16$–$17\,\mathrm{ps/(nm\,km)}$ ($\beta_2 \approx -21\,\mathrm{ps^2/km}$) at 1550 nm and $D = 0$ near 1310 nm. Total dispersion is the sum of material dispersion (the index of glass varies with λ) and waveguide dispersion (the confinement varies with λ/a); special index profiles shift it (Table 15.2).

Example. A 10 Gb/s chirp-free NRZ signal has its main spectral content within $\Delta f \approx R_B = 10\,\text{GHz}$, that is $\Delta\lambda = 0.08\,\text{nm}$. After 200 km of standard fiber $\Delta\tau = 16 \times 200 \times 0.08 = 256\,\text{ps}$, 2.6 bit slots: the pulses overlap completely. In a coherent system at 100 GBd with roll-off 0.3, $\Delta f = 130\,\text{GHz}$, and 1000 km ($\beta_2 L = -21\,000\,\text{ps}^2$) spread each pulse over $2\pi \cdot 130\,\text{GHz} \cdot 21\,000\,\text{ps}^2 \approx 17\,000\,\text{ps}$, 1700 symbols.

Accumulated dispersion. The field is linear in the fiber, so cascaded segments multiply their transfer functions: $H_{F,\text{tot}} = e^{-j2\pi^2(f-f_0)^2 \sum_i \beta_{2,i} L_i}$. Only the accumulated dispersion

$$D_{\text{acc}} = \sum_m D_m L_m \ [\text{ps/nm}], \qquad \beta_{2,\text{acc}} = \sum_m \beta_{2,m} L_m \tag{15.15}$$

matters, also across optical amplifiers. Dispersion-compensating fiber ($D < 0$) can bring D_{acc} back to zero; coherent receivers compensate it digitally instead.

Table 15.2: Fiber types at 1550 nm, available as `fiber_type` of `fiber` and `optical_link` (the slope stays a separate parameter).

`fiber_type`	fiber	α_{dB} [dB/km]	D [ps/(nm km)]	γ [1/(W km)]
`smf`	standard single mode, ITU-T G.652	0.20	16	1.3
`nzdsf`	non-zero dispersion shifted, G.655	0.21	4	2.0
`pscf`	pure silica core	0.17	21	0.8
`dcf`	dispersion compensating	0.45	−100	4.0

Third-order dispersion. For wide WDM grids, or near the zero-dispersion wavelength, the term $\frac{\beta_3}{6}\Delta\omega^3$ matters, with $\beta_3 = (\frac{\lambda}{2\pi c})^2(\lambda^2 S + 2\lambda D)$ from the dispersion slope S (`dispersion_slope_ps_nm2_km`; about $0.057\,\text{ps/(nm}^2\,\text{km)}$ for G.652 fiber [32]). It is 0 by default.

Sign convention. The simulator follows Agrawal's convention, in which the carrier is $e^{-j\omega_0 t}$ and the linear operator reads $\exp(j\frac{\beta_2}{2}\omega^2 z - \frac{\alpha}{2}z)$. The course writes the carrier as $e^{+j\omega_0 t}$, which conjugates every phase: Equation (15.12) and the NLSE of Section 15.2.5 are the same physics. The dispersion, nonlinear and LO phases use one convention consistently, so solitons, SPM, CD compensation and frequency offsets behave correctly. With NumPy's transform a *positive* baseband frequency is then a *lower* optical frequency, which matters for odd-order terms: with slope, the operator is $\exp(j(\frac{\beta_2}{2}\omega^2 - \frac{\beta_3}{6}\omega^3)\ell)$ on the NumPy grid and gives the group delay $\tau = D\ell\,\Delta\lambda + \frac{S\ell}{2}\Delta\lambda^2$, which the test suite checks to 0.05 ps. The accumulated slope is tracked in `cd_slope_ps_nm2` and removed by `cd_compensation` together with $D\ell$.

Code 15.1: From D to β_2. `linkstudio/phy/optical.py`, lines 298–302

```
298 def beta2(dispersion_ps_nm_km: float, wavelength_nm: float) -> float:
299     """Group-velocity dispersion beta2 in s^2/m."""
300     D = dispersion_ps_nm_km * 1e-6  # s/m^2
301     lam = wavelength_nm * 1e-9
302     return -D * lam ** 2 / (2 * np.pi * C_LIGHT)
```

Code 15.2: The dispersion operator. `linkstudio/phy/optical.py`, lines 313–324

```
313 def dispersion_operator(n: int, fs: float, cd_ps_nm: float, wavelength_nm: float,
314                        slope_ps_nm2: float = 0.0) -> np.ndarray:
315     """All-pass dispersion filter for an accumulated D*L (ps/nm) and slope S*L (ps/nm^2).
316 
317     exp(j (beta2 L w^2 / 2 - beta3 L w^3 / 6)) on the NumPy frequency grid. The envelope
```

```
    follows Agrawal's convention (carrier exp(-j w0 t)), so a positive NumPy frequency is a
    lower optical frequency; hence the minus sign of the odd beta3 term.
    """
    w = 2 * np.pi * np.fft.fftfreq(n, 1 / fs)
    b2l = beta2(cd_ps_nm, wavelength_nm) * 1e3  # D*L in ps/nm == (ps/nm/km) * km
    b3l = beta3(cd_ps_nm, slope_ps_nm2, wavelength_nm) * 1e3
    return np.exp(1j * (b2l / 2 * w ** 2 - b3l / 6 * w ** 3))
```

Gaussian pulse. An unchirped Gaussian pulse $A(0,t) = e^{-t^2/(2T_0^2)}$ stays Gaussian and broadens as

$$\frac{T_1(z)}{T_0} = \sqrt{1 + \left(\frac{z}{L_D}\right)^2}, \qquad L_D = \frac{T_0^2}{|\beta_2|}. \tag{15.16}$$

For $T_0 = 5\,\mathrm{ps}$ and $|\beta_2| = 21.68\,\mathrm{ps}^2/\mathrm{km}$, $L_D = 1.153\,\mathrm{km}$. Figure 15.2 compares the solver with this result; the RMS widths agree to all printed digits, as expected from an exact spectral method.

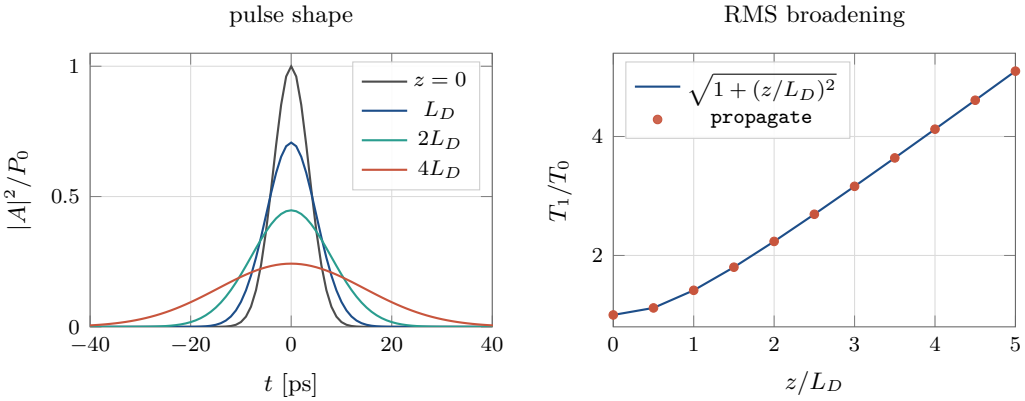


Figure 15.2: Chromatic dispersion of a 5-ps Gaussian pulse in standard single-mode fiber ($D = 17\,\mathrm{ps/nm/km}$, no loss).

15.2.4 Polarization-mode dispersion

Random birefringence rotates the state of polarization and delays the two principal states by the differential group delay (DGD). Because the birefringence changes along the length, the DGD is a random variable: Maxwellian, with mean $\langle\tau\rangle = D_{\mathrm{PMD}}\sqrt{\ell}$, and frequency dependent (higher-order PMD). LinkStudio uses the coarse-step *waveplate* model [34]: a span is a cascade of N birefringent sections (`pmd_sections`, 10 by default), each a fixed DGD $\delta\tau$ between Haar-random rotations,

$$\mathbf{T}(\omega) = \mathbf{U}_{N+1} \prod_{k=N}^{1} \begin{pmatrix} e^{j\omega\delta\tau/2} & 0 \\ 0 & e^{-j\omega\delta\tau/2} \end{pmatrix} \mathbf{U}_k, \qquad \delta\tau = \langle\tau\rangle\sqrt{\frac{3\pi}{8N}}. \tag{15.17}$$

The total DGD then satisfies $\langle\tau^2\rangle = N\delta\tau^2$, and the Maxwellian relation $\langle\tau^2\rangle = \frac{3\pi}{8}\langle\tau\rangle^2$ fixes $\delta\tau$. The rotations come from a QR decomposition of a complex Gaussian matrix with the phases of the diagonal of $\mathbf{R}$ removed. The test suite extracts the DGD of 4000 fibers from the eigenvalues of $j\,\mathbf{T}'\mathbf{T}^\dagger$; the mean is within 1 % of $D_{\mathrm{PMD}}\sqrt{\ell}$ and $\langle\tau^2\rangle/\langle\tau\rangle^2 = 1.168$ against $3\pi/8 = 1.178$ (Figure 15.3).

Code 15.3: Jones matrices of the waveplate PMD model. `linkstudio/phy/optical.py`, lines 426–436

```
    t = np.broadcast_to(_random_unitary(rng, batch)[:, None], (batch, w.size, 2, 2)).copy()
    if mean_dgd_s <= 0 or n_sections == 0:
```

```
    return t
dtau = mean_dgd_s * np.sqrt(3 * np.pi / (8 * n_sections))
d = np.zeros((w.size, 2, 2), dtype=np.complex128)
d[:, 0, 0] = np.exp(1j * w * dtau / 2)
d[:, 1, 1] = np.exp(-1j * w * dtau / 2)
for _ in range(n_sections):
    u = _random_unitary(rng, batch)
    t = np.einsum("nij,bjk,bnkl->bnil", d, u, t)
return np.einsum("bij,bnjk->bnik", _random_unitary(rng, batch), t)
```

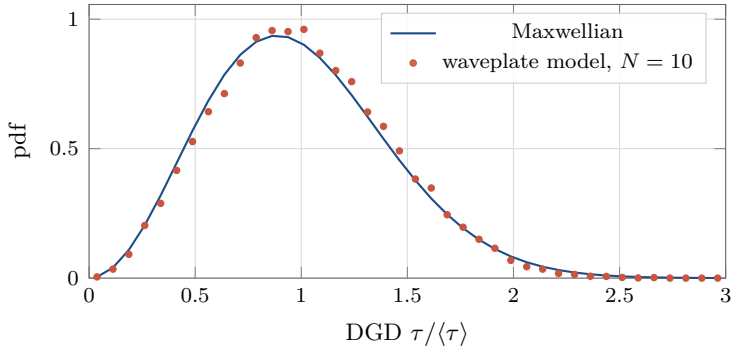


Figure 15.3: DGD distribution of 20 000 fibers built by `pmd_transfer` against the Maxwellian density $\frac{32\tau^2}{\pi^2\langle\tau\rangle^3}e^{-4\tau^2/(\pi\langle\tau\rangle^2)}$.

15.2.5 Kerr nonlinearity and the nonlinear Schrödinger equation

The refractive index of silica grows with the intensity (Kerr effect), so the propagation constant depends on the instantaneous power. The complex envelope obeys the nonlinear Schrödinger equation (NLSE). In the course convention [39],

$$\frac{\partial E}{\partial z} = j\frac{\beta_2}{2}\frac{\partial^2 E}{\partial t^2} - \frac{\alpha}{2}E - j\gamma|E|^2E, \tag{15.18}$$

and in Agrawal's convention, used by the code [2],

$$\frac{\partial A}{\partial z} = \underbrace{-\frac{\alpha}{2}A - j\frac{\beta_2}{2}\frac{\partial^2 A}{\partial t^2}}_{\hat{D}A} + \underbrace{j\gamma|A|^2A}_{\hat{N}A}, \tag{15.19}$$

with power attenuation α, group-velocity dispersion β_2 and Kerr coefficient γ (`gamma_w_km`, about $1.3\,\mathrm{W^{-1}km^{-1}}$ for standard fiber). The two are complex conjugates of each other. The linear part has the exact solution

$$\tilde{A}(z,\omega) = \tilde{A}(0,\omega)\exp\Big(j\frac{\beta_2}{2}\omega^2 z - \frac{\alpha}{2}z\Big), \tag{15.20}$$

which depends only on the accumulated $D\ell$ tracked in `cd_ps_nm`.

Self-phase modulation. Without dispersion and loss, Equation (15.19) gives $A(z,t) = A(0,t)e^{j\gamma P(0,t)z}$: the power modulates its own phase. With loss the power decays as $P(0,t)e^{-\alpha z}$ and the accumulated phase is

$$\phi_{\mathrm{NL}}(t) = \gamma P(0,t)\int_0^z e^{-\alpha z'}dz' = \gamma P(0,t)L_{\mathrm{eff}}, \qquad L_{\mathrm{eff}} = \frac{1-e^{-\alpha z}}{\alpha}, \tag{15.21}$$

so the nonlinearity acts over an effective length shorter than z, tending to $1/\alpha \approx 21.7\,\mathrm{km}$ for long spans. SPM chirps the pulse and broadens its spectrum, with a number of peaks that grows with the peak phase $\phi_{\max} = \gamma P_0 L_{\mathrm{eff}}$ (Figure 15.4).

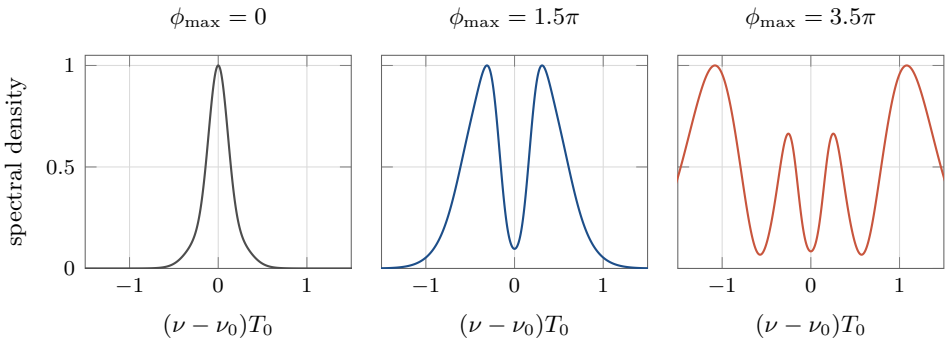


Figure 15.4: SPM-broadened spectra of a Gaussian pulse computed by `propagate` with $D = 0$, the classic figure of Agrawal [2]: $\phi_{\max} = (M - \frac{1}{2})\pi$ produces M peaks.

The split-step Fourier method. With both terms present, Equation (15.19) has no general closed form. Formally $A(z+h) = e^{h(\hat{D}+\hat{N})}A(z)$; the two operators do not commute, but each is trivial on its own: $\hat{D}$ is a multiplication in the frequency domain, $\hat{N}$ in the time domain. The symmetric (Strang) splitting

$$A(z+h) = e^{\frac{h}{2}\hat{D}}\, e^{h\hat{N}}\, e^{\frac{h}{2}\hat{D}} A(z) + \mathcal{O}(h^3) \tag{15.22}$$

is exact to second order, by the Baker–Campbell–Hausdorff formula (the first commutator cancels by symmetry); over a fixed length the global error is $\mathcal{O}(h^2)$. Attenuation and any distributed gain enter the half steps. The nonlinear phase is applied at mid-step with the effective length of the local net loss $a = \alpha - g$,

$$L_{\text{eff}}(h) = \int_{-h/2}^{h/2} e^{-az'}\, dz' = \frac{2\sinh(ah/2)}{a}. \tag{15.23}$$

For dual polarization the Manakov equation replaces $|A|^2$ by $|A_x|^2 + |A_y|^2$ and γ by $\frac{8}{9}\gamma$, the average over fast random polarization evolution.

Code 15.4: The split-step Fourier solver of the NLSE / Manakov equation (`propagate`).
linkstudio/phy/optical.py, lines 333–373

```
333 def propagate(e: np.ndarray, fs: float, length_km: float, att_db_km: float, d_ps_nm_km:
      float,
334               wavelength_nm: float, nonlinear: bool, gamma: float, steps_per_km: float,
335               slope_ps_nm2_km: float = 0.0, nonlinear_phase_rad: float | None = None,
336               gain: RamanGain | None = None) -> np.ndarray:
337     """Split-step Fourier solution of the NLSE (Manakov equation for dual polarization).
338
339     The step is at most 1 / ``steps_per_km`` km. With ``nonlinear_phase_rad`` it is also
340     limited so that the peak nonlinear phase per step, gamma P_peak L_eff, stays below that
341     value (nonlinear-phase-rotation criterion of Sinkin et al. 2003). ``gain`` adds a
342     distributed (Raman) power gain g(z) to the attenuation.
343     """
344     n = e.shape[-1]
345     alpha = att_db_km / (10 * np.log10(np.e)) / 1e3  # 1/m
346     total = length_km * 1e3
347
348     def amp(z1: float, z2: float) -> float:
349         """Field factor exp(int (g - alpha) dz / 2) over [z1, z2]."""
350         return float(np.exp(((gain.integral(z1, z2) if gain else 0.0) - alpha * (z2 - z1))
      / 2))
351
352     if not nonlinear or gamma == 0:
```

```
        lin = dispersion_operator(n, fs, d_ps_nm_km * length_km, wavelength_nm,
    slope_ps_nm2_km * length_km)
        return np.fft.ifft(np.fft.fft(e, axis=-1) * lin * amp(0.0, total), axis=-1)
    max_dz = total / max(1, int(np.ceil(length_km * steps_per_km)))
    g = gamma * 1e-3 * (8 / 9 if e.ndim == 3 else 1.0)  # 1/(W km) -> 1/(W m)
    z = 0.0
    while z < total * (1 - 1e-12):
        dz = min(max_dz, total - z)
        if nonlinear_phase_rad:
            p_peak = float(np.max(np.sum(np.abs(e) ** 2, axis=1) if e.ndim == 3 else np.abs
    (e) ** 2))
            dz = min(dz, max(nonlinear_phase_rad / max(g * p_peak, 1e-30), 1.0))
        half = dispersion_operator(n, fs, d_ps_nm_km * dz / 2e3, wavelength_nm,
    slope_ps_nm2_km * dz / 2e3)
        # Effective length around the step midpoint (where the nonlinear phase is applied),
        # for the local net loss alpha - g.
        a = alpha - (gain.coefficient(z + dz / 2) if gain else 0.0)
        leff = 2 * np.sinh(a * dz / 2) / a if abs(a) > 1e-12 else dz
        e = np.fft.ifft(np.fft.fft(e, axis=-1) * half * amp(z, z + dz / 2), axis=-1)
        p = np.sum(np.abs(e) ** 2, axis=1, keepdims=True) if e.ndim == 3 else np.abs(e) **
    2
        e = e * np.exp(1j * g * p * leff)
        e = np.fft.ifft(np.fft.fft(e, axis=-1) * half * amp(z + dz / 2, z + dz), axis=-1)
        z += dz
    return e
```

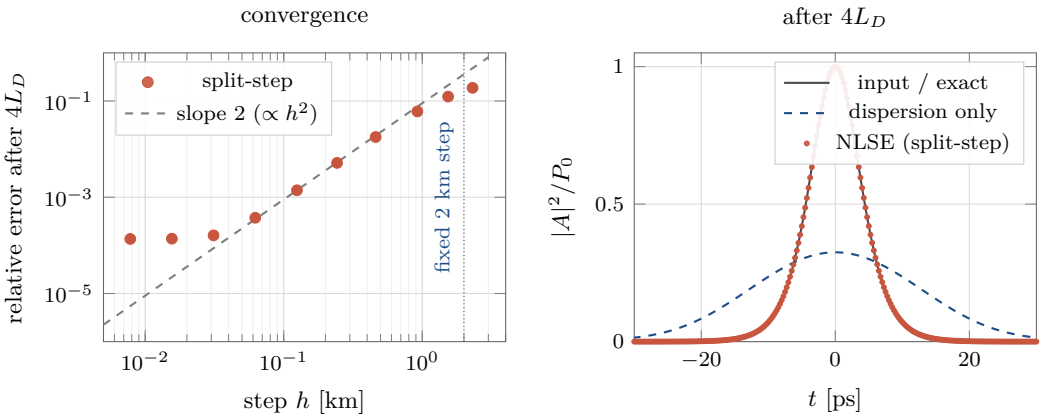


Figure 15.5: Validation of the solver on the fundamental soliton, an exact solution of Equation (15.19) with $\alpha = 0$. With $T_0 = 5\,\text{ps}$, $P_0 = |\beta_2|/(\gamma T_0^2) = 0.667\,\text{W}$ and $L = 4.61\,\text{km} = 4L_D$, the pulse must return unchanged. Left: the relative L^2 error falls as h^2, confirming the order of Equation (15.22). Right: dispersion alone would broaden the pulse, while the balance with SPM preserves it.

Choosing the step. A fixed step suited to 0 dBm per channel, where the nonlinear length $1/(\gamma P)$ is hundreds of km, is far too long for the 28 dBm soliton of Figure 15.5, whose nonlinear length is about 1 km; a fixed 2 km step leaves a 12 % error there. The solver therefore uses the nonlinear-phase-rotation criterion of Sinkin et al. [33]: each step is limited so that the peak nonlinear phase $\gamma P_{\text{peak}} L_{\text{eff}}$ stays below `nonlinear_phase_rad` (0.05 rad by default), and never exceeds $1/$`steps_per_km`. The step lengthens automatically as the power decays. With the defaults the soliton error falls to 3×10^{-4}, while WDM links keep 1–2 km steps. Without `nonlinear` the solver applies one exact FFT for the whole span.

15.3 Semiconductor lasers

15.3.1 Absorption, stimulated and spontaneous emission

Lasers, photodiodes and amplifiers are quantum systems with (at least) two energy levels. Three processes occur, with probabilities that depend on the conditions:

- **absorption**: a photon of energy $hf = E_2 - E_1$ lifts a carrier to the upper level and disappears (the principle of photodiodes);
- **stimulated emission**: a photon makes an excited carrier drop and creates a second photon with the same frequency, phase, polarization and direction, coherently amplifying the field (the principle of lasers and amplifiers); it dominates absorption only with a population inversion, obtained by pumping;
- **spontaneous emission**: an excited carrier drops by itself and emits a photon with random phase and direction, the source of optical noise.

In a semiconductor laser a forward current injects carriers into the active region of a p-n junction; spontaneous emission starts the process, stimulated emission amplifies it, and the optical feedback of the cavity (cleaved facets, a distributed Bragg grating in DFB and DBR lasers, an external filter in ECLs) acts as a narrow filter $|H(f)|^2$ on the broad gain spectrum, so that emission occurs at a single frequency f_c. Fabry–Perot lasers, with several longitudinal modes, are not used for high-quality links.

15.3.2 The CW spectrum: relative intensity noise and phase noise

Ideally a laser in continuous-wave (CW) operation emits $E(t) = a\,e^{j\phi_0}$, a line at f_0. Real lasers have

$$E(t) = (a + n_{\mathrm{RIN}}(t))e^{j\phi_0(t)}, \tag{15.24}$$

with relative intensity noise (RIN) on the amplitude and phase noise on the phase. In good lasers the line shape is dominated by phase noise and specified by its full width at half maximum, the *linewidth* $\Delta\nu$. The simplest model makes the instantaneous frequency a white Gaussian process, so the phase is a Wiener process (Equation (8.2)),

$$\phi_k = \phi_{k-1} + \mathcal{N}\Big(0, \frac{2\pi\Delta\nu}{f_s}\Big), \tag{15.25}$$

which produces a Lorentzian spectrum; an additional $1/f$ (flicker) frequency noise would make it Gaussian. Typical values: DFB lasers 10–20 dBm with linewidths of a few MHz, external-cavity lasers 100–300 kHz (below 100 kHz for coherent transceivers, $\delta f/f_0 \approx 10^{-9}$), special lasers for metrology and quantum communications 1–10 Hz. RIN is the power noise referred to the average power, $\sigma_P^2 = \mathrm{RIN} \cdot \bar{P}^2 \cdot B$; the blocks add it as Gaussian noise over the simulated bandwidth $f_s/2$, as in the IEEE 802.3 link model [24].

Code 15.5: Wiener phase noise of a laser of linewidth $\Delta\nu$. linkstudio/phy/optical.py, lines 51–53

```
51 if linewidth <= 0:
52     return np.zeros((batch, n))
53 return np.cumsum(rng.standard_normal((batch, n)) * np.sqrt(2 * np.pi * linewidth / fs),
       axis=-1)
```

Code 15.6: Relative intensity noise referred to the average power. linkstudio/phy/optical.py, lines 59–61

```
59 if rin_db_hz is None:
60     return power
```

```
return power + p_avg * np.sqrt(10 ** (rin_db_hz / 10) * fs / 2) * rng.standard_normal(power.
    shape)
```

15.3.3 The rate equations

A single-mode laser is described by the number N of carriers in the active region, the number S of photons in the lasing mode and the optical phase ϕ [26]:

$$\frac{dN}{dt} = \frac{I - I_l}{q} - \frac{N}{\tau_n} - G(S)\,(N - N_T)\,S, \tag{15.26}$$

$$\frac{dS}{dt} = \beta\frac{N}{\tau_n} + G(S)\,(N - N_T)\,S - \frac{S}{\tau_p}, \qquad G(S) = \frac{g_0}{1 + \epsilon S}, \tag{15.27}$$

$$\frac{d\phi}{dt} = \frac{\alpha_H}{2}\Big(g_0(N - N_T) - \frac{1}{\tau_p}\Big). \tag{15.28}$$

The terms are: injection of the current I minus a leakage I_l; carrier recombination with lifetime τ_n; stimulated emission with gain coefficient g_0 above the transparency carrier number N_T, compressed by ϵ at high photon numbers; the fraction β of spontaneous emission that falls into the lasing mode; photon loss through the mirrors and absorption with lifetime τ_p. The output power is the emitted share η_{opt} of the photon loss rate,

$$P = \eta_{\text{opt}}\,\frac{h f\, S}{\tau_p}, \tag{15.29}$$

and the phase equation, with the linewidth-enhancement (Henry) factor α_H, expresses that a change of carrier density changes the refractive index together with the gain. All quantities are numbers rather than densities, as in the circuit models of VCSELs [41, 40].

Steady state and the P-I curve. With $dN/dt = dS/dt = 0$, $\beta = 0$ and $\epsilon = 0$, the photon equation either has $S = 0$ or clamps the gain at the losses, $g_0(N - N_T) = 1/\tau_p$. Above threshold the carrier number stays at

$$N_{\text{th}} = N_T + \frac{1}{g_0\tau_p}, \qquad I_{\text{th}} = \frac{qN_{\text{th}}}{\tau_n} + I_l, \tag{15.30}$$

and every additional carrier becomes a photon:

$$P = \eta_{\text{opt}}\frac{hf}{q}\,(I - I_{\text{th}}). \tag{15.31}$$

This is the linear P-I curve of Figure 15.8: below threshold the laser behaves like an LED (spontaneous emission only), above it the slope efficiency is $\eta_{\text{opt}}hf/q$ (W/A), and at high current gain compression and heating bend it down. With $\beta > 0$ the threshold is soft. The test suite checks the threshold and the slope exactly.

In general (with β, ϵ and the spatial terms of Section 15.4) the steady state is found from the photon number: for a given S the two stationary equations for the carriers are linear, so N follows in closed form, and the current that sustains it is $I = q(N/\tau_n + G(S)(N - N_T)S) + I_l$. This current grows monotonically with S, and a bisection on $\ln S$ inverts it.

Code 15.7: Carriers that hold the photon number constant. `linkstudio/phy/lasers.py`, lines 105–109

```
g = re.g0 / (1 + re.epsilon * s)
k = g * s * re.h2 / (g * s * re.h3 + (1 + re.diffusion) / re.tau_n)  # N1 = k (N0 - N_T)
c = re.h0 - re.h1 * k
n0 = (s / re.tau_p + g * s * c * re.n_tr) / (re.beta / re.tau_n + g * s * c)
return n0, k * (n0 - re.n_tr), c * (n0 - re.n_tr)
```

Large-signal response. The blocks integrate Equation (15.28) in time with fourth-order Runge–Kutta, using $\ln S$ instead of S (the photon number spans many decades between below and above threshold) and sub-steps no longer than τ_p. The drive current is linearly interpolated between samples. Each frame starts from the static state; before the frame, the integrator runs over its last $3\tau_n$, so that it starts in the state in which it ends, as the circular FFTs downstream require. When numba is installed the loop is compiled (about 20 ms per frame of a few thousand samples); otherwise the same code runs in NumPy.

Code 15.8: Right-hand side of the rate equations (single mode and VCSEL). linkstudio/phy/lasers.py, lines 142–149

```
s = np.exp(y)
g = g0 / (1.0 + eps * s)
r0 = (n0 - n_tr) * h0 - n1 * h1
dn0 = i / Q_E - n0 / tau_n - g * r0 * s
dn1 = g * ((n0 - n_tr) * h2 - n1 * h3) * s - n1 * (1.0 + hd) / tau_n
dy = beta * n0 / (tau_n * s) + g * r0 - 1.0 / tau_p
dphi = 0.5 * alpha * g0 * (r0 - r_bias)
return dn0, dn1, dy, dphi
```

Small-signal response. Around a bias point the rate equations can be linearized, $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x} + \boldsymbol{b}\,\delta I$, with $\boldsymbol{x}$ the deviations of the carriers and photons, $\mathbf{A}$ the Jacobian and $\boldsymbol{b} = (1/q, 0, 0)^{\mathsf{T}}$. The modulation response is

$$H(j\omega) = \frac{\boldsymbol{e}_S^{\mathsf{T}}(j\omega\mathbf{I} - \mathbf{A})^{-1}\boldsymbol{b}}{\boldsymbol{e}_S^{\mathsf{T}}(-\mathbf{A})^{-1}\boldsymbol{b}}. \tag{15.32}$$

For the single-mode laser it reduces to the classic damped resonance

$$H(f) \approx \frac{f_r^2}{f_r^2 - f^2 + jf\gamma/(2\pi)}, \qquad (2\pi f_r)^2 \approx \frac{g_0 S}{\tau_p(1+\epsilon S)}, \tag{15.33}$$

$$\gamma \approx K f_r^2 + \frac{1}{\tau_n}, \qquad K = 4\pi^2\Big(\tau_p + \frac{\epsilon}{g_0}\Big), \tag{15.34}$$

so the relaxation frequency grows with the square root of the photon number (of $I - I_{\mathrm{th}}$), the damping grows with f_r^2 (the K-factor), and the bandwidth saturates. The RC parasitics of the chip add a pole, $1/(1 + jf/f_p)$ (`parasitic_ghz`), applied to the drive current. This is the response the inspector plots for `dml` and `vcsel`.

Code 15.9: Small-signal response from the linearized rate equations. linkstudio/phy/lasers.py, lines 236–243

```
n0, n1, s = (v[0] for v in steady_state(re, current))
A = jacobian(re, n0, n1, s)
b = np.array([1 / Q_E, 0.0, 0.0])
w = 2j * np.pi * np.asarray(f, dtype=float)
M = w[:, None, None] * np.eye(3) - A
x = np.linalg.solve(M, np.broadcast_to(b, (w.size, 3))[..., None])[..., 0]
dc = np.linalg.solve(-A, b)
return x[:, 2] / dc[2]
```

15.3.4 Chirp of a directly modulated laser

Changing the current changes the carrier density, hence the index and the optical phase (and the temperature, on longer time scales). The phase equation Equation (15.28) gives the frequency deviation. Using the photon equation to eliminate the gain and neglecting spontaneous emission,

$g_0(N - N_T) - 1/\tau_p \approx d\ln S/dt + \epsilon S/\tau_p$, hence

$$\Delta\nu(t) = \frac{\alpha_H}{4\pi}\Big(\frac{d}{dt}\ln P(t) + \kappa P(t)\Big), \qquad \kappa = \frac{\epsilon}{\eta_{\text{opt}} h f}, \tag{15.35}$$

the Koch–Bowers relation [35]. The first term, the *transient* chirp, follows the edges of the power; the second, the *adiabatic* chirp, is a frequency offset proportional to the power, $\alpha_H\kappa/(4\pi)$, about 1.1 GHz/mW for the default DFB. In the block the carrier frequency is defined at the mean current, so the adiabatic term moves the "1" and "0" levels to opposite sides of the carrier (Figure 15.6). The test suite checks the adiabatic step against $\frac{\alpha_H}{4\pi}\epsilon(S_1 - S_0)/\tau_p$ to 2 %.

A bit-synchronous phase modulation broadens the optical spectrum considerably, so a directly modulated laser (DML) is far more sensitive to chromatic dispersion than an external modulator. This is why low-cost high-rate DMLs work in the O band, near the zero-dispersion wavelength: the higher loss and the lack of amplifiers do not matter over 2–40 km, and data-center links use millions of them. The *56 Gb/s PAM-4 with a directly modulated laser* scenario shows it: error-free over 10 km at 1310 nm, closed within 2 km at 1550 nm.

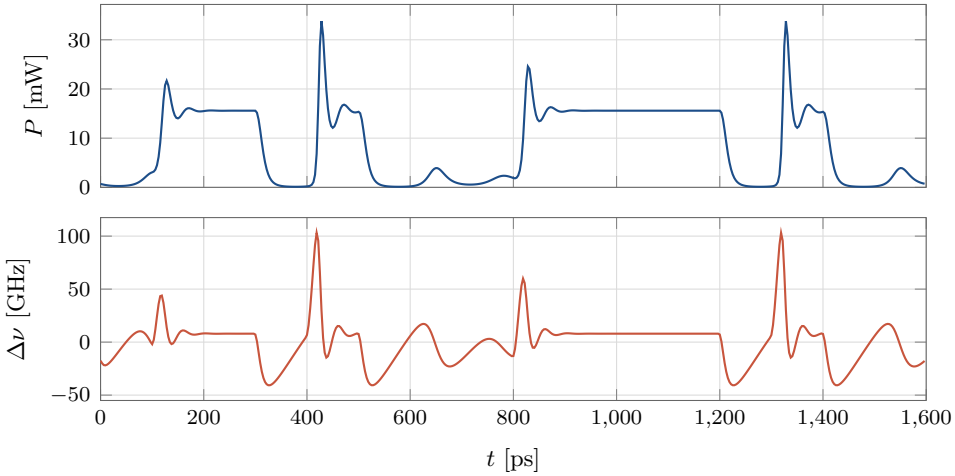


Figure 15.6: Output of `dml` for a 10 Gb/s pattern (40 mA bias, ±25 mA, 25 GHz parasitic pole): relaxation-oscillation ringing on the power, transient chirp spikes on the edges and adiabatic chirp separating the frequencies of the two levels.

15.3.5 The directly modulated laser block

`dml` turns the normalized drive $x \in [-1, 1]$ into the current $I = I_{\text{bias}} + I_{\text{mod}}x$, passes it through the parasitic pole and integrates the rate equations. The defaults describe a 25G-class O-band DFB: threshold 10 mA, 3-dB bandwidth about 28 GHz at 60 mA, $\alpha_H = 4$. The metrics are the threshold current, the average power and the static extinction ratio $P(I_{\text{bias}}{+}I_{\text{mod}})/P(I_{\text{bias}}{-}I_{\text{mod}})$. Directly modulated lasers have low extinction ratios (below 8 dB, 3–4 dB for the fastest), which costs sensitivity (Section 15.7.2); their PAM levels are characterized by the outer optical modulation amplitude, $\text{OMA} = P_{\text{high}} - P_{\text{low}}$. For example, a PAM-4 DML with levels 1, 2.5, 4 and 5.5 mW has $\bar{P} = 3.25$ mW (5.1 dBm), ER = 7.4 dB and OMA = 4.5 mW. The best DMLs reach 40–50 GBd and cost an order of magnitude less than externally modulated transmitters.

Code 15.10: Drive current, rate equations and noise of the laser blocks. `linkstudio/phy/lasers.py`, lines 337–345

```
337 current = p.bias_ma * 1e-3 + p.modulation_ma * 1e-3 * drive
338 if p.parasitic_ghz:
339     current = apply_frequency_response(current, self.parasitic, fs, pad=256)
```

```
340 leak, temp = self.operating_point()
341 preroll = int(round(3 * self.re.tau_n * fs))
342 power, phase = simulate(self.re, np.maximum(current - leak, 0.0), fs, preroll=preroll)
343 p_avg = float(np.mean(power))
344 power = np.maximum(intensity_noise(self.rng, power, p_avg, p.rin_db_hz, fs), 0.0)
345 phase = phase + wiener_phase(self.rng, *power.shape, p.linewidth_hz, fs)
```

What the laser blocks leave out The rate equations are deterministic: RIN and linewidth are added as separate noise terms with the measured values instead of Langevin forces, and the linewidth is not tied to α_H and β. The DFB has no thermal model (a TEC holds its temperature), so the thermal part of the chirp is absent. Mode competition between longitudinal modes is not modelled.

15.4 The VCSEL

A vertical-cavity surface-emitting laser (VCSEL) has its cavity along the growth direction, between two distributed Bragg reflectors (DBRs), with an oxide aperture that confines current and light. The cavity is so short that only one longitudinal mode fits; the planar structure gives a circular beam that couples well into fiber, allows on-wafer testing and two-dimensional arrays, and the small volume gives a high modulation bandwidth at low current. VCSELs at 850 nm are the cheapest lasers for multimode data-center links. Two physical effects shape their behaviour and are missing from the single-mode rate equations [40]:

- **self-heating**: poor heat dissipation and the resistance of the DBRs heat the device, raising the threshold and rolling the output power over at high current;
- **spatial effects**: the transverse profiles of carriers and photons differ; the lasing mode depletes the carriers at the centre (spatial hole burning) and lateral diffusion refills them, which causes mode competition and secondary pulsations at turn-off.

`vcsel` implements the circuit-level model of Shi, Qi, Wang and Hu [40], built on the spatially independent rate equations of Morikuni et al. [42] and the comprehensive model of Mena et al. [41].

15.4.1 Spatially dependent rate equations

Letting the carrier and photon numbers depend on the radial position r in the active disc of radius R,

$$\frac{\partial N(r,t)}{\partial t} = \frac{I(r,t)}{q} - \frac{N(r,t)}{\tau_n} - G(r,t)S(r,t), \tag{15.36}$$

$$\frac{\partial S(r,t)}{\partial t} = \beta\frac{N(r,t)}{\tau_n} + G(r,t)S(r,t) - \frac{S(r,t)}{\tau_s}, \qquad G(r,t) = \frac{G_0\left[N(r,t) - N_T\right]}{1 + \epsilon S_0}, \tag{15.37}$$

with S_0 the magnitude of the photon distribution and τ_s the photon lifetime. Solving these needs a discretization in space and time, too heavy for system simulation.

15.4.2 Bessel expansion and the reduced equations

With cylindrical symmetry the carrier profile is expanded in the eigenfunctions of the disc and the photon profile is fixed:

$$N(r,t) = N_0(t) - \sum_{i\geq 1} N_i(t)\, J_0\!\left(\frac{\sigma_i r}{R}\right), \qquad S(r,t) = S_0(t)\,\psi(r), \tag{15.38}$$

where J_0 is the Bessel function of order zero and σ_i the i-th root of J_1 ($\sigma_0 = 0$, $\sigma_1 = 3.8317$, so that the radial carrier flux vanishes at the edge). The functions are orthogonal on the disc,

$$\int_0^R J_0\Big(\frac{\sigma_m r}{R}\Big) J_0\Big(\frac{\sigma_n r}{R}\Big) r\,dr = \begin{cases} 0 & m \neq n \\ \frac{R^2}{2} J_0^2(\sigma_m) & m = n, \end{cases} \tag{15.39}$$

so projecting the carrier equation on each term gives one ordinary equation per coefficient. Keeping two terms (N_0, the mean carrier number, and N_1, the depth of the hole burnt by the mode) and a uniform current leaves three equations:

$$\frac{dN_0}{dt} = \frac{I - I_l}{q} - \frac{N_0}{\tau_n} - \frac{G_0}{1 + \epsilon S_0}\big[(N_0 - N_T)h_0 - N_1 h_1\big] S_0, \tag{15.40}$$

$$\frac{dN_1}{dt} = \frac{G_0}{1 + \epsilon S_0}\big[(N_0 - N_T)h_2 - N_1 h_3\big] S_0 - \frac{N_1}{\tau_n}(1 + h), \tag{15.41}$$

$$\frac{dS_0}{dt} = \beta\frac{N_0}{\tau_n} + \frac{G_0}{1 + \epsilon S_0}\big[(N_0 - N_T)h_0 - N_1 h_1\big] S_0 - \frac{S_0}{\tau_s}. \tag{15.42}$$

Lateral diffusion with coefficient D turns into the extra decay h/τ_n of the hole, with $h = (\sigma_1 L_D/R)^2$ and the diffusion length $L_D = \sqrt{D\tau_n}$. The coefficients h_i are overlap integrals between the photon and carrier distributions:

$$h_0 = \frac{2}{R^2}\int_0^R \psi(r)\, r\,dr, \qquad h_1 = \frac{2}{R^2}\int_0^R \psi(r) J_0\Big(\frac{\sigma_1 r}{R}\Big) r\,dr, \tag{15.43}$$

$$h_2 = \frac{2}{R^2 J_0^2(\sigma_1)}\int_0^R \psi(r) J_0\Big(\frac{\sigma_1 r}{R}\Big) r\,dr, \qquad h_3 = \frac{2}{R^2 J_0^2(\sigma_1)}\int_0^R \psi(r) J_0^2\Big(\frac{\sigma_1 r}{R}\Big) r\,dr, \tag{15.44}$$

with $J_0(\sigma_1) = -0.4028$. For the Gaussian mode (zero-order Laguerre–Gauss beam of waist R_m)

$$\psi(r) = 2\Big(\frac{R}{R_m}\Big)^2 \exp\Big(-\frac{2r^2}{R_m^2}\Big), \qquad h_0 = 1 - e^{-2R^2/R_m^2}, \tag{15.45}$$

and the other integrals are evaluated numerically. For the default device ($R = 3.5\,\mu\text{m}$, $R_m = 2.5\,\mu\text{m}$, $L_D = 1.5\,\mu\text{m}$) $h_0 = 0.980$, $h_1 = 0.399$, $h_2 = 2.459$, $h_3 = 1.964$ and $h = 2.70$. With $h_1 = h_2 = h_3 = 0$ and $h_0 = 1$ the equations reduce to the single-mode laser of Section 15.3, which is how the code handles both. The test suite checks the closed form of h_0 and the narrow-mode limits $h_1 \to 1 - \sigma_1^2 R_m^2/(8R^2)$, $h_3 \to h_2$.

Code 15.11: Overlap integrals of the Gaussian mode with the Bessel modes. linkstudio/phy/lasers.py, lines 90–99

```
R, Rm = active_radius_um, mode_radius_um
r = np.linspace(0.0, R, 4001)
psi = 2 * (R / Rm) ** 2 * np.exp(-2 * r ** 2 / Rm ** 2)
b = j0(SIGMA1 * r / R)
norm = 2 / R ** 2
jn2 = j0(SIGMA1) ** 2
h0 = norm * np.trapezoid(psi * r, r)
h1 = norm * np.trapezoid(psi * b * r, r)
h3 = norm / jn2 * np.trapezoid(psi * b ** 2 * r, r)
return float(h0), float(h1), float(h1 / jn2), float(h3)
```

Turn-off transient. When the current drops below threshold the photons disappear in a few photon lifetimes, but the carriers lost to hole burning are then refilled by diffusion from the edges of the disc. The carrier number in the mode region momentarily exceeds threshold again and the laser emits a secondary pulse. Figure 15.7 reproduces the square-wave experiment of the paper (0.5 and 5 mA): with hole burning the light lingers and rings after the falling edge, while the uniform model switches off at once. The test suite checks that the emitted energy after turn-off is larger with hole burning.

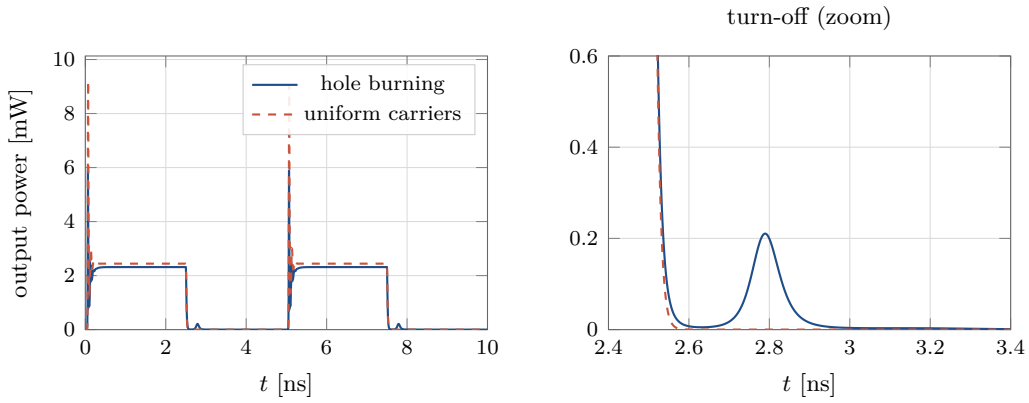


Figure 15.7: Response of `vcsel` to a square current between 0.5 and 5 mA (thermal model and parasitics off), after Fig. 2 of Shi et al. [40]. Right: after the falling edge at 2.5 ns, diffusion refills the burnt hole and the laser emits a secondary pulse 0.3 ns later.

15.4.3 Tucker's thermal model

As the device heats up, the bandgap of the active layer shrinks and the active region no longer confines the carriers: a thermal leakage current, modelled as a polynomial in temperature,

$$I_l(T) = \sum_{i=1}^{n} a_i T^{i-1}, \tag{15.46}$$

is lost from the injection. The power dissipated as heat is the electrical power minus the light, $\Delta P = VI - P_L$, and the junction temperature follows the thermal rate equation

$$T = T_0 + R_{\mathrm{th}} \Delta P - \tau_{\mathrm{th}} \frac{dT}{dt}, \tag{15.47}$$

with ambient temperature T_0, thermal impedance R_{th} and thermal time constant $\tau_{\mathrm{th}} \approx 1\,\mu\mathrm{s}$. The electrical characteristic is that of a diode with series resistance,

$$V = nV_T \ln\left(\frac{I}{I_{ss}} + 1\right) + IR_s, \qquad R_s(T) = \frac{c_1}{T - c_2}, \qquad I_{ss}(T) = b_1 I_{ss0}(T - b_2), \tag{15.48}$$

with ideality factor n, thermal voltage $V_T = kT/q$ and temperature-dependent series resistance and saturation current. In a SPICE implementation the diode, the photon storage ($C_{ph} = qS_c$) and loss ($R_{ph} = \tau_{ph}/C_{ph}$) become circuit elements [40]; LinkStudio solves the same equations directly.

Code 15.12: Diode characteristic with temperature-dependent parameters. `linkstudio/phy/lasers.py`, lines 267–270

```
267 i, t = np.asarray(i, dtype=float), np.asarray(t, dtype=float)
268 vt = K_BOLTZMANN * (t + 273.15) / Q_E
269 iss = self.b1 * self.iss0 * (t - self.b2)
270 return self.ideality * vt * np.log(np.maximum(i, 0.0) / iss + 1) + i * self.c1 / (t - self.
        c2)
```

In the code The thermal time constant is three orders of magnitude longer than a frame (a few ns), so the temperature cannot follow the bits: it settles at the operating point set by the mean current. `vcsel` solves $T = T_0 + R_{\mathrm{th}}(V(I,T)I - P(I - I_l(T)))$ at the bias current by damped fixed-point iteration, subtracts the resulting leakage from the drive and reports the junction temperature as a metric. The same solver at every current gives

the static L-I curves (`light_current`).

Code 15.13: Static operating point of Tucker's thermal model. `linkstudio/phy/lasers.py`, lines 280–292

```
i = np.atleast_1d(np.asarray(current, dtype=float))
t = np.full(i.shape, th.ambient_c)
for _ in range(iterations):
    il = th.leakage_current(t)
    p = static_power(re, i - il)
    target = th.ambient_c + th.r_th * (th.voltage(i, t) * i - p)
    t_new = 0.5 * t + 0.5 * target
    if np.max(np.abs(t_new - t)) < 1e-6:
        t = t_new
        break
    t = t_new
il = th.leakage_current(t)
return t, il, static_power(re, i - il)
```

Figure 15.8 shows the static L-I curves at four ambient temperatures, the counterpart of Fig. 1 of the paper. The threshold grows with temperature because the leakage adds to the threshold current; at high current the self-heating raises the leakage faster than the current, and the power rolls over. At 80 °C the peak power is below a third of its 25 °C value. The default device (25G class, threshold 0.55 mA plus leakage, $R_{\mathrm{th}} = 3\,\mathrm{K/mW}$) runs at 55 °C at its 7 mA bias, with 0.34 mA of leakage.

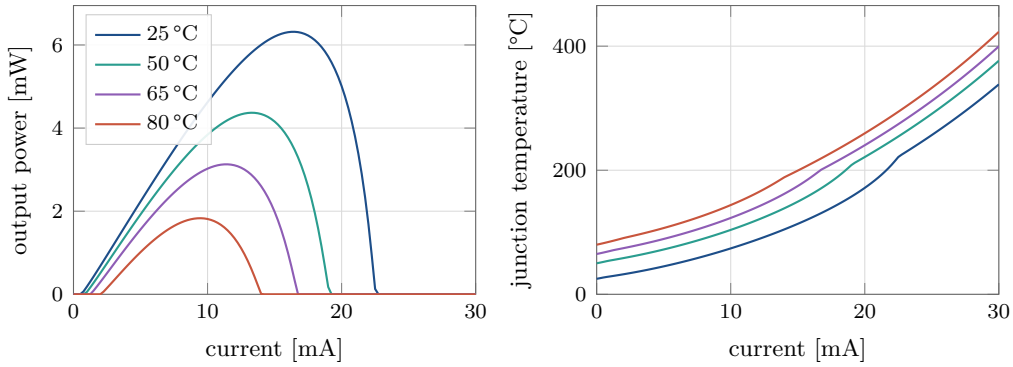


Figure 15.8: Static L-I curves (left) and junction temperature (right) of the default `vcsel` at four ambient temperatures: thermal leakage raises the threshold and rolls the power over.

15.4.4 Modulation response

Linearizing the three equations gives the small-signal response including hole burning and diffusion (Figure 15.9). The resonance moves up and the damping grows with the bias; above about 8 mA the response is overdamped. The markers are the amplitude of the simulated power when the block is driven by a small sinusoid: they lie on the linearized response within 0.01 dB, which validates the integrator and the Jacobian together.

VCSEL parameters Shi et al. take their parameters from Mena et al. [41] without listing them. The defaults of `vcsel` are a self-consistent 25G-class 850 nm device: carrier and photon numbers from an aperture of 7 µm and three quantum wells, gain coefficient chosen for a 15 GHz relaxation frequency at 7 mA, slope efficiency 0.55 W/A, and a cubic leakage polynomial ($I_l = 2 \times 10^{-9} T^3$ A, T in °C) that reproduces the qualitative behaviour of the paper. Fit the parameters to measured L-I and S_{21} curves before drawing quantitative

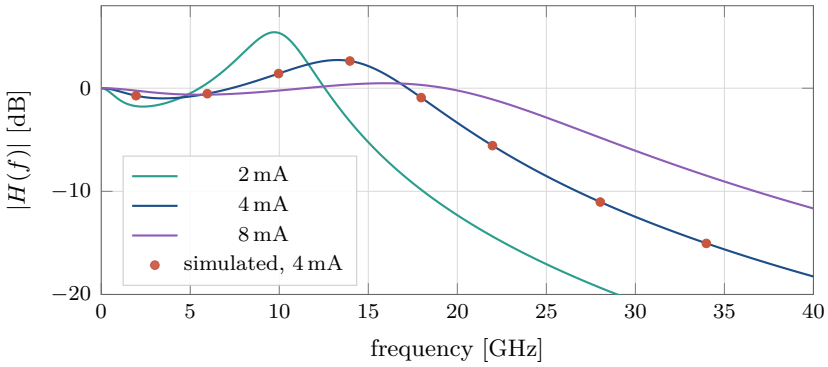


Figure 15.9: Small-signal modulation response of the VCSEL rate equations (hole burning on, thermal model and parasitic pole off) at three bias currents, with large-signal simulation points.

conclusions. The model also keeps a single transverse mode for the photons; the multimode spectrum enters only through `spectral_width_nm` in the multimode fiber.

15.5 External modulators

15.5.1 External modulation

In external modulation the laser runs in CW at constant current, emitting $E_{CW}(t) = \sqrt{P_{CW}}\, e^{j\phi_0}$, and a separate device driven by the electrical signal $V_{\mathrm{mod}}(t)$ imposes the modulation,

$$E_{\mathrm{mod}}(t) = \sqrt{P_{CW}}\, e^{j\phi_0}\, f\big(V_{\mathrm{mod}}(t)\big). \tag{15.49}$$

Most modulators exploit a voltage-dependent refractive index. In a lithium-niobate ($LiNbO_3$) waveguide the electro-optic effect makes the phase linear in the applied voltage, $E_{\mathrm{out}} = E_{\mathrm{in}} e^{jkV}$: a few centimetres of waveguide are an almost perfect phase modulator.

15.5.2 The Mach–Zehnder modulator

Two such phase modulators in the arms of a Mach–Zehnder interferometer, with 3-dB couplers, give

$$E_{\mathrm{out}} = \frac{E_{\mathrm{in}}}{2}\big(e^{jkV_1} + e^{jkV_2}\big) = E_{\mathrm{in}}\, e^{jk\frac{V_1+V_2}{2}} \cos\Big(k\frac{V_1 - V_2}{2}\Big)\,: \tag{15.50}$$

the difference voltage modulates the amplitude, the sum voltage the phase. In *push-pull* operation ($V_1 = -V_2 = V/2$) the phase term vanishes and the modulation is purely amplitude, *chirp-free*. The voltage that moves the output from minimum to maximum is the switching or half-wave voltage V_π ($k = \pi/V_\pi$). With a static bias that shifts one arm by $-\pi/2$ relative to the other, the null is at $V = 0$ and the characteristic is the sine law

$$E_{\mathrm{out}} = E_{\mathrm{in}} \sin\Big(\frac{\pi}{2}\frac{V}{V_\pi}\Big), \qquad P_{\mathrm{out}} = P_{\mathrm{in}} \sin^2\Big(\frac{\pi}{2}\frac{V}{V_\pi}\Big) = \frac{P_{\mathrm{in}}}{2}\Big[1 - \cos\Big(\pi\frac{V}{V_\pi}\Big)\Big] \tag{15.51}$$

(Figure 15.10). The field characteristic is odd and spans $\pm E_{\mathrm{in}}$ for $V \in [-V_\pi, V_\pi]$: biased at null it modulates the field (coherent formats). The power characteristic is a raised cosine: biased at quadrature ($V_\pi/2$) with a swing of $\pm V_\pi/2$ it goes from null to peak, as needed for OOK and PAM-M. The drive of an OOK transmitter is

$$V(t) = V_{\mathrm{bias}} + \frac{V_\pi}{2}\sum_k \alpha_k s(t - kT_s), \qquad \alpha_k = \pm 1,\ \max s(t) = 1, \tag{15.52}$$

and the average output power is $(P^1_{\mathrm{out}} + P^0_{\mathrm{out}})/2$. For PAM-4 the four voltage levels are placed on the $\sin^2$ curve so that the powers are equally spaced: $V_i = \frac{2V_\pi}{\pi}\arcsin\sqrt{P_i/P_{\mathrm{in}}}$.

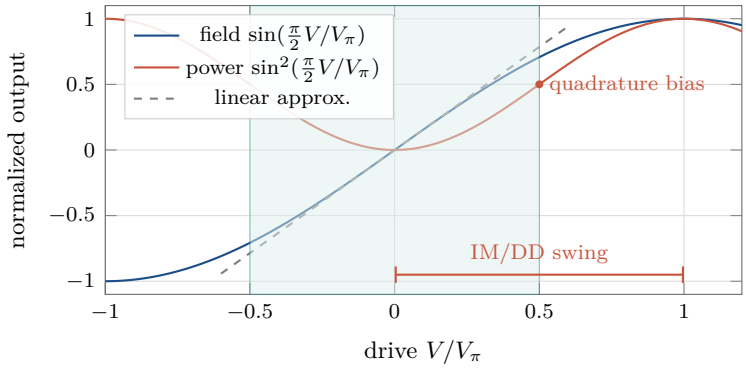


Figure 15.10: Sine-law characteristic of a push-pull MZM. Coherent transmitters bias at null and drive the field (shaded: the default IQ drive swing of $0.5\,V_\pi$); IM/DD transmitters bias at quadrature and swing over half the period of the power.

Parameters. Commercial lithium-niobate MZMs have $V_\pi = 3$–$5\,\mathrm{V}$, extinction ratios $\mathrm{ER} = P^1_{\mathrm{out}}/P^0_{\mathrm{out}}$ of 15–20 dB (up to 30 dB), electro-optic bandwidths of tens of GHz (60–100 GHz for the best) and insertion losses L of 3–5 dB, so $P^1_{\mathrm{out,dBm}} = P_{\mathrm{laser,dBm}} - L_{\mathrm{dB}}$. A 45 mW (16.5 dBm) DFB, an MZM with 4 dB loss and full-swing OOK give an average of $16.5 - 4 - 3 = 9.5$ dBm. The extinction ratio is finite because the arms are never exactly balanced. The electrodes, package and driver limit the bandwidth; their effect is an equivalent electrical response $H_{\mathrm{mod}}(f)$ applied to the drive before the electro-optic conversion. MZMs cost around a thousand euros, largely because of the drivers.

Electro-absorption modulators. An EAM absorbs light below a cutoff wavelength that an applied voltage shifts (Franz–Keldysh effect, quantum-confined Stark effect). It is almost chirp-free, can be integrated with a DFB laser (EML) and is common for 20–40 km links. It is modelled by `intensity_modulator`.

15.5.3 The modulator blocks

`mach_zehnder_modulator` combines a CW laser (power, linewidth, RIN) with an MZM. The drive $x \in [-1, 1]$, filtered by a first-order H_{mod} (`bandwidth_ghz`), sets $V = V_{\mathrm{bias}} + V_{\mathrm{swing}}x$, with quadrature bias and $V_\pi/2$ swing by default. A finite extinction ratio comes from unequal arm amplitudes $(1 \pm \delta)/2$:

$$\frac{E_{\mathrm{out}}}{E_{\mathrm{in}}} = \frac{1}{j}\left(\frac{1+\delta}{2}e^{j\theta} - \frac{1-\delta}{2}e^{-j\theta}\right) = \sin\theta - j\delta\cos\theta, \qquad \theta = \frac{\pi V}{2V_\pi}, \quad \delta = 10^{-\mathrm{ER}/20}, \tag{15.53}$$

so $|E_{\mathrm{out}}|^2 = \sin^2\theta + \delta^2\cos^2\theta$ swings between δ^2 and 1, and the imbalance leaves a small residual chirp. With `drive=single_arm` ($V_1 = V$, $V_2 = 0$) the output becomes $e^{j\theta}\sin\theta$: the phase follows the drive, the chirp of a single-drive modulator. The metrics are the average output power and the modulated extinction ratio.

Code 15.14: Field transfer of the MZM with arm imbalance and drive mode.
linkstudio/phy/optical.py, lines 128–132

```
d = 0.0 if extinction_ratio_db is None else 10 ** (-extinction_ratio_db / 20)
theta = np.pi * v / (2 * v_pi)
if push_pull:
    return ((1 + d) / 2 * np.exp(1j * theta) - (1 - d) / 2 * np.exp(-1j * theta)) / 1j
return ((1 + d) / 2 * np.exp(2j * theta) - (1 - d) / 2) / 1j
```

Code 15.15: CW laser and MZM (`mach_zehnder_modulator`). linkstudio/phy/optical.py, lines 229–233

```
bias, swing = self.voltages()
p_cw = dbm_to_w(p.laser_power_dbm) * 10 ** (-p.insertion_loss_db / 10)
e_in = cw_laser(self.rng, drive.shape, p_cw, p.linewidth_hz, p.rin_db_hz, fs)
push = p.drive == "push_pull"
field = e_in * mzm_field(bias + swing * drive, p.v_pi, p.extinction_ratio_db, push)
```

`intensity_modulator` is the ideal transmitter assumed by the IM/DD formulas of Section 15.7: the drive is mapped linearly to the power,

$$P(t) = \bar{P}(1 + m\,x(t)), \qquad m = \frac{\mathrm{ER} - 1}{\mathrm{ER} + 1}, \tag{15.54}$$

so a PAM-M drive with equally spaced levels gives powers equally spaced between P_1 and $P_M = \mathrm{ER}\,P_1$, with no chirp. It stands for a well-linearized external modulator or EML.

Code 15.16: Linear intensity modulation (`intensity_modulator`). linkstudio/phy/optical.py, lines 172–176

```
er = 10 ** (p.extinction_ratio_db / 10)
p_avg = dbm_to_w(p.power_dbm)
power = p_avg * (1 + (er - 1) / (er + 1) * drive)
power = np.maximum(intensity_noise(self.rng, power, p_avg, p.rin_db_hz, fs), 0.0)
field = np.sqrt(power) * np.exp(1j * wiener_phase(self.rng, *power.shape, p.linewidth_hz,
    fs))
```

15.5.4 The IQ modulator and the polarization-multiplexed transmitter

Two MZMs biased at null, nested in a larger interferometer with a 90° phase shift on one branch, modulate the in-phase and quadrature components of one polarization:

$$E_x(t) = E_I(t) + jE_Q(t) \propto \sin\Big(\frac{\pi}{2}\frac{v_I(t)}{V_\pi}\Big) + j\sin\Big(\frac{\pi}{2}\frac{v_Q(t)}{V_\pi}\Big). \tag{15.55}$$

A polarization-multiplexed (PM-QAM) transmitter splits one laser into two IQ modulators and combines their outputs on orthogonal polarizations with a polarization beam combiner (PBS), $\boldsymbol{E}(t) = E_x(t)\hat{x} + E_y(t)\hat{y}$. The transmitter DSP generates the four drive signals $v_{Ix}, v_{Qx}, v_{Iy}, v_{Qy}$ from pulse-shaped constellation points, often pre-distorted for the sine law, and the drivers amplify them to the V_π range. Small swings are nearly linear; large swings compress the outer constellation points (Figure 15.10).

Signal normalization. The four components are pulse trains, $E_{Ix}(t) = \sum_n a_{Ix,n} g(t - nT)$ and so on, combined into the vector symbols $\boldsymbol{a}_n = a_{x,n}\hat{x} + a_{y,n}\hat{y}$ with $a_{x,n} = a_{Ix,n} + j a_{Qx,n}$. With $\mathbb{E}\{|\boldsymbol{a}_n|^2\} = \mathbb{E}|a_{x,n}|^2 + \mathbb{E}|a_{y,n}|^2 = 1$ and a unit-energy pulse, $\int |g|^2 dt = 1$,

$$\boldsymbol{E}(t) = \sqrt{\frac{P_{\mathrm{sig}}}{R_s}} \sum_n \boldsymbol{a}_n g(t - nT) = \sqrt{E_s} \sum_n \boldsymbol{a}_n g(t - nT), \qquad \langle |\boldsymbol{E}(t)|^2 \rangle = P_{\mathrm{sig}}, \tag{15.56}$$

with $E_s = P_{\mathrm{sig}} T$ the energy per (four-dimensional) symbol. For PM-QPSK with power split equally, $\mathbb{E}|a_{x,n}|^2 = \frac{1}{2}$ and the points are $\frac{1}{2}(\pm 1 \pm j)$ on each polarization.

`iq_modulator` implements this transmitter: the complex drive is either mapped linearly to the field (`model=linear`) or passed through the two sine laws with peak swing `drive_swing`$\times V_\pi$, a finite child-MZM extinction ratio and an error of the 90° phase (`quadrature_error_deg`). The output is scaled to the launch power `power_dbm`, as a booster amplifier would, with the power split equally between polarizations, and the laser phase noise of `linewidth_hz` is applied to both.

Code 15.17: Nested MZMs with quadrature error and power normalization (`iq_modulator`).
linkstudio/phy/optical.py, lines 280–289

```
if p.model == "mzm":
    peak = max(np.max(np.abs(d.real)), np.max(np.abs(d.imag))) or 1.0
    ei = mzm_field(p.drive_swing * d.real / peak, 1.0, p.extinction_ratio_db)
    eq = mzm_field(p.drive_swing * d.imag / peak, 1.0, p.extinction_ratio_db)
    d = ei + np.exp(1j * np.pi / 2 * (1 + p.quadrature_error_deg / 90)) * eq
elif p.quadrature_error_deg:
    d = d.real + np.exp(1j * np.pi / 2 * (1 + p.quadrature_error_deg / 90)) * d.imag
rms = np.sqrt(np.mean(np.abs(d) ** 2, axis=_pol_axes(d), keepdims=True))
p_tot = dbm_to_w(p.power_dbm)
field = d / np.maximum(rms, 1e-30) * np.sqrt(p_tot / npol)
```

15.6 Photodetection

15.6.1 PIN photodiodes

A PIN photodiode (p-doped, intrinsic, n-doped) converts photons into conduction electrons by absorption, ideally one electron per photon, so the current is proportional to the power:

$$i(t) = \mathcal{R}\,P(t). \tag{15.57}$$

In CW the photon rate is P/hf and the electron rate $\eta_{ph}P/hf$, with the quantum efficiency $\eta_{ph} \le 1$, so the responsivity is

$$\mathcal{R} = \frac{q}{hf}\eta_{ph} \quad [\mathrm{A/W}]. \tag{15.58}$$

q/hf is 1.25 A/W at 1550 nm, 1.07 A/W at 1330 nm and 0.68 A/W at 850 nm, so $\mathcal{R} \approx 1$, 0.9 and 0.6 A/W for typical efficiencies (the value close to 1 A/W in the C band is a numerical coincidence). The responsivity grows with wavelength up to the bandgap cutoff of the material (Si, Ge, InGaAs) and then drops. `photodiode` takes `responsivity` or computes it from `quantum_efficiency` and the signal wavelength.

The currents are small. A +1 dBm transmitter, 80 km of fiber at 0.22 dB/km and 2 dB of connectors give −18.6 dBm (14 µW) and, with $\mathcal{R} = 0.9$ A/W, 12.4 µA: an electrical amplifier is indispensable.

15.6.2 The transimpedance amplifier and its noise

A direct-detection receiver is a photodiode followed by a transimpedance amplifier (TIA), $V_{TIA}(t) = G_{TIA}\,i(t)$ with G_{TIA} in ohms (1600–2700 Ω for a 10GbE TIA, giving 25 mV from 12.4 µA), a low-pass filter, sampling at $t_i = t_s + iT_s$ and the decision. The TIA noise is well described by an additive Gaussian current $n_I(t)$ at its input. Datasheets give it as

- the input-referred RMS current σ_I over the amplifier bandwidth, or its variance σ_I^2;
- the one-sided input-referred noise density (IRND) in $\mathrm{A}/\sqrt{\mathrm{Hz}}$, the square root of the noise PSD,

related through the equivalent noise bandwidth B_{eq} of the receiver:

$$\sigma_I = \mathrm{IRND}\sqrt{B_{eq}}, \qquad \sigma_I^2 = \mathrm{IRND}^2 B_{eq}. \tag{15.59}$$

Examples: 1100 $\mathrm{nA_{rms}}$ over 6 GHz is $14\,\mathrm{pA}/\sqrt{\mathrm{Hz}}$; 1.4 µA over 21 GHz is $9.7\,\mathrm{pA}/\sqrt{\mathrm{Hz}}$; research TIAs above 35 GHz have 12–25 $\mathrm{pA}/\sqrt{\mathrm{Hz}}$. With a CW signal and TIA noise only, the electrical SNR is

$$\mathrm{SNR}_e = \frac{i_{Rx}^2}{\sigma_{TIA}^2} = \frac{(\mathcal{R}P_{Rx})^2}{\mathrm{IRND}^2 B_{eq}}. \tag{15.60}$$

For the example above ($i_{Rx} = 12\,\mu\text{A}$, $\sigma_I = 1.1\,\mu\text{A}$) it is 20.8 dB: signal and noise are comparable. The TIA gain does not change the SNR (it multiplies both), and the bandwidth cannot be reduced below what the baud rate needs. Lower received powers need better receivers.

15.6.3 Quantum noise and shot noise

Even with a noiseless amplifier, the quantum nature of light limits detection. The number of photons arriving in a window ΔT is a Poisson variable with mean and variance

$$\lambda = \mathbb{E}\{N_{Rx}\} = \text{Var}\{N_{Rx}\} = \frac{P_{Rx}\Delta T}{hf_0}, \tag{15.61}$$

nearly Gaussian above about 10. The quantum-limited SNR is therefore

$$\text{SNR}_Q = \frac{\mathbb{E}\{N\}^2}{\text{Var}\{N\}} = \frac{P_{Rx}\Delta T}{hf_0}, \tag{15.62}$$

an absolute bound for any photodetection. At 10 Gb/s ($\Delta T = 100$ ps) and -20 dBm it is 39.9 dB, against 17 dB for a PIN + TIA receiver with $20\,\text{pA}/\sqrt{\text{Hz}}$ over 5 GHz; the quantum limit would reach 17 dB at -41.9 dBm. In practice the TIA noise dominates by more than 20 dB.

The same discreteness appears in the current as *shot noise*: $I = q\xi$ with ξ the Poisson electron rate gives the one-sided PSD $2qI$ and, over B_{eq}, $\sigma^2_{\text{shot}} = 2qB_{eq}\mathcal{R}P$. With $B_{eq} = 1/(2\Delta T)$ and $\eta_{ph} = 1$, $\text{SNR}_{\text{shot}} = \text{SNR}_Q$. Both sources together give

$$\text{SNR}_{\text{PIN}} = \frac{(\mathcal{R}P_{Rx})^2}{\text{IRND}^2 B_{eq} + 2qB_{eq}\mathcal{R}P_{Rx}}. \tag{15.63}$$

15.6.4 Avalanche photodiodes

An avalanche photodiode (APD) multiplies each photoelectron by M (10–30) in a high-field region, so $\mathcal{R}_{APD} = M\mathcal{R}$ and $i = M\mathcal{R}P$. The multiplication is itself random: the shot noise is amplified by M^2 and increased by the excess noise factor $F(M) > 1$,

$$\sigma^2_{APD} = 2qB_{eq}M^2F\mathcal{R}P_{Rx}, \qquad \text{SNR}_{\text{APD}} = \frac{(\mathcal{R}MP_{Rx})^2}{\text{IRND}^2 B_{eq} + 2qB_{eq}M^2F\mathcal{R}P_{Rx}}. \tag{15.64}$$

McIntyre's theory gives $F = kM + (1-k)(2 - 1/M)$ with the ionization ratio k; datasheets often give the power law $F = M^x$ (InGaAs: $x \approx 0.7$). The gain raises the signal above the TIA noise but the SNR grows less than M^2; for each power there is an optimum M. APDs are better than PINs at low power (Figure 15.11) but more expensive, need a high reverse bias and have less bandwidth (avalanche build-up time). Single-photon avalanche detectors (SPADs), used in quantum communications, operate far beyond breakdown and fire on single photons.

15.6.5 The photodiode block

`photodiode` sums the powers of both polarizations, multiplies by $M\mathcal{R}$ and adds Gaussian noise generated over the simulated band $f_s/2$: TIA noise $\text{IRND}^2 f_s/2$, shot noise $2qM^2F(\mathcal{R}P + I_d)f_s/2$ with the dark current I_d, and the APD excess noise (McIntyre or power law). The receiver bandwidth then sets B_{eq}: with `bandwidth_ghz` a fourth-order Bessel–Thomson response (the reference receiver of optical Ethernet) filters signal and noise, and the reported `noise_var` is scaled by its noise bandwidth $\frac{1}{f_s}\int |H|^2 df$, so the result does not depend on the sample rate. With `transimpedance_ohm` the output is the TIA voltage.

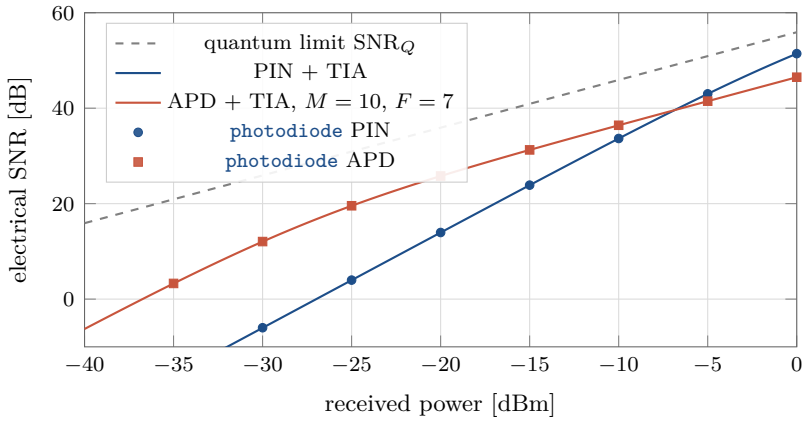


Figure 15.11: Electrical SNR of PIN and APD receivers ($\mathcal{R} = 1\,\mathrm{A/W}$, $20\,\mathrm{pA}/\sqrt{\mathrm{Hz}}$, $B_{eq} = 10\,\mathrm{GHz}$) and the quantum limit for $\Delta T = 1/(2B_{eq})$. The markers are $\bar{i}^2/\operatorname{Var} i$ measured on the output of `photodiode` for a CW input.

Code 15.18: Square-law detection with TIA, shot and APD excess noise. `linkstudio/phy/optical.py`, lines 1005–1010

```
1005 M, F = p.apd_gain, self.excess_noise()
1006 i_primary = self.responsivity_aw(x.meta.get("wavelength_nm", 1550.0)) * power + p.
         dark_current_na * 1e-9
1007 var = (p.thermal_noise_pa_rthz * 1e-12) ** 2 * bw * np.ones_like(power)
1008 if p.shot_noise:
1009     var = var + 2 * Q_E * M ** 2 * F * i_primary * bw
1010 current = M * i_primary + np.sqrt(var) * self.rng.standard_normal(power.shape)
```

15.7 IM/DD system performance

15.7.1 PAM-M with signal-independent noise

IM/DD systems carry the information in M power levels $P_1 < \cdots < P_M$, equally spaced by ΔP, with extinction ratio $\mathrm{ER} = P_M/P_1$. Then

$$P_1 = \frac{(M-1)\Delta P}{\mathrm{ER}-1}, \qquad P_M = \frac{\mathrm{ER}\,(M-1)\Delta P}{\mathrm{ER}-1}, \qquad \bar{P} = \frac{\mathrm{ER}+1}{2(\mathrm{ER}-1)}(M-1)\Delta P, \tag{15.65}$$

so for a given average power the level spacing is

$$\Delta P = \frac{\mathrm{ER}-1}{\mathrm{ER}+1}\,\frac{2\bar{P}}{M-1}. \tag{15.66}$$

The sampled current is $i_k = \mathcal{R}P_k + n$ with Gaussian TIA noise of variance $\sigma^2 = \mathrm{IRND}^2 B_{eq}$, the same on every level (shot noise neglected, as for PIN receivers). With thresholds half-way between levels, every symbol has one or two neighbours at distance $\Delta I/2 = \mathcal{R}\Delta P/2$, and

$$\mathrm{SER} = \frac{M-1}{M}\operatorname{erfc}\left(\sqrt{\frac{\Delta I^2}{8\sigma^2}}\right) = \frac{M-1}{M}\operatorname{erfc}\left(\frac{\mathcal{R}\bar{P}}{M-1}\,\frac{\mathrm{ER}-1}{\mathrm{ER}+1}\,\frac{1}{\mathrm{IRND}\sqrt{2B_{eq}}}\right), \tag{15.67}$$

and with Gray labels $\mathrm{BER} \approx \mathrm{SER}/\log_2 M$. For PAM-2 and PAM-4,

$$\mathrm{BER}_{\mathrm{PAM2}} = \frac{1}{2}\operatorname{erfc}(\gamma\bar{P}), \qquad \mathrm{BER}_{\mathrm{PAM4}} \approx \frac{3}{8}\operatorname{erfc}\left(\frac{\gamma\bar{P}}{3}\right), \qquad \gamma = \frac{\mathcal{R}}{\mathrm{IRND}}\,\frac{\mathrm{ER}-1}{\mathrm{ER}+1}\,\frac{1}{\sqrt{2B_{eq}}}. \tag{15.68}$$

Below 3×10^{-2} the tail $\operatorname{erfc}(y)\approx e^{-y^2}/(\sqrt{\pi}y)$ shows that the BER drops by decades within a few dB. Figure 15.12 compares the chain `intensity_modulator`→`photodiode`→`agc`→`demapper` with Equation (15.67). The *sensitivity* is the average power at the target BER.

Code 15.19: IM/DD PAM-M symbol error rate (`theory.pam_imdd_ser`). `linkstudio/phy/theory.py`, lines 148–150

```
148 er = 10 ** (extinction_ratio_db / 10)
149 dp = 2 * np.asarray(p_avg_w, dtype=float) * (er - 1) / ((er + 1) * (order - 1))
150 return (order - 1) / order * erfc(resp * dp / (2 * np.sqrt(2) * sigma))
```

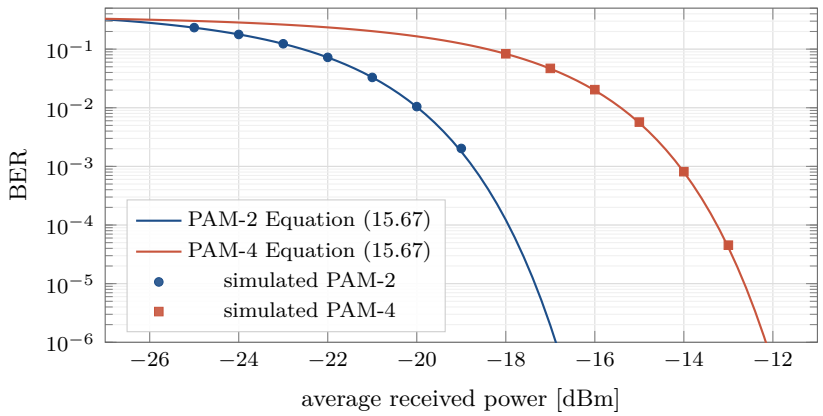


Figure 15.12: 10 GBd PAM-2 and PAM-4 into a PIN + TIA receiver ($\mathcal{R}=0.8$ A/W, $20\,\text{pA}/\sqrt{\text{Hz}}$ over $f_s/2=20$ GHz, ER = 10 dB): simulation against the course formula. PAM-4 needs 4.8 dB more power at the same baud rate.

15.7.2 Penalties and sensitivity

Equation (15.67) gives the power penalties of IM/DD directly.

- **PAM-M at the same baud rate**: the argument scales as $1/(M-1)$, so PAM-M needs $10\log_{10}(M-1)$ more average power than PAM-2 (4.8 dB for PAM-4).
- **PAM-M at the same bit rate**: the baud rate and B_{eq} drop by $n_{\text{bit}}=\log_2 M$, so the penalty is $10\log_{10}\left((M-1)/\sqrt{n_{\text{bit}}}\right)$, 3.3 dB for PAM-4. Unlike coherent formats, multilevel IM/DD always costs power.
- **Finite extinction ratio**: $10\log_{10}\frac{\text{ER}+1}{\text{ER}-1}$ relative to an infinite ER: 0.9 dB at 10 dB, 1.4 dB at 8 dB, 3.7 dB at 4 dB. External modulators reach 10–14 dB, DMLs less than 8 dB.
- **Bandwidth**: $P_{\text{sens}}\propto\sqrt{B_{eq}}$ in theory, between $B^{0.5}$ and $B^{1.5}$ in practice.

The γ approach. Since the BER depends on $\gamma\bar{P}$ only, a receiver datasheet that quotes the sensitivity P_{sens} at a target BER fixes $\gamma=\eta/P_{\text{sens}}$ with $\eta=\operatorname{erfc}^{-1}(2\,\text{BER}_{\text{sens}})$ (PAM-2), from which the BER at any power or for PAM-4 follows. Values of η: 2.19 at 10^{-3}, 3.02 at 10^{-5}, 4.24 at 10^{-9}, 4.97 at 10^{-12}, 5.62 at 10^{-15}. For example a 2.5 Gb/s PIN receiver with −22 dBm sensitivity at 10^{-9} has $\gamma=4.24/6.3\,\mu\text{W}=672\,\text{mW}^{-1}$; a 10 Gb/s APD receiver at −26 dBm has $1688\,\text{mW}^{-1}$. The approach is excellent for PIN + TIA receivers, crude for APDs (signal-dependent noise) and wrong for optically pre-amplified receivers, where ASE beat noise dominates.

15.7.3 Bandwidth limitations and equalization

When the end-to-end bandwidth falls below about half the bit rate, inter-symbol interference closes the eye. With a fourth-order Bessel receiver, $B_{3\mathrm{dB}} = 0.9R_B$ is practically perfect, $0.6R_B$ (the SONET reference filter) is good, $0.3R_B$ halves the eye opening (3 dB penalty). Commercial optoelectronic bandwidths are 20–25 GHz (40–50 GHz in research), so 50–100 GBd links need adaptive equalization after the TIA: a feed-forward FIR filter (`lms_equalizer`, Chapter 9) reduces the ISI at the cost of noise enhancement, and a decision-feedback equalizer (`dfe_equalizer`) cancels post-cursors without it.

15.7.4 Power budget

The power available for losses is $L_{\mathrm{avail}} = P_{TX,\mathrm{dBm}} - P_{\mathrm{sens,dBm}}$. It must cover the fiber, the splices (0.05–0.2 dB each), the connectors (0.1–0.5 dB), bends, the propagation penalties of dispersion and nonlinearity, and a system margin of 1–6 dB for ageing and repairs:

$$P_{TX} - P_{\mathrm{sens}} \geq \alpha_{\mathrm{dB}} L + \sum A_{\mathrm{splice}} + \sum A_{\mathrm{conn}} + A_{\mathrm{penalties}} + \mu. \tag{15.69}$$

For example a 2.5 Gb/s link with 0 dBm, −22 dBm sensitivity, 0.25 dB/km, a 0.1 dB splice every 2 km, four 0.4 dB connectors, 2 dB of bends, 0.25 dB of penalties and 6 dB of margin reaches 40.5 km. Passive components enter the budget too: a $1 \times N$ splitter divides the power by N times an excess loss, $A_{\mathrm{dB}} = 10\log_{10} N + \epsilon_{\mathrm{dB}}$ (passive optical networks work around −25 dBm); a 100 GHz WDM demultiplexer has up to 2.5 dB insertion loss, 26 dB adjacent and 35 dB non-adjacent isolation. `optical_attenuator` models a variable attenuator, a splitter (`split_ways`, `excess_loss_db`) or a connector loss; `fiber` has `connector_loss_db` at its input.

15.7.5 FEC in IM/DD

A Reed–Solomon RS(255,239) code (7 % overhead) turns a pre-FEC BER of 10^{-4} into about 10^{-14}; modern hard- and soft-decision FECs accept 3.8×10^{-3} to 2×10^{-2}. Because the electrical SNR of IM/DD is proportional to the *square* of the optical power, the optical coding gain is $10\log_{10}(\eta_{\mathrm{noFEC}}/\eta_{\mathrm{FEC}})$, half the electrical one: from 10^{-15} to 3.8×10^{-3} it is 4.7 dB optical, while an AWGN channel would gain $20\log_{10}$ of the same ratio, 9.5 dB.

15.7.6 The quantum limit of OOK

An ideal photon counter decides "1" on the arrival of at least one photon. With $P_0 = 0$ errors happen only when a "1", carrying on average $N_1 = 2\bar{P}/(hfR_B)$ photons, delivers none:

$$\mathrm{BER} = \frac{1}{2}e^{-N_1} = \frac{1}{2}\exp\Big(-\frac{2\bar{P}}{hfR_B}\Big). \tag{15.70}$$

10^{-15} needs $N_1 = 34$, about 17 photons per bit on average; 10^{-2} only two. Real receivers are 20–30 dB away, except SPAD receivers at very low rates.

15.7.7 Chromatic dispersion in IM/DD

Dispersion acts linearly on the field, $E_{Rx}(f) = E_{Tx}(f)H_F(f)$, but the photodiode detects $|E_{Rx}(t)|^2$: the end-to-end IM/DD channel is *nonlinear* and cannot be equalized exactly, unlike the coherent channel where the DSP inverts H_F. The course uses a heuristic criterion for chirp-free PAM-2: the eye is acceptable while the differential group delay within the signal spectrum stays below half a bit slot,

$$|\Delta\tau_g| = |D|L\Delta\lambda \leq \frac{T_B}{2} = \frac{1}{2R_B}, \qquad \Delta\lambda = \frac{\lambda^2}{c}\Delta f, \qquad \Delta f \approx \frac{2}{3}R_B, \tag{15.71}$$

the last value found by simulations and experiments for NRZ. At 1550 nm this gives the *dispersion limit*

$$L_{\max} = \frac{1}{10\,|D|\,R_B^2} = \frac{1}{8\,|\beta_2|\,R_B^2} \qquad (R_B \text{ in Tb/s},\ D \text{ in ps/(nm km)},\ \beta_2 \text{ in ps}^2\text{/km},\ L \text{ in km}). \tag{15.72}$$

For standard fiber ($D = 16$) it is 1000 km at 2.5 Gb/s and 62.5 km at 10 Gb/s. $L_{\max} \propto 1/R_B^2$: multiplying the bit rate by N divides the reach by N^2 (16 for SDH steps, 100 for Ethernet steps). For PAM-M the bandwidth and the symbol slot follow the baud rate R_B/n_{bit}, hence

$$L_{\max} = \frac{n_{\text{bit}}^2}{10|D|R_B^2}, \tag{15.73}$$

four times more for PAM-4 than PAM-2 at the same bit rate. With several fiber types only the accumulated dispersion counts, $|D_{\text{acc}}| = |\sum D_m L_m| \le 1/(10R_B^2)$ (1000 ps/nm at 10 Gb/s), which dispersion-compensating fiber can restore. The other remedies are higher-order PAM and operation near the zero-dispersion wavelength (O band).

Figure 15.13 tests the criterion with noise-free simulations of `intensity_modulator` and `fiber`: the eye-opening penalty of 10 and 25 Gb/s NRZ falls on one curve when the length is normalized to Equation (15.72), confirming the R_B^2 scaling, and is about 0.9 dB at $L_{\max}$. PAM-4 at its own $L_{\max}$ already has a 4 dB penalty: its eyes are a third as high, so the n_{bit}^2 rule is optimistic, and a 1 dB penalty is reached at about half that distance. As the course stresses, such formulas are indicative; the joint effect of power budget and dispersion needs simulations such as the *10G OOK with a Mach–Zehnder modulator* scenario. With chirp (Section 15.3.4) the reach is much shorter.

Code 15.20: Frequency-domain CD compensation, possible only with a coherent receiver.
linkstudio/phy/optical.py, lines 1140–1141

```
1140 H = np.conj(dispersion_operator(x.length, fs, cd, lam, slope))
1141 y = np.fft.ifft(np.fft.fft(x.data, axis=-1) * H, axis=-1)
```

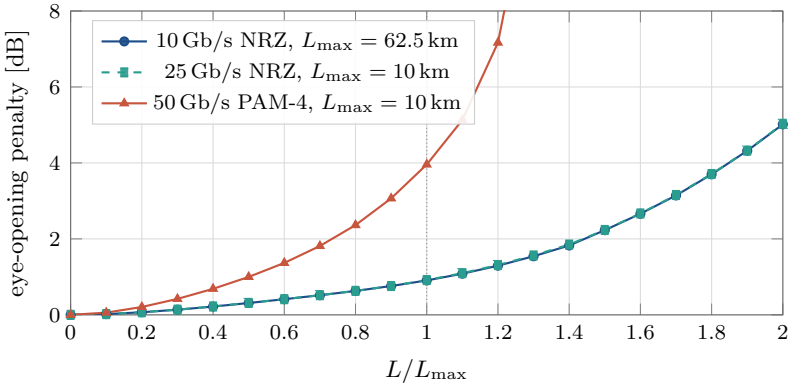


Figure 15.13: Eye-opening penalty of chirp-free IM/DD over standard fiber ($D = 16$ ps/(nm km), raised-cosine pulses, ER = 15 dB, no noise) against the length normalized to the dispersion limit.

15.8 Short-reach links: VCSELs and multimode fiber

Inside a data center most links are shorter than a few hundred metres. They use 850 nm VCSELs over graded-index multimode fiber (MMF). The 50 µm core makes coupling and connectors cheap, but it guides hundreds of modes. 100GBASE-SR4, for example, sends four lanes of

25.78 GBd NRZ over 70 m of OM3 or 100 m of OM4; the reach is set by dispersion, not loss. This section builds the fiber model from the mode theory of graded-index cores [22, 23] and links it to the bandwidth model of the IEEE 802.3 link model [24]. The block is `multimode_fiber`, in `phy/multimode.py`; the transmitter is the `vcsel` of Section 15.4.

15.8.1 Why the fiber acts on power

Write the field in the fiber as a sum of guided modes, $E(\mathbf{r},t)=\sum_m a_m(t)\psi_m(\mathbf{r})$, with orthonormal profiles, $\int \psi_m\psi_n^* dA=\delta_{mn}$. A photodiode that collects the whole core measures $P_{\text{out}}(t)=\int |E|^2 dA=\sum_m |a_m(t)|^2$: the cross terms vanish by orthogonality. Each mode carries a fraction w_m of the launched power and arrives after its own group delay τ_m, so

$$P_{\text{out}}(t)=10^{-\alpha_{\text{dB}}L/10}\sum_m w_m\,P_{\text{in}}(t-\tau_m)=10^{-\alpha_{\text{dB}}L/10}\,(h*P_{\text{in}})(t). \tag{15.74}$$

The fiber is a linear system for the optical *power*, not for the field. Chromatic dispersion fits the same picture. A multimode VCSEL has a spectrum about 0.4 nm wide (RMS, roughly 170 GHz), far wider than the modulation, so its spectral components add incoherently. A component at wavelength λ is delayed by $DL(\lambda-\lambda_c)$, so the intensity response is the source spectrum mapped onto delays: for a Gaussian spectrum of RMS width σ_λ, a Gaussian of RMS width $|D|L\sigma_\lambda$. The total response is the modal response convolved with the chromatic one, $H(f)=H_{\text{modal}}(f)H_{\text{cd}}(f)$, and the variances add.

> **Key idea** SMF links propagate a coherent field and dispersion is an all-pass filter on the field. VCSEL–MMF links propagate incoherent power and dispersion is a low-pass filter on the intensity. `multimode_fiber` therefore filters $|E|^2$ and rebuilds the field as $\sqrt{P_{\text{out}}}$ with the input phase. It is meant for direct detection.

15.8.2 Mode groups of an α-profile core

Graded-index cores follow a power law, $n^2(r)=n_1^2[1-2\Delta(r/a)^\alpha]$ for $r<a$, with core radius a, peak index n_1 and contrast $\Delta=\text{NA}^2/(2n_1^2)$ ($\Delta=0.0093$ for the standard 50 µm, NA = 0.2 fiber). In the WKB approximation [22] the number of modes with propagation constant above β is

$$m(\beta)=M\Big[\frac{n_1^2k^2-\beta^2}{2\Delta n_1^2k^2}\Big]^{\frac{\alpha+2}{\alpha}},\qquad M=\frac{\alpha}{\alpha+2}a^2k^2n_1^2\Delta=\frac{\alpha}{\alpha+2}\,\frac{V^2}{2}, \tag{15.75}$$

with $V=ka\,\text{NA}$. Inverting it gives $\beta=n_1k\sqrt{1-2\Delta x}$ with $x=(m/M)^{\alpha/(\alpha+2)}$.

Group delay. The delay per unit length is $\tau/L=c^{-1}d\beta/dk$. Keeping m fixed and noting $M\propto k^2n_1^2$, $dx/dk=-\frac{\alpha}{\alpha+2}x\frac{2N_1}{n_1k}$ with group index $N_1=d(n_1k)/dk$, and

$$\tau=\frac{LN_1}{c}\,\frac{1-\frac{4\Delta x}{\alpha+2}}{\sqrt{1-2\Delta x}}. \tag{15.76}$$

With $\Delta\approx 0.01$ a second-order expansion loses nothing. Including the profile dispersion $\epsilon=-\frac{2n_1}{N_1}\frac{\lambda}{\Delta}\frac{d\Delta}{d\lambda}$ gives the Olshansky–Keck form [23]:

$$\frac{\tau}{LN_1/c}-1=\Delta C_1x+\Delta^2C_2x^2,\qquad C_1=\frac{\alpha-2-\epsilon}{\alpha+2},\quad C_2=\frac{3\alpha-2-2\epsilon}{2(\alpha+2)}. \tag{15.77}$$

With $\epsilon=0$ it is the Taylor expansion of Equation (15.76); the test suite checks the agreement to $\mathcal{O}(\Delta^3)$.

Overfilled RMS width and the optimum profile. If every mode carries the same power, $u = m/M$ is uniform and $x = u^q$, $q = \alpha/(\alpha+2)$, has moments $\mathbb{E}[x^k] = 1/(kq+1)$, so

$$\frac{\sigma_\tau}{LN_1/c} = \Delta\left[C_1^2 \operatorname{Var} x + 2\Delta C_1 C_2 \operatorname{Cov}(x, x^2) + \Delta^2 C_2^2 \operatorname{Var} x^2\right]^{1/2}. \tag{15.78}$$

The leading term vanishes for $\alpha = 2 + \epsilon$; minimizing the whole expression gives [23]

$$\alpha_{\mathrm{opt}} = 2 + \epsilon - \Delta\frac{(4+\epsilon)(3+\epsilon)}{5+2\epsilon}, \qquad \sigma_{\min} \approx \frac{LN_1\Delta^2}{20\sqrt{3}\,c}, \tag{15.79}$$

$\alpha_{\mathrm{opt}} = 1.9777$ and $\sigma_{\min} = 12.2\,\mathrm{ps/km}$ for this fiber. A parabolic core ($\alpha = 2$) is five times worse, and a profile error of ± 0.01 multiplies the spread by two or three (Figure 15.14): real fibers are therefore graded by measured bandwidth.

Discrete groups. Modes with equal principal mode number are degenerate. A core holds about $G = \sqrt{M}$ groups (18 for OM4 at 850 nm); group μ holds the $2\mu - 1$ modes between $u = ((\mu-1)/G)^2$ and $(\mu/G)^2$. The block evaluates Equation (15.77) at the group centre and removes the mean delay.

Code 15.21: Group delay of Equation (15.77). linkstudio/phy/multimode.py, lines 63–66

```
x = np.asarray(u, dtype=float) ** (alpha / (alpha + 2))
c1 = (alpha - 2 - eps) / (alpha + 2)
c2 = (3 * alpha - 2 - 2 * eps) / (2 * (alpha + 2))
return delta * c1 * x + delta ** 2 * c2 * x ** 2
```

Code 15.22: Group delays and launch weights. linkstudio/phy/multimode.py, lines 194–208

```
g = max(1, int(round(np.sqrt(mode_count(self.radius_um, self.na, wavelength_nm, alpha)))))
mu = np.arange(1, g + 1)
delays = group_delay(((mu - 0.5) / g) ** 2, alpha, self.delta, p.profile_dispersion)
if launch == "overfilled":
    w = (2 * mu - 1).astype(float)  # group mu holds 2 mu - 1 of the g^2 modes
elif launch == "gaussian":
    w0 = np.sqrt(self.radius_um * 1e-6 * wavelength_nm * 1e-9 / (np.pi * self.na)) * 1e6
    w = gaussian_launch_weights(g, p.launch_waist_um, p.launch_offset_um, w0)
else:
    if not p.group_weights:
        raise BlockError("Custom launch needs group_weights.")
    w = np.zeros(g)
    k = min(g, len(p.group_weights))
    w[:k] = np.asarray(p.group_weights[:k], dtype=float)
return w, delays
```

15.8.3 Launch conditions

The power distribution w_μ depends on the launch.

- **Overfilled launch** (every mode equally): $w_\mu \propto 2\mu - 1$, the definition of the overfilled-launch (OFL) bandwidth.
- **Gaussian beam** (`launch=gaussian`). In a parabolic core the modes are the Hermite–Gauss modes of a two-dimensional harmonic oscillator with fundamental field radius $w_0 = \sqrt{a\lambda/(\pi\,\mathrm{NA})} \approx 5.8\,\mu\mathrm{m}$; group $p = \mu - 1$ collects the modes with $n_x + n_y = p$, and a beam of radius w displaced by r_0 separates into x and y overlap integrals.

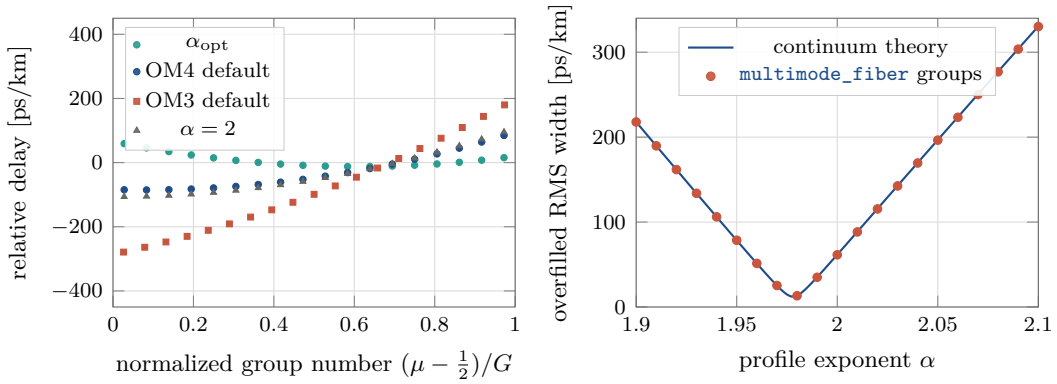


Figure 15.14: Left: mode-group delays over 1 km for the optimum profile, the default OM4 and OM3 profiles and a parabolic core. Right: overfilled RMS width against α: continuum formula and discrete-group block.

Two exact cases check the computation (Figure 15.15): a matched beam displaced by r_0 is a coherent state, with Poisson group powers of mean r_0^2/w_0^2; a centred beam of the wrong size is a squeezed state, which excites only even groups with weights $(1-t)t^{p/2}$, $t = \left((w^2-w_0^2)/(w^2+w_0^2)\right)^2$. A centred launch puts the power in the low groups, which travel close together: this is why the effective modal bandwidth (EMB) of VCSEL links exceeds the OFL bandwidth (Table 15.3).

Code 15.23: Gaussian-beam coupling into the principal mode groups. `linkstudio/phy/multimode.py`, lines 108–125

```
xi = np.linspace(-12, 12, 4001)  # in units of w0 / sqrt(2) (oscillator length)
dx = xi[1] - xi[0]
s = waist_um / fundamental_um
d = offset_um / fundamental_um * np.sqrt(2)
hermite = np.zeros((n_groups, xi.size))  # normalized oscillator eigenfunctions
hermite[0] = np.pi ** -0.25 * np.exp(-xi ** 2 / 2)
if n_groups > 1:
    hermite[1] = np.sqrt(2) * xi * hermite[0]
for k in range(2, n_groups):
    hermite[k] = np.sqrt(2 / k) * xi * hermite[k - 1] - np.sqrt((k - 1) / k) * hermite[k -
    2]

def beam(centre: float) -> np.ndarray:
    g = np.exp(-(xi - centre) ** 2 / (2 * s ** 2))
    return g / np.sqrt(np.sum(g ** 2) * dx)

cx = (hermite @ beam(d) * dx) ** 2
cy = (hermite @ beam(0.0) * dx) ** 2
return np.array([np.sum(cx[:p + 1] * cy[p::-1]) for p in range(n_groups)])
```

15.8.4 Fiber classes and the bandwidth model

Gaussian bandwidth model (`model=bandwidth`, the default). The IEEE 802.3 link model assumes Gaussian responses: for $H(f) = \exp(-2\pi^2\sigma^2 f^2)$, $H(B) = \frac{1}{2}$ gives $B\sigma = \sqrt{\ln 2/2}/\pi \approx 0.187$. The modal width is $\sigma_{\mathrm{modal}} = 0.187\,L/\mathrm{EMB}$. The chromatic width uses the dispersion at the source wavelength, $D_1 = \frac{S_0\lambda}{4}\left(1 - (\lambda_0/\lambda)^4\right)$ (about $-100\,\mathrm{ps/(nm\,km)}$ at 850 nm), plus a spectral-width term $D_2 = 0.7 S_0 \sigma_\lambda$: $\sigma_{\mathrm{cd}} = L\sigma_\lambda\sqrt{D_1^2 + D_2^2}$.

Code 15.24: Chromatic RMS width of the IEEE 802.3 link model. `linkstudio/phy/multimode.py`, lines 94–96

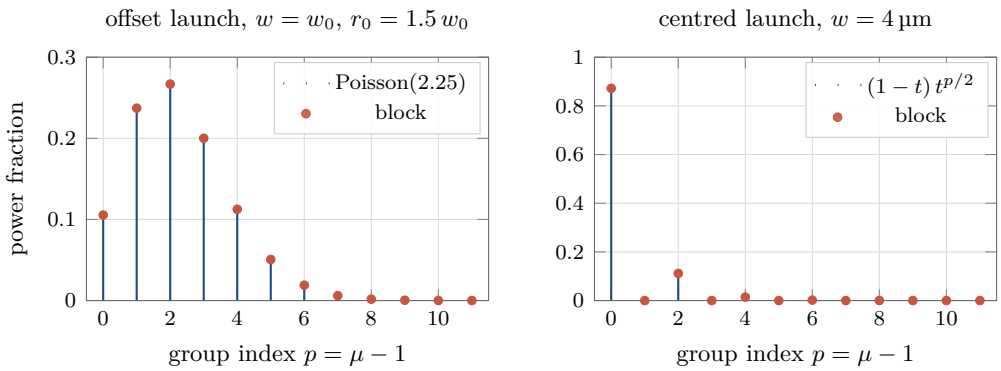


Figure 15.15: Mode-group powers from `gaussian_launch_weights` against the coherent-state (left) and squeezed-state (right) closed forms.

Table 15.3: Multimode fiber classes at 850 nm (ISO/IEC 11801, TIA-568.3, IEC 60793-2-10) as used by `multimode_fiber`.

class	core [µm]	NA	OFL BW [MHz km]	EMB [MHz km]	loss [dB/km]
OM1	62.5	0.275	200	–	3.5
OM2	50	0.200	500	–	3.5
OM3	50	0.200	1500	2000	3.5
OM4	50	0.200	3500	4700	3.5
OM5	50	0.200	3500	4700	3.5

```
    d1 = s0 * wavelength_nm / 4 * (1 - (lambda0_nm / wavelength_nm) ** 4)
    d2 = 0.7 * s0 * rms_width_nm
    return rms_width_nm * float(np.hypot(d1, d2))
```

Mode-group model (`model=mode_groups`). The response is the sum of delayed impulses of Equation (15.74) times the Gaussian chromatic response. Without an explicit `alpha`, the block detunes the profile until the overfilled RMS width of the groups equals 0.187/OFL: $\alpha = 1.9971$ for OM4 and 2.0235 for OM3. The fiber then meets its class by construction, and the launch options show how much a real transmitter improves on it.

Code 15.25: Power transfer function of the fiber. `linkstudio/phy/multimode.py`, lines 237–243

```
    p = self.params
    h = np.exp(-2 * (np.pi * f * sigma_cd_s) ** 2)
    if self.groups is None:
        return h * np.exp(-2 * (np.pi * f * self.modal_rms_s()) ** 2)
    w, d = self.groups
    modal = np.exp(-2j * np.pi * np.multiply.outer(f, d * p.length_m)) @ (w / w.sum())
    return h * modal
```

15.8.5 A 25G data-center lane

The *25G VCSEL multimode data-center link* scenario chains `pulse_shaper` (NRZ), `vcsel` (7 mA bias, ±3 mA), a 6 dB coupling loss, `multimode_fiber`, `photodiode` with the reference receiver ($0.75R_s$ Bessel–Thomson) and `agc`. Figure 15.16 shows the received eyes. At 100 m of OM4 the fiber hardly matters: the eye shows the VCSEL relaxation ringing and the receiver filter. At 300 m the modal and chromatic widths grow to about 12 ps each, a third of the 38.8 ps

symbol, and the eye starts closing; OM3 at 300 m, with less than half the EMB, closes it. The test suite checks BER below 10^{-3} for OM4 at 100 m and above 10^{-2} for OM3 at 300 m.

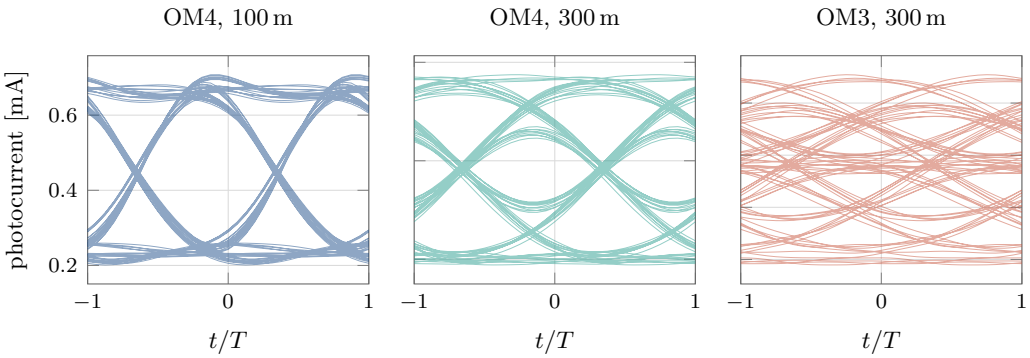


Figure 15.16: Received eyes of the data-center lane (25.78 GBd NRZ, rate-equation VCSEL, 16 samples per symbol, RIN off for clarity), cut from the `photodiode` output.

What the multimode model leaves out The IEEE link model also budgets mode-partition noise (the VCSEL modes fluctuate in power while the total stays constant, which dispersion turns into noise) and modal noise (speckle filtered by connector offsets); neither is modelled. The mode-group model ignores the coupling between modal and chromatic dispersion (each group has its own wavelength dependence [25]), differential mode attenuation and mode coupling. Statistically realistic fibers would need DMD templates measured according to IEC 60793-1-49.

15.9 Optical amplifiers

Optical amplifiers amplify the light directly, without conversion to the electrical domain, and are fiber-pigtailed so they can be inserted anywhere in a link. Commercial since about 1990, they made multi-span links of thousands of kilometres possible. Three types are used: erbium-doped fiber amplifiers (EDFAs), semiconductor optical amplifiers (SOAs) and Raman amplifiers (Section 15.11).

15.9.1 Erbium-doped fiber amplifiers

The core of a 5–30 m fiber is doped with Er^{3+} ions (100–1000 ppm), a quasi three-level system. A pump laser at 980 nm lifts electrons from the ground level E_1 to the pump band E_3, from which they decay in about 1 µs to the metastable level E_2; its long lifetime ($\tau_{21} \approx 8$–12 ms) lets the population of E_2 build up until it is inverted with respect to E_1. The $E_2 \to E_1$ transition matches 1530–1570 nm, so C-band signal photons are amplified by stimulated emission. Pumping at 1480 nm excites the top of the E_2 manifold directly. The levels are broadened by the glass host (Stark splitting), so the gain covers the whole C band (and, with longer fibers, the L band), with a peak near 1530 nm. A basic EDFA has an isolator, a WDM coupler that adds the pump, the doped fiber, a coupler that removes the residual pump and an output isolator; commercial units add several stages, counter-propagating pumps, gain-flattening filters (GFF), power monitors, a spectrum analyzer and a control CPU.

15.9.2 Semiconductor optical amplifiers

An SOA is a laser chip without feedback: the current injected into a forward-biased junction creates the inversion. It is cheap, small, integrable and can cover the O to L bands, but has

a higher noise figure (7–8 dB), polarization dependence and fast carrier dynamics (ns), which cause cross-gain modulation and four-wave mixing between WDM channels. SOAs are used as single-channel boosters and preamplifiers and for switching and wavelength conversion.

15.9.3 System model and ASE noise

On the field, an amplifier of power gain G is linear (below saturation) and adds noise:

$$\boldsymbol{E}_{\text{out}}(t) = \sqrt{G}\,\boldsymbol{E}_{\text{in}}(t) + \boldsymbol{n}(t). \tag{15.80}$$

The noise is amplified spontaneous emission (ASE), emitted on both polarizations. After gain flattening it is white and Gaussian on all four quadratures, $\boldsymbol{n} = (n_{Rx} + jn_{Ix})\hat{x} + (n_{Ry} + jn_{Iy})\hat{y}$, each with power spectral density $N_0/2$,

$$N_0 = hf\,(G-1)\,n_{sp}, \tag{15.81}$$

where $n_{sp} \geq 1$ is the spontaneous emission (inversion) factor. System specifications use the noise figure $F = 2n_{sp}$, so the quantum limit is $F \geq 2$ (3 dB). Very good C-band EDFAs have 4.5–5 dB ($n_{sp} = 1.4$–1.6), cheaper ones 5–6 dB, L-band EDFAs about 1 dB more, SOAs 7–8 dB. The ASE power in a bandwidth B (both polarizations, four quadratures) is

$$P_{\text{ASE}} = 4 \cdot \frac{N_0}{2} B = 2N_0 B = hf\,(G-1)\,F\,B. \tag{15.82}$$

The density per polarization of the complex envelope is $S_{\text{ASE}} = N_0$; this is the quantity carried in `ase_psd`. (The exact quantum-optical definition, $F = 2n_{sp}(G-1)/G + 1/G$, coincides with $F = 2n_{sp}$ at high gain; LinkStudio follows the course definition.)

Code 15.26: ASE density per polarization, $n_{sp} = F/2$. `linkstudio/phy/optical.py`, lines 521–530

```
521 def ase_psd(gain_lin: float, nf_db: float, wavelength_nm: float) -> float:
522     """ASE power spectral density per polarization (W/Hz): S_ASE = N_0 = n_sp h f (G - 1),
        n_sp = F / 2.
523
524     Each of the four field quadratures carries white noise of density N_0 / 2, so the ASE
525     power in a bandwidth B is P_ASE = 2 N_0 B = h f (G - 1) F B.
526     """
527     if gain_lin <= 1:
528         return 0.0
529     nu = C_LIGHT / (wavelength_nm * 1e-9)
530     return 10 ** (nf_db / 10) / 2 * H_PLANCK * nu * (gain_lin - 1)
```

Code 15.27: ASE as circular Gaussian noise in the simulated bandwidth. `linkstudio/phy/optical.py`, lines 533–535

```
533 def _add_ase(rng: np.random.Generator, e: np.ndarray, psd: float, fs: float) -> np.ndarray:
534     sigma = np.sqrt(psd * fs / 2)
535     return e + sigma * (rng.standard_normal(e.shape) + 1j * rng.standard_normal(e.shape))
```

15.9.4 OSNR of one amplifier

The generalized signal-to-noise ratio GSNR is the ratio of the signal power to all the noise added by the link (ASE, nonlinear interference); when only ASE counts it is the *optical* SNR,

$$\text{OSNR} = \frac{P_{\text{sig}}}{P_{\text{ASE}}} = \frac{G\,P_{\text{in}}}{hf\,B\,(G-1)\,F} \approx \frac{P_{\text{in}}}{hf\,B\,F} \quad (G \gg 1). \tag{15.83}$$

At high gain the OSNR does not depend on G, only on the input power and the noise figure: the amplifier behaves like an input-referred noise source $P_{\mathrm{ASE,in}} \approx hfFB$, the *noise floor*, about $-58\,\mathrm{dBm}$ in $0.1\,\mathrm{nm}$ ($12.5\,\mathrm{GHz}$) or $-48.9\,\mathrm{dBm}$ in $100\,\mathrm{GHz}$, plus F_{dB}. Hence the "58 dB rule"

$$\mathrm{OSNR}_{0.1\,\mathrm{nm,dB}} \approx 58 + P_{\mathrm{in,dBm}} - F_{\mathrm{dB}}. \tag{15.84}$$

To keep a high OSNR the signal must not fall too close to the floor before it is amplified. The simulator reports the OSNR both in $0.1\,\mathrm{nm}$ (`osnr_db`) and in $B = R_s$ (`osnr_rs_db`), the reference bandwidth of coherent systems.

Gain saturation. The gain of a real amplifier depends on the total input power. For a homogeneously broadened amplifier with small-signal gain G_0 and saturation power P_{sat},

$$G = G_0 \exp\Big(-\frac{(G-1)P_{\mathrm{in}}}{P_{\mathrm{sat}}}\Big), \tag{15.85}$$

solved numerically when `saturation_power_dbm` is set. EDFAs saturate on the average power (their ms lifetime ignores the bits); in long chains working at constant total output power, the accumulated ASE steals gain from the signal, which then decays along the link (the "droop" effect).

Code 15.28: Saturated gain. `linkstudio/phy/optical.py`, lines 541–543

```
541 if p_in_w <= 0:
542     return g0
543 return float(brentq(lambda g: g - g0 * np.exp(-(g - 1) * p_in_w / p_sat_w), 1.0, g0))
```

`optical_amplifier` works in `gain` mode (fixed or saturated gain) or `power` mode (output power held at `target_power_dbm`) and reports the OSNR and the gain.

15.10 Multi-span links and the OSNR

Long-haul links are chains of spans, each a stretch of fiber followed by an amplifier. The analysis below refers to the power of one WDM channel, uses $B = R_s$ as noise bandwidth and counts only ASE. Let span i have loss $A_i < 1$ (fiber and lumped losses) and amplifier gain G_i and noise figure F_i, with P_{Tx} launched into the first span. The ASE of amplifier i is amplified and attenuated by all the following spans, so at the receiver

$$P_{\mathrm{ASE,tot}} = hfB \sum_{i=1}^{M} \Big[F_i(G_i-1) \prod_{j=i+1}^{M} G_j A_j\Big], \qquad P_{Rx} = P_{Tx} \prod_{k=1}^{M} A_k G_k, \tag{15.86}$$

and $\mathrm{OSNR} = P_{Rx}/P_{\mathrm{ASE,tot}}$.

Transparent spans. If every amplifier compensates exactly the loss of its span, $G_i A_i = 1$, the products are 1, the launch power into every span is P_{Tx} and

$$\mathrm{OSNR} = \frac{P_{Tx}}{hfB \sum_i F_i (G_i - 1)}. \tag{15.87}$$

With identical spans,

$$\mathrm{OSNR} = \frac{P_{Tx}}{M\,hfBF(G-1)} \approx \frac{A\,P_{Tx}}{M\,hfBF} = \frac{P_{\mathrm{in,EDFA}}}{M\,hfBF}\,: \tag{15.88}$$

the signal is restored after every span while the ASE accumulates linearly, so the OSNR falls as $1/M$: $-3\,\mathrm{dB}$ for every doubling of the number of spans (or of the length at fixed span length). This mild scaling is what makes transoceanic links possible.

Non-transparent spans. With identical spans but $GA \neq 1$ the power drifts as $(AG)^M$ and

$$P_{\mathrm{ASE}} = hfBF(G-1)\frac{1-(AG)^M}{1-AG}, \qquad \mathrm{OSNR} = \frac{P_{Tx}}{hfBF(G-1)}\frac{(AG)^M(1-AG)}{1-(AG)^M}. \tag{15.89}$$

`optical_link` uses transparent spans by default and a fixed gain with `amplifier_gain_db`; Figure 15.17 shows both against the formulas.

The course exercise. A 3000 km link of 40 spans of 75 km (0.2 dB/km, so $A = -15$ dB and $G = 15$ dB), EDFAs with $F = 5$ dB, 0 dBm per channel at 1550 nm and $B = R_s = 64$ GHz. After the first span $\mathrm{OSNR}_1 = 31.0$ dB; after 40 spans $\mathrm{OSNR}_{40} = \mathrm{OSNR}_1 - 10\log_{10} 40 = 15.0$ dB. A 64 GBd PM-16QAM transceiver needs about 14 dB, so the link works with 1 dB of margin, and could grow to $10^{(31-14)/10} \approx 50$ spans (3750 km). Margin can be created with shorter spans, lower-loss fiber, better amplifiers, more launch power (up to the nonlinear limit of Section 15.13) or a lower symbol rate. The *3000 km PM-16QAM OSNR budget* scenario runs this link end to end.

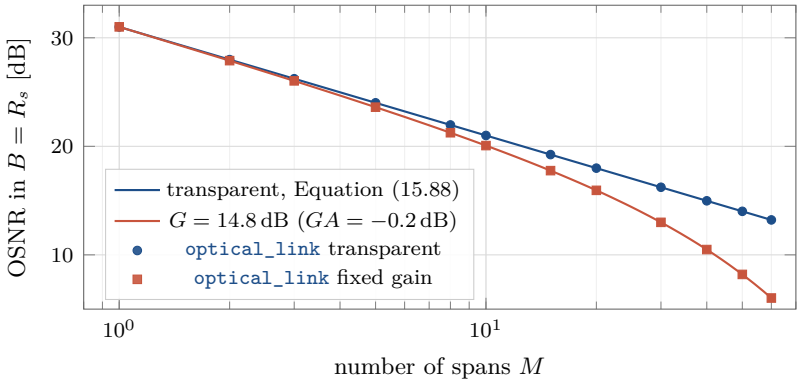


Figure 15.17: OSNR of the course exercise (75 km spans, 0.2 dB/km, $F = 5$ dB, 0 dBm, $R_s = 64$ GBd) against the number of spans, for transparent spans and for amplifiers 0.2 dB short of the span loss. The block metric equals the formulas.

Span length. For a fixed total length L_{tot} divided into M spans, $G = 10^{\alpha_{\mathrm{dB}} L_{\mathrm{tot}}/(10M)}$ falls exponentially while M grows linearly. As $M \to \infty$, $M(G-1) \to \ln(10)\alpha_{\mathrm{dB}} L_{\mathrm{tot}}/10 = \alpha_{Np} L_{\mathrm{tot}}$ (the total loss in nepers), and

$$\mathrm{OSNR}_{\mathrm{distrib}} = \frac{P_{Tx}}{hfR_sF\,\alpha_{Np}L_{\mathrm{tot}}}, \tag{15.90}$$

the limit of ideal distributed amplification: 27.3 dB for 1000 km at 0 dBm, $F = 5$ dB and $B = 100$ GHz. Long spans lose OSNR quickly: 100 km spans are about 12 dB below the limit, 200 km spans more than 30 dB (Figure 15.18).

From OSNR to E_s/N_0. For a coherent receiver whose noise is the ASE, the density per polarization is $N_0 = S_{\mathrm{ASE}}$ and the signal power per polarization is P/p for p polarizations:

$$\frac{E_s}{N_0} = \frac{P/p}{S_{\mathrm{ASE}}R_s} = \mathrm{OSNR}_{0.1\,\mathrm{nm}} \cdot \frac{2B_{\mathrm{ref}}}{p\,R_s} = \mathrm{OSNR}_{B=R_s} \quad (p=2), \tag{15.91}$$

implemented as `osnr_to_esn0_db`. The OSNR in $B = R_s$ of a dual-polarization signal is thus directly the constellation SNR of Section 15.12.4.

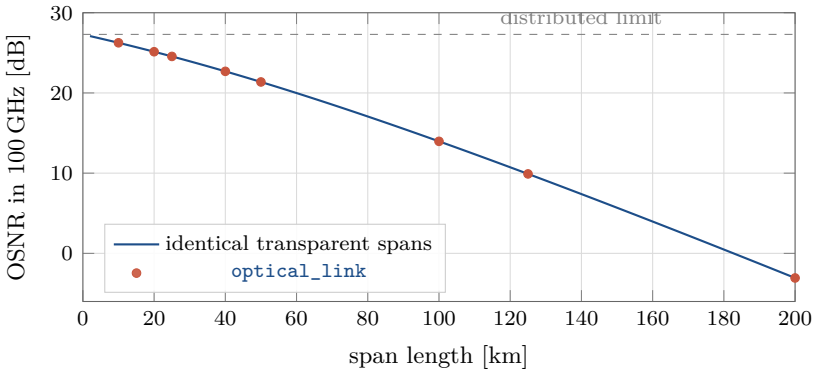


Figure 15.18: OSNR of a 1000 km link (0 dBm, $F = 5$ dB) against the span length.

> **In the code** `optical_link` repeats, for each span: lumped loss A_{pre} (`connector_loss_db`), fiber propagation (linear FFT or split-step, with Raman gain if pumped, PMD), Raman ASE, amplifier gain and ASE. The metadata track the signal power and the accumulated ASE density, $S \leftarrow (SA_{\text{pre}}A_F G_R + S_R)G + S_{\text{EDFA}}$, so the OSNR metric is exact.

Code 15.29: The multi-span loop of `optical_link`. linkstudio/phy/optical.py, lines 687–696

```
687 for _ in range(p.n_spans):
688     e = propagate(e * np.sqrt(a_pre), fs, p.span_km, att, d, lam, p.nonlinear, gamma, p.
        steps_per_km,
689                   p.dispersion_slope_ps_nm2_km, p.nonlinear_phase_rad, gain=raman)
690     if e.ndim == 3 and (p.polarization_rotation or p.pmd_ps_sqrt_km):
691         e = apply_pmd(self.rng, e, fs, p.pmd_ps_sqrt_km * 1e-12 * np.sqrt(p.span_km), p.
        pmd_sections)
692     if s_r:
693         e = _add_ase(self.rng, e, s_r, fs)
694     e = _add_ase(self.rng, e * np.sqrt(g), s, fs)
695     ase = (ase * a_pre * a_f * g_r + s_r) * g + s
696     power = power * a_pre * a_f * g_r * g
```

Code 15.30: OSNR after identical spans (`theory.osnr_link_db`). linkstudio/phy/theory.py, lines 202–207

```
202 a = 10 ** (-span_loss_db / 10)
203 g = 1 / a if gain_db is None else 10 ** (gain_db / 10)
204 ag = a * g
205 acc = n_spans if abs(ag - 1) < 1e-12 else (1 - ag ** n_spans) / (1 - ag)
206 p_rx = 1e-3 * 10 ** (p_tx_dbm / 10) * ag ** n_spans
207 return float(10 * np.log10(p_rx / (ase_power(g, nf_db, bandwidth, wavelength_nm) * acc)))
```

15.11 Raman amplification

Raman amplifiers, commercial since about 2010, use the transmission fiber itself as gain medium, through stimulated Raman scattering (SRS): a strong pump scatters on the vibrations of the silica molecules and transfers energy to photons at lower frequency. The gain peaks about 13 THz below the pump, with a gain efficiency up to $C_R \approx 0.4\,\text{W}^{-1}\text{km}^{-1}$ in standard fiber; a pump near 206 THz (1455 nm) amplifies the C band. Pumps of 0.5–1 W are typical, and several pumps at different wavelengths add their gain profiles to cover the C and L bands. The pump is usually injected backwards from the span end (counter-propagating), which averages its power fluctuations and reduces the noise transferred to the signal.

The gain is distributed: the signal does not fall as low as in a passive span. The difference between the output power with the pump on and off is the *on/off gain*. The main benefit is on the noise: the signal reaches the following EDFA well above its noise floor, so the Raman + EDFA stage has a lower equivalent noise figure than the EDFA alone. Equivalent noise figures of 0 dB and below are possible without violating any law: the amplification starts before the signal has fully attenuated, which shortens the effective loss of the span (distributed amplification has its own quantum limit).

Model. With a counter-propagating pump of input power P_p and pump loss α_p, the signal power gain coefficient along the span is

$$g(z) = C_R P_p \, e^{-\alpha_p (L - z)}, \qquad \frac{dP}{dz} = (g(z) - \alpha) P, \tag{15.92}$$

$$G_R = \exp\left(C_R P_p L_{\mathrm{eff},p}\right), \qquad L_{\mathrm{eff},p} = \frac{1 - e^{-\alpha_p L}}{\alpha_p}, \tag{15.93}$$

where G_R is the on/off gain. Spontaneous Raman scattering adds noise with density per polarization

$$S_R = n_{sp} h f \int_0^L g(z)\, G(z, L)\, dz, \qquad G(z, L) = \exp \int_z^L \left(g(z') - \alpha\right) dz', \tag{15.94}$$

the ASE born at z amplified to the span end, with $n_{sp} = 1/(1 - e^{-h\Delta\nu/kT}) \approx 1.14$ at the 13 THz shift and room temperature. Without fiber and pump loss it reduces to the lumped $n_{sp} h f (G_R - 1)$, which the test suite checks. In `optical_link` the gain enters the linear half-steps of the split-step solver (so the higher power in the second half of the span also raises the Kerr nonlinearity), the Raman ASE is added at the span end, and the EDFA gain is reduced to keep the span transparent. The span's equivalent noise figure is reported as

$$F_{\mathrm{eq}} = \frac{2(S_R G + S_{\mathrm{EDFA}})}{h f \, (G_{\mathrm{span}} - 1)}, \tag{15.95}$$

the noise figure of a single lumped amplifier that would compensate the passive span loss G_{span} with the same total ASE.

Code 15.31: Distributed Raman ASE. linkstudio/phy/optical.py, lines 411–415

```
alpha = att_db_km / (10 * np.log10(np.e)) / 1e3
z = np.linspace(0.0, self.length, n_points)
net = np.array([self.integral(zi, self.length) for zi in z]) - alpha * (self.length - z)
nu = C_LIGHT / (wavelength_nm * 1e-9)
return float(RAMAN_NSP * H_PLANCK * nu * np.trapezoid(self.coefficient(z) * np.exp(net), z)
    )
```

Raman effects not modelled Pump depletion, the Raman tilt between WDM channels (inter-channel SRS), double Rayleigh backscattering and the transfer of pump RIN to the signal are not included. The GN estimate of Section 15.13 is not reported for Raman-pumped links, whose power profile breaks its assumptions.

15.12 Coherent systems

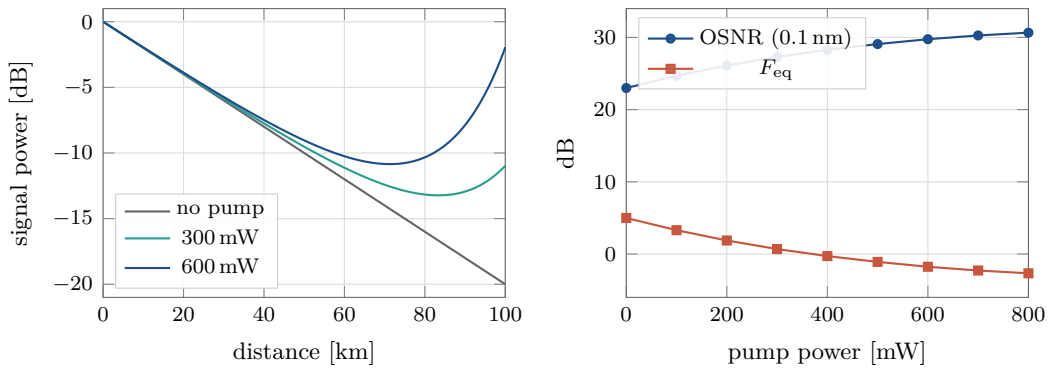


Figure 15.19: Left: signal power along a 100 km span with a counter-propagating pump ($C_R = 0.4\,\mathrm{W^{-1}km^{-1}}$, pump loss 0.25 dB/km); the gap between each curve and the passive one at 100 km is the on/off gain. Right: OSNR after 10×100 km (0 dBm, EDFA $F = 5$ dB) and equivalent span noise figure against the pump power, from `optical_link`.

15.12.1 Throughput and spectral efficiency

Backbone links carry from a few to more than 100 Tb/s per fiber over hundreds to more than 10 000 km, with wavelength-division multiplexing (one laser per channel) and coherent transmission. Since traffic grows by 25–30 % per year, the goal is to maximize the throughput

$$\text{Throughput} = \text{SE} \cdot B, \tag{15.96}$$

the product of the spectral efficiency and the optical bandwidth (5 THz in the C band, 10 THz in C+L, 12 THz and more in extended bands). For a channel of bit rate R_b^{ch} occupying B_{ch}, $\text{SE}_{\text{ch}} = R_b^{\text{ch}}/B_{\text{ch}}$; for N_{ch} identical channels spaced by guard bands δ_f,

$$\text{SE}_{\text{WDM}} = \frac{N_{\text{ch}} R_b^{\text{ch}}}{N_{\text{ch}} B_{\text{ch}} + (N_{\text{ch}} - 1)\delta_f} \approx \frac{R_b^{\text{ch}}}{B_{\text{ch}} + \delta_f}. \tag{15.97}$$

With $B_{\text{ch}} = (1+\rho)R_s$ (roll-off ρ, 0.1–0.2 today) and $R_b^{\text{ch}} = R_s \cdot \text{IR}$, where the information rate IR is the number of bits per symbol,

$$\text{SE}_{\text{WDM}} \approx \frac{\text{IR}}{(1+\rho) + \delta_f/R_s}, \tag{15.98}$$

so raising the symbol rate alone barely helps: the spectral efficiency grows with IR. IM/DD has IR = 1 (OOK) and poor spectral control. Coherent systems modulate the four quadratures: PM-BPSK has IR = 2, PM-QPSK 4, PM-8QAM 6, PM-16QAM 8, PM-64QAM 12. At 64 GBd, PM-16QAM carries 512 Gb/s gross; net rates subtract the FEC overhead (15–25 %) and framing (OTN, Ethernet). The throughput of a multi-fiber cable is $N_{\text{fiber}} B\,\text{SE}_{\text{WDM}}$, with the bracketed factor $1/((1+\rho) + \delta_f/R_s) \approx 0.8$–$0.9$.

Three generations of commercial systems illustrate the trend: 10–32 GBd PM-QPSK and PM-16QAM on a 50 GHz grid (100–200 Gb/s, 10–20 Tb/s per fiber, from 2010); up to 64 GBd and multi-format transceivers adapting from PM-BPSK to PM-64QAM to the link GSNR (from 2017); about 140 GBd with probabilistic constellation shaping and up to 1.2 Tb/s per transceiver (from 2025), with fewer wavelengths for the same capacity. Pluggable coherent modules (CFP2-DCO, QSFP-DD, OSFP) serve data-center interconnect and metro.

15.12.2 The coherent receiver

A single photodiode detects $i \propto |\boldsymbol{E}|^2 = |E_{Rx}|^2 + |E_{Ix}|^2 + |E_{Ry}|^2 + |E_{Iy}|^2$: phase and polarization are lost. Coherent detection adds a strong local oscillator (LO) before the photodiode:

$$i \propto |\boldsymbol{E} + \boldsymbol{E}_{LO}|^2 = |\boldsymbol{E}|^2 + |\boldsymbol{E}_{LO}|^2 + 2\Re\{\boldsymbol{E} \cdot \boldsymbol{E}_{LO}^*\}. \tag{15.99}$$

With $\boldsymbol{E}_{LO} = E_{LOx}\hat{x}$ real and in phase with the carrier, $i \propto P_{ph} + P_{LO} + 2E_{LOx}E_{Rx}(t)$: the beat term is proportional to one quadrature, boosted by the LO amplitude. *Balanced detection* removes the common terms: an optical hybrid feeds two photodiodes with $\boldsymbol{E} + E_{LOx}\hat{x}$ and $\boldsymbol{E} - E_{LOx}\hat{x}$, and the difference current is

$$i_{\text{bal}} = i_1 - i_2 \propto 4E_{LOx}E_{Rx}(t). \tag{15.100}$$

An LO shifted by 90° ($jE_{LOx}\hat{x}$) detects E_{Ix}, and the y polarization is handled alike. The coherent front end (Figure 15.20) splits signal and LO with polarization beam splitters, mixes each polarization with the LO in a 90° hybrid (outputs $E_x \pm E_{LOx}$, $E_x \pm jE_{LOx}$) and detects the four outputs with balanced photodetectors; integrated coherent receivers contain all of it. The LO is a free-running laser: with an arbitrary phase α the "I" output is $E_{LO}[E_{Rx}\cos\alpha + E_{Ix}\sin\alpha]$, a rotation of the quadratures. The fiber also rotates the polarization, so the received quadratures are

$$\begin{pmatrix} E'_{Rx} \\ E'_{Ix} \\ E'_{Ry} \\ E'_{Iy} \end{pmatrix} = \mathbf{M}(t) \begin{pmatrix} E_{Rx} \\ E_{Ix} \\ E_{Ry} \\ E_{Iy} \end{pmatrix}, \tag{15.101}$$

with $\mathbf{M}(t)$ a time-varying 4×4 orthogonal matrix (a 2×2 unitary Jones matrix) for a lossless fiber without polarization-dependent loss. Estimating and inverting it, together with the LO phase, is the job of the receiver DSP (Section 15.12.7).

`coherent_receiver` outputs the complex photocurrent per polarization,

$$i_I + j\, i_Q = \mathcal{R}\sqrt{P_{LO}}\, E(t)\, e^{-j(2\pi\Delta f\, t + \phi_{LO}(t))} + n(t), \tag{15.102}$$

with LO frequency offset Δf and phase noise ϕ_{LO}, LO shot noise and TIA noise. With dual polarization a splitter feeds each hybrid with half of the LO. With a strong LO the LO shot noise $2q\mathcal{R}P_{LO}B$ dominates the TIA noise; signal and shot noise then scale together, so the split does not change the shot-noise-limited SNR, only the margin over TIA noise. `hybrid_phase_error_deg` models a hybrid that is not exactly 90° (I/Q skew).

> **In the code** The simulator processes each frame with circular FFTs (fiber, dispersion compensation). An LO phase that does not return to its initial value at the end of the frame would create a discontinuity, spread over the frame by the dispersion compensator. The block therefore rounds the frequency offset to the frame's grid f_s/n and turns the Wiener phase into a Brownian bridge, $\phi_k - \frac{k}{n}\phi_n$, which has the same local increments but returns to zero.

Code 15.32: Coherent detection with a periodic LO phase (`coherent_receiver`).
`linkstudio/phy/optical.py`, lines 1054–1073

```
1054 walk = wiener_phase(self.rng, B, n + 1, p.lo_linewidth_hz, fs)
1055 bridge = walk[:, :n] - np.arange(n) / n * walk[:, n:n + 1]
1056 fo = np.round(p.frequency_offset_hz * n / fs) * fs / n
1057 phase = bridge + 2 * np.pi * fo * np.arange(n) / fs
1058 lo = np.exp(-1j * (phase[:, None, :] if e.ndim == 3 else phase))
```

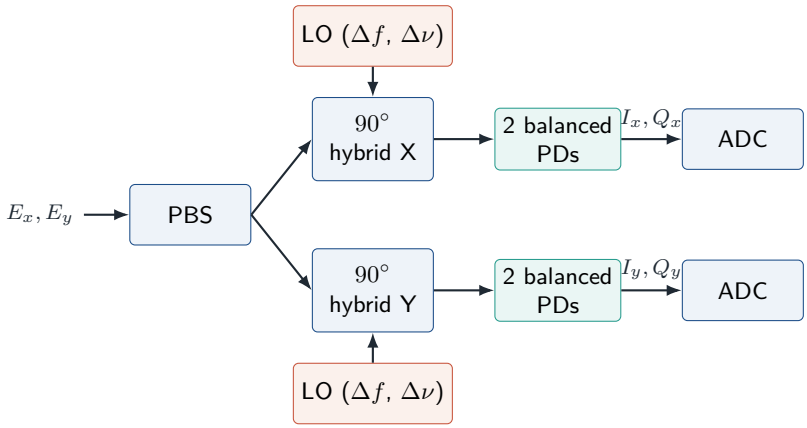


Figure 15.20: Polarization-diverse coherent front end, modelled by `coherent_receiver`. One LO laser is split between the two hybrids.

```
npol = e.shape[1] if e.ndim == 3 else 1
p_lo = dbm_to_w(p.lo_power_dbm) / npol  # a polarization splitter feeds one hybrid per polarization
scale = p.responsivity * np.sqrt(p_lo)
bw = fs / 2
var = 2 * (p.thermal_noise_pa_rthz * 1e-12) ** 2 * bw
if p.shot_noise:
    var += 2 * Q_E * p.responsivity * p_lo * bw
beat = scale * e * lo
if p.hybrid_phase_error_deg:  # the Q output sees the LO at 90 + error degrees
    err = np.deg2rad(p.hybrid_phase_error_deg)
    beat = beat.real + 1j * (beat.imag * np.cos(err) - beat.real * np.sin(err))
sig_power = scale ** 2 * (x.meta.get("power") or 1e-3) / npol
if p.internal_snr_db is not None:  # P_N,int = chi P_sig with chi = 1 / ISNR
    var += sig_power * 10 ** (-p.internal_snr_db / 10)
noise = np.sqrt(var / 2) * (self.rng.standard_normal(e.shape) + 1j * self.rng.standard_normal(e.shape))
```

15.12.3 Noise, matched filtering and decision variables

At the receiver input the field is signal plus ASE,

$$\boldsymbol{E}_{ph}(t) = \sqrt{\frac{P_s}{R_s}} \sum_n \boldsymbol{a}_n g(t - nT) + \boldsymbol{n}(t), \tag{15.103}$$

with the four ASE components white with density $N_0/2$. An optical band-pass filter (a WDM demultiplexer, much wider than the channel) shapes them as $\frac{N_0}{2}|H(f)|^2$, essentially flat over the channel. The front end delivers the four noisy quadratures, and the optimum detector for AWGN is a matched filter $h(t) = g^*(-t)$ (made causal with a delay t_d), implemented in the DSP. For a root-raised-cosine $g(t)$ the overall pulse $p = g * g^*(-t)$ is raised cosine, $P(f) = |G(f)|^2$, which satisfies the Nyquist criterion: no ISI at $t = kT$. Sampling gives the decision variables

$$d_{k,x} = \sqrt{E_{s,x}}\, a_{k,x} + n_{F,x}, \qquad d_{k,y} = \sqrt{E_{s,y}}\, a_{k,y} + n_{F,y}, \tag{15.104}$$

with $E_{s,x} = E_{s,y} = E_s/2$ for equal power, and four independent Gaussian noise components of variance $\sigma^2 = \frac{N_0}{2}\int |g|^2 = \frac{N_0}{2}$.

15.12.4 BER of PM-QPSK and the constellation SNR

For PM-QPSK each quadrature is an independent binary signal of amplitude $\pm\sqrt{E_s/2}\cdot\frac{1}{\sqrt{2}} = \pm\frac{\sqrt{E_s}}{2}$ in Gaussian noise of variance $N_0/2$, so

$$P_e = \int_{-\infty}^{-\sqrt{E_s}/2} \frac{e^{-x^2/N_0}}{\sqrt{\pi N_0}}\,dx = \frac{1}{2}\operatorname{erfc}\Big(\sqrt{\frac{E_s}{4N_0}}\Big), \tag{15.105}$$

the BER of each of the four Gray-coded bits. The SNR is the same whether measured on one real component ($\frac{E_s/4}{N_0/2}$), on one polarization ($\frac{E_s/2}{N_0}$) or on the four-dimensional symbol ($\frac{E_s}{2N_0}$):

$$\mathrm{SNR} = \frac{E_s}{2N_0}, \qquad \mathrm{BER}_{\text{PM-QPSK}} = \frac{1}{2}\operatorname{erfc}\Big(\sqrt{\frac{\mathrm{SNR}}{2}}\Big). \tag{15.106}$$

This "constellation SNR" is measurable optically. The ASE power in B over both polarizations is $4\frac{N_0}{2}B = 2N_0B$, so the GSNR in the reference bandwidth $B = R_s$ is

$$\mathrm{GSNR}_{B=R_s} = \frac{P_{\text{sig}}}{2N_0R_s} = \frac{E_sR_s}{2N_0R_s} = \frac{E_s}{2N_0} = \mathrm{SNR}_{\text{constellation}}, \tag{15.107}$$

in the absence of transceiver noise. The apparent factor of 2 with the wireless formula $\frac{1}{2}\operatorname{erfc}\sqrt{E_s/(2N_0)}$ for QPSK disappears with consistent definitions: there E_s is the energy per polarization and N_0 the density of the complex noise. In LinkStudio the link metric `osnr_rs_db` is this constellation SNR, equal to E_s/N_0 per polarization in the convention of the baseband chapters.

15.12.5 BER of PM-QAM

For AWGN with Gray labels the course gives, in terms of the constellation SNR per polarization:

$$\mathrm{BER}_{\text{PM-BPSK}} = \frac{1}{2}\operatorname{erfc}\sqrt{\mathrm{SNR}}, \qquad \mathrm{BER}_{\text{PM-QPSK}} = \frac{1}{2}\operatorname{erfc}\sqrt{\frac{\mathrm{SNR}}{2}}, \tag{15.108}$$

$$\mathrm{BER}_{M\text{-QAM}} \approx \frac{L-1}{L\log_2 L}\operatorname{erfc}\sqrt{\frac{3\,\mathrm{SNR}}{2(L^2-1)}} \quad (M = L^2), \tag{15.109}$$

$$\mathrm{BER}_{8\text{QAM}} \approx \frac{2}{3}\operatorname{erfc}\sqrt{\frac{3}{14}\mathrm{SNR}}, \qquad \mathrm{BER}_{16\text{QAM}} \approx \frac{3}{8}\operatorname{erfc}\sqrt{\frac{\mathrm{SNR}}{10}}, \tag{15.110}$$

$$\mathrm{BER}_{32\text{QAM}} \approx \frac{13}{40}\operatorname{erfc}\sqrt{\frac{\mathrm{SNR}}{20}}, \quad \mathrm{BER}_{64\text{QAM}} \approx \frac{7}{24}\operatorname{erfc}\sqrt{\frac{\mathrm{SNR}}{42}}, \tag{15.111}$$

accurate between 3×10^{-2} and 10^{-3} and below. The general square-QAM expression, written here with the prefactor and argument that reproduce the specific cases ($L = 2, 4, 8$), is the nearest-neighbour approximation of the exact result of Chapter 6. They are available as `theory.ber_pm_qam`. Figure 15.21 compares them with back-to-back simulations in which an attenuator and an amplifier load the signal with ASE and the SNR is the amplifier's `osnr_rs_db` metric.

The information rate actually achieved is below the nominal IR and depends on the SNR: 16-QAM approaches 4 bit/symbol per polarization only above about 15 dB, and at 10 dB carries about 3.2. A constellation is used near its maximum IR, where a FEC with 15–27 % overhead can correct the pre-FEC BER (up to a few 10^{-2} for soft decision). Typical operating SNRs are 5–8 dB for QPSK, 8–11 dB for 8-QAM, 11–14 dB for 16-QAM, 14–17 dB for 32-QAM and 17–20 dB for 64-QAM, so higher IR requires higher SNR. Soft-decision FEC and probabilistic shaping let the IR follow the channel capacity more finely.

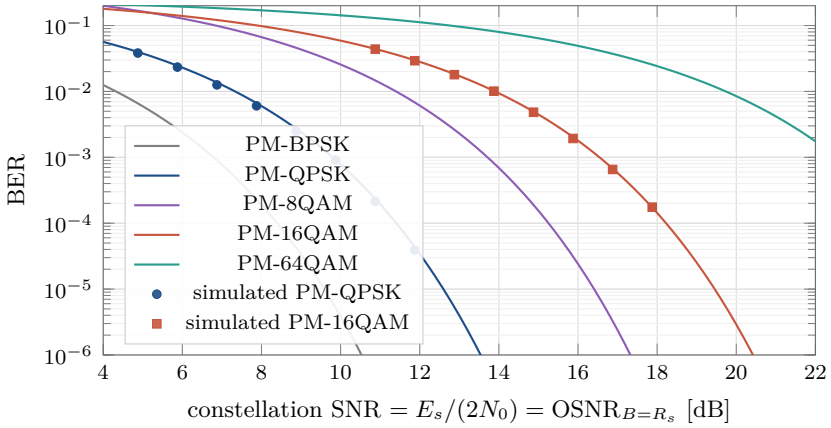


Figure 15.21: BER of PM-QAM formats against the constellation SNR: course formulas (lines) and back-to-back 32 GBd simulations with ASE noise loading (markers).

15.12.6 Transceiver noise: ISNR and GSNR

The SNR on the recovered constellation contains link noise and noise generated inside the transceiver,

$$\mathrm{SNR} = \frac{P_{\mathrm{sig}}}{P_{N,\mathrm{link}} + P_{N,\mathrm{int}}}, \qquad P_{N,\mathrm{link}} = P_{\mathrm{ASE}} + \eta_{\mathrm{NLI}} P_{\mathrm{sig}}^3, \qquad P_{N,\mathrm{int}} = \chi_{N,\mathrm{int}} P_{\mathrm{sig}}. \tag{15.112}$$

The link noise is ASE plus nonlinear interference (Section 15.13); the internal noise (electrical amplifiers, DAC and ADC quantization, modulator and other imperfections) scales with the signal. Back to back the receiver measures the internal SNR, ISNR $= 1/\chi_{N,\mathrm{int}}$, a property of the transceiver pair (20–21 dB for state-of-the-art equipment); the link alone is characterized by GSNR $= P_{\mathrm{sig}}/P_{N,\mathrm{link}}$, which reduces to the OSNR when only ASE counts. Then

$$\frac{1}{\mathrm{SNR}} = \frac{1}{\mathrm{GSNR}} + \frac{1}{\mathrm{ISNR}}, \qquad \mathrm{SNR} = \frac{\mathrm{GSNR} \cdot \mathrm{ISNR}}{\mathrm{GSNR} + \mathrm{ISNR}}: \tag{15.113}$$

the link dominates when ISNR $\gg$ GSNR, the transceiver when GSNR $\gg$ ISNR. The ISNR caps the usable constellation (around PM-64QAM for long haul today). `internal_snr_db` of `coherent_receiver` adds Gaussian noise of variance $P_{\mathrm{sig}}/\mathrm{ISNR}$ to the photocurrents; the test suite checks that a noiseless input then shows exactly that SNR.

Code 15.33: Hybrid phase error and transceiver noise. `linkstudio/phy/optical.py`, lines 1067–1072

```
1067 if p.hybrid_phase_error_deg:  # the Q output sees the LO at 90 + error degrees
1068     err = np.deg2rad(p.hybrid_phase_error_deg)
1069     beat = beat.real + 1j * (beat.imag * np.cos(err) - beat.real * np.sin(err))
1070 sig_power = scale ** 2 * (x.meta.get("power") or 1e-3) / npol
1071 if p.internal_snr_db is not None:  # P_N,int = chi P_sig with chi = 1 / ISNR
1072     var += sig_power * 10 ** (-p.internal_snr_db / 10)
```

15.12.7 Coherent digital signal processing

The chain follows the architecture described by Savory [17]. Figure 15.22 follows a 128 Gb/s DP-QPSK signal through it after 20 × 80 km, with random polarization, a 1 GHz LO offset and 100 kHz lasers.

Chromatic dispersion compensation. The accumulated dispersion is known from the metadata (or given), and its all-pass response Equation (15.20) is inverted exactly by multiplying

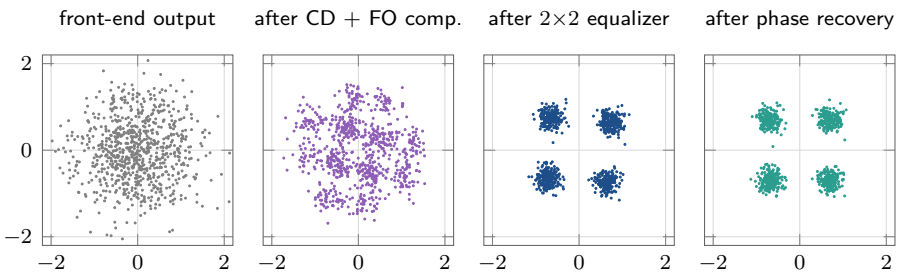


Figure 15.22: Polarization X of the *Coherent DP-QPSK long haul* scenario at four points of the DSP chain (normalized to unit power). After 1600 km of uncompensated dispersion the front-end output is noise-like (Section 15.13).

with its conjugate in the frequency domain (Listing 15.20). In modern systems there is no optical dispersion compensation: uncompensated transmission (UT).

Frequency-offset estimation. Raising QPSK to the fourth power removes the modulation: $(s\,e^{j2\pi\Delta f t})^4 = s^4 e^{j2\pi 4\Delta f t}$ with $s^4 = -1$ for every QPSK symbol, and roughly constant on average for square QAM. The spectrum of r^4 has a line at $4\Delta f$; the estimate is its peak divided by 4, refined by zero-padding and computed on a low-pass copy. In dual polarization a rotated mixture $as_1 + bs_2$ has a fourth-power line proportional to $a^4 + b^4$, which can vanish, so the estimator also uses $(x_0 + x_1)/\sqrt{2}$ and $(x_0 + jx_1)/\sqrt{2}$. Compensating the offset *after* the dispersion compensator leaves the residual $\exp\left(j\frac{\beta_2 L}{2}(2\omega\Delta\omega + \Delta\omega^2)\right)$, a bulk delay $\beta_2 L\Delta\omega$ plus a constant phase, which the block removes.

Code 15.34: Fourth-power FO estimation over several polarization combinations.
linkstudio/phy/optical.py, lines 1180–1187

```
nfft = int(2 ** np.ceil(np.log2(4 * n)))
spec = np.abs(np.fft.fft(lp ** m, n=nfft, axis=-1)) ** 2
if spec.ndim == 3:
    spec = spec.sum(axis=1)
f = np.fft.fftfreq(nfft, 1 / fs)
df = f[np.argmax(spec, axis=-1)] / m  # (B,)
self._estimates.extend(df.tolist())
rot = np.exp(-2j * np.pi * df[:, None] * np.arange(n) / fs)
```

Adaptive 2×2 equalization. Polarization rotation, PMD and residual dispersion form a 2×2 MIMO channel, inverted by a fractionally spaced butterfly of four FIR filters (Figure 15.23):

$$y_p[k] = \sum_{q\in\{x,y\}} \boldsymbol{w}_{pq}^{\mathsf{T}}\,\boldsymbol{u}_q[k], \qquad \boldsymbol{w}_{pq} \leftarrow \boldsymbol{w}_{pq} + \mu\, e_p[k]\,\boldsymbol{u}_q^*[k]. \tag{15.114}$$

The error e_p is either the constant-modulus error $y_p(R - |y_p|^2)$ with $R = \mathbb{E}|s|^4/\mathbb{E}|s|^2$ (Godard's CMA [15]), blind and insensitive to the carrier phase, or the LMS error $d_p - y_p$, trained on known symbols and then decision directed, with a larger step during training.

Code 15.35: Butterfly update with CMA or trained/decision-directed LMS.
linkstudio/phy/optical.py, lines 1251–1265

```
for k in range(n_sym):
    c = start + k * sps + D  # centre index in the padded signal
    u = pad[:, :, c - D:c + D + 1][:, :, ::-1]  # (B, P, T)
    y = np.einsum("bpqt,bqt->bp", W, u)
    out[:, :, k] = y
    if not use_lms:  # blind: phase-insensitive, leaves the carrier phase to phase_recovery
```

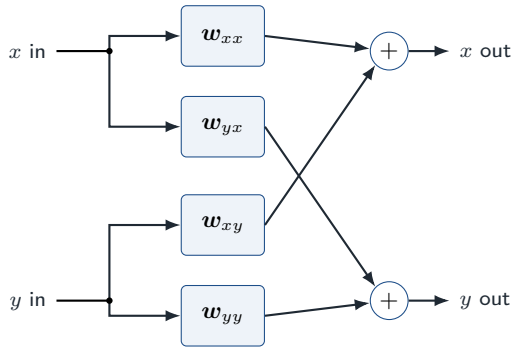


Figure 15.23: The butterfly equalizer (2 samples per symbol, `n_taps` taps per filter).

```
        e = y * (radius - np.abs(y) ** 2)
    elif k < p.n_train:
        e = refd[:, :, k] - y
    else:
        e = const.points[const.nearest(y)] - y if const is not None else refd[:, :, k] - y
        err += float(np.sum(np.abs(e) ** 2))
        count += e.size
    mu = p.train_step_size if use_lms and k < p.n_train else p.step_size
    W += mu * e[:, :, None, None] * np.conj(u)[:, None, :, :]
```

Carrier phase recovery. Laser phase noise of $2 \times 100\,\mathrm{kHz}$ at $32\,\mathrm{GBd}$ is slow but needs feed-forward tracking. The Viterbi–Viterbi estimator removes the QPSK modulation with the fourth power, $\hat{\phi} = \frac{1}{4}\big(\arg \sum_{|i|\le W} y_{k+i}^4 - \pi\big)$; blind phase search (BPS) [16] works for any square QAM: it rotates the symbols by B test phases in $[-\pi/4, \pi/4)$, sums over a window the squared distances to the nearest points and keeps the best phase. Both are defined modulo $\pi/2$ and are unwrapped on $4\hat{\phi}$; the quadrant ambiguity is resolved once per frame with a short block of known symbols after the equalizer has converged.

Code 15.36: Viterbi–Viterbi and blind phase search. `linkstudio/phy/optical.py`, lines 1306–1319

```
if p.method == "viterbi_viterbi":
    s = smooth(y ** 4)
    phi = (np.angle(s) - np.pi) / 4
else:
    if const is None:
        raise BlockError("BPS needs the constellation (mapper upstream).")
    phases = np.arange(p.n_test_phases) / p.n_test_phases * np.pi / 2 - np.pi / 4
    dist = np.empty((*y.shape, phases.size))
    for i, ph in enumerate(phases):
        r = y * np.exp(-1j * ph)
        dist[..., i] = np.abs(r - const.points[const.nearest(r)]) ** 2
    dist = np.moveaxis(smooth(np.moveaxis(dist, -1, 0)), 0, -1)
    phi = phases[np.argmin(dist, axis=-1)]
phi = np.unwrap(4 * phi, axis=-1) / 4
```

15.13 Nonlinear interference and the GN model

15.13.1 Gaussianity of dispersed signals

In uncompensated transmission the accumulated dispersion is enormous: a $100\,\mathrm{GBd}$ signal spreads over 17 symbols after $10\,\mathrm{km}$ and over 1700 after $1000\,\mathrm{km}$. The four quadratures of the received field are then sums of many independent symbols and take an approximately zero-mean,

uncorrelated Gaussian distribution, with the power spectral density of the transmitted signal ($|H_F| = 1$): the constellation before DSP looks like a Gaussian cloud (Figure 15.22, left). The Kerr nonlinearity acts on this noise-like signal. The nonlinear disturbance that the WDM channels generate on each other and on themselves, the *nonlinear interference* (NLI), is therefore itself approximately additive Gaussian noise, like ASE, and adds to it:

$$\mathrm{GSNR} = \frac{P_{\mathrm{ch}}}{P_{\mathrm{ASE}} + P_{\mathrm{NLI}}}. \tag{15.115}$$

After CD compensation the NLI appears as a cloud around the constellation points, and the BER follows from the GSNR with the AWGN formulas.

15.13.2 The Gaussian-noise model

The GN model [43] gives the NLI power in closed form under these assumptions: identical channels (symbol rate R_s, power P_{ch}, compact spectra), equal spacing Δf, identical transparent spans of length L_s with loss coefficient α, dispersion β_2 and nonlinearity γ. For one span, with α the *field* loss coefficient ($\alpha_{\mathrm{dB}} = 8.686\,\alpha$),

$$P_{\mathrm{NLI}}^{(1)} = \eta^{(1)} P_{\mathrm{ch}}^3, \qquad \eta^{(1)} = \frac{8}{27}\,\frac{\gamma^2 L_{\mathrm{eff}}^2}{\pi |\beta_2| L_{\mathrm{eff},a} R_s^2}\,\operatorname{asinh}\!\left(\frac{\pi^2}{2}|\beta_2| L_{\mathrm{eff},a} R_s^2 N_{\mathrm{ch}}^{2R_s/\Delta f}\right), \tag{15.116}$$

with $L_{\mathrm{eff}} = (1 - e^{-2\alpha L_s})/(2\alpha)$ and $L_{\mathrm{eff},a} = 1/(2\alpha)$. For long spans $L_{\mathrm{eff}} \to 1/(2\alpha)$ and it reduces to the form of the course,

$$\eta^{(1)} \approx \frac{4}{27\pi}\,\frac{\gamma^2}{|\beta_2|\alpha R_s^2}\,\operatorname{asinh}\!\left(\frac{\pi^2}{4\alpha}|\beta_2| R_s^2 N_{\mathrm{ch}}^{2R_s/\Delta f}\right) \approx \frac{4}{27\pi}\,\frac{\gamma^2}{|\beta_2|\alpha R_s^2}\,\ln\!\left(\frac{\pi^2}{2\alpha}|\beta_2| R_s^2 N_{\mathrm{ch}}^{2R_s/\Delta f}\right), \tag{15.117}$$

where $\operatorname{asinh} x \approx \ln 2x$ for $x > 3$, always the case in practice. The exact effective length of Equation (15.116) also makes the NLI vanish correctly for short spans (the course uses the factor $1 - e^{-4\alpha L_s}$ for the same purpose). The formula holds for $R_s \geq 25\,\mathrm{GBd}$, span loss $\geq 7\,\mathrm{dB}$, $|\beta_2| > 4\,\mathrm{ps}^2/\mathrm{km}$ and roll-off below 0.5. Its main features: the NLI grows as P_{ch}^3 and as γ^2, and decreases with $|\beta_2|$, because higher dispersion decorrelates the interacting signals faster. Assuming incoherent accumulation, M identical spans give $P_{\mathrm{NLI,tot}} = M\eta^{(1)} P_{\mathrm{ch}}^3$.

Code 15.37: The GN-model coefficient η of one span (`theory.gn_eta`). linkstudio/phy/theory.py, lines 233–240

```
alpha = attenuation_db_km / (20 * np.log10(np.e)) / 1e3  # field, 1/m
gamma = gamma_w_km * 1e-3
b2 = abs(beta2_ps2_km) * 1e-27  # s^2/m
leff = (1 - np.exp(-2 * alpha * span_km * 1e3)) / (2 * alpha)
la = 1 / (2 * alpha)
df = spacing_hz or symbol_rate
arg = np.pi ** 2 / 2 * b2 * la * symbol_rate ** 2 * n_channels ** (2 * symbol_rate / df)
return float(8 / 27 * gamma ** 2 * leff ** 2 * np.arcsinh(arg) / (np.pi * b2 * la *
    symbol_rate ** 2))
```

15.13.3 Optimum launch power and the 3 dB rule

For M identical spans with ASE $P_{\mathrm{ASE}}^{(1)} = hfR_sF(G-1)$ per span,

$$\mathrm{GSNR} = \frac{P_{\mathrm{ch}}}{M\left(P_{\mathrm{ASE}}^{(1)} + \eta^{(1)} P_{\mathrm{ch}}^3\right)}. \tag{15.118}$$

At low power the ASE dominates and the GSNR grows with P_{ch}; at high power the NLI dominates and it falls as $1/P_{\mathrm{ch}}^2$. Setting $d\,\mathrm{GSNR}/dP_{\mathrm{ch}} = 0$ gives $P_{\mathrm{ASE}}^{(1)} - 2\eta^{(1)} P_{\mathrm{ch}}^3 = 0$:

$$P_{\mathrm{ch,opt}} = \sqrt[3]{\frac{P_{\mathrm{ASE}}^{(1)}}{2\eta^{(1)}}}, \qquad \mathrm{GSNR}_{\max} \cdot M = \frac{2}{3\sqrt[3]{2\eta^{(1)}\left[P_{\mathrm{ASE}}^{(1)}\right]^2}}, \tag{15.119}$$

independent of the number of spans; the same power maximizes the reach for a target GSNR. At the optimum $P_{\mathrm{NLI}} = \frac{1}{2} P_{\mathrm{ASE}}$ (span by span and in total): the ASE is 3 dB above the NLI, and the GSNR counting only NLI is twice the OSNR. Lumped losses before the fiber A_{pre} reduce the power entering the fiber and give $\tilde{\eta}^{(1)} = \eta^{(1)} A_{\mathrm{pre}}^2$ referred to the span input.

Figure 15.24 checks the model against the split-step solver on the *WDM with fiber nonlinearity* scenario: five 32 GBd DP-QPSK channels at 50 GHz over 10×80 km of standard fiber. The markers are the MER of the centre channel after the full DSP chain; the line is the GSNR of Equation (15.116) plus ASE ($\eta = 536\,\mathrm{W}^{-2}$ per span, $P_{\mathrm{opt}} = -1.1$ dBm). Both show the optimum launch power and the -2 dB/dB slope above it; the simulated optimum is within 1 dB of the predicted one. Near the optimum the simulation is about 1 dB below the model because the MER includes the adaptive equalizer and phase recovery penalties; at high power it is about 1 dB above, because the GN model overestimates the NLI of non-Gaussian formats such as QPSK (the correction of the enhanced GN model [44]).

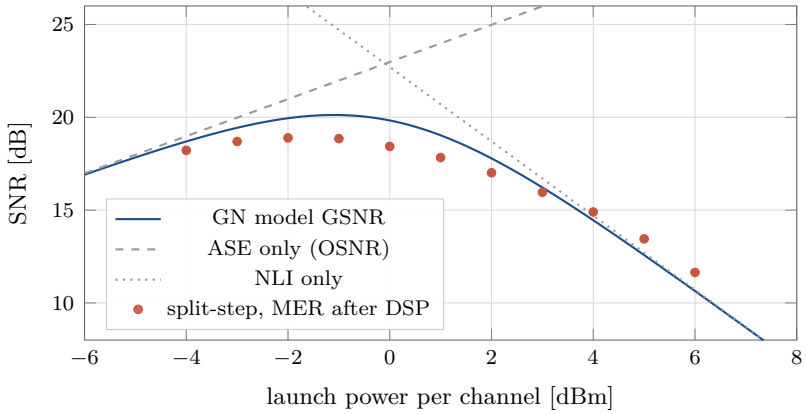


Figure 15.24: Received SNR of the centre channel of a 5-channel WDM comb against the launch power: split-step simulation and GN model.

15.13.4 Non-identical spans and the LOGO principle

Terrestrial links have spans of different lengths and fibers, and the WDM comb can change along the route. With span i described by its input power $P_{\mathrm{ch},i}^{\mathrm{in}}$, lumped losses $A_{\mathrm{pre},i}$, $A_{\mathrm{post},i}$, fiber loss $A_{F,i} = e^{-2\alpha_i L_i}$ and amplifier G_i, F_i, the noise at the receiver is

$$P_{\mathrm{ASE,tot}} = \sum_{i=1}^{M} P_{\mathrm{ASE},i}^{(1)} \prod_{j=i+1}^{M} A_{\mathrm{span},j} G_j, \qquad P_{\mathrm{NLI,tot}} = \sum_{i=1}^{M} P_{\mathrm{NLI},i}^{(1)} \prod_{j=i+1}^{M} A_{\mathrm{span},j} G_j, \tag{15.120}$$

with $P_{\mathrm{ASE},i}^{(1)} = hfBF_i(G_i - 1)$ and $P_{\mathrm{NLI},i}^{(1)} = \tilde{\eta}_i (P_{\mathrm{ch},i}^{\mathrm{in}})^3 A_{F,i} A_{\mathrm{post},i} G_i$. With transparency the products are 1 and $\mathrm{GSNR} = P_{\mathrm{ch}} / (\sum_i hfBF_i(G_i - 1) + P_{\mathrm{ch}}^3 \sum_i \tilde{\eta}_i)$, but transparency is not optimal for different spans. The total GSNR is the "parallel" combination of span-local GSNRs, each the signal at the span output over the noise the span adds:

$$\frac{1}{\mathrm{GSNR}_{\mathrm{tot}}} = \sum_{i=1}^{M} \frac{1}{\mathrm{GSNR}_i}, \qquad \mathrm{GSNR}_i = \frac{P_{\mathrm{ch},i}^{\mathrm{in}} A_{\mathrm{span},i} G_i}{P_{\mathrm{ASE},i}^{(1)} + \tilde{\eta}_i (P_{\mathrm{ch},i}^{\mathrm{in}})^3}. \tag{15.121}$$

The *LOGO* principle (local optimization, global optimization) follows: maximizing each GSNR_i maximizes the total, with

$$P^{\mathrm{in}}_{\mathrm{ch},i,\mathrm{opt}} = \sqrt[3]{\frac{P^{(1)}_{\mathrm{ASE},i}}{2\tilde{\eta}_i}}, \qquad \mathrm{GSNR}_{i,\max} = \frac{2A_{\mathrm{span},i}G_i}{3\sqrt[3]{2\tilde{\eta}_i\left[P^{(1)}_{\mathrm{ASE},i}\right]^2}}. \tag{15.122}$$

The apparent circularity (the optimum input power depends on the gain, which sets the next input power) disappears for $G_i \gg 1$: the optimum powers follow from the span properties, and the gains are then set to deliver them. The 3 dB rule holds span by span. LOGO makes the management of large heterogeneous networks tractable. For arbitrary combs the NLI on a channel splits into self-channel interference (SCI, related to SPM) and cross-channel interference (XCI, XPM and FWM), given by the EGN model.

> **In the code** The split-step solver computes the NLI directly. For EDFA-only transparent links, `optical_link` also reports the GN estimate for the WDM comb in the `wdm` metadata (`gn_nli_dbm`, `gsnr_gn_db`, `gn_optimum_launch_dbm`), useful to choose the launch power before running the heavy nonlinear simulation.

Code 15.38: GN-model metrics of `optical_link`. `linkstudio/phy/optical.py`, lines 709–716

```
wdm = x.meta.get("wdm") or {}
eta = gn_eta(gamma, att, beta2(d, lam) * 1e27, rs, p.span_km, wdm.get("n_channels", 1),
             wdm.get("spacing_hz")) * a_pre ** 2  # launch power enters the fiber after A_pre
p_ase = 2 * s * rs  # ASE of one span in B = R_s, both polarizations
p_nli = p.n_spans * eta * launch ** 3
self._last.update({"gn_nli_dbm": w_to_dbm(p_nli),
                   "gsnr_gn_db": 10 * np.log10(launch / (p.n_spans * p_ase + p_nli)),
                   "gn_optimum_launch_dbm": w_to_dbm((p_ase / (2 * eta)) ** (1 / 3))})
```

15.14 Wavelength-division multiplexing

WDM is frequency-division multiplexing of optical carriers: each channel has its own laser and the channels share the fiber and the amplifiers. In the C band (about 191.7–196.1 THz) a 50 GHz grid holds about 100 channels, an 87.5 GHz grid about 60. Optical filters select channels with a transfer function on the field, $E_{\mathrm{out}}(f) = H_{\mathrm{opt}}(f)E_{\mathrm{in}}(f)$, as rectangular as possible: low loss in band, strong rejection outside. A guard band is needed for the filters (wavelength-selective switches in reconfigurable networks), which is why Nyquist WDM ($\delta_f \to 0$) is hard to reach.

`wdm_multiplexer` surrounds the channel under test with $N_{\mathrm{ch}} - 1$ neighbours at multiples of the spacing, carrying independent symbols with the same constellation, pulse shape and power. Propagating the composite field through a nonlinear fiber produces cross-phase modulation and four-wave mixing. `wdm_demultiplexer` selects a channel with a second-order super-Gaussian filter, $|H(f)| = \exp(-\frac{\ln 2}{2}(2f/B)^4)$, 3 dB down at $\pm B/2$. The sample rate must cover all channels.

Code 15.39: Channel selection by frequency shift and optical filtering. `linkstudio/phy/optical.py`, lines 794–800

```
shift = np.exp(-2j * np.pi * self.params.channel * spacing * np.arange(n) / fs)
f = np.fft.fftfreq(n, 1 / fs)
if self.params.shape == "rect":
    H = (np.abs(f) <= bw / 2).astype(float)
else:
    H = np.exp(-np.log(2) / 2 * (2 * f / bw) ** 4)  # -3 dB at +-bw/2
```

```
y = np.fft.ifft(np.fft.fft(x.data * shift, axis=-1) * H, axis=-1)
```

15.15 Free-space optics

15.15.1 Scintillation

Atmospheric turbulence makes the received intensity fluctuate (scintillation). `fso_turbulence` multiplies the field by $\sqrt{h}$, where the normalized irradiance h ($\mathbb{E}h = 1$) is constant over a frame and follows one of two models.

- **Log-normal** (weak turbulence): $\ln h \sim \mathcal{N}(-\sigma^2/2, \sigma^2)$.
- **Gamma-gamma** (moderate to strong turbulence): h is the product of two independent unit-mean gamma variables with parameters α (large eddies) and β (small eddies). Its density is

$$f(h) = \frac{2(\alpha\beta)^{\frac{\alpha+\beta}{2}}}{\Gamma(\alpha)\Gamma(\beta)}\, h^{\frac{\alpha+\beta}{2}-1} K_{\alpha-\beta}\!\left(2\sqrt{\alpha\beta h}\right). \tag{15.123}$$

Code 15.40: Scintillation models. `linkstudio/phy/optical.py`, lines 866–869

```
s2, a, b = self.model_parameters()
if self.params.model == "lognormal":
    return np.exp(self.rng.normal(-s2 / 2, np.sqrt(s2), n))
return self.rng.gamma(a, 1 / a, n) * self.rng.gamma(b, 1 / b, n)
```

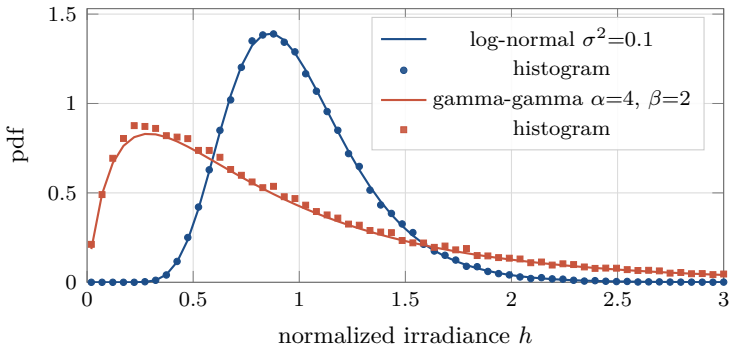


Figure 15.25: Histograms of h from `fso_turbulence` (60 000 frames) against the closed-form densities.

The parameters follow from the turbulence strength. For a plane wave the Rytov variance is $\sigma_R^2 = 1.23\, C_n^2 k^{7/6} L^{11/6}$, with C_n^2 the refractive-index structure parameter (about 10^{-16} to $10^{-13}\,\mathrm{m}^{-2/3}$ at optical wavelengths [37]). Al-Habash, Andrews and Phillips [36] give

$$\alpha = \left[e^{0.49\sigma_R^2/(1+1.11\sigma_R^{12/5})^{7/6}} - 1\right]^{-1}, \qquad \beta = \left[e^{0.51\sigma_R^2/(1+0.69\sigma_R^{12/5})^{5/6}} - 1\right]^{-1}. \tag{15.124}$$

For example, $\sigma_R^2 = 1$ gives $\alpha = 4.39$ and $\beta = 2.56$. The scintillation index of the gamma-gamma model is $\sigma_I^2 = 1/\alpha + 1/\beta + 1/(\alpha\beta)$. With `rytov_variance` set, `fso_turbulence` derives α and β, and the log-normal variance $\ln(1+\sigma_I^2)$, so both models share the same scintillation index. In weak turbulence that index tends to σ_R^2.

15.15.2 The complete link: `fso_channel`

`fso_channel` computes the whole budget of ITU-R P.1814 [37] from geometry and weather.

- **Atmospheric attenuation** from the visibility V (2 % contrast threshold, km), with Kim's size-distribution exponent:

$$\gamma_{\mathrm{sp}} = \frac{17}{V}\Big(\frac{0.55\,\mu\mathrm{m}}{\lambda}\Big)^{q}\ \mathrm{dB/km}, \qquad q = \begin{cases} 1.6 & V > 50 \\ 1.3 & 6 < V \le 50 \\ 0.16V + 0.34 & 1 \le V \le 6 \\ V - 0.5 & 0.5 \le V < 1 \\ 0 & V < 0.5. \end{cases} \tag{15.125}$$

  Light fog ($V = 1\,\mathrm{km}$) costs 10 dB/km at 1550 nm; haze at $V = 5\,\mathrm{km}$ about 1 dB/km.

- **Geometric loss**: the beam of full divergence θ covers the area $\frac{\pi}{4}(d\theta)^2$, so a receiver of diameter D_R collects $A_{\mathrm{geo}} = 20\log_{10}(d\theta/D_R)$ dB, or 0 when the aperture catches the whole beam.

- **Scintillation** from C_n^2 via σ_R^2, as above.

- **Pointing errors** [38]. Building sway gives a radial beam offset $r \sim \mathrm{Rayleigh}(\sigma_s)$ on the receiver plane. A Gaussian beam of radius $w_z = d\theta/2$ on an aperture of radius a then collects $h_p = e^{-2r^2/w_{z,\mathrm{eq}}^2}$ relative to perfect alignment, with $w_{z,\mathrm{eq}}^2 = w_z^2\sqrt{\pi}\,\mathrm{erf}(v)/(2v\,e^{-v^2})$ and $v = \sqrt{\pi/2}\,a/w_z$. On average $\mathbb{E}h_p = w_{z,\mathrm{eq}}^2/(w_{z,\mathrm{eq}}^2 + 4\sigma_s^2)$, which the test suite checks.

All fading terms are quasi-static (one value per frame) and appear on `csi`. The metrics give each loss term and σ_R^2. Aperture averaging, beam wander and time-correlated fading are not modelled. The *Free-space optical link budget* scenario runs a 1 Gb/s OOK link over 1 km with all four effects.

Code 15.41: Losses and fading of `fso_channel`. `linkstudio/phy/optical.py`, lines 915–935

```
atm = kim_attenuation_db_km(p.visibility_km, lam) * p.distance_m / 1e3 if p.visibility_km else 0.0
loss = 10 ** (-(atm + geometric_loss_db(p.distance_m, p.divergence_mrad, p.rx_aperture_m)) / 10)
h = np.ones(B)
s2r = rytov_variance(p.cn2, lam, p.distance_m)
if p.turbulence != "none" and s2r > 0:
    a, b = gamma_gamma_parameters(s2r)
    if p.turbulence == "gamma_gamma":
        h = self.rng.gamma(a, 1 / a, B) * self.rng.gamma(b, 1 / b, B)
    else:
        s2 = np.log1p(1 / a + 1 / b + 1 / (a * b))
        h = np.exp(self.rng.normal(-s2 / 2, np.sqrt(s2), B))
if p.pointing_jitter_m > 0:
    wz = p.distance_m * p.divergence_mrad * 1e-3 / 2
    v = np.sqrt(np.pi / 2) * (p.rx_aperture_m / 2) / wz
    wzeq2 = wz ** 2 * np.sqrt(np.pi) * erf(v) / (2 * v * np.exp(-v ** 2))
    r = p.pointing_jitter_m * np.sqrt(-2 * np.log(1 - self.rng.random(B)))
    hp = np.exp(-2 * r ** 2 / wzeq2)
    self._hp.extend(hp.tolist())
    h = h * hp
shape = (B,) + (1,) * (x.data.ndim - 1)
out = x.data * np.sqrt(loss) * np.sqrt(h).reshape(shape)
```

15.16 Summary of the optical models

Table 15.4 lists the optical blocks with the theory they implement and the validation that ties them to it.

Table 15.4: Optical blocks, models and validation.

block	model	validation
`dml`	single-mode rate equations, transient and adiabatic chirp, parasitic pole	threshold, slope, linearization, Koch–Bowers
`vcsel`	Shi et al.: Bessel expansion (hole burning, diffusion), Tucker thermal model	h_0, turn-off, L-I rollover, Figure 15.9
`intensity_modulator`	ideal PAM-M power levels from the extinction ratio	PAM levels, Figure 15.12
`mach_zehnder_modulator`	CW laser + MZM sine law, push-pull / single arm, ER, IL	sine law, ER, chirp
`iq_modulator`	nested MZMs, PM transmitter, quadrature error	Figure 15.21
`fiber`	NLSE / Manakov by split-step, PMD, fiber types	Figures 15.2 to 15.5
`optical_amplifier`	$\sqrt{G}E + n$, $n_{sp} = F/2$, saturation	OSNR formula
`optical_link`	multi-span, $GA \neq 1$, Raman, GN metrics	Figures 15.17 to 15.19 and 15.24
`optical_attenuator`	VOA, splitter, connectors	loss
`photodiode`	PIN/APD + TIA, IRND, shot, excess noise, reference receiver	Figures 15.11 and 15.12
`coherent_receiver`	90° hybrid, balanced PDs, LO noise, ISNR	ISNR, LO split
`multimode_fiber`	mode groups / IEEE link model	Figures 15.14 to 15.16

Chapter 16

Validation

A simulator is only useful if its numbers can be trusted. LinkStudio is validated at three levels:

- unit tests of the engine and the DSP helpers;
- *physics tests*, which compare complete chains with closed-form results;
- a test that loads and runs every shipped scenario.

The demo scripts repeat the physics checks in readable form, and this manual's figures are a third, visual check of the same models.

16.1 Running the checks

```
cd backend
python -m pytest                 # all tests (about 220, a few minutes)
python -m pytest tests/test_physics.py -q
python scripts/main.py           # demos: simulated vs theory, PASS/FAIL per check
python scripts/main.py --quick   # fewer errors per point, faster
```

Physics tests stop after 300–500 errors and accept a relative deviation of 20–25 %. By Section 5.3 this is several standard deviations, so false alarms are negligible, while a systematic error of 1 dB (a factor of 2–3 in BER on steep curves) is caught.

16.2 Physics checks

Model	Reference	Checked in
AWGN, PSK/QAM/PAM	exact Gray BER (Section 6.3)	`test_physics.py`, Figure 6.2
Soft demapping	max-log decisions = hard decisions	`test_physics.py`
Pulse shaping	RRC link BER = symbol-rate theory; noise metadata after MF	`test_physics.py`
OFDM over AWGN	per-subcarrier E_s/N_0 theory	`test_physics.py`
OFDM multipath	LS and perfect CSI equalization	`test_physics.py`, Figure 11.3
Rayleigh + ZF	Equation (9.6)	`test_physics.py`, Figure 12.2
ISI equalizers	ZF/MMSE/LMS remove ISI	`test_physics.py`
Soft vs hard Viterbi	soft better by ≈ 2 dB	`test_physics.py`
Synchronization	receiver close to theory with delay and CFO; fails without sync	`test_advanced.py`
Carrier recovery	tracks phase noise	`test_advanced.py`
Timing recovery	estimates the fractional delay	`test_advanced.py`
LDPC	codewords satisfy $\mathbf{H}\boldsymbol{c}^{\mathsf{T}} = 0$; corrects noise; coding gain	`test_advanced.py`
Polar	noiseless decoding; frozen set; coding gain	`test_advanced.py`
Puncturing	rate 3/4; decodable	`test_advanced.py`
SIMO MRC	Equation (12.3), $L = 2$	`test_advanced.py`, Figure 12.2

Model	Reference	Checked in
Spatial multiplexing	ZF 2×2 = SISO Rayleigh; ML < MMSE < ZF	`test_advanced.py`, Figure 12.1
Alamouti	MRC $L = 2$ shifted by 3 dB	`test_advanced.py`, Figure 12.2
Time-varying multipath	OFDM with Jakes taps	`test_advanced.py`
Line codes	round trips; 4B/5B and 64B/66B rates; error multiplication	`test_wireline_optical.py`
Real NRZ over AWGN	BPSK theory	`test_wireline_optical.py`
Cable	insertion loss equals the preset formula	`test_wireline_optical.py`, Figure 14.4
SerDes	CTLE peaking; CTLE + DFE open the eye	`test_wireline_optical.py`, Figure 14.5
DMT	water-filling bit loading	`test_wireline_optical.py`, Figure 14.6
Crosstalk, echo	coupling level; canceller reduces echo	`test_wireline_optical.py`
Jitter	zero jitter is identity; RJ degrades	`test_wireline_optical.py`
Fiber loss, metadata	$P\,e^{-\alpha L}$, $D\ell$	`test_wireline_optical.py`
Dispersion	Gaussian broadening; exact compensation	`test_wireline_optical.py`, Figure 15.2
SPM	CW phase $\gamma P L_{\text{eff}}$	`test_wireline_optical.py`, Figure 15.4
Split-step solver	second-order convergence on the soliton	Figure 15.5
EDFA, multi-span link	OSNR formulas ($n_{sp} = F/2$, $GA \neq 1$, course exercise), saturation	`test_optical_models.py`, Figures 15.17 and 15.18
Raman	on/off gain, lumped-limit ASE, $F_{\text{eq}} <$ 3 dB	`test_optical_models.py`, Figure 15.19
GN model	long-span form, 3 dB rule, link metrics; split-step comparison	`test_optical_models.py`, Figure 15.24
Lasers	threshold and slope, settling, linearized response, adiabatic chirp, VCSEL overlaps, thermal rollover, hole-burning turn-off	`test_lasers.py`, Figures 15.7 to 15.9
Modulators, detectors	MZM sine law and ER, PAM levels, TIA output, APD power law, ISNR	`test_optical_models.py`, Figure 15.11
PM-QAM	course BER formulas vs exact; back-to-back simulation	`test_optical_models.py`, Figure 15.21
IM/DD OOK, PAM-4	Q-factor BER; PAM-M formula of the course; dispersion limit	`test_wireline_optical.py`, `test_optical_models.py`, Figures 15.12 and 15.13
IM/DD PAM-4	dispersion-induced power fading	`test_wireline_optical.py`
Coherent DP-QPSK	BER vs OSNR theory within $\approx$ 0.3 dB; FO estimate	`test_wireline_optical.py`
FSO	mean and variance of the scintillation	`test_wireline_optical.py`, Figure 15.25
WDM	XPM lowers the MER of the centre channel	`test_wireline_optical.py`
Multimode fiber	α_{opt}, $\sigma_{\min}$, WKB delay, continuum limit, class bandwidths, launch closed forms, variance additivity	`test_multimode.py`, Figures 15.14 and 15.15

Model	Reference	Checked in
Data-center lane	Bessel–Thomson noise bandwidth; reach by fiber class	`test_multimode.py`, Figure 15.16
Fading generator	Zheng–Xiao: $R_h = J_0$, uncorrelated I/Q	`test_channels.py`, Figure 9.1
NR TDL profiles	unit normalized delay spread; TDL-D $K_1 = 13.3$ dB; fractional delay	`test_channels.py`
Path loss	P.525; TR 38.901 breakpoint continuity, NLOS $\geq$ LOS, shadowing, LOS probability	`test_propagation.py`, Figure 9.2
Thermal noise	$k_B T F f_s$; free-space QPSK link budget BER	`test_propagation.py`
Copper loops	G.996.1 Mid-C insertion loss within 0.35 dB; causal dielectric	`test_wireline_optical.py`, Figure 14.3
Dispersion slope	group delay $D\ell\Delta\lambda + \frac{S\ell}{2}\Delta\lambda^2$; CD+slope compensation	`test_wireline_optical.py`
PMD	Maxwellian DGD (mean, $\langle\tau^2\rangle/\langle\tau\rangle^2$)	`test_wireline_optical.py`, Figure 15.3
Adaptive split step	soliton error $< 10^{-3}$ with default criterion	`test_wireline_optical.py`
DML, coherent RX	adiabatic chirp frequency step; LO split per polarization	`test_wireline_optical.py`
FSO channel	α, β from σ_R^2; Kim, geometric and pointing losses	`test_wireline_optical.py`
CP-based OFDM sync	recovers delay and CFO	`test_wireline_optical.py`
Confidence interval	Wilson bounds contain the BER	`test_wireline_optical.py`
All scenarios	load, validate, run; BER below a threshold	`test_scenarios.py`

16.3 Known limitations

Every model in this manual states its assumptions; the most important simplifications are collected here.

- Laser dynamics are small-signal (no large-signal rate equations). The multimode fiber omits mode-partition and modal noise.
- FSO fading is quasi-static (no time correlation, aperture averaging or beam wander).
- WDM simulates the full multi-channel field but demodulates only one channel.
- The OFDM CP synchronizer estimates the fractional CFO only.
- DMT adapts bits to a flat transmit PSD (no power water-filling).
- MIMO channels are flat; frequency-selective MIMO-OFDM is not yet a block.
- Subsystems are edited in their YAML files, not in the GUI.
- Error counts assume independent errors; for coded and block-fading links, also judge the accuracy by the number of frame errors.

Part III

Reference

Chapter A

Block reference

This appendix lists the 94 registered blocks, grouped by functional class in signal-chain order as in the GUI palette. It is generated by `manual/tools/generate.py` from `block_catalogue()`, the same source as the GUI palette and `python -m linkstudio describe`. Each entry gives:

- the first paragraph of the block documentation;
- the input and output ports with their accepted kinds;
- the parameters with their defaults. *auto* means the value is taken from upstream metadata or measured; *required* means the parameter has no default.

A.1 SOURCES

prbs Pseudo-random binary sequence (PRBS-7/9/11/15/20/23/31), continuous across frames.

in: - out: out:BITS

order=9; n_bits=1024

random_bits Random bits, a new random frame for every batch row.

in: - out: out:BITS

n_bits=1024; p_one=0.5; transition_probability=auto

A.2 ENCODERS

alamouti_encoder Alamouti space-time block code: 2 transmit antennas, rate 1, full diversity.

in: in:SYMBOLS out: out:SYMBOLS

–

conv_encoder Feed-forward convolutional encoder of rate 1/n.

in: in:BITS out: out:BITS

generators=[171, 133]; constraint_length=7; terminate=True

crc_encoder Append CRC parity bits to every frame.

in: in:BITS out: out:BITS

polynomial=crc16

hamming_encoder Systematic Hamming encoder (frame length must be a multiple of k).

in: in:BITS out: out:BITS

m=3

ldpc_encoder Systematic quasi-cyclic LDPC encoder (frame length must be a multiple of k).

in: in:BITS out: out:BITS

k_blocks=8; m_blocks=8; z=48; col_weight=3; seed=1

line_block_encoder 4B/5B (FDDI, 100BASE-TX) or 64B/66B (10G/100G Ethernet) block line code.

in: in:BITS out: out:BITS

code=64b66b

polar_encoder Polar encoder (frame length must be a multiple of k).

in: in:BITS out: out:BITS

block_length=256; k=128; design_snr_db=0

puncturer Remove coded bits according to a periodic pattern (raises the code rate).

in: `in:BITS` out: `out:BITS`

`pattern`=[1, 1, 1, 0, 0, 1]

repetition_encoder Repeat every bit `n` times (rate 1/n).

in: `in:BITS` out: `out:BITS`

`n`=3

A.3 BIT PROCESSING

interleaver Permute the bits (or LLRs) of every frame.

in: `in:BITS|LLR|SYMBOLS` out: `out:same as 'in'`

`kind`=random; `rows`=16; `seed`=1234; `inverse`=False

scrambler XOR every frame with the same LFSR sequence.

in: `in:BITS|LLR` out: `out:same as 'in'`

`taps`=[7, 4]; `seed`=93

A.4 MAPPERS

line_encoder Baseband line code: bits -> real symbols.

in: `in:BITS` out: `out:SYMBOLS`

`code`=nrz; `symbol_rate`=1e+09

mapper Map groups of bits onto Gray-labelled constellation points (unit average energy).

in: `in:BITS` out: `out:SYMBOLS`

`scheme`=qam; `order`=4; `phase_offset_deg`=auto; `points`=auto; `symbol_rate`=1e+06

A.5 MULTIPLEXERS

stream_mux Inverse of `stream_demux`: interleave the streams back into one sequence.

in: `in:SYMBOLS` out: `out:SYMBOLS`

wdm_multiplexer Wavelength-division multiplexing: add neighbouring channels around the channel under test.

in: `in:SAMPLES` out: `out:SAMPLES`

`n_channels`=5; `spacing_ghz`=50; `interferer_power_db`=0

A.6 FRAMING

preamble Prepend a known preamble and append guard symbols to every frame.

in: `in:SYMBOLS` out: `out:SYMBOLS`

`length`=64; `guard`=16; `seed`=2024

A.7 MODULATORS

dml Directly modulated single-mode laser (DFB) solved from its rate equations.

in: in:SYMBOLS|SAMPLES out: out:SAMPLES

carrier_lifetime_ns=1; photon_lifetime_ps=2.5; gain_coefficient=80000; transparency_carriers=5.74e+07; gain_compression=1.6e-07; spontaneous_coupling=1e-05; linewidth_enhancement=4; optical_efficiency=0.3; rin_db_hz=auto; linewidth_hz=0; bias_ma=60; modulation_ma=36; wavelength_nm=1310; parasitic_ghz=35

dmt_modulator Discrete multitone (ADSL/VDSL): real IFFT with Hermitian symmetry, per-tone bit loading.

in: in:BITS out: out:SAMPLES

n_fft=512; cp_len=40; first_tone=33; last_tone=255; sample_rate=2.208e+06; loading=waterfill; uniform_bits=4; max_bits=14; gap_db=9.8; margin_db=3; snr_db=60; line_preset=dsl_0.4mm; line_length_m=2000

intensity_modulator Ideal intensity modulation for IM/DD links (PAM-M power levels set by the extinction ratio).

in: in:SYMBOLS|SAMPLES out: out:SAMPLES

power_dbm=0; extinction_ratio_db=10; bandwidth_ghz=auto; rin_db_hz=auto; linewidth_hz=0; wavelength_nm=1550

iq_modulator Laser + nested IQ Mach-Zehnder modulator for coherent transmission (single or dual polarization).

in: in:SYMBOLS|SAMPLES out: out:SAMPLES

power_dbm=0; model=linear; drive_swing=0.5; extinction_ratio_db=auto; quadrature_error_deg=0; bandwidth_ghz=auto; linewidth_hz=100000; wavelength_nm=1550

mach_zehnder_modulator CW laser and external Mach-Zehnder modulator for IM/DD (lithium-niobate MZM).

in: in:SYMBOLS|SAMPLES out: out:SAMPLES

laser_power_dbm=13; linewidth_hz=1e+06; rin_db_hz=auto; v_pi=4; bias_v=auto; swing_v=auto; drive=push_pull; extinction_ratio_db=25; insertion_loss_db=4; bandwidth_ghz=auto; wavelength_nm=1550

ofdm_modulator Map data symbols and pilots on subcarriers, IFFT and add the cyclic prefix.

in: in:SYMBOLS out: out:SAMPLES

n_fft=64; cp_len=16; n_active=52; pilot_spacing=6; subcarrier_spacing=15000; tail_zeros=0

pulse_shaper Upsample symbols by sps and filter them with a unit-energy pulse.

in: in:SYMBOLS out: out:SAMPLES

shape=rrc; sps=8; span=10; rolloff=0.25; bt=0.3

vcsel Directly modulated VCSEL with spatial hole burning and self-heating (Shi et al. 2009).

in: in:SYMBOLS|SAMPLES out: out:SAMPLES

carrier_lifetime_ns=1; photon_lifetime_ps=1.8; gain_coefficient=360000; transparency_carriers=1.85e+06; gain_compression=1.5e-06; spontaneous_coupling=0.0001; linewidth_enhancement=3; optical_efficiency=0.38; rin_db_hz=-140; linewidth_hz=0; bias_ma=7; modulation_ma=3; wavelength_nm=850; spectral_width_nm=0.4; parasitic_ghz=25; spatial_hole_burning=True; active_radius_um=3.5; mode_radius_um=2.5; diffusion_length_um=1.5; ambient_c=25; thermal=True; thermal_resistance_k_mw=3; leakage_coefficients=[0.0, 0.0, 0.0, 2e-09]; diode_ideality=2; saturation_current_a=1e-15; saturation_b1=0.02; saturation_b2=-50; series_c1=6250; series_c2=-100

A.8 FILTERS AND RESAMPLERS

downsampler Keep one sample out of `factor` starting at `offset`.

in: `in:SYMBOLS|SAMPLES` out: `out:SYMBOLS|SAMPLES`

`factor`=2; `offset`=0; `output_kind`=samples

fir_filter Generic FIR filter with user-supplied taps (causal, output length preserved).

in: `in:SYMBOLS|SAMPLES` out: `out:same as 'in'`

`taps`=[1.0]

upsampler Insert `factor - 1` zeros between samples.

in: `in:SYMBOLS|SAMPLES` out: `out:SAMPLES`

`factor`=2

A.9 AMPLIFIERS AND ATTENUATORS

agc Automatic gain control: remove the mean (optional) and scale every frame and stream to the nominal symbol power (`power` metadata of the upstream mapper / line code).

in: `in:SYMBOLS|SAMPLES` out: `out:same as 'in'`

`remove_dc`=True; `target_power`=auto

gain Scale the signal by `gain_db` (e.g. path loss). Noise metadata is scaled too.

in: `in:SYMBOLS|SAMPLES` out: `out:same as 'in'`

`gain_db`=0

optical_amplifier Optical amplifier (EDFA, SOA) with amplified spontaneous emission noise.

in: `in:SAMPLES` out: `out:SAMPLES`

`mode`=power; `gain_db`=20; `target_power_dbm`=0; `noise_figure_db`=5; `saturation_power_dbm`=auto

optical_attenuator Passive loss: variable optical attenuator, 1xN power splitter, connectors.

in: `in:SAMPLES` out: `out:SAMPLES`

`loss_db`=0; `split_ways`=1; `excess_loss_db`=0

A.10 TRANSMISSION MEDIA

awgn Add circularly-symmetric complex Gaussian noise.

in: `in:SYMBOLS|SAMPLES` out: `out:same as 'in'`

`snr_db`=10; `mode`=ebn0; `power_reference`=nominal; `signal_power`=auto; `bits_per_symbol`=auto; `code_rate`=auto; `noise_figure_db`=5; `temperature_k`=290

copper_line Cable channel with frequency-dependent loss (twisted pair, coax, PCB trace).

in: `in:SYMBOLS|SAMPLES` out: `out:same as 'in', csi:CSI`

`preset`=cat5e; `length_m`=100; `termination_ohm`=100; `a`=auto; `b`=auto; `max_taps`=256

crosstalk Near-end (NEXT) or far-end (FEXT) crosstalk from `n_disturbers` similar pairs.

in: `in:SYMBOLS|SAMPLES, tx:SYMBOLS|SAMPLES` out: `out:same as 'in'`

`model`=next; `n_disturbers`=10; `preset`=dsl_0.4mm; `length_m`=1000

fiber Single-mode fiber span: attenuation, chromatic dispersion, optional Kerr nonlinearity and PMD.

in: `in:SAMPLES` out: `out:SAMPLES`

`fiber_type`=custom; `attenuation_db_km`=0.2; `dispersion_ps_nm_km`=17; `nonlinear`=False; `dispersion_slope_ps_nm2_km`=0; `gamma_w_km`=1.3; `connector_loss_db`=0; `steps_per_km`=0.5; `nonlinear_phase_rad`=0.05; `pmd_ps_sqrt_km`=0; `pmd_sections`=10; `polarization_rotation`=False; `length_km`=80

flat_fading Frequency-flat Rayleigh or Rician fading, E|h|^2 = 1.

in: in:SYMBOLS|SAMPLES out: out:same as 'in', csi:CSI

model=rayleigh; k_factor_db=0; variation=block; doppler_hz=100; n_sinusoids=16

fso_channel Terrestrial free-space optical link from geometry and weather.

in: in:SAMPLES out: out:SAMPLES, csi:CSI

distance_m=1000; visibility_km=auto; divergence_mrad=2; rx_aperture_m=0.08; cn2=1e-14; turbulence=gamma_gamma; pointing_jitter_m=0

fso_turbulence Free-space optical scintillation: log-normal (weak) or gamma-gamma (moderate/strong).

in: in:SAMPLES out: out:SAMPLES, csi:CSI

model=lognormal; log_intensity_variance=0.1; alpha=4; beta=2; rytov_variance=auto

hybrid_echo Leakage of the local transmitter into the receiver through the hybrid (full duplex).

in: in:SYMBOLS|SAMPLES, tx:SYMBOLS|SAMPLES out: out:same as 'in'

echo_db=-10; taps=[1.0, 0.6, 0.3, 0.1]

impulse_noise Bursts of strong Gaussian noise (electrical switching, appliances).

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

burst_probability=0.001; burst_samples=32; power_db=10

mimo_channel Flat Rayleigh MIMO channel y = H x, H of size Nr x Nt, normalized by 1/sqrt(Nt).

in: in:SYMBOLS|SAMPLES out: out:same as 'in', csi:CSI

n_rx=2; variation=block; tx_correlation=0; rx_correlation=0

multimode_fiber Graded-index multimode fiber (OM1-OM5) for short-reach IM/DD links.

in: in:SAMPLES out: out:SAMPLES

fiber_type=OM4; length_m=100; model=bandwidth; modal_bandwidth_mhz_km=auto; attenuation_db_km=auto; alpha=auto; profile_dispersion=0; launch=overfilled; launch_waist_um=4; launch_offset_um=0; group_weights=[]; core_index=1.467; group_index=1.482; zero_dispersion_nm=1320; dispersion_slope=0.101; source_rms_width_nm=auto

multipath Tapped-delay-line channel (frequency selective).

in: in:SYMBOLS|SAMPLES out: out:same as 'in', csi:CSI

preset=auto; delay_spread_ns=100; fractional_delays=True; delays=[0.0, 1.0, 2.0]; powers_db=[0.0, -3.0, -6.0]; delay_unit=samples; fading=True; variation=block; doppler_hz=50; normalize=True

optical_link Multi-span amplified link: `n_spans` x (connectors + fiber span + optional Raman + EDFA).

in: in:SAMPLES out: out:SAMPLES

fiber_type=custom; attenuation_db_km=0.2; dispersion_ps_nm_km=17; nonlinear=False; dispersion_slope_ps_nm2_km=0; gamma_w_km=1.3; connector_loss_db=0; steps_per_km=0.5; nonlinear_phase_rad=0.05; pmd_ps_sqrt_km=0; pmd_sections=10; polarization_rotation=False; n_spans=10; span_km=80; noise_figure_db=5; amplifier_gain_db=auto; launch_power_dbm=auto; raman_pump_mw=0; raman_gain_w_km=0.4; raman_pump_attenuation_db_km=0.25

path_loss Distance-dependent path loss and log-normal shadow fading (one value per frame).

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

model=free_space; frequency_ghz=3.5; distance_m=100; h_bs_m=auto; h_ut_m=auto; condition=probabilistic; shadow_fading=True

A.11 IMPAIRMENTS

carrier_offset Carrier frequency offset and phase rotation: y = x * exp(j(2 pi cfo t + phi)).

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

cfo_hz=0; phase_deg=0; random_phase=False

iq_imbalance Receiver IQ imbalance and DC offset: y = K1 x + K2 conj(x) + dc.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

amplitude_db=0; phase_deg=0; dc_i=0; dc_q=0

jitter Sampling-clock jitter: random (RJ, per symbol) and sinusoidal (SJ), in unit intervals.

in: in:SAMPLES out: out:same as 'in'

rj_rms_ui=0.02; sj_amplitude_ui=0; sj_frequency_hz=1e+06

phase_noise Oscillator phase noise modelled as a Wiener process with 3-dB linewidth_hz.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

linewidth_hz=100

power_amplifier Memoryless PA nonlinearity (Rapp, Saleh or ideal clipper).

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

model=rapp; input_backoff_db=6; smoothness=2; saleh_beta_a=1.1517; saleh_alpha_p=4.0033; saleh_beta_p=9.104; normalize_output=True

quantizer Uniform mid-rise ADC on I and Q with clipping.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

bits=8; full_scale=auto; loading_factor=4

timing_offset Delay the signal by a (fractional) number of samples, keeping its length.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

delay_samples=0

A.12 DEMODULATORS

coherent_receiver Coherent front-end: local oscillator, 90-degree hybrid and balanced photodiodes.

in: in:SAMPLES out: out:SAMPLES

responsivity=0.8; thermal_noise_pa_rthz=20; shot_noise=True; lo_power_dbm=13; lo_linewidth_hz=100000; frequency_offset_hz=0; hybrid_phase_error_deg=0; internal_snr_db=auto

dmt_demodulator Remove the cyclic prefix, FFT, one-tap frequency-domain equalization and per-tone demapping.

in: in:SAMPLES, csi:CSI out: out:BITS

matched_filter Receive filter + symbol-rate sampling.

in: in:SAMPLES out: out:SYMBOLS|SAMPLES

shape=matched; sps=auto; span=10; rolloff=0.25; bt=0.3; normalize=True; decimate=True

ofdm_demodulator Remove the cyclic prefix and FFT every OFDM symbol into a resource grid.

in: in:SAMPLES out: out:GRID

n_fft=auto; cp_len=16; n_active=52; pilot_spacing=6; fft_offset=0

photodiode Direct-detection receiver: PIN or APD photodiode and transimpedance amplifier (TIA).

in: in:SAMPLES out: out:SAMPLES

responsivity=0.8; thermal_noise_pa_rthz=20; shot_noise=True; quantum_efficiency=auto; dark_current_na=5; apd_gain=1; apd_excess_noise=mcintyre; apd_k=0.2; apd_x=0.7; bandwidth_ghz=auto; transimpedance_ohm=auto

A.13 SYNCHRONIZATION

carrier_recovery Decision-directed second-order PLL tracking residual phase and frequency offsets.

in: in:SYMBOLS out: out:SYMBOLS

loop_bandwidth=0.005; damping=0.707

fo_compensation Blind frequency-offset estimation from the spectral peak of the M-th power (M = 4 for QPSK / square QAM), joint over polarizations, and correction.

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

power=4; estimation_bandwidth=0.75

frame_sync Preamble-based frame synchronization.

in: in:SYMBOLS out: out:SYMBOLS

search=32; correct_cfo=True

ofdm_sync Blind OFDM timing and fractional CFO synchronization from the cyclic prefix (van de Beek).

in: in:SAMPLES out: out:SAMPLES

max_delay=auto; backoff=auto; correct_cfo=True

phase_recovery Feed-forward carrier phase recovery for large laser linewidths.

in: in:SYMBOLS, ref:SYMBOLS out: out:SYMBOLS

method=bps; n_test_phases=32; half_window=16; n_ref=64; ref_start=600

timing_recovery Symbol timing recovery: Gardner detector, cubic interpolator, second-order loop.

in: in:SAMPLES out: out:SYMBOLS

loop_bandwidth=0.01; damping=0.707; detector_gain=1; initial_estimate=True; estimate_symbols=128

A.14 EQUALIZERS

adaptive_mimo_equalizer Fractionally spaced 2x2 (NxN) butterfly FIR equalizer for coherent receivers.

in: in:SAMPLES, ref:SYMBOLS out: out:SYMBOLS

n_taps=15; step_size=0.001; train_step_size=0.01; algorithm=lms; n_train=512; sps=2

cd_compensation Frequency-domain chromatic dispersion compensation (inverse all-pass filter).

in: in:SAMPLES out: out:SAMPLES

dispersion_ps_nm=auto; slope_ps_nm2=auto; wavelength_nm=auto

ctle Continuous-time linear equalizer: H(s) = A (1 + s/wz) / ((1 + s/wp1)(1 + s/wp2)).

in: in:SYMBOLS|SAMPLES out: out:same as 'in'

zero_hz=1e+09; pole1_hz=6e+09; pole2_hz=2.5e+10; dc_gain_db=0

dfe_equalizer Adaptive feed-forward + decision-feedback equalizer (NLMS), symbol spaced.

in: in:SYMBOLS, ref:SYMBOLS out: out:SYMBOLS

ffe_taps=7; dfe_taps=4; step_size=0.05; n_train=300

echo_canceller NLMS adaptive filter estimating the echo from the local `tx` signal and subtracting it.

in: in:SYMBOLS|SAMPLES, tx:SYMBOLS|SAMPLES out: out:same as 'in'

n_taps=16; train_step_size=0.05; step_size=0.002; train_samples=1000

linear_equalizer FIR equalizer designed from the known channel impulse response (`csi` input).

in: in:SYMBOLS, csi:CSI out: out:SYMBOLS

n_taps=15; method=mmse; delay=auto

lms_equalizer Adaptive (N)LMS FIR equalizer: trained on known symbols, then decision directed.
in: in:SYMBOLS, ref:SYMBOLS out: out:SYMBOLS
n_taps=11; step_size=0.1; normalized=True; n_train=200; decision_directed=True

ofdm_equalizer Extract the data subcarriers and equalize them (one-tap ZF per subcarrier).
in: in:GRID, csi:CSI out: out:SYMBOLS

one_tap_equalizer Coherent equalization of flat fading with known gains (csi input).
in: in:SYMBOLS, csi:CSI out: out:SYMBOLS
method=zf

A.15 DETECTORS AND ESTIMATORS

mimo_detector Linear (ZF, MMSE) or maximum-likelihood MIMO detection with known channel.
in: in:SYMBOLS, csi:CSI out: out:SYMBOLS
method=mmse

ofdm_channel_estimator Estimate the channel frequency response on every subcarrier.
in: in:GRID, csi:CSI out: out:CSI
method=ls; average_symbols=False

A.16 DEMULTIPLEXERS

stream_demux Spatial multiplexing: distribute symbols round-robin over n_streams antennas.
in: in:SYMBOLS out: out:SYMBOLS
n_streams=2

wdm_demultiplexer Select one WDM channel with an optical band-pass filter (rectangular or 2nd-order super-Gaussian).
in: in:SAMPLES out: out:SAMPLES
channel=0; bandwidth_ghz=auto; shape=supergaussian

A.17 DEMAPPERS

demapper Hard or soft (LLR) demapping.
in: in:SYMBOLS out: out:BITS|LLR
method=maxlog; scheme=auto; order=4; phase_offset_deg=auto; points=auto; noise_var=auto

line_decoder Hard decisions for NRZ / Manchester / AMI / MLT-3 symbols (the real part is used).
in: in:SYMBOLS|SAMPLES out: out:BITS
code=nrz; threshold=0.5

A.18 DECODERS

alamouti_decoder Linear Alamouti combining with known (block-fading) channel and any number of RX antennas.
in: in:SYMBOLS, csi:CSI out: out:SYMBOLS

crc_decoder Check and strip the CRC; reports the frame error rate detected by the CRC.
in: in:BITS|LLR out: out:BITS, valid:FLAGS
polynomial=crc16

depuncturer Re-insert punctured positions as erasures (LLR = 0) before decoding.

in: in:BITS|LLR out: out:LLR

pattern=[1, 1, 1, 0, 0, 1]

hamming_decoder Syndrome decoding (hard decision; LLR inputs are sliced first).

in: in:BITS|LLR out: out:BITS

m=3

ldpc_decoder Belief-propagation LDPC decoder (normalized min-sum or sum-product) with early stopping.

in: in:BITS|LLR out: out:BITS

k_blocks=8; m_blocks=8; z=48; col_weight=3; seed=1; max_iter=30; method=min_sum; alpha=0.8

line_block_decoder Inverse of `line_block_encoder`; counts invalid code words / sync headers.

in: in:BITS|LLR out: out:BITS

code=64b66b

polar_decoder Successive-cancellation polar decoder (soft LLR or hard input).

in: in:BITS|LLR out: out:BITS

block_length=256; k=128; design_snr_db=0

repetition_decoder Soft (LLR sum) or hard (majority vote) repetition decoding.

in: in:BITS|LLR out: out:BITS

n=3

viterbi_decoder Maximum-likelihood sequence decoder; soft decisions with LLR input, hard with bits.

in: in:BITS|LLR out: out:BITS

generators=[171, 133]; constraint_length=7; terminate=True

A.19 MEASUREMENTS

ber_meter Count bit, symbol and frame errors between transmitted and received bits.

in: tx:BITS, rx:BITS|LLR out: -

bits_per_symbol=auto; skip_bits=0

evm_meter Error vector magnitude and modulation error ratio between reference and received symbols.

in: ref:SYMBOLS, in:SYMBOLS out: -

skip_symbols=0

optical_power_meter Average optical power (dBm) and OSNR (0.1 nm) of an optical signal.

in: in:SAMPLES out: -

power_meter Average power and peak-to-average power ratio of a waveform.

in: in:SYMBOLS|SAMPLES out: -

scope Plots a tapped signal; the available views depend on its kind.

in: in1:BITS|LLR|SYMBOLS|SAMPLES|GRID|CSI, in2:BITS|LLR|SYMBOLS|SAMPLES|GRID|CSI, in3:BITS|LLR|SYMBOLS|SAMPLES|GRID|CSI, in4:BITS|LLR|SYMBOLS|SAMPLES|GRID|CSI out: -

channel=1; view=auto

Chapter B

Metadata keys

Metadata travels with every signal (Section 2.8). The table lists the keys used by the built-in library, the blocks that set them and the blocks that read them.

Key	Meaning	Set by / read by
`bits_per_symbol`	bits per constellation symbol k	mapper, line encoder / awgn (E_b/N_0), BER meter
`constellation`	the `Constellation` object	mapper / demapper, equalizers (decisions), PLL, BPS, AGC, WDM
`code_rate`	cumulative code rate R (with tails, CRC, puncturing)	encoders, CRC, puncturer, block line codes / awgn
`power`	nominal mean power per sample	mapper, pulse shaper, OFDM/DMT, optical blocks / awgn (SNR reference), AGC, OSNR
`sps`	samples per symbol N_{sps}	pulse shaper, rate changers, OFDM (N/N_{act}) / awgn, matched filter, timing recovery, equalizers
`delay`	known processing delay in samples	pulse shaper, matched filter, FIR / matched filter, timing recovery, MIMO equalizer
`pulse_taps`	transmit pulse	pulse shaper / matched filter, WDM multiplexer
`n_symbols`	symbols per frame	mapper, pulse shaper, preamble, MIMO / matched filter, equalizers
`noise_var`	noise variance per complex sample or symbol (scalar or array)	awgn, equalizers, detectors, photodiode, coherent receiver / soft demapper, AGC, MIMO detector
`spatial_rate`	streams per channel use N_s	stream demux / awgn
`csi_type`	form of CSI: `gain`, `cir`, `cir_tv`, `freq`, `mimo`	channels, estimators / equalizers, detectors
`ofdm`	`OFDMLayout`	OFDM modulator / demodulator, estimator, equalizer, sync
`n_data_symbols`, `data_power`	payload size and data-subcarrier power	OFDM modulator / OFDM equalizer
`preamble`, `n_payload_symbols`	preamble sequence and payload size	preamble / frame sync
`punctured_length`	mother-code length before puncturing	puncturer / depuncturer
`line_code`	line code name	line encoder / AGC (target power)
`dmt`, `n_bits`	DMT modulator and payload size	DMT modulator / DMT demodulator
`optical`	the signal is an optical field	optical transmitters / detectors
`wavelength_nm`	carrier wavelength	optical transmitters / fiber, EDFA, CD and FO compensation
`cd_ps_nm`	accumulated dispersion $D\ell$	fiber, link / CD compensation

Key	Meaning	Set by / read by
`cd_compensated_ps_nm`	dispersion removed so far	CD compensation / FO compensation (bulk-delay fix)
`ase_psd`	accumulated ASE density per polarization	EDFA, link, fiber / OSNR metrics
`wdm`	channel count and spacing	WDM multiplexer / WDM demultiplexer
`unit`	physical unit of the samples ($\sqrt{\text{W}}$, A); absent when normalized	optical transmitters, photodiode, coherent receiver (cleared by AGC and the MIMO equalizer) / probe views

Chapter C

Scenario library

The 42 scenario files shipped in `backend/scenarios/` (subsystem files excluded) are listed below. The table is generated from the files themselves. Open them from the GUI scenario browser (Ctrl+O) or run them with `python -m linkstudio run <file>`. Each scenario's `description` field explains what to observe and which parameters to sweep.

Category	File	Name	Blocks	Mode
Tutorials	`tutorials/01_bpsk_awgn.yaml`	1 - BPSK over AWGN	5	simulate
Tutorials	`tutorials/02_qam_soft_demapping.yaml`	2 - 16-QAM and soft demapping	6	simulate
Tutorials	`tutorials/03_pulse_shaping.yaml`	3 - Pulse shaping and the eye diagram	8	simulate
Tutorials	`tutorials/04_rayleigh_fading.yaml`	4 - Rayleigh fading	7	simulate
Tutorials	`tutorials/05_coding_gain.yaml`	5 - Coding gain (two branches)	11	simulate
Tutorials	`tutorials/06_isi_equalization.yaml`	6 - ISI and adaptive equalization	7	simulate
Wireless	`wireless/alamouti_2x1.yaml`	Alamouti 2x1 transmit diversity	8	simulate
Wireless	`wireless/link_budget_umi.yaml`	Urban micro-cell link budget	7	simulate
Wireless	`wireless/lte_like_ofdm.yaml`	LTE-like OFDM, mobile channel	14	simulate
Wireless	`wireless/mimo_2x2_smux.yaml`	2x2 MIMO spatial multiplexing	9	simulate
Wireless	`wireless/ofdm_eva_ls.yaml`	OFDM over EVA multipath	11	simulate
Wireless	`wireless/ofdm_synchronization.yaml`	OFDM timing and CFO synchronization	12	simulate
Wireless	`wireless/satellite_8psk.yaml`	Satellite link, 8-PSK with TWTA	12	simulate
Wireless	`wireless/single_carrier_16qam.yaml`	RRC 16-QAM link	11	simulate
Wireless	`wireless/synchronized_receiver.yaml`	Synchronized QPSK receiver	14	simulate
Wireless	`wireless/wifi_like_ofdm.yaml`	Wi-Fi-like OFDM (802.11a/g)	17	simulate
Coding	`coding/crc_conv_fading.yaml`	CRC + convolutional code over fading	13	simulate
Coding	`coding/hamming_block_code.yaml`	Hamming code, hard decisions	7	simulate
Coding	`coding/ldpc_qpsk.yaml`	LDPC-coded QPSK	7	simulate
Coding	`coding/polar_sc.yaml`	Polar code with SC decoding	7	simulate
Coding	`coding/punctured_conv.yaml`	Punctured convolutional code (rate 3/4)	9	simulate
Wireline	`wireline/10gbase_kr_backplane.yaml`	10GBASE-KR backplane	13	simulate
Wireline	`wireline/adsl_dmt.yaml`	ADSL-like DMT over a telephone pair	7	simulate
Wireline	`wireline/ethernet_100base_tx.yaml`	Fast Ethernet 100BASE-TX	14	simulate

Category	File	Name	Blocks	Mode
Wireline	`wireline/line_code_spectra.yaml`	Line code spectra	9	simulate
Wireline	`wireline/serdes_pam4_backplane.yaml`	SerDes 25 GBd PAM-4 backplane	12	simulate
Wireline	`wireline/vdsl_fext.yaml`	VDSL-like DMT with far-end crosstalk	7	simulate
Optical	`optical/apd_access_link.yaml`	O-band access link with APD	9	simulate
Optical	`optical/coherent_dp_16qam.yaml`	Coherent DP-16QAM metro link	17	simulate
Optical	`optical/coherent_dp_qpsk.yaml`	Coherent DP-QPSK long haul	17	simulate
Optical	`optical/datacenter_vcsel_mmf.yaml`	25G VCSEL multimode data-center link	12	simulate
Optical	`optical/fso_link_budget.yaml`	Free-space optical link budget	9	simulate
Optical	`optical/fso_turbulence.yaml`	Free-space optical link with turbulence	8	simulate
Optical	`optical/imdd_mzm_dispersion_limit.yaml`	10G OOK with a Mach-Zehnder modulator and the dispersion limit	11	simulate
Optical	`optical/imdd_pam4.yaml`	56 Gb/s PAM-4 with a directly modulated laser	11	simulate
Optical	`optical/long_haul_pm16qam_osnr.yaml`	3000 km PM-16QAM OSNR budget	16	simulate
Optical	`optical/ook_receiver_sensitivity.yaml`	10G OOK receiver sensitivity	7	simulate
Optical	`optical/raman_hybrid_link.yaml`	Raman-assisted long-haul link	15	simulate
Optical	`optical/wdm_nonlinear.yaml`	WDM with fiber nonlinearity	19	simulate
Examples (CLI)	`coded_link_subsystems.yaml`	coded-16qam-subsystems	4	simulate
Examples (CLI)	`qpsk_awgn_sweep.yaml`	qpsk-awgn-sweep	5	sweep
Examples (CLI)	`rrc_16qam_impairments.yaml`	rrc-16qam-impairments	12	simulate

Two reusable subsystems, `subsystems/coded_qam_tx.yaml` and `subsystems/coded_qam_rx.yaml`, are used by `coded_link_subsystems.yaml`.

Bibliography

[1] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. McGraw-Hill, 2008.

[2] G. P. Agrawal, *Nonlinear Fiber Optics*, 6th ed. Academic Press, 2019.

[3] U. Mengali and A. N. D'Andrea, *Synchronization Techniques for Digital Receivers*. Plenum Press, 1997.

[4] K. Cho and D. Yoon, "On the general BER expression of one- and two-dimensional amplitude modulations," *IEEE Trans. Commun.*, vol. 50, no. 7, pp. 1074–1080, 2002.

[5] J. W. Craig, "A new, simple and exact result for calculating the probability of error for two-dimensional signal constellations," in *Proc. IEEE MILCOM*, 1991.

[6] A. A. M. Saleh, "Frequency-independent and frequency-dependent nonlinear models of TWT amplifiers," *IEEE Trans. Commun.*, vol. 29, no. 11, pp. 1715–1720, 1981.

[7] F. M. Gardner, "A BPSK/QPSK timing-error detector for sampled receivers," *IEEE Trans. Commun.*, vol. 34, no. 5, pp. 423–429, 1986.

[8] M. Oerder and H. Meyr, "Digital filter and square timing recovery," *IEEE Trans. Commun.*, vol. 36, no. 5, pp. 605–612, 1988.

[9] T. M. Schmidl and D. C. Cox, "Robust frequency and timing synchronization for OFDM," *IEEE Trans. Commun.*, vol. 45, no. 12, pp. 1613–1621, 1997.

[10] J.-J. van de Beek, M. Sandell, and P. O. Börjesson, "ML estimation of time and frequency offset in OFDM systems," *IEEE Trans. Signal Process.*, vol. 45, no. 7, pp. 1800–1805, 1997.

[11] S. M. Alamouti, "A simple transmit diversity technique for wireless communications," *IEEE J. Sel. Areas Commun.*, vol. 16, no. 8, pp. 1451–1458, 1998.

[12] E. Arıkan, "Channel polarization: a method for constructing capacity-achieving codes for symmetric binary-input memoryless channels," *IEEE Trans. Inf. Theory*, vol. 55, no. 7, pp. 3051–3073, 2009.

[13] R. G. Gallager, "Low-density parity-check codes," *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, 1962.

[14] T. Starr, J. M. Cioffi, and P. J. Silverman, *Understanding Digital Subscriber Line Technology*. Prentice Hall, 1999.

[15] D. N. Godard, "Self-recovering equalization and carrier tracking in two-dimensional data communication systems," *IEEE Trans. Commun.*, vol. 28, no. 11, pp. 1867–1875, 1980.

[16] T. Pfau, S. Hoffmann, and R. Noé, "Hardware-efficient coherent digital receiver concept with feedforward carrier recovery for M-QAM constellations," *J. Lightw. Technol.*, vol. 27, no. 8, pp. 989–999, 2009.

[17] S. J. Savory, "Digital coherent optical receivers: algorithms and subsystems," *IEEE J. Sel. Topics Quantum Electron.*, vol. 16, no. 5, pp. 1164–1179, 2010.

[18] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[19] R. H. Clarke, “A statistical theory of mobile-radio reception,” *Bell Syst. Tech. J.*, vol. 47, no. 6, pp. 957–1000, 1968.

[20] 3GPP TS 36.104, *Evolved Universal Terrestrial Radio Access (E-UTRA); Base Station (BS) radio transmission and reception*, Annex B.

[21] ANSI T1.417-2001, *Spectrum Management for Loop Transmission Systems*.

[22] D. Gloge and E. A. J. Marcatili, “Multimode theory of graded-core fibers,” *Bell Syst. Tech. J.*, vol. 52, no. 9, pp. 1563–1578, 1973.

[23] R. Olshansky and D. B. Keck, “Pulse broadening in graded-index optical fibers,” *Appl. Opt.*, vol. 15, no. 2, pp. 483–491, 1976.

[24] D. Cunningham and P. Dawe, *Review of the 10 Gigabit Ethernet Link Model*, white paper, Agilent Technologies, 2002.

[25] S. E. Golowich, P. Pepeljugoski *et al.*, “Modeling and simulation of next-generation multimode fiber links,” *J. Lightw. Technol.*, vol. 21, no. 5, 2003.

[26] L. A. Coldren, S. W. Corzine, and M. L. Mašanović, *Diode Lasers and Photonic Integrated Circuits*, 2nd ed. Wiley, 2012.

[27] Y. R. Zheng and C. Xiao, “Simulation models with correct statistical properties for Rayleigh fading channels,” *IEEE Trans. Commun.*, vol. 51, no. 6, pp. 920–928, 2003.

[28] 3GPP TR 38.901 v19.1.0, *Study on channel model for frequencies from 0.5 to 100 GHz*, 2025.

[29] ITU-R Recommendation P.525-5, *Calculation of free-space attenuation*, 2024.

[30] ITU-T Recommendation G.996.1, *Test procedures for digital subscriber line (DSL) transceivers*, 2001.

[31] A. R. Djordjevic, R. M. Biljic, V. D. Likar-Smiljanic, and T. K. Sarkar, “Wideband frequency-domain characterization of FR-4 and time-domain causality,” *IEEE Trans. Electromagn. Compat.*, vol. 43, no. 4, pp. 662–667, 2001.

[32] ITU-T Recommendation G.652, *Characteristics of a single-mode optical fibre and cable*, 2024.

[33] O. V. Sinkin, R. Holzlöhner, J. Zweck, and C. R. Menyuk, “Optimization of the split-step Fourier method in modeling optical-fiber communications systems,” *J. Lightw. Technol.*, vol. 21, no. 1, pp. 61–68, 2003.

[34] D. Marcuse, C. R. Menyuk, and P. K. A. Wai, “Application of the Manakov-PMD equation to studies of signal propagation in optical fibers with randomly varying birefringence,” *J. Lightw. Technol.*, vol. 15, no. 9, pp. 1735–1746, 1997.

[35] T. L. Koch and J. E. Bowers, “Nature of wavelength chirping in directly modulated semiconductor lasers,” *Electron. Lett.*, vol. 20, no. 25, pp. 1038–1040, 1984.

[36] M. A. Al-Habash, L. C. Andrews, and R. L. Phillips, “Mathematical model for the irradiance probability density function of a laser beam propagating through turbulent media,” *Opt. Eng.*, vol. 40, no. 8, pp. 1554–1562, 2001.

[37] ITU-R Recommendation P.1814-1, *Prediction methods required for the design of terrestrial free-space optical links*, 2025.

[38] A. A. Farid and S. Hranilovic, "Outage capacity optimization for free-space optical links with pointing errors," *J. Lightw. Technol.*, vol. 25, no. 7, pp. 1702–1710, 2007.

[39] *Optical Fiber Communications*, course notes, OptCom group, Politecnico di Torino, 2024–2025. Part I: IM/DD systems; Part II: coherent systems, optical amplification, nonlinear propagation and Raman amplification.

[40] X. Shi, C. Qi, G. Wang, and J. Hu, "Rate-equation-based VCSEL model and simulation," in *Proc. IEEE Int. Conf. on Microwave Technology and Computational Electromagnetics*, 2009, pp. 503–506.

[41] P. V. Mena, J. J. Morikuni, S.-M. Kang, A. V. Harton, and K. W. Wyatt, "A comprehensive circuit-level model of vertical-cavity surface-emitting lasers," *J. Lightw. Technol.*, vol. 17, no. 12, pp. 2612–2632, 1999.

[42] J. J. Morikuni, P. V. Mena, A. V. Harton, K. W. Wyatt, and S.-M. Kang, "Spatially independent VCSEL models for the simulation of diffusive turn-off transients," *J. Lightw. Technol.*, vol. 17, no. 1, pp. 95–102, 1999.

[43] P. Poggiolini, "The GN model of non-linear propagation in uncompensated coherent optical systems," *J. Lightw. Technol.*, vol. 30, no. 24, pp. 3857–3879, 2012.

[44] A. Carena, G. Bosco, V. Curri, Y. Jiang, P. Poggiolini, and F. Forghieri, "EGN model of non-linear fiber propagation," *Opt. Express*, vol. 22, no. 13, pp. 16335–16362, 2014.